
SOFTWARE ENGINEERING

NOTEBOOK

Table of Contents

Table of Contents	2
I C++ Foundations	17
1 C++	19
1.1 C++: General Knowledge and Good Practices	19
1.1.1 Rule of three/five/zero	19
1.1.2 The Most Vexing Parse	21
1.1.3 General Topics	22
1.1.4 Classes and Inheritance	31
1.1.5 Beyond Classes	49
1.1.6 Value Categories	56
1.1.7 Reference Binding	58
1.1.8 lvalue references	58
1.1.9 Reference Collapsing	59
1.2 Contemporary C++: new language and library features	60
1.2.1 C++11 new language features	60
1.2.2 C++11 new library features	78
1.2.3 C++14 new language features	85
1.2.4 C++14 new library features	88
1.2.5 C++17 new language features	89
1.2.6 C++17 new library features:	97
1.2.7 C++20 new language features	104
1.2.8 C++20 new library features	114
1.2.9 C++23 (to expand)	120
C++ Bibliography	121
2 Idioms	123
2.1 C++ Idioms	123
2.1.1 PImpl Idiom	123
2.1.2 Fast PImpl Idiom	124
2.1.3 Handle/Object Idiom	125

2.1.4	Copy and Swap Idiom	125
2.1.5	RAII - Resource Acquisition is Initialization	126
2.1.6	DBDII - Dynamic Binding During Initialization Idiom	127
2.1.7	Named Constructor Idiom	127
2.1.8	Construct On First Use Idiom	128
2.1.9	Nifty Counter Idiom	128
2.1.10	Named Parameter Idiom	129
2.1.11	Virtual Friend Function Idiom	129
2.1.12	C RTP - Curiously recurring template pattern	130
2.1.13	Type Erasure	131
	Idioms Bibliography	132
3	Memory Management	133
3.1	Memory Management	133
3.1.1	Memory Allocation and Deallocation in C	135
3.1.2	Memory Allocation and Deallocation in C++	135
	Memory Management Bibliography	141
4	Smart Pointers	143
4.1	Smart Pointers in C++	143
4.1.1	std::unique_ptr	145
4.1.2	std::shared_ptr	148
4.1.3	std::weak_ptr	150
4.1.4	std::auto_ptr	152
	Smart Pointers Bibliography	153
5	Containers	155
5.1	Container catalog	157
5.1.1	Sequence Containers	157
5.1.2	Associative Containers	158
5.1.3	Unordered Associative Containers	158
5.1.4	Ordered vs unordered associative	159
5.1.5	How a hash table works (sketch)	159
5.1.6	Complexity cheat sheet	160
5.1.7	Container Adaptors	161
5.1.8	Views	162
5.2	Invalidation of Iterators and References	163
	Containers Bibliography	164
6	Concurrency	165
6.1	Threads	165

6.1.1	Functions managing the current thread	166
6.2	Atomicity	166
6.2.1	Atomic operations	166
6.2.2	Atomic types	167
6.2.3	Operations on atomic types	168
6.2.4	Flag type and operations	169
6.2.5	Initialization	169
6.2.6	Memory synchronization ordering	170
6.2.7	Mutual exclusion	170
6.2.8	Generic mutex management	170
6.2.9	Generic locking management	172
6.2.10	Call once	173
6.2.11	Condition variables	173
6.2.12	Futures	175
6.2.13	Future errors	175
6.2.14	Memory Order Semantics	176
	Concurrency Bibliography	177

II Systems and Low Latency 179

7 Linux Processes and Threads 181

7.1	Processes vs Threads	181
7.1.1	Processes	181
7.1.2	Threads	181
7.2	Address Space	182
7.3	User Mode vs Kernel Mode	183
7.4	Linux Scheduler	183
7.4.1	Context Switch	183
7.4.2	Scheduling Classes	184
7.4.3	Real-Time Scheduling	184
7.5	Thread Affinity	185
7.5.1	CPU Affinity and Thread Pinning	185
7.5.2	Topology	185
7.5.3	IRQ Overview	186
7.5.4	Pinning APIs	186
	Linux Processes and Threads Bibliography	189

8 Memory Model and Synchronization 191

8.1	C++ Memory Model	191
-----	----------------------------	-----

8.1.1	Data Race	191
8.1.2	Race Condition	192
8.1.3	Happens-Before	192
8.1.4	Sequential Consistency	192
8.2	Memory Orders	193
8.2.1	C++ Reference	193
8.2.2	Release-Acquire Pairing	193
8.2.3	When to Weaken Ordering	194
8.3	Lock-Free Ring Buffer (SPSC)	194
8.3.1	SPSC Model	194
8.3.2	Head and Tail Indices	194
8.3.3	Producer and Consumer Ordering	195
8.3.4	Full and Empty Conditions	195
8.3.5	Cache-Line Layout	195
8.3.6	Implementation	195
8.4	Lock-Free Stack (Treiber)	196
8.4.1	Push and Pop	196
8.4.2	Implementation Sketch	197
8.4.3	ABA and Memory Reclamation	198
8.5	Lock-Free Queue (Michael–Scott)	198
8.5.1	Structure	198
8.5.2	Enqueue / Dequeue (logic)	199
8.5.3	Implementation Sketch	199
8.5.4	HFT Takeaway	201
8.6	Synchronization	202
8.6.1	C++ Reference	202
8.6.2	Spinlock	202
8.7	Performance Considerations	202
8.7.1	Lock Contention	202
8.7.2	Lock Convoy	203
8.7.3	Synchronization Scalability	203
	Memory Model and Synchronization Bibliography	204
9	Cache Coherence and Performance	205
9.1	CPU Cache Hierarchy	205
9.1.1	L1	205
9.1.2	L2	206
9.1.3	L3	206
9.2	Cache Coherence	206
9.2.1	MESI	206

9.3	False Sharing	207
9.3.1	What It Is	208
9.3.2	Why It Slows Things Down	208
9.3.3	Cache-Line Ping-Pong	208
9.3.4	Solutions	208
9.4	Profiling	209
9.4.1	perf stat	209
9.4.2	perf record and perf annotate	209
9.4.3	Cache Miss Events	210
9.4.4	Interpreting Results	210
9.4.5	Detecting False Sharing	210
9.5	Cache Locality	210
9.5.1	Spatial Locality	210
9.5.2	Temporal Locality	211
9.6	TLB	211
9.6.1	Translation Lookaside Buffer	211
9.6.2	TLB Misses	211
	Cache Coherence and Performance Bibliography	212
10	NUMA Architectures	213
10.1	NUMA Fundamentals	213
10.1.1	Local Memory	213
10.1.2	Remote Memory	214
10.2	NUMA Performance	214
10.2.1	Memory Access Latency	214
10.3	NUMA-Aware Programming	214
10.3.1	Allocation Locality	214
10.3.2	NUMA Thread and CPU Binding	215
	NUMA Architectures Bibliography	217
11	Linux Networking Fundamentals	219
11.1	TCP vs UDP	219
11.2	Which Calls Block?	220
11.3	Socket Lifecycle	220
11.3.1	socket()	221
11.3.2	bind()	221
11.3.3	listen()	221
11.3.4	accept()	221
11.3.5	connect()	222
11.3.6	send()	222

11.3.7	<code>recv()</code>	222
11.3.8	<code>close()</code>	222
11.4	Blocking vs Non-Blocking I/O	223
11.4.1	Blocking Sockets	223
11.4.2	Non-Blocking Sockets	223
11.4.3	<code>O_NONBLOCK</code>	223
11.4.4	<code>MSG_DONTWAIT</code>	223
11.5	Latency Knobs	223
11.5.1	<code>TCP_NODELAY</code>	223
11.5.2	<code>MSG_NOSIGNAL</code>	224
11.6	Common Errors	224
11.6.1	<code>EAGAIN / EWOULDBLOCK</code>	224
11.6.2	<code>EINPROGRESS</code>	224
11.6.3	<code>EADDRINUSE</code>	224
11.6.4	<code>ECONNREFUSED</code>	225
11.6.5	<code>ETIMEDOUT</code>	225
11.6.6	<code>EPIPE / ECONNRESET</code>	225
	Linux Networking Fundamentals Bibliography	226
12	Event Driven I/O	227
12.1	<code>select()</code>	227
12.1.1	How It Works	227
12.1.2	Limits	228
12.2	<code>poll()</code>	228
12.2.1	Improvements over <code>select()</code>	228
12.2.2	Still Linear	229
12.3	<code>epoll()</code>	229
12.3.1	Level Triggered	229
12.3.2	Edge Triggered	229
12.3.3	Epoll Scalability	230
12.4	Event Loop Architecture	230
	Event Driven I/O Bibliography	232
13	Memory Mapping and Shared Memory	233
13.1	File I/O	233
13.1.1	<code>open()</code>	233
13.1.2	<code>read()</code>	234
13.1.3	<code>write()</code>	234
13.1.4	<code>close()</code>	234
13.1.5	<code>lseek()</code>	234

13.2	Memory Mapping	234
13.2.1	mmap()	235
13.2.2	munmap()	235
13.3	Concepts	235
13.3.1	Memory-Mapped Files	235
13.3.2	Shared Memory	236
13.3.3	Lazy Loading	236
13.3.4	Page Faults	237
13.3.5	Copy-on-Write	237
	Memory Mapping and Shared Memory Bibliography	238
14	Kernel Bypass and Low Latency Networking	239
14.1	Why Kernel Bypass	239
14.1.1	Kernel Networking Overhead	239
14.1.2	System Call Overhead	240
14.1.3	Context Switches	240
14.2	Technologies	240
14.2.1	DPDK	240
14.2.2	Solarflare OpenOnload	240
14.2.3	AF_XDP	241
14.3	Zero-Copy Market-Data Parsing	241
14.4	When To Use Them	242
14.4.1	Trading	242
14.4.2	HPC	243
14.4.3	Ultra-Low Latency Systems	243
	Kernel Bypass and Low Latency Networking Bibliography	244
15	Low Latency Engineering	245
15.1	CPU Isolation	245
15.1.1	isolcpus: scheduling exclusion	245
15.1.2	nohz_full: adaptive-ticks / tickless	246
15.1.3	rcu_nocbs: RCU callback offload	246
15.1.4	irqaffinity / smp_affinity: hard-IRQ routing	246
15.2	Busy Polling	247
15.3	Cache Optimization	247
15.4	Avoiding Allocations	248
15.5	Latency vs Throughput	248
	Low Latency Engineering Bibliography	250
16	Quant Basics for Developers	251

16.1	What a Quant Developer Does	252
16.2	Trade Lifecycle: From Intent to P&L	253
16.3	Market Wire Protocols	254
16.3.1	Two planes	254
16.3.2	FIX	254
16.3.3	Native binary protocols	255
16.3.4	SBE and other compact encodings	256
16.3.5	Crypto and modern HTTP APIs	256
16.3.6	Orientation table	256
16.4	Market Structure	256
16.4.1	What is a financial market?	257
16.4.2	Market participants	257
16.4.3	Exchange vs OTC	257
16.4.4	Major venues (orientation only)	258
16.5	Asset Classes and Instruments	258
16.5.1	Asset classes (tour sheet)	258
16.5.2	Spot	259
16.5.3	Forward contracts	259
16.5.4	Futures	259
16.5.5	Options (light intro)	260
16.5.6	Swaps (panorama)	260
16.5.7	Why derivatives exist	260
16.5.8	Spot vs futures — interview staple	261
16.6	Spot vs Futures Infrastructure	261
16.6.1	Spot workflows	261
16.6.2	Futures workflows	262
16.6.3	Implied orders and synthetic liquidity	262
16.7	Crypto Markets (Compact)	263
16.8	Microstructure Essentials	263
16.8.1	Bid, ask, mid, spread, liquidity	264
16.8.2	L1 / L2 / L3 and matching	264
16.8.3	Order types and time-in-force	264
16.8.4	Market-data shapes (recap)	265
16.9	Stocks and Options in Plain Language	265
16.9.1	Stock	265
16.9.2	Derivative / option	265
16.9.3	Why anyone buys an option	266
16.10	Interest Rates, Dividends, and Forward Price	266
16.11	Volatility	267

16.12	Modelling the Stock: Random Walk and GBM	267
16.12.1	Discrete warm-up	267
16.12.2	Risk-neutral measure (pricing only)	267
16.13	Black–Scholes: Assumptions and the Big Idea	267
16.14	The Black–Scholes PDE	268
16.15	Closed-Form Price (European Call and Put)	268
16.15.1	Put–Call Parity	268
16.16	The Greeks: Sensitivities of the Price	269
16.16.1	Delta hedging	269
16.16.2	Gamma and theta (long option)	269
16.17	P&L Explain: Accounting for Daily Profit and Loss	269
16.18	Implied Volatility and the Smile	270
16.19	Beyond Vanilla (Short Glossary)	270
16.20	Computing Greeks in Code	271
16.21	Implementation Pitfalls	272
16.22	Monte Carlo and PDE (When BS Is Not Enough)	272
16.23	Complex Derivatives: Systems Notes	272
16.23.1	IRS and CDS (systems view)	272
16.23.2	American options	273
16.23.3	Streaming greeks on a large options universe	273
16.24	Production Architecture (Lite)	274
16.25	Testing and Python/C++ Split	274
16.26	Interview Cheat Sheet (Read Last)	275
16.27	Typical Interview Questions	276
16.27.1	Markets and instruments	276
16.27.2	Market data and the order book	276
16.27.3	Latency and systems (quant-dev)	279
16.27.4	Pricing, risk, and greeks	281
16.27.5	Behavioural / role	281
	Quant Basics for Developers Bibliography	282

III Algorithms 283

17 Algorithms 285

17.1	Time Complexity	285
17.1.1	Amortized vs average-case	285
17.1.2	Big O notation	287
17.1.3	Omega notation	287
17.1.4	Theta notation	287

17.1.5	Meaning of previous notations - Insertion sort case	287
17.1.6	Running times of well known algorithms	289
17.1.7	Methods for solving recurrences	290
17.1.8	Comparison-sort lower bound	291
17.2	Space Complexity	291
17.3	Order Book	292
17.3.1	Price Levels and Market Depth	292
17.3.2	Bids and Asks	292
17.3.3	Data Structures	292
17.3.4	Add, Update, and Remove	294
17.3.5	Best Bid and Offer	295
17.3.6	Hot-Path Design	296
	Algorithms Bibliography	297

18 Algorithmic Problem-Solving Patterns 299

18.1	Arrays and Hash Maps	299
18.1.1	Frequency map	300
18.1.2	Lookup / complement	300
18.1.3	Prefix sum	300
18.1.4	Prefix / suffix aggregation	301
18.2	Two Pointers	301
18.2.1	Opposite-direction pointers	302
18.2.2	Same-direction pointers	302
18.2.3	Fast / slow pointers	302
18.3	Sliding Window	303
18.3.1	Fixed window	303
18.3.2	Variable window	303
18.3.3	Frequency map + window	304
18.4	Stack and Monotonic Stack	304
18.4.1	Matching / parsing	304
18.4.2	Next greater / smaller	305
18.4.3	Monotonic stack	305
18.5	Queue and Deque	306
18.5.1	BFS	306
18.5.2	Monotonic deque	307
18.6	Binary Search	307
18.6.1	Exact search	308
18.6.2	lower_bound / upper_bound	308
18.6.3	First / last valid element	308
18.6.4	Binary search on answer	309

18.7	Sorting-Based Patterns	309
18.7.1	Sort + scan	309
18.7.2	Sort + two pointers	310
18.7.3	Custom comparator	310
18.7.4	Partial sorting / <code>nth_element</code>	311
18.8	Intervals	311
18.8.1	Merge intervals	311
18.8.2	Overlap detection	312
18.8.3	Interval scheduling	312
18.8.4	Sweep line	312
18.9	Linked Lists	313
18.9.1	Reverse	313
18.9.2	Fast / slow	313
18.9.3	Cycle detection	314
18.9.4	Merge	314
18.10	Trees	315
18.10.1	DFS	315
18.10.2	BFS	315
18.10.3	BST	316
18.10.4	LCA	316
18.11	Graphs	316
18.11.1	DFS / BFS	316
18.11.2	Connected components	317
18.11.3	Cycle detection	317
18.11.4	Topological sort	317
18.11.5	Union–Find (DSU)	318
18.11.6	Shortest paths	319
18.12	Heaps	319
18.12.1	Top- K	319
18.12.2	K-way merge	320
18.12.3	Running median	320
18.12.4	Scheduling	321
18.13	Backtracking	321
18.13.1	Subsets	321
18.13.2	Permutations	322
18.13.3	Combinations	322
18.13.4	Pruning	323
18.14	Dynamic Programming	323
18.14.1	Memoization	323

18.14.2 Tabulation	323
18.14.3 1D / 2D DP	324
18.15Where C++ / systems interview topics live	324
18.16Interview Drill Sheet	325
Algorithmic Patterns Bibliography	326

IV Software Design 327

19 Design Patterns 329

19.1 Creational Patterns	330
19.1.1 Factory Method	330
19.1.2 Abstract Factory	332
19.1.3 Builder	335
19.1.4 Prototype	337
19.1.5 Singleton	339
19.2 Behavioral Patterns	341
19.2.1 Chain of Responsibility	341
19.2.2 Command	343
19.2.3 Iterator	345
19.2.4 Mediator	348
19.2.5 Memento	351
19.2.6 Observer	353
19.2.7 State	355
19.2.8 Strategy	357
19.2.9 Template Method	359
19.2.10 Visitor	361
19.3 Structural Patterns	363
19.3.1 Adapter	363
19.3.2 Bridge	365
19.3.3 Composite	367
19.3.4 Decorator	369
19.3.5 Facade	371
19.3.6 Flyweight	373
19.3.7 Proxy	375
19.4 Relations among Patterns (summary)	377
Design Patterns Bibliography	379

20 Solid Principles 381

20.1 Single-Responsibility	382
--------------------------------------	-----

20.2	Open-Close	382
20.3	Liskov Substitution	382
20.4	Interface Segregation	382
20.5	Dependency Inversion	383
	Solid Principles Bibliography	384
21	UML and OOP	385
21.1	UML	385
21.1.1	Class Diagram	385
21.1.2	Sequence Diagram	389
21.2	Object Oriented Programming	396
21.2.1	Polymorphism	396
21.2.2	Interface vs Abstract Class	397
	UML and OOP Bibliography	399
V	Other Topics	401
22	Python	403
22.1	PEP	403
22.2	Built-in Data Types	403
22.2.1	dict	403
22.2.2	list	403
22.2.3	set	403
22.2.4	frozenset	403
22.2.5	tuple	403
22.2.6	string	403
22.2.7	bytes	403
22.2.8	bytearray	403
22.3	unittest Module	404
22.3.1	Classes and functions	404
22.4	Decorator	405
22.5	Special Instance Method	406
22.6	Providing Multiple Ctors	406
	Python Bibliography	407
23	Software Testing	409
23.1	Seven Tesing Principles	409
23.2	SDLC vs. STLC	409
23.3	Python Unit Testing	409
23.3.1	unittest	409

23.3.2	doctest	412
23.4	C++ Unit Testing	412
23.4.1	gMock	412
23.5	Integration Test	412
23.6	Regression Test	413
23.7	Behavior-Driven Development	413
23.8	terms from the youtube video	413
23.9	Continous Integration Systems	413
	Testing Bibliography	415
24	AI	417
	AI Bibliography	418
VI	Reference	419
25	Dictionary	421
25.0.1	A	421
25.0.2	B	422
25.0.3	C	423
25.0.4	D	424
25.0.5	E	425
25.0.6	F	426
25.0.7	G	427
25.0.8	H	428
25.0.9	I	429
25.0.10	J	430
25.0.11	K	431
25.0.12	L	432
25.0.13	M	433
25.0.14	N	434
25.0.15	O	435
25.0.16	P	436
25.0.17	R	437
25.0.18	S	438
25.0.19	T	439
25.0.20	U	440
25.0.21	V	441
25.0.22	W	442
25.0.23	X	443

25.0.24 Z	444
Dictionary Bibliography	445
26 References	446



PART

C++ Foundations

1.1 C++: General Knowledge and Good Practices

1.1.1 Rule of three/five/zero

Rule of three

If a class requires a *user-defined destructor*, a user-defined *copy constructor*, or a user-defined *copy assignment operator*, it almost certainly requires all three.

Because C++ copies and copy-assigns objects of user-defined types in various situations (passing/returning by value, manipulating a container, etc), these special member functions will be called, if accessible, and if they are not user-defined, they are implicitly-defined by the compiler.

The implicitly-defined special member functions are typically incorrect if the class manages a resource whose handle is an object of non-class type (raw pointer, POSIX file descriptor, etc), whose destructor does nothing and copy constructor/assignment operator performs a "shallow copy" (copy the value of the handle, without duplicating the underlying resource).

```
1 class Buffer {  
2     char* data_;  
3     std::size_t size_;  
4 public:  
5     explicit Buffer(std::size_t n);  
6     ~Buffer(); // free data_  
7     Buffer(const Buffer& other); // deep copy
```

```

8     Buffer& operator=(const Buffer& other); // deep copy / self-
      assign safe
9 };

```

Classes that manage non-copyable resources through copyable handles may have to declare copy assignment and copy constructor private and not provide their definitions or define them as deleted. This is another application of the rule of three: deleting one and leaving the other to be implicitly-defined will most likely result in errors.

Rule of five

Because the presence of a user-defined destructor, copy-constructor, or copy-assignment operator prevents the implicit definition of the move constructor and the move assignment operator, *any class for which move semantics are desirable*, has to declare all five special member functions.

```

1 class Buffer {
2     char* data_;
3     std::size_t size_;
4 public:
5     explicit Buffer(std::size_t n);
6     ~Buffer();
7     Buffer(const Buffer&);
8     Buffer& operator=(const Buffer&);
9     Buffer(Buffer&&) noexcept;           // steal pointer
10    Buffer& operator=(Buffer&&) noexcept; // steal pointer
11 };

```

Unlike Rule of Three, failing to provide move constructor and move assignment is usually not an error, but a missed optimization opportunity.

Rule of zero

Classes that have custom destructors, copy/move constructors or copy/move assignment operators should deal exclusively with ownership (which follows from the Single Responsibility Principle). Other classes should not have custom destructors, copy/move constructors or copy/move assignment operators. (see [2])

This rule also appears in the C++ Core Guidelines as C.20: If you can avoid defining default operations, do.

When a *base class is intended for polymorphic use*, its destructor may have to be declared public and virtual. This blocks implicit moves (and deprecates implicit copies), and so the special member functions have to be declared as defaulted (see [3])

However, this makes the class prone to slicing, which is why polymorphic classes often define copy as deleted (see [4]), which leads to the following generic wording for the Rule of Five: If you define or =delete any default operation, define or =delete them all (see [5])

1.1.2 The Most Vexing Parse

Examples

The term "[most vexing parse](#)" was first used by Scott Meyers in 2001. While unusual in C, the phenomenon was quite common in C++ until the introduction of uniform initialization in C++11.

This issue is about ambiguous code line like

```
1 void f(double my_dbl)
2 {
3     int i(int(my_dbl));
4 }
```

in which the second line can be interpreted both as [a declaration of a variable](#) `i` whose initial value is obtained by converting `my_dbl` into an `int` and as [a function declaration](#) equivalent to

```
1 int i(int my_dbl);
```

A similar ambiguity arises for

```
1 struct Foo {};
2
3 struct FooKeeper {
4     explicit FooKeeper(Foo f);
5     int get_foo();
6 };
7
8 int main() {
9     FooKeeper foo_keeper(Foo());
10    return foo_keeper.get_foo();
11 }
```

in which the line

```
1 TimeKeeper time_keeper(Timer());
```

can be interpreted both as a variable definition and a function declaration. The C++ standard requires the second interpretation, which is not consistent with the previous code.

How to fix it

It is possible to solve this parsing issue in different ways.

In the type conversion example

- C-style cast

```
1     int i((int)my_dbl);
2
```

- C++ named cast (to prefer to the previous solution)

```

1      int i(static_cast<int>(my_dbl));
2

```

In the variable declaration example, since C++11 the preferred method is to use uniform brace initialization

```

1  //Any of the following work:
2  FooKeeper foo_keeper(Foo{});
3  FooKeeper foo_keeper{Foo()};
4  FooKeeper foo_keeper{Foo{}};
5  FooKeeper foo_keeper( {});
6  FooKeeper foo_keeper{ {} };

```

Prior to C++11, extra parenthesis or copy-initialization solves the most vexing parse issue

```

1  FooKeeper foo_keeper( /*Avoid MVP*/ (Foo()) );
2  FooKeeper foo_keeper = FooKeeper(Foo());

```

1.1.3 General Topics

Built-in / Intrinsic / Primitive Data Types

A **POD type** is a C++ type that has an equivalent in C, and that uses the same rules as C uses for initialization, copying, layout, and addressing.

The actual definition of a POD type is recursive and gets a little gnarly.

Here's a slightly simplified definition of POD:

- a **POD type's non-static data members** must be **public** and can be of any of these types: **bool**, **any numeric type** including the various char variants, **any enumeration type**, **any data-pointer type** (that is, any type convertible to void*), **any pointer-to-function type**, or **any POD type**, including **arrays of any of these**.
- **data-pointers and pointers-to-function** are okay, but **pointers-to-member** are not.
- **references** are not allowed
- a POD type can't have **constructors**, **virtual functions**, **base classes**, or an **overloaded assignment operator**.

When **initializing non-static data members** of built-in / intrinsic / primitive types, it is always better to initialize all non-static data members in the constructor's **initialization list**.

In order to initialize your built-in / intrinsic / primitive variable by an expression that the compiler doesn't evaluate solely at compile-time, concern should be put on the initialization order fiasco, explained in 1.1.4.

A function should return

- **void** if there is no need for a return value;

- local by value
- local by pointer
- data member by value
- data member by pointer
- data member by reference-to-nonconst
- data member by referenec-to-const
- shared_ptr
- local unique_ptr or shared_ptr
- other
-

Input/Output via <iostream> and <cstdio>

There are several reasons why <iostream> should be preferred to <cstdio>

- type-safeness is better guaranteed since <iostream> lets the compiler know the I/O object statically, while <cstdio>
- less error prone
- extensibility
- inheritance

check for invalid input

```

1  #include <iostream>
2  int main()
3  {
4      std::cout << "Enter a number, or -1 to quit: ";
5      int i = 0;
6      while (std::cin >> i) {      // GOOD FORM
7          if (i == -1) break;
8          std::cout << "You entered " << i << '\n';
9      }
10     // ...
11 }
```

skip invalid input

```

1  // ...
2  while ((std::cout << "How old are you? ")
3      && (!(std::cin >> age) || age < 1 || age > 200)) {
4      std::cout << "That's not a number between 1 and 200; ";
5      std::cin.clear();
```

```

6 std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n')
  );
7 }
8 // ...

```

Functioning of (std::cin >> foo)

stop processing past the end of file

Ending output lines with std::endl of '\n'

Providing printing for an entire hierarchy of classes

printOn() method vs. friend function

providing input for a class

reopen std::cin and std::cout in binary mode

writing/reading objects to/from a data file

sending objects to another computer

printing a char as a number

Const Correctness

ISO Cpp FAQ about [const correctness](#).

Const correctness consists in using the keyword `const` to prevent const objects from being mutated.

const and pointer combinations

Reading it right-to-left

<code>const X* p</code>	p is a pointer to an X that is constant.
<code>X const* p</code>	equivalent to <code>const X* p</code> .
<code>X* const p</code>	p is a const pointer to an X that is not const.
<code>const X* const p</code>	p is a const pointer to an X that is const.

The second combination is equivalent to the first one: the [const-on-the-right style](#) requires to write the const specifier on the right of the object it makes const. This style is also called [consistent const](#) since it is consistent with the way of defining a const member function. Moreover, it is also more consistent with pointer declarations.

const and reference combinations

Reading it right-to-left

<code>const X& r</code>	r aliases an X object, which can not be changed via r
<code>X const& r</code>	equivalent to <code>const X& r</code>
<code>X& const r</code>	nonsense functionally equivalent to <code>X& r</code>
<code>const X& const r</code>	nonsense functionally equivalent to <code>const X& r</code>

The equivalence between the second and the first combination is due to the same reason valid for the pointers.

The last two combinations are nonsense and should be avoided. They are redundant as a reference is always constant, in the sense that you can never reseal a reference to make it refer to a different object.

Logical and Physical State

When reasoning in terms of const-correctness, it's important to distinguish the logical and the physical state of an object.

On the inside, an object has a **physical state**, also called *concrete* or *bitwise* state. This state is the one visible to the programmer who implements the software. From the outside, the client has an access to the object restricted to public member functions and friend functions. What is visible is the **logical** or *abstract* or *meaningwise state*.

A lookup functionality for a collection-object is a good example to show how there could be bits in the object's physical state that have no corresponding elements in the object's logical state. The lookup functionality might improve the performance using a cache to store the already looked-up items. The implementation of the cache is part of the object's physical state but the implementation is internal and the details will probably not be exposed to the users, i.e. it is not part of the object's logical state.

constness of public member functions

The constness of a public method must make sense to the object's users, and those users can see only the object's logical state. Therefore, in this context the constness to take care of is the logical one.

- If a method changes any part of the object's logical state, it logically is a mutator; it should not be const even if (as actually happens!) the method doesn't change any physical bits of the object's concrete state.
- Conversely, a method is logically an inspector and should be const if it never changes any part of the object's logical state, even if (as actually happens!) the method changes physical bits of the object's concrete state.

return-by-reference and const member function

A const member function must not change nor allow its caller to change the logical state of the this object it's been called on.

If an inspector method needs to return a data member of a this object, it should return by const reference or by copy. For example

```
1 class Person
2 {
3     public:
4         const std::string& get_name() const;    // good one
5         std::string& get_lastname() const;     // wrong one
6         int get_age();                          // good one
7 }
```

Compilers won't always catch such a mistake, therefore it is responsibility of the programmer.

const overloading

const overloading is a way to achieve const correctness when there are an inspector and mutator method with the same name. One of the most common case is due to the subscript operators, which usually come in pair as in the following

```
1 class PersonList
2 {
3     public:
4         // Subscript operators often come in pairs
5         const Person& operator[] (unsigned index) const;
6         Person& operator[] (unsigned index);
7         // ...
8 };
```

Of course const-overloading can be used for other things than subscript operator.

Following some links to the isocpp FAQs

- [Subscript operator for Matric class](#)
- [Matrix class made simpler](#)
- [Matrix using templates](#)
- [Matrix using templates and vectors](#)
- [Class Template's syntax and semantics](#)

mutable keyword

mutable keyword helps achieving the const correctness, when the code needs to make changes to a const object's physical state. An example could be again the cached previous lookup in 1.1.3. In this case, the cache would be marked as mutable and the compiler would let the programmer change it even in case of a const object because it would know that a change of that attribute would reflect in a change in the physical state, leaving the logical state as it was before.

Using const is better than removing the constness with a `const_cast`

Error converting $F^{**} \rightarrow \text{const } F^{**}$

C++ allows the conversion $F^{**} \rightarrow F \text{ const}^{*}$ but gives an error if a program tries to implicitly convert $F^{**} \rightarrow \text{const } F^{**}$.

The reason why this conversion is not legal in C++ is that it would let silently and accidentally modify a const Foo object without a cast.

```

1  class Foo
2  {
3      public:
4      void modify(); // make some modification to the this object
5  };
6
7  int main()
8  {
9      const Foo x;
10     Foo* p;
11     const Foo** q = &p; // q now points to p;
12                        // this is (fortunately!) an error
13     *q = &x;           // p now points to x
14     p->modify();        // Ouch: modifies a const Foo!!
15     // ...
16 }

```

In case the third line of the main were legal, *q* would be pointing at *p* and the next line it would change *p* to point at *x*. This would made the `const` qualifier disappear. The last line would then use *p* ability to modify its referent and the program would end up modifying a `const Foo`.

In case of necessity, the solution is to change `const Foo**` to `const Foo* const*`.

References

A [reference](#) is an alias for an object. At the assembly implementation level, a reference *i* to an object *x* is typically the machine address of the object *x*. When the programmer writes an instruction like `i++`, the compiler generates the code to increment the value of the referenced object *x*.

Assigning to a reference changes the state of the referenced object.

In case a function returns a reference, the function call can appear on the left hand of an assignment operator:

```

1  class Array {
2  public:
3      int size() const;
4      float& operator[] (int index);
5      // ...
6  };
7  int main()
8  {
9      Array a;
10     for (int i = 0; i < a.size(); ++i)
11         a[i] = 7; // This line invokes Array::operator[](int)
12     // ...
13 }

```

Method chaining like `object.method1().method2()`, in which the returned object from `method1()` is the `this` object for `method2()` are possible because of return by ref-

erence.

The method chaining is used in the [Named Parameter Idiom](#), described in 2.1.10 (see also [68]).

Memory Management

The best techniques to prevent memory leaks rely on hiding allocation and deallocation inside more manageable types. Single objects can be managed through `make_unique` and `make_shared`, multiple objects can be managed through the container `vector` or `unordered_map`. Their memory management frees the programmer from explicit memory management, macros, casts, overflow checks, explicit size limits, pointers and similia.

Templates and the standard libraries make this use of containers, resource handles, etc., much easier than it was even a few years ago. The use of exceptions makes it close to essential.

In the following code, a `unique_ptr` is used as a resource handle in order not to cope with allocation/deallocation responsibilities when returning an object allocated on the freestore from a function

```
1  #include<memory>
2  #include<iostream>
3  using namespace std;
4  struct S {
5      S() { cout << "make an S\n"; }
6      ~S() { cout << "destroy an S\n"; }
7      S(const S&) { cout << "copy initialize an S\n"; }
8      S& operator=(const S&) { cout << "copy assign an S\n"; }
9  };
10 S* f()
11 {
12     return new S;    // who is responsible for deleting this S?
13 };
14 unique_ptr<S> g()
15 {
16     return make_unique<S>();    // explicitly transfer
17                                 // responsibility for deleting this S
18 }
19 int main()
20 {
21     cout << "start main\n";
22     S* p = f();
23     cout << "after f() before g()\n";
24     //S* q = g(); // this error would be caught by the compiler
25     unique_ptr<S> q = g();
26     cout << "exit main\n";
27     //leaks *p
28     //implicitly deletes *q
29 }
```

In case containers are not available, it is possible to use a memory leak detector as part of your standard development procedure, or plug in a garbage collector.

In C++11 a literal `nullptr` has been introduced and should be used as the null pointer value in place of the previous solutions (namely `NULL` and `0`).

The function of `delete` is deleting the data pointed by its operand. It must be used not more than once on the same pointer allocated with a `new`, assuming that pointer has not gone through a successive new assignment.

The `malloc()/free()` operations can not be mixed with `new/delete` operations. The reasons are that they do not operate on the same level: they allocate on `different heaps` and `the former work with raw memory` while `the latter work with constructed objects` whose proper construction and destruction are guaranteed.

More specifically, `malloc()` is a function that takes a number (of bytes) as its argument; it returns a `void*` pointing to uninitialized storage and this pointer needs to be converted to a proper type. `new` is an operator that takes a type and (optionally) a set of initializers for that type as its arguments; it returns a pointer to an (optionally) initialized object of its type.

`malloc()` reports memory exhaustion by returning `0`. `new` reports allocation and initialization errors by throwing exceptions (`bad_alloc`).

Another important reason to use `new` and `delete` is their `overridability`.

In case there is a need to use `realloc()`, it is better to use vector containers. When `realloc()` has to copy the allocation, it uses a bitwise copy operation, which will tear many C++ objects to shreds. C++ objects should be allowed to copy themselves.

C++ provides the programmer with `set_new_handler`, a function called by allocation functions whenever a memory allocation attempt fails.

In C++ there is not the need to check whether a pointer is null before delete-ing it. The process goes through two steps

```
1 // Original code: delete p;
2 if (p) {      // or "if (p != nullptr)"
3     p->~Fred();
4     operator delete(p);
5 }
```

in which `operator delete(p)` calls the memory deallocation primitive, `void operator delete(void* p)`. C++ does not guarantee that `delete` also null out its operand. One reason is that the operand of `delete` is not required to be an lvalue.

In case it is important to null out the pointer, a possible implementation can be

```
1 template<class T> inline void destroy(T*& p) { delete p; p = 0; }
```

Passing a reference has also the effect to prevent the a call of this function template on an rvalue.

For what concerns the call to the destructor at the end of the scope, heap allocation should be used only if an object needs to live beyond the lifetime of the scope you create it in. Even then, `make_unique` or `make_shared`. In rare cases in which heap allocation is really needed and `new` is used to create an object, `delete` must be used to destroy the

object, taking into account that code is no longer exception-safe.

In case an object must live in a scope only, heap allocation is not needed at all.

In case an array of objects is created with `new`, the `delete[]` operator must be used to destroy the array. There are several ways a compiler can keep memory of the number of elements to destroy

<https://isocpp.org/wiki/faq/compiler-dependencies#num-elems-in-new-array-overalloc>

<https://isocpp.org/wiki/faq/compiler-dependencies#num-elems-in-new-array-assocarray>

In case of interest, there are several code example to

- [allocating multidimensional arrays using new](#)
- [allocating multidimensional arrays encapsulating pointers in a class](#)
- [reworking the previous example in terms of templates](#)
- [building a multidimensiona array using `std::vector`](#)

In case a member function needs to delete this following requirements must be satisfied

- this object was allocated via `new` (not by `new[]`, nor by placement `new`, nor a local object on the stack, nor a namespace-scope / global, nor a member of another object)
- member function will be the last member function invoked on this object.
- the rest of your member function (after the `delete` this line) doesn't touch any piece of this object (including calling any other member functions or touching any data members). This includes code that will run in destructors for any objects allocated on the stack that are still alive.
- no one even touches the `this` pointer itself after the `delete` this line. In other words, you must not examine it, compare it with another pointer, compare it with `nullptr`, print it, cast it, do anything with it.

It's also possible to use the [Named Constructor Idiom](#) (see 2.1.7 and [64]) to force a class to be created via `new` rather than as local, namespace-scope, global or static

```
1 class Fred {
2 public:
3     // The create() methods are the "named constructors":
4     static Fred* create() { return new Fred(); }
5     static Fred* create(int i) { return new Fred(i); }
6     static Fred* create(const Fred& fred) { return new Fred(fred); }
7     // ...
8 private:
9     // The constructors themselves are private or protected:
```

```

10  Fred();
11  Fred(int i);
12  Fred(const Fred& fred);
13  // ...
14  };

```

In case Fred class is expected to have derived classes, the constructors must be in protected section of the class.

1.1.4 Classes and Inheritance

Classes and Objects

A **class** is the building block of Object Oriented programming. It defines a data type, that consists of a set of states and a set of operations which transition between those states. A class provides a set of (usually public) operations, and a set of (usually non-public) data bits representing the abstract values that instances of the type can have.

An object is a region of storage with associated semantics defined in the class it belongs to.

An interface is good when

- unnecessary details are intentionally hidden.
This reduces the user's defect-rate.
- users don't need to learn a new set of words and concepts.
This reduces the user's learning curve.

Encapsulation is the mechanism used to prevent unauthorized access to some piece of information or functionality of a class/object.

The key money-saving insight is to separate the volatile part of some chunk of software from the stable part.

The **volatile parts** are the implementation details. In case of a single class this part is encapsulated in the **private/protected** part of the class. Inheritance can also be used as a form of encapsulation.

The **stable parts** are the interfaces. Good ones provide simplified view in the vocabulary of users, and are designed from the outside-in. !!! PUT LINK TO OPERATOR OVERLOADING !!!

If there is a single class, the interface is simply the class's public member functions and friend functions. The private and/or protected parts of a class contain the class's implementation, which is typically where the data lives.

The clean interface separates the interface from its implementation, and encapsulation forces users to use the interface.

Encapsulation does not prevent a user to read the private and/or protected parts of a class, but does prevent the code from being dependent on the insides of the class.

A class method can directly access the non-public members of another instance of its class. This rule of C++ preserves the encapsulation. Otherwise there would be the need

to provide a public `get_method` for each private variable used by the assignment operator, the copy constructor, the comparison operators, the binary arithmetic and bitwise operators, and many others.

Methods and friends of a derived class can access the protected base class members of any of its own objects, but not others. Suppose the classes D1 and D2 inherit from class B, that has a private `x` member. In this scenario, the compiler let D1's members and friend directly access the `x` member of any D1. The compiler gives a compile-time error if D1 member or friend tries to directly access the `x` member of anything it does not know is at least a D1, such as via a `B*` pointer, a `B&` reference, a `B` object, a `D2*` pointer, a `D2&` reference, a `D2` object.

The members and base classes of a struct are public by default, while in class, they default to private. For the other aspects are functionally equivalent.

To define an in-class constant useable in a compile time constant expression, there are three choices

```

1  class X {
2      constexpr int c1 = 42; // preferred since C++11
3      static const int c2 = 7;
4      enum { c3 = 19 };
5      array<char, c1> v1;
6      array<char, c2> v2;
7      array<char, c3> v3;
8      // ...
9  };

```

In case the constant isn't needed in a compile time constant expression

```

1  class Z {
2      static char* p; // initialize in definition
3      const int i; // initialize in constructor
4  public:
5      Z(int ii) : i(ii) { }
6  };
7  char* Z::p = "hello, there";

```

It is possible to bind a reference to a static data member (to take its address), if and only if it has an out-of-the-class definition

```

1  class AE {
2      // ...
3  public:
4      static const int c6 = 7;
5      static const int c7 = 31;
6  };
7  const int AE::c7; // definition
8      void byref(const int&);
9  int f()
10 {
11     byref(AE::c6); // error: c6 not an
        lvalue
12     byref(AE::c7); // ok

```



```

13     const int* p1 = &AE::c6;    // error: c6 not an lvalue
14     const int* p2 = &AE::c7;    // ok
15     // ...
16 }

```

Like C, C++ doesn't define layouts, just semantic constraints that must be met.

The size of an empty class isn't zero to ensure that two different objects will have different addresses. For the same reason, new always return pointers to different objects.

The Empty Base Optimization (see [42]) is a mandatory requirement on class layout since C++11, and requires that the size of any object or member subobject is required to be at least 1 even if the type is an empty class type'

Constructors

Destructors

Destructors are used to release any resource allocated by the object, like a semaphore, a file and so on. A very common case is delete-ing the object created using new.

Destructors are invoked on a local scope in reverse order of construction in the code.

The objects stored in an array are destructed in reverse order of construction as well.

The sub-objects are destructed in reverse order of construction as well. For example, having this structure

```

1  struct derived: base0, base1
2  {
3      member0 m0_;
4      member1 m1_;
5      ~derived()
6      {
7          local0 l0;
8          local1 l1;
9      }
10 }

```

when an derived object goes out of scope, destructors are called in this order: local1(), local0(), member1(), member0(), base1(), base0()

Destructors can not be overloaded.

Destructors need not to be called on local variables. In case it is needed to destroy a local object in a specific point of a local scope, it is possible to use an artificial block {...}. In case it's not possible to wrap the object with an artificial block, the solution is to create a function with similar duties.

If the object to destroy was allocated via new, a call to the destructor isn't needed unless it was a placement new

```

1  void someCode()
2  {
3      char memory[sizeof(Fred)];
4      void* p = memory;
5      Fred* f = new(p) Fred();

```

```

6 // ...
7 f->~Fred(); // Explicitly call the destructor for the placed
  object
8 }

```

Is there a placement delete?

From a class destructor, it is not needed to explicitly call destructor of neither member objects nor base classes.

Is there a way to force new to allocate memory from a specific memory area?

Assignment Operator

Operator Overloading

Friends

Cppreference documentation about Friends

A **friend** can be a **function**, a **class**, and both their **template** version.

The friend declaration appears in a class body and grants a function or another class access to **private** and **protected** members of the class where the friend declaration appears.

friend declaration helps enforcing encapsulation. Indeed, granting access to private and protected members removes the burden of creating public **getters** and **setters**, since in most of the cases these methods destroy encapsulation, exposing private and protected details which make no sense outside of the class.

The disadvantage of friend functions is that they require extra code when dynamic binding is needed. Indeed, the friend function should call a hidden (usually protected) virtual member function, as explained in details in 2.1.11.

Friendship *is not inherited, transitive or reciprocal*:

- a derived class of a friend class is not a friend class;
- a friend of a friend is not necessarily a friend;
- if class A declares class B its friend, B objects have access to A objects' private data, but the opposite is not true.

Usually, it is better to rely on member function if possible and friend ones when it is really needed.

Inheritance - Basics

static initialization

Inheritance - virtual member functions

A **virtual member function** is the way of C++ to allow derived classes to replace the implementation provided by the base class. The compiler makes sure the correct derived class' implementation is called, even when the object is accessed through a pointer to the base class.

The reason why the member function are not virtual by default is that many classes are not designed to be used as a base class. Moreover, the virtual function call mechanism needs requires an overhead of memory, typically one word per object.

Dynamic Binding and Static Typing

A pointer to an object may actually point to a class that is derived from the class of the pointer. Thus there are two different types to consider: the (static) type of the pointer and the (dynamic) type of the actual pointed object.

Static typing means that the legality of a member function invocation is checked as early as possible, i.e. at compile time by the compiler itself. In this kind of check, the compiler uses the **static type** of the pointer to determine whether the member function invocation is correct, meaning that the type pointed by the pointer can handle the member function.

Dynamic binding is the virtual function call mechanism, which checks the correctness of the function call at the last possible moment, i.e. at run time and based on the dynamic type of the object pointed to.

Member Function call: virtual vs. non-virtual

The practical difference is that non-virtual member functions are resolved statically, i.e. the compiler checks the **type of the pointer or reference to the object** and selects the correct member function statically at compile time.

On the other side, a virtual member function call is resolved dynamically.

virtual Member Function Call - Overhead Estimation

As explained, a non-virtual member function is resolved statically based on the type of the pointer or reference to the object.

In contrast, **dynamic binding** select the correct virtual member function at run time following a compiler-based variant of the following technique: for each virtual function, the compiler create in each object an hidden **v-pointer** (virtual pointer) pointing to a global table called **v-table** (virtual table). One single **v-table** is created for each class which contains at least a **virtual** function.

To resolve a call to a **virtual** function, the run-time procedure follows the object's **v-pointer** to the class's **v-table**, and then follows the appropriate slot in the **v-table** to the method code.

The **space-cost** overhead consists of an extra pointer per object needing to be dynamically binded and another extra pointer per virtual method. The **time-space** overhead

compared to a normal function call consists of the two extra fetches to get the value of the *v-pointer* and a second one to get the address of the method.

Suppose to have a *Base* class with 5 *virtual* functions

```
1 // Original C++ source code
2 class Base
3 {
4     public:
5         virtual return_type virt0( /*...arbitrary params...*/ );
6         virtual return_type virt1( /*...arbitrary params...*/ );
7         virtual return_type virt2( /*...arbitrary params...*/ );
8         virtual return_type virt3( /*...arbitrary params...*/ );
9         virtual return_type virt4( /*...arbitrary params...*/ );
10        // ...
11};
```

There are three steps the compiler goes through

First step: building a static table

Compiler build a static table containing 5 function-pointers, most likely while compiling the .cpp that define the Base's first non-inline virtual function.

Let's call this table *Base::__vtable*. If a function pointer fits into one machine word, the *v-table* adds 5 hidden words of memory overall.

```
1 // FunctionPtr is a generic pointer to a generic member function
2 FunctionPtr Base::__vtable[5] = {
3     &Base::virt0, &Base::virt1, &Base::virt2,
4     &Base::virt3, &Base::virt4 };

```

Second step: adding an hidden pointer

An hidden pointer called *v-pointer* is added by the compiler to *each object of Base* class as it was an hidden data member.

```
1 // Original C++ source code
2 class Base
3 {
4     public:
5         // ...
6         FunctionPtr* __vptr; // Supplied by the compiler,
7                               // hidden from the programmer
8         // ...
9 };

```

Third step: initializing this->__vptr

Compiler initializes *this->__vptr* within each constructor in order to let each object's *v-pointer* to point at its class *v-table*. This step could be thought of as if the following instruction is added in each constructor's *init list*.

```

1 Base::Base( /*...arbitrary params...*/ )
2 : __vptr(&Base::__vtable[0]) // Supplied by the compiler,
3                               // hidden from the programmer
4 // ...
5 {
6 // ...
7 }

```

When the code defines a `Der` class which inherits from the `Base` class, the compiler repeats the first and third steps, but *skips the second step*.

In the **first step**, compiler creates another hidden *v-table*, in which the compiler replaces the virtual methods overridden in the derived class. For example, in case `Der` overrides `virt0()...virt2()` the `Der`'s *v-table* looks like

```

1 // Pseudo-code (not C++, not C)
2 // for a static table defined within file Der.cpp
3 FunctionPtr Der::__vtable[5] = {
4     &Der::virt0, &Der::virt1, &Der::virt2,
5     &Base::virt3, &Base::virt4 };

```

In the **third step**, the compilers for each `Der` object changes the *v-pointer* in order to make it point to its class *v-table*.

At this point, it is possible to call a virtual function with code like

```

1 void mycode(Base* p)
2 {
3     p->virt3();
4 }

```

The compiler rewrites the code like

```

1 void mycode(Base* p)
2 {
3     p->__vptr[3](p);
4 }

```

At this point

- a *first load* gets the v-pointer and stores it in a register which we call r1
- a *second load* gets the work at $r1 + 3 \times 4$ (if function-pointers are 4-bytes long) and stores it in a second register r2
- *compiler calls* the code at location r2

Therefore, it is possible to state that:

- a *minimal space-overhead* is needed to manage virtual functions
- virtual function calls are almost *as fast as* non-virtual function calls
- there is *no chaining effect*, i.e. the fetch-fetch-call mechanism is not dependent on how deep the inheritance gets.

Calling a Base Class virtual member function

When a virtual function is called by its *fully-qualified name*, the compiler skip the virtual call mechanism and uses the same mechanism used for non-virtual functions. Therefore, the fully-qualified virtual function is called by name rather than by slot-number.

To call the virtual function defined in the Base within the derived class, it is possible to write

```
1 void Der::f()
2 {
3     Base::f();
4 }
```

virtual destructor

A virtual destructor is required any time someone deletes a derived-class object through a base-class pointer. In this case, indeed, the destructor invoked matches the type of the pointed object ignoring the type of the pointer itself.

In general, a virtual destructor is needed

- if the class is meant to be *derived from*
- *and* if code uses `new Derived`
- *and* if code uses `delete p` in which p is a pointer to Base pointing to an actual object of Derived class.

To be shorter, destructor needs to be virtual any time there is a virtual method in the class. Indeed

- it's a general safety measure in case the class will be used as a base class;
- it costs nothing since the

virtual constructor

An idiom that allows to achieve a behavior that C++ does not really support.

[virtual-ctors](#)

[copy-of-abc-via-clone](#)

Inheritance - Proper Inheritance and Substitutability

Converting `Derived**` → `Base**`

Differently from the conversion `Derived*` → `Base*`, the conversion mentioned in the title does not compile, because it would be dangerous.

Even though a `Derived` object is a kind of `Base` object, in case the conversion in the title was possible the `Base**` could be dereferenced and it could be possible to make `Base*` point to an object of a *different derived class*.

This prohibition has similarities with 1.1.3.

container of `Thing` == kind-of container of `Everything`?

As a consequence of 1.1.4, a container of a certain `class Thing` objects is not a container of `class Anything` objects, even if `Thing` is a kind of `Anything`.

Is an array of `Derived` a kind-of array of `Base`?

In the same perspective of the previous two paragraphs, the answer is negative and easy to explain: a `Derived` object is larger than a `Base` object and it would mess up the pointer arithmetics in case it was possible to store both the kinds of objects in the same array.

Inheritance - Abstract Base Classes

At the design level, an `ABC` corresponds to an abstract concept. At the programming language level, an `ABC` is a class containing one or more pure `virtual` functions, as in the following example

```
1 class Shape
2 {
3 public:
4     virtual void draw() const = 0; // = 0 means "pure virtual"
5     // ...
6 };
```

Even though it is possible to provide a definition of a pure `virtual` function, it could be confusing.

Copy Constructor and Assignment Operator for a class containing a pointer to a (abstract) base class

If a class owns 1.1.4

Inheritance - What your mother never told you

final classes and virtual methods

Calling virtual function from non-virtual functions of base class

Inheritance - private and protected inheritance

Private and protected inheritance are expressed using respectively the keyword `private` and `protected` in place of `public` in the definition of the derived class.

`private` inheritance is a syntactic variant of the concepts of composition or aggregation. For example, the "`Car` has-a `Engine`" relationship can be implemented both using single composition and `private` inheritance. For the latter, an example is

```

1 class Car : private Engine
2 {
3     public:
4     Car() : Engine(8) {} // in case Engine has a ctor
5                           // which takes an int
6     using Engine::start;
7 }

```

`private` inheritance has both similarities and differences when compared to composition. Some of the similarities are

- a single `Engine` is present in both cases;
- in neither case a conversion `Car* → Engine*` can be done by users;
- `Car` has a `start()` method that calls `Engine`'s `start` method.

The differences are

- simple-composition is needed in case multiple `Engine` are contained in a `Car`;
- `private` inheritance can lead to unnecessary highlighting of multiple inheritance
- `private` inheritance allows member of `Car` to convert `Car* → Engine*`
- `private` inheritance allows access to `protected` members of the base class
- `private` inheritance allows `Car` to override `Engine`'s `virtual` functions
- `private` inheritance make it simpler to implement a `Car`'s `start()` method that calls the `Engine`'s `start()` method.

Which should I prefer: composition or private inheritance?

Composition should be generally preferred to `private` inheritance. The latter gives access to the internals and there could be many class to expose. This aspect could make the program harder to maintain and easy to break when something in the code changes. A scenario in which `private` inheritance is *preferred* is when a `class Foo` uses code in `Class Bar` and the code in the latter class invokes member functions in the former class. The solution is to let `class Foo` call non-virtual in `class Bar`, and `class Bar` calls its own (pure) `virtual`s, which are overridden by `class Foo`.

Should I pointer-cast from a private derived class to its base class?

It is generally not a good practice.

The conversion `PrivatelyDer* → Base*` (and likewise for the references) is safe and no cast is needed or recommended. However, users of `PrivatelyDer` should avoid this conversion, since a private decision is subject to change without notice.

How is protected inheritance related to private inheritance?

Similarly, `protected` inheritance allows to override `virtual` functions in the base class and does not claim the derived is a kind-of-base class.

Dissimilarly, `protected` inheritance allows derived classes of derived classes to know about the inheritance relationship. The benefit is the capability of the `protected derived class` to exploit the relationship to the base class. The *cost* is the fact that changes the relationship in the protected derived class can potentially break further derived classes.

Access rules with private and protected inheritance

Considering the following code

```
1 class B { /*...*/ };
2 class D_priv : private B { /*...*/ };
3 class D_prot : protected B { /*...*/ };
4 class D_publ : public B { /*...*/ };
5 class UserClass { B b; /*...*/ };
```

- none of the derived classes can access anything private in B;
- in `D_priv`, the `public` and `protected` of B parts are `private`
- in `D_prot` the public and protected parts of B are protected;
- in `D_publ` the `public` parts of B are public and the `protected` parts are protected (`D_publ` is-a-kind-of-a B);
- in `class UserClass` can access only the `public` parts of B.

The correct way to make a `public` member `f(int, float)` of B public in `D_prot` or in `D_priv` makes use of the `scope operator` as the in the following code

```
1 class D_prot : protected B
2 {
3     public:
4         using B::f;
5 }
```

Inheritance - Multiple and Virtual Inheritance

Do we really need multiple inheritance?

Every modern language with static type checking and inheritance provides multiple inheritance. In C++, an *ABC* serves usually as an interface and class can need many. Other language simply have a separate name for pure abstract classes interfaces. Using multiple inheritance is not really common but it is better than using workarounds.

Disciplines for using multiple inheritance

- Inheritance should be used if it removes `if/switch` statements. Indeed, while composition is meant for code reuse, inheritance's primary purpose is dynamic binding and the flexibility which comes with it.
- `pure ABCs` play a crucial role in the context of `multiple inheritance`, since classes with as little data as possible and almost all pure virtual methods help avoiding inheritance for code reuse and using it for interface sustainability'
- The **bridge pattern** or **nested generalization** are possible alternatives and a wise designer checks out all the ways before choosing the best.

Example for the guidelines

Imagine a context in which there are multiple variants of a similar concepts, for example land, water, air and space vehicles, and different power sources, for example gas, wind, nuclear power and so on.

Before tying up everything using multiple inheritance, there are two aspects to take into account choosing a good strategy:

- users need to call methods on a `Vehicle`-reference and expect the implementation of those methods to be specific to a concrete vehicle, like `LandVehicle`;
- users need a `Vehicle`-reference that refers to a concrete vehicle, for example `GasPoweredVehicle`, and want to call methods on that `Vehicle`-reference expecting the implementation to get overridden by `GasPoweredVehicle`.

In case both the conditions stand, multiple inheritance could be the correct choice. Better alternatives could still be available depending on a few more decision criteria.

In case there are **N** geographies (land, water, air, space, e.c.) and **M** power sources, there are at least 3 options.

- **Bridge Pattern**

`Vehicle` and `Engine` are two pure `ABCs`, concrete vehicles and engines are derived from. `Vehicle` interface has an `Engine*` and everything is matched at run-time.

This way the code grows *gracefully*, because there are only **N+M** derived classes to implement. There are several disadvantages as well. A first one is due to having at most **N+M** overrides and therefore the same number of concrete algorithms and adding more could be very difficult.

Bridge Pattern is not able to solve nonsensical choices, as a pedal powered space vehicles and to solve this issue extra checks are needed as well.

Another issue is the absence of an interface for all the gas powered vehicles, which are considered just a kind of `Vehicle`. On the other side, all gas powered vehicles share their code in derived class `GasPoweredEngine`.

- **Nested Generalization**

One of the hierarchies is chosen as primary, and suppose to choose the geography as primary in the current example.

Concrete classes as `LandVehicle` and `WaterVehicle` are derived from `Vehicle` and each would have further derived classes, one per power source. For example, `LandVehicle` would be the base class for `GasPoweredLandVehicle`, `NuclearPoweredVehicle` and so on.

This way, the codebase does not grow gracefully because $N \times M$ different derived classes need to be implemented, the advantage is the possibility to have the same number of different algorithms.

This approach provides with a fine granular control removing the nonsensical combination because the combinations are decided in the design and should be reasonable.

Analogously to *Bridge Pattern*, there is not a common base class for the specific vehicles, and finally it is not obvious how to share code between different vehicles with same power source.

- **Multiple Inheritance**

There are two different hierarchies as in the *Bridge Pattern*, the `Engine*` is no longer a data member of the vehicle classes though, and in its place roughly $N \times M$ derived classes are implemented below both the hierarchy of the geographies and the hierarchy of engines.

In this context, the concept of Engine changes and vehicle classes should be renamed, e.g. `GasPoweredEngine` would be renamed to `GasPoweredVehicle`. Moreover, `GasPoweredLandVehicle` multiply inherits from `GasPoweredVehicle` and `LandVehicle`. At the cost of not having a graceful growth, the gain is a fine granular control simply because nonsensical combinations like `PedalPoweredSpaceVehicle` are not created for the user.

In this model, it is also possible for any gas powered vehicle to share a common interface. Unlike the *Bridge Pattern*, the derived class can override the shared code to replace parts that are not ideal for the new class.

Visualization of tradeoffs

Given N and M , the tradeoffs of the approaches described in 1.1.4 are summarized in table 1.1

	Grow Gracefully	Low Code Bulk	Fine-Grained Control	Static Detect Bad Combos	Polimorphic on both sides	Share Code
Bridge						
Nested Gen.						
Multiple In.						

Table 1.1: *Bridge Pattern, Nested Generalization and Multiple Inheritance's tradeoffs.*

The meaning of the columns are the following

- **Grow Gracefully**

For the example of the previous paragraph, the growth is due to adding more geographies or power sources and in this case it is linear.

- **Low Code Bulk**

- **Fine Grained Control**

It takes into account the option of having a different algorithm for any of the different NxM possibilities or being stuck with a single solution for all the concrete classes of vehicles.

- **Static Detect Bad Combos**

- **Polymorphic on Both Sides**

- **Share Common Code**

Second Example for the guidelines

This example is more symmetric than the previous one, resulting in being better managed in using a multiple-inheritance solution.

Suppose to start with two categories of vehicles, for example `LandVehicle` and `WaterVehicle`, and later an `AmphibiousVehicle` is needed as well.

Given this conditions

- it is really needed to have an amphibious vehicle, instead of using the other classes with a bit of changes;
- users need a `LandVehicle` reference that refers to an `AmphibiousVehicle` and need to call methods on the reference expecting the actual implementation is specific to the referenced object;
- same as the previous point, with `WaterVehicle`.

in this case the *multiple inheritance* would be the best choice. Answering the question seen in 1.1.4 could help being more confident with this choice.

The Dreaded Diamond

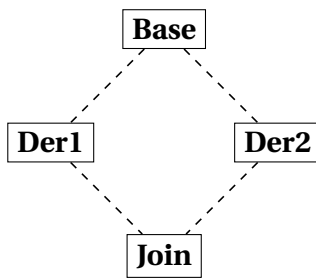


Figure 1.1: *A Dreaded Diamond.*

A *dreaded diamond* describes a class structure in which a particular class appears more than once in an inheritance hierarchy and looks like the diagram in figure 1.1.

Actually, C++ provides techniques to deal with the dreads of this triangle and this situation is not really dreaded but just something to be aware of.

The issue is due to `Base` being inherited twice with the consequence that any data member declared in `Base` appears twice within a `Join` object.

This duality creates ambiguities: when `Join` change a `Base`'s data member, it is not clear which derived class to go through for the change; likewise, a conversion `Join* → Base*` is ambiguous.

C++ lets resolve the ambiguity with the *scope operator*, there is a better solution though.

virtual Inheritance

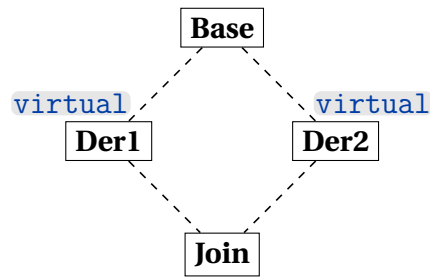


Figure 1.2: *A Nicer Diamond.*

To avoid duplicates in the `Join` class, every class that derives from `Base` needs to `virtual`-inherit. This way, an instance of `Join` will contain only one `Base` subobject and this solution is better than using fully qualified names.

delegate to a sister class

`virtual` inheritance helps achieving delegation to a sister class.

```
1  class Base
2  {
3  public:
4      virtual void foo() = 0;
5      virtual void bar() = 0;
6  };
7
8  class Der1 : public virtual Base
9  {
10 public:
11     virtual void foo();
12 };
13 void Der1::foo()
14 { bar(); }
15
16 class Der2 : public virtual Base
17 {
18 public:
19     virtual void bar();
20 };
21
22 class Join : public Der1, public Der2
23 {
24 public:
25     // ...
26 };
```

`Join` objects contain the `Der1`'s overridden `foo()` and the `Der2`'s overridden `bar()`. For this reason, when `Der1::foo()` calls `this->bar()` it ends up calling `Der2::bar()`.

Considerations needed when

- **using virtual inheritance**

Usually, `virtual` base classes and the ones that directly inherits from them are pure abstract classes, and contain very little if any data.

- **inheriting from a class that uses virtual inheritance**

The initialization list of the most-derived-class's constructor directly invokes the virtual base class's constructor.

C++ has a set of rules to make sure the `virtual` base class's constructor and destructor get called once per instance

- `virtual` base classes are constructed before all non-virtual base classes.
- constructors for `virtual` base classes anywhere in your class's inheritance hierarchy are called by the most derived class's constructor.

A consequence is the need to pass whatever parameters are required to call the `virtual` base class's constructor. In case of multiple `virtual` base classes, the most-derived class's constructor needs more parameters than other contexts require.

The practice of leaving the `virtual` base class without any data to initialize implies that the constructor probably takes no parameters.

- **using a class that uses virtual inheritance**

`dynamic_cast` is preferable to C-style downcasts.

Order of constructors in a multiple and/or virtual inheritance

First constructors to be executed are the virtual base classes' ones, in the order of appearance of base class names in the declaration of the derived class.

After the previous step, the construction order is from base class to derived class. Indeed, the compiler at first makes an hidden call to the non-virtual base classes' constructor in the derived class's constructor.

The previous rule is applied recursively: if `B1` inherits from `B1a` and `B1b`, and `B2` inherits from `B2a` and `B2b`, the final order of initialization is:

`B1a → B1b → B1 → B2a → B2b → B2 → D`.

The last order is determined by the order that the base classes appear in the declaration of the class.

Order of destructors in a multiple and/or virtual inheritance

For what concerns the non-virtual inheritance, the destructor order is the opposite of the constructor order:

`D → B2 → B2b → B2a → B1 → B1b → B1a`

Afterwards, the destructor of the virtual base classes that appear in the hierarchy are called, taking into account a class only when it uniquely appear for the first time.

1.1.5 Beyond Classes

Templates

A class template is a description of how to build a family of classes that all look basically the same, and a function template describes how to build a family of similar looking functions.

A class template is sometimes called [parameterized type](#) or [genericity](#).

Class templates are often used to build type safe containers (although this only scratches the surface for how they can be used).

```
1 // This would go into a header file such as "Array.h"
2 template<typename T>
3 class Array {
4 public:
5     Array(int len=10)
6         : len_(len), data_(new T[len]) { }
7     ~Array() { delete[] data_; }
8     int len() const { return len_; }
9     const T& operator[](int i) const { return data_[check(i)]; }
10    T& operator[](int i) { return data_[check(i)]; }
11    Array(const Array<T>&);
12    Array(Array<T>&&);
13    Array<T>& operator= (const Array<T>&);
14    Array<T>& operator= (Array<T>&&);
15 private:
16     int len_;
17     T* data_;
18     int check(int i) const {
19         assert(i >= 0 && i < len_);
20         return i;
21     }
22 };
```

The template in the previous example removes the need to rewrite the same class for different data-types, using a general identifier called T in the example.

Like for a normal class, templates' methods can be defined outside the class.

Instantiations of class templates need to be explicit about the parameters over which they are instantiating, for example

```
1 int main()
2 {
3     Array<int> ai;
4     Array<Array<int>> aai; // before C++11,
5                             // compiler would see a >> (right-shift)
6 }
```

The syntax for a function template is the following

```
1 template<typename T>
2 void swap(T& x, T& y)
```

```

3 {
4     T tmp = x;
5     x = y;
6     y = tmp;
7 }

```

Template functions usually do not need to be explicit about the parameters over which they are instantiating. The compiler can usually determine them automatically. Every time we used `swap()` with a given pair of types, the compiler will go to the above definition and will create yet another “template function” as an instantiation of the above.

Every once in a while it can be needed to explicitly select which version of a function template to use, because the compiler cannot deduce the type at all, or it would deduce the wrong type.

For example, the function template might have not any parameter of its template argument types and calls to this function need to explicitly select the template version.

It could be useful to force the compiler to select a function template that requires certain promotions on arguments. With a template like the following

```

1 template<typename T>
2 void g(T x)
3 {
4     // ...
5 }

```

`g("xyz")` would select `g<char*>(char*)` but to call the template for strings it is possible to use `g<std::string>("xyz")`.

It might also happen that the function template has two parameters of the same type but the user calls it with two different types

```

1 template<typename T>
2 void g(T x, T y);
3 int m = 0;
4 long n = 1;
5 g<int>(m, n);

```

In this case, the compiler can not deduce itself, and an explicit selection must be done.

It is possible to use [template specialization](#), in case the function template’s observable behavior is consistent for all the `T` types with the differences limited to implementation details. For example, a function template that represent an object as a string could make use of `std::boolalpha` to represent a `bool` as `false` and `true`, or `std::precision` to represent a `double`, and so on.

In case the function template has common code along with a relatively small amount of `T`-specific code,

Function templates participate in [name resolution for overloaded functions](#), the rules are different though. For a template to be considered in overload resolution, the type has to match exactly. If the types do not match exactly, the conversions are not considered and the template is simply dropped from the set of viable functions. That’s what is known as [SFINAE](#) — Substitution Failure Is Not An Error.

The reasons why ... are

- A template is not a class or a function. A template is a “pattern” that the compiler uses to generate a family of classes or functions.
- In order for the compiler to generate the code, it must see both the template definition (not just declaration) and the specific types/whatever used to “fill in” the template. For example, if you’re trying to use a `Foo<int>`, the compiler must see both the `Foo` template and the fact that you’re trying to make a specific `Foo<int>`.
- Your compiler probably doesn’t remember the details of one `.cpp` file while it is compiling another `.cpp` file. It could, but most do not.
This is called the [separate compilation model](#).

Container Classes

A contiguous array is the optimal construct for accessing a sequence of objects in memory, in terms of space and time.

In C++ there are better alternatives to plain C arrays, easier to write to read and less prone to errors but with same speed. Two of the main issues with C-style arrays are

- C array doesn’t know its own size
- the name of a C array converts to its first element almost always

The first issue implies that there can not be array assignment. Moreover, an array size must be determined at compile-time.

The second issue makes array interacts badly with inheritance. In the following code

```
1 class Base { void fct(); /* ... */ };
2 class Derived : Base { /* ... */ };
3 void f(Base* p, int sz)
4 {
5     for (int i=0; i<sz; ++i) p[i].fct();
6 }
7 Base ab[20];
8 Derived ad[20];
9 void g()
10 {
11     f(ab,20);
12     f(ad,20);    // disaster!
13 }
```

the array of `Derived` is treated as an array of `Base` and the different sizes of the two break the subscripting without the compiler clearly notifying it, as it would do with vectors. Some of the benefits that come with the container classes are the following

- it is possible to insert an element in almost any place without the need
-

Some of the things that can be done using arrays are

- `std::map` can be used as a lookup table;
- `std::map` can be used as a sparse array or matrix;
- `std::array` can be used in place of plain arrays, since it provides the user with boundary checks, memory management in case of exceptions
-
-

The storage for a `std::vector<T>` or `std::array<T, N>` is guaranteed to be [contiguous](#). This means that for any non-empty `std::vector<T>` or `std::array<T, N>` named `v` and an integer `n` in the range `0 ... v.size()-1`, it is always sure that `&v[0] + n == &v[n]`.

On the other side, `v.begin()` is not guaranteed to be a `T*` therefore it is not guaranteed to have the same value as `&v[0]`. Using `v.data()` is always safe though.

Containers provided by C++ are homogeneous, in the sense that they contain elements of the same type. In order to hold elements of several different types, it is possible to use a container of pointers to a polymorphic type. Homogeneous container are the easiest to use, give the best compile-time error messages and have no unnecessary run-time overhead.

To have an heterogeneous container, it is possible to define a common interface for all the elements and make a container of pointers:

```
1 class Io_obj { /* ... */ };    // the interface needed to take
    part in object I/O
2 vector<Io_obj*> v1o;           // if you want to manage the
    pointers directly
3 vector<shared_ptr<Io_obj>> v2; // if you want a "smart pointer"
    to handle the objects
```

An alternative is to use libraries like [boost::any](#).

If a pointer is used, objects in the container can be accessed through the pointers themselves, leveraging the polymorphic behavior as well. To know the exact type of the object, it is possible to use `dynamic_cast<>` or `typeid()`. To copy the container of different objects, the [virtual constructor idiom](#) is probably needed (see 1.1.4).

In order to make things easier, it is possible to rely on a `shared_ptr`.

It could be necessary to have a container of disjoint classes, that do not share a common base class. In this case, it is possible to create a handle class and make a container of handle objects. This approach comes with the benefit of encapsulating most of the memory management and object lifetime, but also with the downside of maintenance every time there are changes to the set of elements that can be contained.

[check and include](#)

Class Libraries

The [STL](#) is a library that consists mainly of container classes, along with iterators and algorithms to work with the content of the containers. Technically speaking it is no longer meaningful since the classes it provides have been integrated into the standard library. Sometimes, it is described as a separate library though.

A good resource about STL is

[Apache C++ Standard Library](#)

The National-Institute-of-Health's-class-library, [NIHCL](#), is a C++ translation of the Smalltalk class library. It can be retrieved [here](#).

For what concerns Numerical Recipes, following some resources

[LAPACK: Linear Algebra Subroutine Library](#).

[Netlib](#) is a collection of mathematical software, papers, and databases.

Other information about C++ class libraries can be found in the following resources

[Available C++ Libraries FAQ](#) maintained by Nikki Locke.

[Mathtools.net](#) A Resource for the Technical Computing Community.

[boost C++ libraries](#)

Exceptions and Error Handlings

Reference and Value Semantics

[virtual](#) data allows a derived class to change the exact class of a base class's member object. It is not directly supported by C++, it can be simulated though.

To [simulate a virtual data](#),

- the base class must have a [pointer to the member object](#);
- the derived class must provide a [new object to be pointed to by the base class's pointer](#)

The base class would also have one or more constructors that provide their own referent

Inline Functions

Conceptually similar to what happens with a `#define` macro, the compiler inline-expands a function call, inserting the function's code directly into the caller's code stream.

Because of the [procedurally integration](#) the optimizer can make on the called code, this expansion might improve the performances. Considering the following code

```
1 void f()  
2 {  
3     int x = /*...*/;  
4     int y = /*...*/;  
5     int z = /*...*/;  
6     // ...code that uses x, y and z...  
7     g(x, y, z);  
8     // ...more code that uses x, y and z...  
9 }
```

in a typical C++ implementation with registers and a stack, registers and parameters get written on the stack just before the call to $g(x)$, then the parameters get read from the stack into $g(x)$ and read again to restore the register after $g(x)$ returns to $f(x)$.

If the compiler inline-expands the call to $g(x)$, there would not be a function call and the not needed writes and reads that come with it.

It is not automatic that an inline function can improve the performance, since it might depend on multiple factors.

-
-
-

inline functions should be used instead of macros

<https://isocpp.org/wiki/faq/misc-technical-issues#macros-with-if>

<https://isocpp.org/wiki/faq/misc-technical-issues#macros-with-multi-stmts>

<https://isocpp.org/wiki/faq/misc-technical-issues#macros-with-token-pasting>

<https://isocpp.org/wiki/faq/big-picture#use-evil-things-sometimes>

In order to make both a non-member and function inline, the declaration stays the same but the definition must be prepended with the keyword `inline` and written in a [header file](#). In case the definition is placed in a `.cpp` file, unless the function is used only in that single file there will be an unresolved external error from the linker.

Another way to tell the compiler that a function is inline is to define the member function in the class body without the `inline` keyword in the definition. This way might be confusing for readers though, since it mixes the what (behavior) with the how (implementation). Therefore, in case of a highly reused class the definition outside the class should be preferred.

In case of an inline member function defined outside the class, the keyword must be put on the definition only, as this separation makes it easier for the user to read the header file to determine the [observable semantics](#) or [external behavior](#).

Pointers to Member Functions

Pointers to member functions are different than pointers to simple functions.

Considering the function

```
1 int f(char a, float b);
```

its type changes depending on whether it is an ordinary function or a non-static member of an imaginary Fred class.

```
1 int (*)(char, float);           // in case of an ordinary function
2                               // or a static member function
3 int (Fred::*)(char, float)     // in case of a member function
4                               // of class Fred
```

Pointers to member functions require elaborate techniques to be passed to signal handler, X event callback, system call that starts a thread/task and so on, since the interrupts are unable to provide the member function with a `this` pointer.

One possible solution is to use a static member as interrupt.

When creating pointers to member a function for a class, for example

```
1 class Fred
2 {
3 public:
4     int f(char x, float y);
5     // ...
6 };
```

it useful to rely on `typedef` to create a pointer `FredMemFn` to any member of `Fred` class that takes a `(char, float)`

```
1 typedef int (Fred::*FredMemFn)(char x, float y);
```

Consequently, it is trivial to declare a member-function pointer

```
1 int main()
2 {
3     FredMemFn p = &Fred::f;
4     // ...
5 }
```

or a function that receive a member-function pointer

```
1 void userCode(FredMemFn p)
2 { /*...*/ }
```

or a function that returns a member-function pointer

```
1 void userCode(FredMemFn p)
2 { /*...*/ }
```

C++17 provides the programmer with an `std::invoke`

```
1 void userCode(Fred& fred, FredMemFn p)
2 {
3     int ans = std::invoke(p, fred, 'x', 3.14);
4     // Would normally be: int ans = (fred.*p)('x', 3.14);
5     // ...
6 }
```

In case `std::invoke` is not accessible, a macro like the following

```
1 #define CALL_MEMBER_FN(object, ptrToMember) ((object).*(
    ptrToMember))
```

could help reducing code complexity

```
1 void userCode(Fred& fred, FredMemFn p)
2 {
3     int ans = CALL_MEMBER_FN(fred, p)('x', 3.14);
4     // Would normally be: int ans = (fred.*p)('x', 3.14);
```

```

5 // ...
6 }

```

In case the member function is constant, the typedef should look like

```

1 typedef int (Fred::*FredMemFn)(int) const;

```

[pointer-to-member access operators](#) are the last resource, in case neither the `std::invoke` nor a macro are possible options

```

1 class Fred { /*...*/ };
2 typedef int (Fred::*FredMemFn)(int i, double d);
3 void sample(Fred x, Fred& y, Fred* z, FredMemFn func)
4 {
5     x.*func(42, 3.14);
6     y.*func(42, 3.14);
7     z->*func(42, 3.14);
8 }

```

Neither a pointer to a member function nor a pointer to a function can be converted to a `void*`

A [functionoid](#) is similar to a function pointer but with more flexibility and thread safety.

Serialization and Unserialization

1.1.6 Value Categories

[Cppreference documentation about value categories.](#)

Due to the expressiveness of the C++ language, the terms [lvalue](#) and [rvalue](#) references evolved in a more complex set.

glvalues

[Generalized](#) lvalue is used to refer to something that could be either a [lvalue](#) or [xvalue](#).

rvalues

This term is inherited from C, in which [rvalues](#) can be on the [right](#) side of an assignment. In C++ this term can refer to things that are either [xvalues](#) and [prvalues](#).

lvalues

This term is also inherited from C, in which [lvalues](#) can be on the [left](#) side of an assignment. Therefore, the simplest one is the name of a variable in a declaration.

Any expression resulting in an [lvalue reference](#) is also an [lvalue](#).

```

1 int v;           // v is an lvalue of type int
2 A* p1;           // *p1 is an lvalue
3 A& r1 = v1;      // r1 is an lvalue
4 A& f();           // f() is an lvalue

```

Accessing members of objects through pointers or references yields [lvalues](#). For example, with the following code


```

1 struct Foo
2 {
3     Bar a;
4     Bar b;
5 };
6
7 Foo& f2();
8 Foo* p2;
9 Foo v2;

```

`f2().a`, `p2->b` and `v2.a` are all *lvalues* of type `Bar`.

String literals are *lvalues* of type array.

To conclude, a **named object** declared with an *rvalue reference* is an *lvalue*. The name *rvalue reference* indicates that it can bind to an *rvalue*, and once you've declared a variable and given it a name it is an *lvalue*.

```

1 void Foo(A&& bar)
2 {
3     // within Foo, bar is an lvalue
4     // which will only bind to rvalues
5 }

```

xvalues

This term indicates **eXpiring** value: an unnamed object that will be destroyed soon.

xvalues can be treated either as *glvalues* or *rvalues*, based on the context.

Usually, *xvalues* only arise through *explicit casts* and *function calls*. If an expression is **cast to an rvalue reference to some type T** then the result is an *xvalue of type T*

```

1 static_cast<A&&>(v1) // yields an xvalue of type A

```

If the return type of a function is an *rvalue reference* to some type **T**, the result is an *xvalue* of type **T**. For example, the `std::move` is declared as

```

1 template <typename T>
2 constexpr remove_reference_t<T>&& move(T&& t) noexcept;

```

`std::move(v1)` is an **xvalue** of type `A`. The type deduction rules deduce **T** to be `A&` because `v1` is an *lvalue*.

prvalues

Pure rvalue is an *rvalue* which is not an *xvalue*

Literals other than string literals are *prvalues*. For example, `42` is a *prvalue* of type `int` and `3.141f` is a *prvalue* of type `float`

Temporaries are *prvalues*. For example any temporaries created as a result of an implicit conversion is also a *prvalue*. In the following code

```

1 int consume_string(std::string&& s);
2 int i = consume_string("hello");

```

the string literal in the function call will implicitly convert to a temporary of type `std::string`, which can [bind to the rvalue reference](#) used for the function parameter since the temporary is a [prvalue](#)

1.1.7 Reference Binding

[cppreference documentation](#) about reference declaration.

The [main difference](#) between [lvalues](#) and [rvalues](#) is the way they bind to references. The differences in type deduction rules can have a big impact too.

1.1.8 lvalue references

These references are declared with a single ampersand &.

A [non-const lvalue reference will only bind to non-const lvalues](#) of the same type, or a class derived from the referenced type

```
1  class C : public A{};
2
3  int i = 42;
4  A a;
5  B b;
6  C c;
7  const A ca{};
8
9  A& r1 = a;
10 A& r2 = c;
11 A& r3 = b;      // error, wrong type
12 int& r4 = i;
13 int& r5 = 42;    // error, cannot bind rvalue
14 A& r6 = ca;     // error, cannot bind const object
15                // to non const ref
16 A& r7 = r1;
17 A& r8 = A();    // error, cannot bind rvalue
18 A& r9 = B().a;  // error, cannot bind rvalue
19 A& r10 = C();   // error, cannot bind rvalue
```

A [const lvalue reference can also bind to rvalues](#). The object bound to the reference must have the same type as the referenced type or a class derived from the referenced type. It is possible to bind both const and non-const values to a const lvalue reference.

```
1  const A& cr1 = a;
2  const A& cr2 = c;
3  const A& cr3 = b;      // error, wrong type
4  const int& cr4 = i;
5  const int& cr5 = 42;    // rvalue can bind OK
6  const A& cr6 = ca;     // OK, can bind const object to const
7                          ref
8  const A& cr7 = cr1;
9  const A& cr8 = A();    // OK, can bind rvalue
10 const A& cr9 = B().a;  // OK, can bind rvalue
```

```

10 const A&    cr10 = C();    // OK, can bind rvalue
11 const A&    cr11 = r1;

```

Binding a temporary object (i.e. a *prvalue*) to a const lvalue reference *extends the lifetime* of that temporary to the lifetime of the reference. In the previous example, it is possible to use r8, r9 and r10 later in the code.

Lifetime extension does not extend to references initialized from the first reference. If a function parameter is a const lvalue reference and gets bound to a temporary passed to the function call, the temporary is destroyed when the function returns. This happens also if the reference was stored in a longer-lived variable, such as a member of a newly constructed object.

volatile and *const volatile* lvalue references are much less used but behave as expected: *volatile T&* bind to a volatile or non-volatile, non-const lvalue of type T or a class derived from T. However, *volatile const* lvalue references do not bind to rvalues.

1.1.9 Reference Collapsing

In templates and typedef/using aliases it is possible to form a “reference to reference.” The *reference collapsing* rules then apply: an rvalue reference to an rvalue reference collapses to an rvalue reference; every other combination collapses to an lvalue reference.

```

1  using lref = int&;
2  using rref = int&&;
3  int n;
4
5  lref&   r1 = n; // int&
6  lref&& r2 = n; // int&
7  rref&   r3 = n; // int&
8  rref&& r4 = 1; // int&&

```

This is the mechanism behind forwarding references (*T&&* in a deduced context) and `std::forward`. See also Section 1.1.6.

1.2 Contemporary C++: new language and library features

This section summarizes language and library features by standard (C++11 onward). Entries are adapted from the open cheatsheet [AnthonyCalandra/modern-cpp-features](#), with links back to the source for full detail. Where this book has a dedicated chapter (smart pointers, concurrency, containers), prefer that chapter for depth and use the entries here as a standard-by-standard index.

1.2.1 C++11 new language features

Following a summary of the new language features introduced with C++11.

move semantics

Move semantics is a new way of moving resources around in an optimal way, since it helps *avoiding unnecessary copies* of temporary objects. It is based on rvalue references and plays an *important role in exception safety* for C++.

Moving an object means to transfer ownership of some resource it manages to another object.

The first benefit of move semantics is performance optimization. When an object is about to reach the end of its lifetime, either because it's a temporary or by explicitly calling `std::move`, a move is often a cheaper way to transfer resources. For example, moving a `std::vector` is just copying some pointers and internal state over to the new vector – copying would involve having to copy every single contained element in the vector, which is expensive and unnecessary if the old vector will soon be destroyed.

Moves also make it possible for non-copyable types such as `std::unique_ptr`s (smart pointers) to guarantee at the language level that there is only ever one instance of a resource being managed at a time, while being able to transfer an instance between scopes.

variadic templates

The `...` syntax creates a parameter pack or expands one.

A template parameter pack is a template parameter that accepts zero or more template arguments (non-types, types, or templates).

A template with at least one parameter pack is called a variadic template.

```
1 template <typename... T>
2 struct arity {
3     constexpr static int value = sizeof...(T);
4 };
5 static_assert(arity<>::value == 0);
6 static_assert(arity<char, short, int>::value == 3);
```

An interesting use for this is creating an initializer list from a parameter pack in order to iterate over variadic function arguments.

```

1 template <typename First, typename... Args>
2 auto sum(const First first, const Args... args) -> decltype(first
   ) {
3     const auto values = {first, args...};
4     return std::accumulate(values.begin(), values.end(), First{0});
5 }
6
7 sum(1, 2, 3, 4, 5); // 15
8 sum(1, 2, 3);      // 6
9 sum(1.5, 2.0, 3.7); // 7.2

```

rvalue references

An rvalue reference to T, which is a non-template type parameter (such as int, or a user-defined type), is created with the syntax T&&. Rvalue references only bind to rvalues. An [rvalue reference](#) can be used to extend the lifetime of temporary objects, making the referenced object modifiable through them. The reference must be const in order to bind to a const object, though const rvalue reference are much rarer.

```

1 const A make_const_A();
2 A&& rr1 = a; // error, cannot bind lvalue
3 // to rvalue reference
4 A&& rr2 = A();
5 A&& rr3 = B(); // error, wrong type
6 int&& rr4 = i; // error, cannot bind lvalue
7 int&& rr5 = 42;
8 A&& rr6 = make_const_A(); // error, cannot bind
9 // const object
10 // to non const ref
11 const A&& rr7 = A();
12 const A&& rr8 = make_const_A();
13 A&& rr9 = B().a;
14 A&& rr10 = C();
15 A&& rr11 = std::move(a); // std::move returns an rvalue
16 A&& rr12 = rr11; // error rvalue references
17 // are lvalues

```

rvalue references extend the lifetime of temporary objects in the same way const lvalue references do, therefore temporaries associated with rr2, rr5, rr7, rr8, rr9 and rr10 remain valid until the corresponding references are destroyed.

forwarding references

Cppreference documentation about [forward](#) utility.

Cppreference about [forwarding references](#).

Perfect forwarding is an important C++0x feature built on top of rvalue references. It provides the programmer with the capability automatically apply the move semantics, even when there are intervening function calls to separate the source and the destination of a move.

Examples include [constructors](#) and [setter functions](#), that can forward arguments they receive directly to the data member they are initializing or setting. Standard Library functions like [make_shared](#) "perfect-forwards" its arguments to the class constructor of whatever object the to-be-created `shared_ptr` is to point to.

The idiomatic expression of the perfect forwarding make use of the `std::forward`

```
1 template <typename T>
2 void wrapper(T&& arg)
3 {
4     wrapped(std::forward<T1>(t1), std::forward<T2>(t2));
5 }
```

initializer lists

Cppreference documentation about Initializer list

An object of type `std::initializer_list<T>` is a lightweight proxy object that provides access to an array of objects of type `const T`.

Uniform initialization syntax and semantics

C++98 provides the users with several ways of initializing an object. These ways depend on the type and the initialization context and have different ways of working.

Some examples can be the following code lines

```
1 string a[] = { "foo", " bar" };           // ok: initialize array
2 vector<string> v = { "foo", " bar" };     // error: initializer list
3                                           // for non-aggregate
4
5     vector
6 void f(string a[]);
7 f( { "foo", " bar" } );                  // syntax error:
8                                           // block as argument
```

and

```
1 int a = 2;                               // "assignment style"
2 int aa[] = { 2, 3 };                     // assignment style with list
3 complex z(1,2);                           // "functional style" initialization
4 x = Ptr(y);                               // "functional style" for
5                                           // conversion/cast/construction
6
7 int a(1);                                 // variable definition
8 int b();                                  // function declaration
9 int b(foo);                               // variable definition or function
10                                           declaration
```

C++11 introduces a [{}-initializer](#), which provides a much simpler and consistent way to perform initialization, for all different initialization context as in the following example

```
1 X x1 = X{1,2};
2 X x2 = {1,2};    // the = is optional
3 X x3{1,2};
```

```

4 X* p = new X{1,2};
5 struct D : X
6 {
7     D(int x, int y) :X{x,y} { /* ... */ };
8 };
9 struct S
10 {
11     int a[3];
12     S(int x, int y, int z) :a{x,y,z} { /* ... */ }; // solution
13                                     to
14                                     // old
15                                     problem
16 };

```

`{}`-initializer produces the same result in any context

```

1 X x{a};
2 X* p = new X{a};
3 z = X{a};           // use as cast
4 f({a});             // function argument (of type X)
5 return {a};         // function return value (function returning X)

```

`{}`-initializer is also useful to solve the **most vexing parse** problem, explained in 1.1.2.

static assertions

Assertions that are evaluated at compile-time.

```

1 constexpr int x = 0;
2 constexpr int y = 1;
3 static_assert(x == y, "x != y");

```

auto

Cppreference documentation about **auto**.

References in bibliography: [17], [18], [19].

This placeholder type specifier has the following meanings:

- for **variables**:
the variable that is being declared will be automatically deduced from its initializer;
- for **functions**:
the return type will be deduced from its return statements;
- for **non-type template parameters**:
the type will be deduced from the argument.

Following a summary of **contextes in which auto may appear**.

In the summary, the **rules for template argument deduction** are referenced.

- if used **in the type specifier of a variable** the variable type is deduced from the initializer list using the template argument deduction:

```
1 auto x = expr;
2
```

In case **no pointer or reference** is attached to auto, both const and references are ignored:

```
1 int y = 10;
2 int& r = y;
3 auto x = r;           // type is int (reference ignored)
4
5 const int y = 10;
6 auto x = y;           // type is int (const ignored)
7
8 int y = 10;
9 const int& r = y;
10 auto x = r;           // type is int
11                        // (const and reference ignored)
12
13 const int a[10] = {};
14 auto x = a;           // type is int*
15                        // (array to pointer conversion)
16
```

Modifiers which may accompany auto will participate in the type deduction. For example, in the code

```
1 const auto& i = expr;
2 template<class U> void f(constU& u);
3
```

the type deduced for the first code line is the same deduced for the argument u of the second line if the function call f(expr) was compiled.

Therefore **auto&&** may be deduced both as an **lvalue** reference and an **rvalue** reference.

In case **auto** is used to declare multiple variables, the deduced type must match.

- If used **in a function defined with the trailing return type syntax**, it is just part of the syntax and automatic type deduction is not performed

```
1 auto function_name(parameter_list) -> return_type
2 {
3     // Function implementation
4 }
5
```

- If used **as return type of a function defined without the trailing return type**, or **as a return type of a lambda expression**, the keyword auto indicates that the re-

turn type will be deduced from the return statement, using the rules for template argument deduction.

- If used **in combination with decltype** and the declared type of the variable is `decltype(auto)`, the keyword `auto` is replaced with the expression (or expression list) of its initializer and the actual type is deduced using the rules for `decltype`.
- If used **along with decltype in the return type of a function** and the return type of the function is `decltype(auto)`, the keyword `auto` is replaced with the operand of its return statement, and the actual return type is deduced using the rules of `decltype`.
- If used in the **type-id of a new expression**, the type is deduced from the initializer.
- If used **in the parameter declaration of a non-type template parameter**, for example `template<auto I> struct A;`, its type is deduced from the corresponding argument.
- if used **as a nested scope like in `auto::`** ???
- if used **in the parameter declaration of a lambda-expression**, such lambda expression is a *generic lambda*. This means that the following generic lambda will be written by the compiler as an instance of the templates which follows

```
1 auto g_lambda = [] (auto a) { return a; } ;
2 class /* unnamed */
3 {
4 public:
5     template<typename T>
6     T operator () (T a) const { return a; }
7 };
8
```

- if used **in a function parameter declaration**, like `void f(auto);`, the function declaration introduces an abbreviated function template [29]

auto-typed variables are deduced by the compiler according to the type of their initializer.

```
1 auto a = 3.14; // double
2 auto b = 1; // int
3 auto& c = b; // int&
4 auto d = { 0 }; // std::initializer_list<int>
5 auto&& e = 1; // int&&
6 auto&& f = b; // int&&
7 auto g = new auto(123); // int*
8 const auto h = 1; // const int
9 auto i = 1, j = 2, k = 3; // int, int, int
10 auto l = 1, m = true, n = 1.61; // error -- 'l' deduced to be int
    , 'm' is bool
11 auto o; // error -- 'o' requires initializer
```

Extremely useful for readability, especially for complicated types:

```
1 std::vector<int> v = ...;
2 std::vector<int>::const_iterator cit = v.cbegin();
3 // vs.
4 auto cit = v.cbegin();
```

Functions can also deduce the return type using auto. In C++11, a return type must be specified either explicitly, or using decltype like so:

```
1 template <typename X, typename Y>
2 auto add(X x, Y y) -> decltype(x + y) {
3     return x + y;
4 }
5 add(1, 2); // == 3
6 add(1, 2.0); // == 3.0
7 add(1.5, 1.5); // == 3.0
```

The trailing return type in the above example is the declared type (see section on decltype) of the expression `x + y`. For example, if `x` is an integer and `y` is a double, `decltype(x + y)` is a double. Therefore, the above function will deduce the type depending on what type the expression `x + y` yields. Notice that the trailing return type has access to its parameters, and this when appropriate.

lambda expressions

Cppreference about lambda expressions

A [lambda expression](#) produces a [closure](#), i.e. an unnamed function object of a unique compiler-generated class type that can capture variables from the enclosing scope.

captures	a comma-separated list of zero or more captures, optionally beginning with a capture-default
	A lambda expression can use a variable without capturing it if the variable • is a non-local variable or has static or thread local storage duration (in which case the variable cannot be captured), or • is a reference that has been initialized with a constant expression. A lambda expression can read the value of a variable without capturing it if the variable • has const non-volatile integral or enumeration type and has been initialized with a constant expression, or • is constexpr and has no mutable members.
tparams	a non-empty comma-separated list of template parameters used to provide names to the template parameters of a generic lambda
t-requires	adds constraints to tparams
front-attr	since C++23 study
params	the parameter list of operator() of the closure type
specs	A list of the following specifiers, each specifier is allowed at most once in each sequence. If not provided, the objects captured by copy are const • mutable allows body to modify the objects captured by copy • constexpr since C++17 • constexpr since C++20 • static since C++23
exception	provides the dynamic exception specification or (until C++20) the noexcept specifier for operator() of the closure type
back-attr	an attribute specifier sequence applies to the type of operator() of the closure type (and thus the <code>[[noreturn]]</code> attribute cannot be used)
trailing-type	-> ret where ret specifies the return type
requires	adds constraints to operator() of the closure type
body	function body

decltype

`decltype` inspects the declared type of an entity or the type and value category of an expression.

Parentheses matter:

- `decltype(entity)` — if the operand is an unparenthesized id-expression or class member access naming an entity, the result is the *declared* type of that entity (including references and cv-qualifiers as declared).
- `decltype((expression))` — if the operand is any other expression (including a parenthesized id-expression), the result follows the expression's value category:
 - lvalue $\rightarrow T\&$
 - xvalue $\rightarrow T\&\&$
 - prvalue $\rightarrow T$

```
1 int a = 1;
2 decltype(a) b = a;           // int (declared type of a)
3 decltype((a)) c = a;        // int& (a is an lvalue expression)
4 const int& r = a;
5 decltype(r) d = a;           // const int&
6 decltype((r)) e = a;         // const int& (still an lvalue)
```

See [30]

type aliases

Semantically similar to using a typedef however, type aliases with using are easier to read and are compatible with templates.

```
1 template <typename T>
2 using Vec = std::vector<T>;
3 Vec<int> v; // std::vector<int>
4
5 using String = std::string;
6 String s {"foo"};
```

nullptr

Cppreference documentation about `nullptr`

The keyword `nullptr` denotes the pointer literal. It is a `prvalue` of type `std::nullptr_t`. There exist implicit conversions from `nullptr` to null pointer value of any pointer type and any pointer to member type. Similar conversions exist for any null pointer constant, which includes values of type `std::nullptr_t` as well as the macro `NULL`.

strongly-typed enums

Type-safe enums that solve a variety of problems with C-style enums including: implicit conversions, inability to specify the underlying type, scope pollution.

```
1 // Specifying underlying type as 'unsigned int'
2 enum class Color : unsigned int { Red = 0xff0000, Green = 0xff00,
   Blue = 0xff };
3 // 'Red'/'Green' in 'Alert' don't conflict with 'Color'
4 enum class Alert : bool { Red, Green };
5 Color c = Color::Red;
```

attributes

Attributes provide a universal syntax over `__attribute__((...))`, `__declspec`, etc.

```
1 // 'noreturn' attribute indicates 'f' doesn't return.
2 [[ noreturn ]] void f() {
3     throw "error";
4 }
```

constexpr

Cppreference about constexpr

The `constexpr` specifier declares that it is possible to evaluate the value of the function or variable at compile time. Such variables and functions can then be used where only compile time constant expressions are allowed (provided that appropriate function arguments are given).

delegating constructors

Constructors can now call other constructors in the same class using an initializer list.

```
1 struct Foo {
2     int foo;
3     Foo(int foo) : foo{foo} {}
4     Foo() : Foo(0) {}
5 };
6
7 Foo foo;
8 foo.foo; // == 0
```

user-defined literals

User-defined literals allow you to extend the language and add your own syntax. To create a literal, define a `T operator "" X(...)` function that returns a type `T`, with a name `X`. Note that the name of this function defines the name of the literal. Any literal names not starting with an underscore are reserved and won't be invoked. There are rules on what parameters a user-defined literal function should accept, according to

what type the literal is called on.

Converting Celsius to Fahrenheit

```
1 // 'unsigned long long' parameter required for integer literal.
2 long long operator "" _celsius(unsigned long long tempCelsius) {
3     return std::llround(tempCelsius * 1.8 + 32);
4 }
5 24_celsius; // == 75
```

String to integer conversion

```
1 // 'const char*' and 'std::size_t' required as parameters.
2 int operator "" _int(const char* str, std::size_t) {
3     return std::stoi(str);
4 }
5
6 "123"_int; // == 123, with type 'int'
```

explicit virtual overrides

Specifies that a virtual function overrides another virtual function. If the virtual function does not override a parent's virtual function, throws a compiler error.

```
1 struct A {
2     virtual void foo();
3     void bar();
4 };
5
6 struct B : A {
7     void foo() override; // correct -- B::foo overrides A::foo
8     void bar() override; // error -- A::bar is not virtual
9     void baz() override; // error -- B::baz does not override A::
10     baz
11 };
```

final specifier

Specifies that a virtual function cannot be overridden in a derived class or that a class cannot be inherited from.

```
1 struct A {
2     virtual void foo();
3 };
4
5 struct B : A {
6     virtual void foo() final;
7 };
8
9 struct C : B {
10     virtual void foo(); // error -- declaration of 'foo' overrides
11     a 'final' function
12 };
```

Class cannot be inherited from.

```
1 struct A final {};  
2 struct B : A {}; // error -- base 'A' is marked 'final'
```

default functions

A more elegant, efficient way to provide a default implementation of a function, such as a constructor.

```
1 struct A {  
2     A() = default;  
3     A(int x) : x{x} {}  
4     int x {1};  
5 };  
6 A a; // a.x == 1  
7 A a2 {123}; // a.x == 123
```

With inheritance:

```
1 struct B {  
2     B() : x{1} {}  
3     int x;  
4 };  
5  
6 struct C : B {  
7     // Calls B::B  
8     C() = default;  
9 };  
10  
11 C c; // c.x == 1
```

deleted functions

A more elegant, efficient way to provide a deleted implementation of a function. Useful for preventing copies on objects.

```
1 class A {  
2     int x;  
3  
4 public:  
5     A(int x) : x{x} {};  
6     A(const A&) = delete;  
7     A& operator=(const A&) = delete;  
8 };  
9  
10 A x {123};  
11 A y = x; // error -- call to deleted copy constructor  
12 y = x; // error -- operator= deleted
```

range-based for loops

A [range-for statement](#) applies to all standard containers and string as well, providing a less-typing way to for-loop over a *range* of elements.

An example of usage in the following code

```
1  std::array<int, 5> a {1, 2, 3, 4, 5};
2  for (int& x : a) x *= 2;
3  // a == { 2, 4, 6, 8, 10 }
4
5  std::array<int, 5> a {1, 2, 3, 4, 5};
6  for (int x : a) x *= 2;
7  // a == { 1, 2, 3, 4, 5 }
```

In the previous loops, note the difference when using `int` and `int&`.

special member functions for move semantics

The copy constructor and copy assignment operator are called when copies are made, and with C++11's introduction of move semantics, there is now a move constructor and move assignment operator for moves.

```
1  struct A {
2      std::string s;
3      A() : s{"test"} {}
4      A(const A& o) : s{o.s} {}
5      A(A&& o) : s{std::move(o.s)} {}
6      A& operator=(A&& o) {
7          s = std::move(o.s);
8          return *this;
9      }
10 };
11
12 A f(A a) {
13     return a;
14 }
15
16 A a1 = f(A{}); // move-constructed from rvalue temporary
17 A a2 = std::move(a1); // move-constructed using std::move
18 A a3 = A{};
19 a2 = std::move(a3); // move-assignment using std::move
20 a1 = f(A{}); // move-assignment from rvalue temporary
```

converting constructors

Converting constructors will convert values of braced list syntax into constructor arguments.

```
1  struct A {
2      A(int) {}
3      A(int, int) {}
4      A(int, int, int) {}
```



```

5 };
6
7 A a {0, 0}; // calls A::A(int, int)
8 A b(0, 0); // calls A::A(int, int)
9 A c = {0, 0}; // calls A::A(int, int)
10 A d {0, 0, 0}; // calls A::A(int, int, int)

```

Note that the braced list syntax does not allow narrowing:

```

1 struct A {
2     A(int) {}
3 };
4
5 A a(1.1); // OK
6 A b {1.1}; // Error narrowing conversion from double to int

```

Note that if a constructor accepts a `std::initializer_list`, it will be called instead:

```

1 struct A {
2     A(int) {}
3     A(int, int) {}
4     A(int, int, int) {}
5     A(std::initializer_list<int>) {}
6 };
7
8 A a {0, 0}; // calls A::A(std::initializer_list<int>)
9 A b(0, 0); // calls A::A(int, int)
10 A c = {0, 0}; // calls A::A(std::initializer_list<int>)
11 A d {0, 0, 0}; // calls A::A(std::initializer_list<int>)

```

explicit conversion functions

Conversion functions can now be made explicit using the `explicit` specifier.

```

1 struct A {
2     operator bool() const { return true; }
3 };
4
5 struct B {
6     explicit operator bool() const { return true; }
7 };
8
9 A a;
10 if (a); // OK calls A::operator bool()
11 bool ba = a; // OK copy-initialization selects A::operator bool()
12
13 B b;
14 if (b); // OK calls B::operator bool()
15 bool bb = b; // error copy-initialization does not consider B::
               // operator bool()

```

inline-namespaces

All members of an inline namespace are treated as if they were part of its parent namespace, allowing specialization of functions and easing the process of versioning. This is a transitive property, if A contains B, which in turn contains C and both B and C are inline namespaces, C's members can be used as if they were on A.

```
1 namespace Program {
2     namespace Version1 {
3         int getVersion() { return 1; }
4         bool isFirstVersion() { return true; }
5     }
6     inline namespace Version2 {
7         int getVersion() { return 2; }
8     }
9 }
10
11 int version {Program::getVersion()};           // Uses
12         getVersion() from Version2
13 int oldVersion {Program::Version1::getVersion()}; // Uses
14         getVersion() from Version1
15 bool firstVersion {Program::isFirstVersion()}; // Does not
16         compile when Version2 is added
```

non-static data member initializers

Allows non-static data members to be initialized where they are declared, potentially cleaning up constructors of default initializations.

```
1 // Default initialization prior to C++11
2 class Human {
3     Human() : age{0} {}
4     private:
5         unsigned age;
6 };
7 // Default initialization on C++11
8 class Human {
9     private:
10         unsigned age {0};
11 };
```

right angle brackets

C++11 is now able to infer when a series of right angle brackets is used as an operator or as a closing statement of typedef, without having to add whitespace.

```
1 typedef std::map<int, std::map <int, std::map <int, int> > >
2     cpp98LongTypedef;
3 typedef std::map<int, std::map <int, std::map <int, int>>>
4     cpp11LongTypedef;
```

ref-qualified member functions

Member functions can now be qualified depending on whether `*this` is an lvalue or rvalue reference.

```
1 struct Bar {
2     // ...
3 };
4
5 struct Foo {
6     Bar getBar() & { return bar; }
7     Bar getBar() const& { return bar; }
8     Bar getBar() && { return std::move(bar); }
9 private:
10    Bar bar;
11 };
12
13 Foo foo{};
14 Bar bar = foo.getBar(); // calls 'Bar getBar() &'
15
16 const Foo foo2{};
17 Bar bar2 = foo2.getBar(); // calls 'Bar Foo::getBar() const&'
18
19 Foo{}.getBar(); // calls 'Bar Foo::getBar() &&'
20 std::move(foo).getBar(); // calls 'Bar Foo::getBar() &&'
21
22 std::move(foo2).getBar(); // calls 'Bar Foo::getBar() const&&'
```

trailing return types

C++11 allows functions and lambdas an alternative syntax for specifying their return types.

```
1 int f() {
2     return 123;
3 }
4 // vs.
5 auto f() -> int {
6     return 123;
7 }
8
9 auto g = []() -> int {
10     return 123;
11 };
```

This feature is especially useful when certain return types cannot be resolved:

```
1 // NOTE: This does not compile!
2 template <typename T, typename U>
3 decltype(a + b) add(T a, U b) {
4     return a + b;
5 }
```

```

6
7 // Trailing return types allows this:
8 template <typename T, typename U>
9 auto add(T a, U b) -> decltype(a + b) {
10     return a + b;
11 }

```

In C++14, `decltype(auto)` (C++14) can be used instead.

noexcept specifier

Cppreference about `noexcept` The `noexcept` specifier specifies whether a function could throw exceptions. It is an improved version of `throw()`.

```

1 void func1() noexcept;           // does not throw
2 void func2() noexcept(true);    // does not throw
3 void func3() throw();           // does not throw
4
5 void func4() noexcept(false);   // may throw

```

Non-throwing functions are permitted to call potentially-throwing functions. Whenever an exception is thrown and the search for a handler encounters the outermost block of a non-throwing function, the function `std::terminate` is called.

```

1 extern void f(); // potentially-throwing
2 void g() noexcept {
3     f();         // valid, even if f throws
4     throw 42;    // valid, effectively a call to std::terminate
5 }

```

char32_t and char16_t

Provides standard types for representing UTF-8 strings.

```

1 char32_t utf8_str[] = U"\u0123";
2 char16_t utf8_str[] = u"\u0123";

```

raw string literals

C++11 introduces a new way to declare string literals as "raw string literals". Characters issued from an escape sequence (tabs, line feeds, single backslashes, etc.) can be inputted raw while preserving formatting. This is useful, for example, to write literary text, which might contain a lot of quotes or special formatting. This can make your string literals easier to read and maintain.

A raw string literal is declared using the following syntax:

```

1 R"delimiter(raw_characters)delimiter"

```

where:

- `delimiter` is an optional sequence of characters made of any source character except parentheses, backslashes and spaces.
- `raw_characters` is any raw character sequence; must not contain the closing sequence `)delimiter`.

Example:

```
1 // msg1 and msg2 are equivalent.
2 const char* msg1 = "\nHello,\n\tworld!\n";
3 const char* msg2 = R"(
4 Hello,
5     world!
6 )";
```

1.2.2 C++11 new library features

std::move

std::move indicates that the object passed to it may have its resources transferred. Using objects that have been moved from should be used with care, as they can be left in an unspecified state (see: <http://stackoverflow.com/questions/7027523/what-can-i-do-with-a-moved-from-object>).

A definition of std::move (performing a move is nothing more than casting to an rvalue reference):

```
1 template <typename T>
2 typename remove_reference<T>::type&& move(T&& arg) {
3     return static_cast<typename remove_reference<T>::type&&>(arg);
4 }
```

Transferring std::unique_ptrs:

```
1 std::unique_ptr<int> p1 {new int{0}}; // in practice, use std::
    make_unique
2 std::unique_ptr<int> p2 = p1; // error -- cannot copy unique
    pointers
3 std::unique_ptr<int> p3 = std::move(p1); // move 'p1' into 'p3'
4                                     // now unsafe to
    dereference object held by 'p1'
```

std::forward

Returns the arguments passed to it while maintaining their value category and cv-qualifiers. Useful for generic code and factories. Used in conjunction with forwarding references.

A definition of std::forward:

```
1 template <typename T>
2 T&& forward(typename remove_reference<T>::type& arg) {
3     return static_cast<T&&>(arg);
4 }
```

An example of a function wrapper which just forwards other A objects to a new A object's copy or move constructor:

```
1 struct A {
2     A() = default;
3     A(const A& o) { std::cout << "copied" << std::endl; }
4     A(A&& o) { std::cout << "moved" << std::endl; }
5 };
6
7 template <typename T>
8 A wrapper(T&& arg) {
9     return A{std::forward<T>(arg)};
10 }
11
12 wrapper(A{}); // moved
```

```

13 A a;
14 wrapper(a); // copied
15 wrapper(std::move(a)); // moved

```

See also: forwarding references, rvalue references.

std::thread

The `std::thread` library provides a standard way to control threads, such as spawning and killing them. In the example below, multiple threads are spawned to do different calculations and then the program waits for all of them to finish.

```

1 void foo(bool clause) { /* do something... */ }
2
3 std::vector<std::thread> threadsVector;
4 threadsVector.emplace_back([]() {
5     // Lambda function that will be invoked
6 });
7 threadsVector.emplace_back(foo, true); // thread will run foo(
    true)
8 for (auto& thread : threadsVector) {
9     thread.join(); // Wait for threads to finish
10 }

```

std::to_string

Converts a numeric argument to a `std::string`.

```

1 std::to_string(1.2); // == "1.2"
2 std::to_string(123); // == "123"

```

type traits

Type traits defines a compile-time template-based interface to query or modify the properties of types.

```

1 static_assert(std::is_integral<int>::value);
2 static_assert(std::is_same<int, int>::value);
3 static_assert(std::is_same<std::conditional<true, int, double>::
    type, int>::value);

```

smart pointers

C++11 introduces new smart pointers: `std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`. `std::auto_ptr` now becomes deprecated and then eventually removed in C++17.

`std::unique_ptr` is a non-copyable, movable pointer that manages its own heap-allocated memory. Note: Prefer using the `std::make_X` helper functions as opposed to using constructors. See §1.2.4 (`std::make_unique`) and §1.2.2 (`std::make_shared`).

```

1 std::unique_ptr<Foo> p1 { new Foo{} }; // 'p1' owns 'Foo'
2 if (p1) {
3     p1->bar();
4 }
5
6 {
7     std::unique_ptr<Foo> p2 {std::move(p1)}; // Now 'p2' owns 'Foo'
8     f(*p2);
9
10    p1 = std::move(p2); // Ownership returns to 'p1' -- 'p2' gets
    destroyed
11 }
12
13 if (p1) {
14     p1->bar();
15 }
16 // 'Foo' instance is destroyed when 'p1' goes out of scope

```

A `std::shared_ptr` is a smart pointer that manages a resource that is shared across multiple owners. A shared pointer holds a control block which has a few components such as the managed object and a reference counter. All control block access is thread-safe, however, manipulating the managed object itself is not thread-safe.

```

1 void foo(std::shared_ptr<T> t) {
2     // Do something with 't'...
3 }
4
5 void bar(std::shared_ptr<T> t) {
6     // Do something with 't'...
7 }
8
9 void baz(std::shared_ptr<T> t) {
10    // Do something with 't'...
11 }
12
13 std::shared_ptr<T> p1 {new T{}};
14 // Perhaps these take place in another threads?
15 foo(p1);
16 bar(p1);
17 baz(p1);

```

std::chrono

The chrono library contains a set of utility functions and types that deal with durations, clocks, and time points. One use case of this library is benchmarking code:

```

1 std::chrono::time_point<std::chrono::steady_clock> start, end;
2 start = std::chrono::steady_clock::now();
3 // Some computations...
4 end = std::chrono::steady_clock::now();

```



```

5
6 std::chrono::duration<double> elapsed_seconds = end - start;
7 double t = elapsed_seconds.count(); // t number of seconds,
    represented as a 'double'

```

tuples

Tuples are a fixed-size collection of heterogeneous values. Access the elements of a `std::tuple` by unpacking using `std::tie`, or using `std::get`.

```

1 // 'playerProfile' has type 'std::tuple<int, const char*, const
    char*>'.
2 auto playerProfile = std::make_tuple(51, "Frans Nielsen", "NYI");
3 std::get<0>(playerProfile); // 51
4 std::get<1>(playerProfile); // "Frans Nielsen"
5 std::get<2>(playerProfile); // "NYI"

```

std::tie

Creates a tuple of lvalue references. Useful for unpacking `std::pair` and `std::tuple` objects. Use `std::ignore` as a placeholder for ignored values. In C++17, structured bindings should be used instead.

```

1 // With tuples...
2 std::string playerName;
3 std::tie(std::ignore, playerName, std::ignore) = std::make_tuple
    (91, "John Tavares", "NYI");
4
5 // With pairs...
6 std::string yes, no;
7 std::tie(yes, no) = std::make_pair("yes", "no");

```

std::array

`std::array` is a container built on top of a C-style array. Supports common container operations such as sorting.

```

1 std::array<int, 3> a = {2, 1, 3};
2 std::sort(a.begin(), a.end()); // a == { 1, 2, 3 }
3 for (int& x : a) x *= 2; // a == { 2, 4, 6 }

```

unordered containers

These containers maintain average constant-time complexity for search, insert, and remove operations. In order to achieve constant-time complexity, sacrifices order for speed by hashing elements into buckets. There are four unordered containers:

- `unordered_set`

- `unordered_multiset`
- `unordered_map`
- `unordered_multimap`

`std::make_shared`

`std::make_shared` is the recommended way to create instances of `std::shared_ptr`s due to the following reasons:

- Avoid having to use the `new` operator.
- Prevents code repetition when specifying the underlying type the pointer shall hold.
- It provides exception-safety. Suppose we were calling a function `foo` like so:

```
1 foo(std::shared_ptr<T>{new T{}}, function_that_throws(), std
   ::shared_ptr<T>{new T{}});
```

The compiler is free to call `new T{}`, then `function_that_throws()`, and so on... Since we have allocated data on the heap in the first construction of a `T`, we have introduced a leak here. With `std::make_shared`, we are given exception-safety:

```
1 foo(std::make_shared<T>(), function_that_throws(), std::
   make_shared<T>());
```

- Prevents having to do two allocations. When calling `std::shared_ptr{ new T{} }`, we have to allocate memory for `T`, then in the shared pointer we have to allocate memory for the control block within the pointer.

See the section on smart pointers for more information on `std::unique_ptr` and `std::shared_ptr`.

`std::ref`

`std::ref(val)` is used to create object of type `std::reference_wrapper` that holds reference of `val`. Used in cases when usual reference passing using `&` does not compile or `&` is dropped due to type deduction. `std::cref` is similar but created reference wrapper holds a const reference to `val`.

```
1 // create a container to store reference of objects.
2 auto val = 99;
3 auto _ref = std::ref(val);
4 _ref++;
5 auto _cref = std::cref(val);
6 //_cref++; does not compile
7 std::vector<std::reference_wrapper<int>>vec; // vector<int&>vec
   does not compile
8 vec.push_back(_ref); // vec.push_back(&i) does not compile
```

```

9 cout << val << endl; // prints 100
10 cout << vec[0] << endl; // prints 100
11 cout << _cref; // prints 100

```

memory model

C++11 introduces a memory model for C++, which means library support for threading and atomic operations. Some of these operations include (but aren't limited to) atomic loads/stores, compare-and-swap, atomic flags, promises, futures, locks, and condition variables.

std::async

`std::async` runs the given function either asynchronously or lazily-evaluated, then returns a `std::future` which holds the result of that function call.

The first parameter is the policy which can be:

- `std::launch::async` | `std::launch::deferred` It is up to the implementation whether to perform asynchronous execution or lazy evaluation.
- `std::launch::async` Run the callable object on a new thread.
- `std::launch::deferred` Perform lazy evaluation on the current thread.

```

1 int foo() {
2     /* Do something here, then return the result. */
3     return 1000;
4 }
5
6 auto handle = std::async(std::launch::async, foo); // create an
    async task
7 auto result = handle.get(); // wait for the result

```

std::begin/end

`std::begin` and `std::end` free functions were added to return begin and end iterators of a container generically. These functions also work with raw arrays which do not have begin and end member functions.

```

1 template <typename T>
2 int CountTwos(const T& container) {
3     return std::count_if(std::begin(container), std::end(container)
4         , [](int item) {
5             return item == 2;
6         });
7 }
8
9 std::vector<int> vec = {2, 2, 43, 435, 4543, 534};
10 int arr[8] = {2, 43, 45, 435, 32, 32, 32, 32};

```

```
10 auto a = CountTwos(vec); // 2
11 auto b = CountTwos(arr); // 1
```

1.2.3 C++14 new language features

Following a summary of the new language features introduced with C++14.

binary literals

Binary literals provide a convenient way to represent a base-2 number. It is possible to separate digits with '.

```
1 0b110  // == 6
2 0b1111'1111  // == 255
```

generic lambda expressions

C++14 now allows the auto type-specifier in the parameter list, enabling polymorphic lambdas.

```
1 auto identity = [](auto x) { return x; };
2 int three = identity(3); // == 3
3 std::string foo = identity("foo"); // == "foo"
```

lambda capture initializers

This allows creating lambda captures initialized with arbitrary expressions. The name given to the captured value does not need to be related to any variables in the enclosing scopes and introduces a new name inside the lambda body. The initializing expression is evaluated when the lambda is created (not when it is invoked).

```
1 int factory(int i) { return i * 10; }
2 auto f = [x = factory(2)] { return x; }; // returns 20
3
4 auto generator = [x = 0] () mutable {
5     // this would not compile without 'mutable' as we are modifying
6     // x on each call
7     return x++;
8 };
9 auto a = generator(); // == 0
10 auto b = generator(); // == 1
11 auto c = generator(); // == 2
```

Because it is now possible to move (or forward) values into a lambda that could previously be only captured by copy or reference we can now capture move-only types in a lambda by value. Note that in the below example the p in the capture-list of task2 on the left-hand-side of = is a new variable private to the lambda body and does not refer to the original p.

```
1 auto p = std::make_unique<int>(1);
2
3 auto task1 = [=] { *p = 5; }; // ERROR: std::unique_ptr cannot be
4                               // copied
```

```

4 // vs.
5 auto task2 = [p = std::move(p)] { *p = 5; }; // OK: p is move-
        constructed into the closure object
6 // the original p is empty after task2 is created

```

Using this reference-captures can have different names than the referenced variable.

```

1 auto x = 1;
2 auto f = [&r = x, x = x * 10] {
3     ++r;
4     return r + x;
5 };
6 f(); // sets x to 2 and returns 12

```

return type deduction

Using an auto return type in C++14, the compiler will attempt to deduce the type for you. With lambdas, you can now deduce its return type using auto, which makes returning a deduced reference or rvalue reference possible.

```

1 // Deduce return type as 'int'.
2 auto f(int i) {
3     return i;
4 }
5
6 template <typename T>
7 auto& f(T& t) {
8     return t;
9 }
10
11 // Returns a reference to a deduced type.
12 auto g = [](auto& x) -> auto& { return f(x); };
13 int y = 123;
14 int& z = g(y); // reference to 'y'

```

decltype(auto)

The `decltype(auto)` type-specifier also deduces a type like auto does. However, it deduces return types while keeping their references and cv-qualifiers, while auto will not.

```

1 const int x = 0;
2 auto x1 = x; // int
3 decltype(auto) x2 = x; // const int
4 int y = 0;
5 int& y1 = y;
6 auto y2 = y1; // int
7 decltype(auto) y3 = y1; // int&
8 int&& z = 0;
9 auto z1 = std::move(z); // int
10 decltype(auto) z2 = std::move(z); // int&&
11

```

```

12 // Note: Especially useful for generic code!
13
14 // Return type is 'int'.
15 auto f(const int& i) {
16     return i;
17 }
18
19 // Return type is 'const int&'.
20 decltype(auto) g(const int& i) {
21     return i;
22 }
23
24 int x = 123;
25 static_assert(std::is_same<const int&, decltype(f(x))>::value ==
    0);
26 static_assert(std::is_same<int, decltype(f(x))>::value == 1);
27 static_assert(std::is_same<const int&, decltype(g(x))>::value ==
    1);

```

relaxing constraints on constexpr functions

In C++11, constexpr function bodies could only contain a very limited set of syntaxes, including (but not limited to): typedefs, usings, and a single return statement. In C++14, the set of allowable syntaxes expands greatly to include the most common syntax such as if statements, multiple returns, loops, etc.

```

1 constexpr int factorial(int n) {
2     if (n <= 1) {
3         return 1;
4     } else {
5         return n * factorial(n - 1);
6     }
7 }
8 factorial(5); // == 120

```

variable templates

C++14 allows variables to be templated:

```

1 template<class T>
2 constexpr T pi = T(3.1415926535897932385);
3 template<class T>
4 constexpr T e = T(2.7182818284590452353);

```

[[deprecated]] attribute

C++14 introduces the `[[deprecated]]` attribute to indicate that a unit (function, class, etc.) is discouraged and likely yield compilation warnings. If a reason is provided, it will be included in the warnings.

```

1  [[deprecated]]
2  void old_method();
3  [[deprecated("Use new_method instead")]]
4  void legacy_method();

```

1.2.4 C++14 new library features

Following a summary of the new library features introduced with C++14.

user-defined literals for standard library types

New user-defined literals for standard library types, including new built-in literals for chrono and basic_string. These can be constexpr meaning they can be used at compile-time. Some uses for these literals include compile-time integer parsing, binary literals, and imaginary number literals.

```

1  using namespace std::chrono_literals;
2  auto day = 24h;
3  day.count(); // == 24
4  std::chrono::duration_cast<std::chrono::minutes>(day).count(); //
    == 1440

```

compile-time integer sequences

The class template `std::integer_sequence` represents a compile-time sequence of integers. There are a few helpers built on top:

- `std::make_integer_sequence<T, N>` - creates a sequence of 0, ..., N - 1 with type T.
- `std::index_sequence_for<T...>` - converts a template parameter pack into an integer sequence.

Convert an array into a tuple:

```

1  template<typename Array, std::size_t... I>
2  decltype(auto) a2t_impl(const Array& a, std::integer_sequence<std::
    ::size_t, I...>) {
3      return std::make_tuple(a[I]...);
4  }
5
6  template<typename T, std::size_t N, typename Indices = std::
    make_index_sequence<N>>
7  decltype(auto) a2t(const std::array<T, N>& a) {
8      return a2t_impl(a, Indices());
9  }

```


std::make_unique

std::make_unique is the recommended way to create instances of std::unique_ptr due to the following reasons:

- Avoid having to use the new operator.
- Prevents code repetition when specifying the underlying type the pointer shall hold.
- Most importantly, it provides exception-safety. Suppose we were calling a function foo like so:

```
1 foo(std::unique_ptr<T>{new T{}}, function_that_throws(),
2     std::unique_ptr<T>{new T{}});
```

The compiler is free to call new T{}, then function_that_throws(), and so on...

Since we have allocated data on the heap in the first construction of a T, we have introduced a leak here. With std::make_unique, we are given exception-safety:

```
1 foo(std::make_unique<T>(), function_that_throws(),
2     std::make_unique<T>());
```

See the section on smart pointers (C++11) for more information on std::unique_ptr and std::shared_ptr.

1.2.5 C++17 new language features

Following a summary of the new language features introduced with C++17.

template argument deduction for class templates

Automatic template argument deduction much like how it's done for functions, but now including class constructors.

```
1 template <typename T = float>
2 struct MyContainer {
3     T val;
4     MyContainer() : val{} {}
5     MyContainer(T val) : val{val} {}
6     // ...
7 };
8 MyContainer c1 {1}; // OK MyContainer<int>
9 MyContainer c2; // OK MyContainer<float>
```

declaring non-type template parameters with auto

Following the deduction rules of auto, while respecting the non-type template parameter list of allowable types[*], template arguments can be deduced from the types of its arguments:

```

1 template <auto... seq>
2 struct my_integer_sequence {
3     // Implementation here ...
4 };
5
6 // Explicitly pass type 'int' as template argument.
7 auto seq = std::integer_sequence<int, 0, 1, 2>();
8 // Type is deduced to be 'int'.
9 auto seq2 = my_integer_sequence<0, 1, 2>();

```

* - For example, you cannot use a double as a template parameter type, which also makes this an invalid deduction using auto.

folding expressions

A fold expression performs a fold of a template parameter pack over a binary operator.

- An expression of the form `(... op e)` or `(e op ...)`, where `op` is a fold-operator and `e` is an unexpanded parameter pack, are called unary folds.
- An expression of the form `(e1 op ... op e2)`, where `op` are fold-operators, is called a binary fold. Either `e1` or `e2` is an unexpanded parameter pack, but not both.

```

1 template <typename... Args>
2 bool logicalAnd(Args... args) {
3     // Binary folding.
4     return (true && ... && args);
5 }
6 bool b = true;
7 bool& b2 = b;
8 logicalAnd(b, b2, true); // == true
9
10 template <typename... Args>
11 auto sum(Args... args) {
12     // Unary folding.
13     return (... + args);
14 }
15 sum(1.0, 2.0f, 3); // == 6.0

```

new rules for auto deduction from braced-init-list

Changes to auto deduction when used with the uniform initialization syntax. Previously, `auto x {3};` deduces a `std::initializer_list<int>`, which now deduces to `int`.

```

1
2 auto x1 {1, 2, 3}; // error: not a single element
3 auto x2 = {1, 2, 3}; // x2 is std::initializer_list<int>
4 auto x3 {3}; // x3 is int
5 auto x4 {3.0}; // x4 is double

```

constexpr lambda

Compile-time lambdas using constexpr.

```
1 auto identity = [](int n) constexpr { return n; };
2 static_assert(identity(123) == 123);
```

```
1 constexpr auto add = [](int x, int y) {
2     auto L = [=] { return x; };
3     auto R = [=] { return y; };
4     return [=] { return L() + R(); };
5 };
6
7 static_assert(add(1, 2)() == 3);
8 \begin{lstlisting}[language=C++]
9
10 constexpr int addOne(int n) {
11     return [n] { return n + 1; }();
12 }
13
14 static_assert(addOne(1) == 2);
```

lambda capture this by value

Capturing this in a lambda's environment was previously reference-only. An example of where this is problematic is asynchronous code using callbacks that require an object to be available, potentially past its lifetime. `*this` (C++17) will now make a copy of the current object, while `this` (C++11) continues to capture by reference.

```
1 struct MyObj {
2     int value {123};
3     auto getValueCopy() {
4         return [*this] { return value; };
5     }
6     auto getValueRef() {
7         return [this] { return value; };
8     }
9 };
10 MyObj mo;
11 auto valueCopy = mo.getValueCopy();
12 auto valueRef = mo.getValueRef();
13 mo.value = 321;
14 valueCopy(); // 123
15 valueRef(); // 321
```

inline variables

The inline specifier can be applied to variables as well as to functions. A variable declared inline has the same semantics as a function declared inline.

```

1 // Disassembly example using compiler explorer.
2 struct S { int x; };
3 inline S x1 = S{321}; // mov esi, dword ptr [x1]
4                       // x1: .long 321
5
6 S x2 = S{123};        // mov eax, dword ptr [.L_ZZ4mainE2x2]
7                       // mov dword ptr [rbp - 8], eax
8                       // .L_ZZ4mainE2x2: .long 123

```

It can also be used to declare and define a static member variable, such that it does not need to be initialized in the source file.

```

1 struct S {
2     S() : id{count++} {}
3     ~S() { count--; }
4     int id;
5     static inline int count{0}; // declare and initialize count to
6     0 within the class
7 };

```

nested namespaces

Using the namespace resolution operator to create nested namespace definitions.

```

1 namespace A {
2     namespace B {
3         namespace C {
4             int i;
5         }
6     }
7 }

```

The code above can be written like this:

```

1 namespace A::B::C {
2     int i;
3 }

```

structured bindings

A proposal for de-structuring initialization, that would allow writing `auto [ x, y, z ] = expr;` where the type of `expr` was a tuple-like object, whose elements would be bound to the variables `x`, `y`, and `z` (which this construct declares). Tuple-like objects include `std::tuple`, `std::pair`, `std::array`, and aggregate structures.

```

1 using Coordinate = std::pair<int, int>;
2 Coordinate origin() {
3     return Coordinate{0, 0};
4 }
5
6 const auto [ x, y ] = origin();

```

```

7 x; // == 0
8 y; // == 0

```

```

1 std::unordered_map<std::string, int> mapping {
2     {"a", 1},
3     {"b", 2},
4     {"c", 3}
5 };
6
7 // Destructure by reference.
8 for (const auto& [key, value] : mapping) {
9     // Do something with key and value
10 }

```

selection statements with initializer

New versions of the if and switch statements which simplify common code patterns and help users keep scopes tight.

```

1 {
2     std::lock_guard<std::mutex> lk(mx);
3     if (v.empty()) v.push_back(val);
4 }
5 // vs.
6 if (std::lock_guard<std::mutex> lk(mx); v.empty()) {
7     v.push_back(val);
8 }

```

```

1 Foo gadget(args);
2 switch (auto s = gadget.status()) {
3     case OK: gadget.zip(); break;
4     case Bad: throw BadFoo(s.message());
5 }
6 // vs.
7 switch (Foo gadget(args); auto s = gadget.status()) {
8     case OK: gadget.zip(); break;
9     case Bad: throw BadFoo(s.message());
10 }

```

constexpr if

Write code that is instantiated depending on a compile-time condition.

```

1 template <typename T>
2 constexpr bool isIntegral() {
3     if constexpr (std::is_integral<T>::value) {
4         return true;
5     } else {
6         return false;
7     }
8 }

```

```

9 static_assert(isIntegral<int>() == true);
10 static_assert(isIntegral<char>() == true);
11 static_assert(isIntegral<double>() == false);
12 struct S {};
13 static_assert(isIntegral<S>() == false);

```

utf-8 character literals

A character literal that begins with u8 is a character literal of type char. The value of a UTF-8 character literal is equal to its ISO 10646 code point value.

```

1 char x = u8'x';

```

direct-list-initialization of enums

Enums can now be initialized using braced syntax.

```

1
2 enum byte : unsigned char {};
3 byte b {0}; // OK
4 byte c {-1}; // ERROR
5 byte d = byte{1}; // OK
6 byte e = byte{256}; // ERROR

```

[[fallthrough]], [[nodiscard]], [[maybe_unused]] attributes

C++17 introduces three new attributes:

[[fallthrough]], [[nodiscard]] and [[maybe_unused]].

- [[fallthrough]] indicates to the compiler that falling through in a switch statement is intended behavior. This attribute may only be used in a switch statement, and must be placed before the next case/default label.

```

1 switch (n) {
2     case 1:
3         // ...
4         [[fallthrough]];
5     case 2:
6         // ...
7         break;
8     case 3:
9         // ...
10        [[fallthrough]];
11    default:
12        // ...
13 }

```

- [[nodiscard]] issues a warning when either a function or class has this attribute and its return value is discarded.

```

1 [[nodiscard]] bool do_something() {
2     return is_success; // true for success, false for failure
3 }
4
5 do_something(); // warning: ignoring return value of 'bool
6               do_something()',
               // declared with attribute 'nodiscard'

```

```

1 // Only issues a warning when 'error_info' is returned by
2 value.
3 struct [[nodiscard]] error_info {
4     // ...
5 };
6
7 error_info do_something() {
8     error_info ei;
9     // ...
10    return ei;
11 }
12
13 do_something(); // warning: ignoring returned value of type
               'error_info',
               // declared with attribute 'nodiscard'

```

- `[[maybe_unused]]` indicates to the compiler that a variable or parameter might be unused and is intended.

```

1 void my_callback(std::string msg, [[maybe_unused]] bool
2     error) {
3     // Don't care if 'msg' is an error message, just log it.
4     log(msg);
5 }

```

__has_include

`__has_include` (operand) operator may be used in `#if` and `#elif` expressions to check whether a header or source file (operand) is available for inclusion or not.

One use case of this would be using two libraries that work the same way, using the backup/experimental one if the preferred one is not found on the system.

```

1 #ifdef __has_include
2 #   if __has_include(<optional>)
3 #       include <optional>
4 #       define have_optional 1
5 #   elif __has_include(<experimental/optional>)
6 #       include <experimental/optional>
7 #       define have_optional 1
8 #       define experimental_optional
9 #   else
10 #       define have_optional 0

```

```

11 # endif
12 #endif

```

It can also be used to include headers existing under different names or locations on various platforms, without knowing which platform the program is running on, OpenGL headers are a good example for this which are located in OpenGL directory on macOS and GL on other platforms.

```

1 #ifdef __has_include
2 # if __has_include(<OpenGL/gl.h>)
3 #   include <OpenGL/gl.h>
4 #   include <OpenGL/glu.h>
5 # elif __has_include(<GL/gl.h>)
6 #   include <GL/gl.h>
7 #   include <GL/glu.h>
8 # else
9 #   error No suitable OpenGL headers found.
10 # endif
11 #endif

```

class template argument deduction

Class template argument deduction (CTAD) allows the compiler to deduce template arguments from constructor arguments.

```

1 std::vector v{ 1, 2, 3 }; // deduces std::vector<int>
2
3 std::mutex mtx;
4 auto lck = std::lock_guard{ mtx }; // deduces to std::lock_guard<
    std::mutex>
5
6 auto p = new std::pair{ 1.0, 2.0 }; // deduces to std::pair<
    double, double>

```

For user-defined types, deduction guides can be used to guide the compiler how to deduce template arguments if applicable:

```

1 template <typename T>
2 struct container {
3     container(T t) {}
4
5     template <typename Iter>
6     container(Iter beg, Iter end);
7 };
8
9 // deduction guide
10 template <typename Iter>
11 container(Iter b, Iter e) -> container<typename std::
    iterator_traits<Iter>::value_type>;
12
13 container a{ 7 }; // OK: deduces container<int>
14

```



```

15 std::vector<double> v{ 1.0, 2.0, 3.0 };
16 auto b = container{ v.begin(), v.end() }; // OK: deduces
    container<double>
17
18 container c{ 5, 6 }; // ERROR: std::iterator_traits<int>::
    value_type is not a type

```

1.2.6 C++17 new library features:

Following a summary of the new library features introduced with C++17.

std::variant

The class template `std::variant` represents a type-safe union. An instance of `std::variant` at any given time holds a value of one of its alternative types (it's also possible for it to be valueless).

```

1 std::variant<int, double> v{ 12 };
2 std::get<int>(v); // == 12
3 std::get<0>(v); // == 12
4 v = 12.0;
5 std::get<double>(v); // == 12.0
6 std::get<1>(v); // == 12.0

```

std::optional

The class template `std::optional` manages an optional contained value, i.e. a value that may or may not be present. A common use case for `optional` is the return value of a function that may fail.

```

1 std::optional<std::string> create(bool b) {
2     if (b) {
3         return "Godzilla";
4     } else {
5         return {};
6     }
7 }
8
9 create(false).value_or("empty"); // == "empty"
10 create(true).value(); // == "Godzilla"
11 // optional-returning factory functions are usable as conditions
    of while and if
12 if (auto str = create(true)) {
13     // ...
14 }

```

std::any

A type-safe container for single values of any type.

```

1 std::any x {5};
2 x.has_value() // == true
3 std::any_cast<int>(x) // == 5
4 std::any_cast<int&>(x) = 10;
5 std::any_cast<int>(x) // == 10

```

std::string_view

A non-owning reference to a string. Useful for providing an abstraction on top of strings (e.g. for parsing).

```

1 // Regular strings.
2 std::string_view cppstr {"foo"};
3 // Wide strings.
4 std::wstring_view wctr_v {L"baz"};
5 // Character arrays.
6 char array[3] = {'b', 'a', 'r'};
7 std::string_view array_v(array, std::size(array));

```

```

1 std::string str {"  trim me"};
2 std::string_view v {str};
3 v.remove_prefix(std::min(v.find_first_not_of(" "), v.size()));
4 str; // == "  trim me"
5 v; // == "trim me"

```

std::invoke

Invoke a Callable object with parameters. Examples of callable objects are std::function or lambdas; objects that can be called similarly to a regular function.

```

1 template <typename Callable>
2 class Proxy {
3     Callable c_;
4
5 public:
6     Proxy(Callable c) : c_{ std::move(c) } {}
7
8     template <typename... Args>
9     decltype(auto) operator()(Args&&... args) {
10         // ...
11         return std::invoke(c_, std::forward<Args>(args)...);
12     }
13 };
14
15 const auto add = [](int x, int y) { return x + y; };
16 Proxy p{ add };
17 p(1, 2); // == 3

```

std::apply

Invoke a Callable object with a tuple of arguments.

```
1 auto add = [](int x, int y) {
2     return x + y;
3 };
4 std::apply(add, std::make_tuple(1, 2)); // == 3
```

std::filesystem

The new std::filesystem library provides a standard way to manipulate files, directories, and paths in a filesystem.

Here, a big file is copied to a temporary path if there is available space:

```
1 const auto bigFilePath {"bigFileToCopy"};
2 if (std::filesystem::exists(bigFilePath)) {
3     const auto bigFileSize {std::filesystem::file_size(bigFilePath)
4     };
5     std::filesystem::path tmpPath {"tmp"};
6     if (std::filesystem::space(tmpPath).available > bigFileSize) {
7         std::filesystem::create_directory(tmpPath.append("example"));
8         std::filesystem::copy_file(bigFilePath, tmpPath.append("
9         newFile"));
10    }
11 }
```

std::byte

The new std::byte type provides a standard way of representing data as a byte. Benefits of using std::byte over char or unsigned char is that it is not a character type, and is also not an arithmetic type; while the only operator overloads available are bitwise operations.

S

```
1 std::byte a {0};
2 std::byte b {0xFF};
3 int i = std::to_integer<int>(b); // 0xFF
4 std::byte c = a & b;
5 int j = std::to_integer<int>(c); // 0
```

Note that std::byte is simply an enum, and braced initialization of enums become possible thanks to direct-list-initialization of enums.

splicing for maps and sets

Moving nodes and merging containers without the overhead of expensive copies, moves, or heap allocations/deallocations.

Moving elements from one map to another:

```

1 std::map<int, string> src {{1, "one"}, {2, "two"}, {3, "buckle my
   shoe"}};
2 std::map<int, string> dst {{3, "three"}};
3 dst.insert(src.extract(src.find(1))); // Cheap remove and insert
   of { 1, "one" } from 'src' to 'dst'.
4 dst.insert(src.extract(2)); // Cheap remove and insert of { 2, "
   two" } from 'src' to 'dst'.
5 // dst == { { 1, "one" }, { 2, "two" }, { 3, "three" } };

```

Inserting an entire set:

```

1 std::set<int> src {1, 3, 5};
2 std::set<int> dst {2, 4, 5};
3 dst.merge(src);
4 // src == { 5 }
5 // dst == { 1, 2, 3, 4, 5 }

```

Inserting elements which outlive the container:

```

1 auto elementFactory() {
2     std::set<...> s;
3     s.emplace(...);
4     return s.extract(s.begin());
5 }
6 s2.insert(elementFactory());

```

Changing the key of a map element:

```

1 std::map<int, string> m {{1, "one"}, {2, "two"}, {3, "three"}};
2 auto e = m.extract(2);
3 e.key() = 4;
4 m.insert(std::move(e));
5 // m == { { 1, "one" }, { 3, "three" }, { 4, "two" } }

```

parallel algorithms

Many of the STL algorithms, such as the copy, find and sort methods, started to support the parallel execution policies: seq, par and par_unseq which translate to "sequentially", "parallel" and "parallel unsequenced".

```

1 std::vector<int> longVector;
2 // Find element using parallel execution policy
3 auto result1 = std::find(std::execution::par, std::begin(
   longVector), std::end(longVector), 2);
4 // Sort elements using sequential execution policy
5 auto result2 = std::sort(std::execution::seq, std::begin(
   longVector), std::end(longVector));

```

std::sample

Samples n elements in the given sequence (without replacement) where every element has an equal chance of being selected.

```

1  const std::string ALLOWED_CHARS = "
    abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
    ";
2  std::string guid;
3  // Sample 5 characters from ALLOWED_CHARS.
4  std::sample(ALLOWED_CHARS.begin(), ALLOWED_CHARS.end(), std::
    back_inserter(guid),
5    5, std::mt19937{ std::random_device{}() }));
6
7  std::cout << guid; // e.g. G1fW2

```

std::clamp

Clamp given value between a lower and upper bound.

```

1  std::clamp(42, -1, 1); // == 1
2  std::clamp(-42, -1, 1); // == -1
3  std::clamp(0, -1, 1); // == 0
4
5  // 'std::clamp' also accepts a custom comparator:
6  std::clamp(0, -1, 1, std::less<>{}); // == 0

```

std::reduce

Fold over a given range of elements. Conceptually similar to `std::accumulate`, but `std::reduce` will perform the fold in parallel. Due to the fold being done in parallel, if you specify a binary operation, it is required to be associative and commutative. A given binary operation also should not change any element or invalidate any iterators within the given range.

The default binary operation is `std::plus` with an initial value of 0.

```

1  const std::array<int, 3> a{ 1, 2, 3 };
2  std::reduce(std::cbegin(a), std::cend(a)); // == 6
3  // Using a custom binary op:
4  std::reduce(std::cbegin(a), std::cend(a), 1, std::multiplies<>{}
    ); // == 6

```

Additionally you can specify transformations for reducers:

```

1  std::transform_reduce(std::cbegin(a), std::cend(a), 0, std::plus
    <>{}, times_ten); // == 60
2
3  const std::array<int, 3> b{ 1, 2, 3 };
4  const auto product_times_ten = [](const auto a, const auto b) {
    return a * b * 10; };
5
6  std::transform_reduce(std::cbegin(a), std::cend(a), std::cbegin(b
    ), 0, std::plus<>{}, product_times_ten); // == 140

```

prefix sum algorithms

Support for prefix sums (both inclusive and exclusive scans) along with transformations.

```
1  const std::array<int, 3> a{ 1, 2, 3 };
2
3  std::inclusive_scan(std::cbegin(a), std::cend(a),
4                      std::ostream_iterator<int>{ std::cout, " " }, std::plus<>{});
5                      // 1 3 6
6
7  std::exclusive_scan(std::cbegin(a), std::cend(a),
8                      std::ostream_iterator<int>{ std::cout, " " }, 0, std::plus
9                      <>{}); // 0 1 3
10
11 const auto times_ten = [](const auto n) { return n * 10; };
12
13 std::transform_inclusive_scan(std::cbegin(a), std::cend(a),
14                               std::ostream_iterator<int>{ std::cout, " " }, std::plus<>{},
15                               times_ten); // 10 30 60
16
17 std::transform_exclusive_scan(std::cbegin(a), std::cend(a),
18                               std::ostream_iterator<int>{ std::cout, " " }, 0, std::plus
19                               <>{}, times_ten); // 0 10 30
```

gcd and lcm

Greatest common divisor (GCD) and least common multiple (LCM).

```
1  const int p = 9;
2  const int q = 3;
3  std::gcd(p, q); // == 3
4  std::lcm(p, q); // == 9
```

std::not_fn

Utility function that returns the negation of the result of the given function.

```
1  const std::ostream_iterator<int> ostream_it{ std::cout, " " };
2  const auto is_even = [](const auto n) { return n % 2 == 0; };
3  std::vector<int> v{ 0, 1, 2, 3, 4 };
4
5  // Print all even numbers.
6  std::copy_if(std::cbegin(v), std::cend(v), ostream_it, is_even);
7  // 0 2 4
8  // Print all odd (not even) numbers.
9  std::copy_if(std::cbegin(v), std::cend(v), ostream_it, std::
10               not_fn(is_even)); // 1 3
```

string conversion to/from numbers

Convert integrals and floats to a string or vice-versa. Conversions are non-throwing, do not allocate, and are more secure than the equivalents from the C standard library.

Users are responsible for allocating enough storage required for `std::to_chars`, or the function will fail by setting the error code object in its return value.

These functions allow you to optionally pass a base (defaults to base-10) or a format specifier for floating type input.

`std::to_chars` returns a (non-const) char pointer which is one-past-the-end of the string that the function wrote to inside the given buffer, and an error code object. `std::from_chars` returns a const char pointer which on success is equal to the end pointer passed to the function, and an error code object.

Both error code objects returned from these functions are equal to the default-initialized error code object on success.

Convert the number 123 to a `std::string`:

```
1  const int n = 123;
2
3  // Can use any container, string, array, etc.
4  std::string str;
5  str.resize(3); // hold enough storage for each digit of 'n'
6
7  const auto [ ptr, ec ] = std::to_chars(str.data(), str.data() +
8      str.size(), n);
9
10 if (ec == std::errc{}) { std::cout << str << std::endl; } // 123
11 else { /* handle failure */ }
```

Convert from a `std::string` with value "123" to an integer:

```
1  const std::string str{ "123" };
2  int n;
3
4  const auto [ ptr, ec ] = std::from_chars(str.data(), str.data() +
5      str.size(), n);
6
7  if (ec == std::errc{}) { std::cout << n << std::endl; } // 123
8  else { /* handle failure */ }
```

rounding functions for chrono durations and timepoints

Provides `abs`, `round`, `ceil`, and `floor` helper functions for `std::chrono::duration` and `std::chrono::time_point`.

```
1  std::chrono::milliseconds a{ -5500 };
2  std::chrono::milliseconds d = std::chrono::abs(a); // == 5500ms
3  std::chrono::round<seconds>(d); // == 6s
4  std::chrono::ceil<seconds>(d); // == 6s
5  std::chrono::floor<seconds>(d); // == 5s
```

1.2.7 C++20 new language features

Following a summary of the new language features introduced with C++20.

coroutines

Coroutines are special functions that can have their execution suspended and resumed. To define a coroutine, the `co_return`, `co_await`, or `co_yield` keywords must be present in the function's body. C++20's coroutines are stackless; unless optimized out by the compiler, their state is allocated on the heap.

An example of a coroutine is a generator function, which yields (i.e. generates) a value at each invocation:

```
1 generator<int> range(int start, int end) {  
2     while (start < end) {  
3         co_yield start;  
4         start++;  
5     }  
6  
7     // Implicit co_return at the end of this function:  
8     // co_return;  
9 }  
10  
11 for (int n : range(0, 10)) {  
12     std::cout << n << std::endl;  
13 }
```

The above range generator function generates values starting at start until end (exclusive), with each iteration step yielding the current value stored in start. The generator maintains its state across each invocation of range (in this case, the invocation is for each iteration in the for loop). `co_yield` takes the given expression, yields (i.e. returns) its value, and suspends the coroutine at that point. Upon resuming, execution continues after the `co_yield`.

Another example of a coroutine is a task, which is an asynchronous computation that is executed when the task is awaited:

```
1 task<void> echo(socket s) {  
2     for (;;) {  
3         auto data = co_await s.async_read();  
4         co_await async_write(s, data);  
5     }  
6  
7     // Implicit co_return at the end of this function:  
8     // co_return;  
9 }
```

In this example, the `co_await` keyword is introduced. This keyword takes an expression and suspends execution if the thing you're awaiting on (in this case, the read or write) is not ready, otherwise you continue execution. (Note that under the hood, `co_yield` uses `co_await`.)

Using a task to lazily evaluate a value:

```
1 task<int> calculate_meaning_of_life() {
2     co_return 42;
3 }
4
5 auto meaning_of_life = calculate_meaning_of_life();
6 // ...
7 co_await meaning_of_life; // == 42
```

Note: While these examples illustrate how to use coroutines at a basic level, there is lots more going on when the code is compiled. These examples are not meant to be complete coverage of C++20's coroutines. Since the generator and task classes are not provided by the standard library yet, I used the cppcoro library to compile these examples.

concepts

Concepts are named compile-time predicates which constrain types. They take the following form:

```
1 template < template-parameter-list >
2 concept concept-name = constraint-expression;
```

where constraint-expression evaluates to a constexpr Boolean. Constraints should model semantic requirements, such as whether a type is a numeric or hashable. A compiler error results if a given type does not satisfy the concept it's bound by (i.e. constraint-expression returns false). Because constraints are evaluated at compile-time, they can provide more meaningful error messages and runtime safety.

```
1 // 'T' is not limited by any constraints.
2 template <typename T>
3 concept always_satisfied = true;
4 // Limit 'T' to integrals.
5 template <typename T>
6 concept integral = std::is_integral_v<T>;
7 // Limit 'T' to both the 'integral' constraint and signedness.
8 template <typename T>
9 concept signed_integral = integral<T> && std::is_signed_v<T>;
10 // Limit 'T' to both the 'integral' constraint and the negation
    of the 'signed_integral' constraint.
11 template <typename T>
12 concept unsigned_integral = integral<T> && !signed_integral<T>;
```

There are a variety of syntactic forms for enforcing concepts:

```
1 // Forms for function parameters:
2 // 'T' is a constrained type template parameter.
3 template <my_concept T>
4 void f(T v);
5
6 // 'T' is a constrained type template parameter.
```

```

7  template <typename T>
8      requires my_concept<T>
9  void f(T v);
10
11  // 'T' is a constrained type template parameter.
12  template <typename T>
13  void f(T v) requires my_concept<T>;
14
15  // 'v' is a constrained deduced parameter.
16  void f(my_concept auto v);
17
18  // 'v' is a constrained non-type template parameter.
19  template <my_concept auto v>
20  void g();
21
22  // Forms for auto-deduced variables:
23  // 'foo' is a constrained auto-deduced value.
24  my_concept auto foo = ...;
25
26  // Forms for lambdas:
27  // 'T' is a constrained type template parameter.
28  auto f = []<my_concept T> (T v) {
29      // ...
30  };
31  // 'T' is a constrained type template parameter.
32  auto f = []<typename T> requires my_concept<T> (T v) {
33      // ...
34  };
35  // 'T' is a constrained type template parameter.
36  auto f = []<typename T> (T v) requires my_concept<T> {
37      // ...
38  };
39  // 'v' is a constrained deduced parameter.
40  auto f = [](my_concept auto v) {
41      // ...
42  };
43  // 'v' is a constrained non-type template parameter.
44  auto g = []<my_concept auto v> () {
45      // ...
46  };

```

The `requires` keyword is used either to start a `requires` clause or a `requires` expression:

```

1  template <typename T>
2      requires my_concept<T> // 'requires' clause.
3  void f(T);
4
5  template <typename T>
6  concept callable = requires (T f) { f(); }; // 'requires'
7      expression.
8
9  template <typename T>

```

```

9   requires requires (T x) { x + x; } // 'requires' clause and
    expression on same line.
10  T add(T a, T b) {
11      return a + b;
12  }

```

Note that the parameter list in a `requires` expression is optional. Each requirement in a `requires` expression is one of the following:

- Simple requirements — asserts that the given expression is valid.

```

1  template <typename T>
2  concept callable = requires (T f) { f(); };

```

- Type requirements — denoted by the `typename` keyword followed by a type name, asserts that the given type name is valid.

```

1  struct foo {
2      int foo;
3  };
4
5  struct bar {
6      using value = int;
7      value data;
8  };
9
10 struct baz {
11     using value = int;
12     value data;
13 };
14
15 // Using SFINAE, enable if 'T' is a 'baz'.
16 template <typename T, typename = std::enable_if_t<std::
    is_same_v<T, baz>>>
17 struct S {};
18
19 template <typename T>
20 using Ref = T&;
21
22 template <typename T>
23 concept C = requires {
24     // Requirements on type 'T':
25     typename T::value; // A) has an inner member named 'value'
26     typename S<T>;     // B) must have a valid class template
    specialization for 'S'
27     typename Ref<T>;  // C) must be a valid alias template
    substitution
28 };
29
30 template <C T>
31 void g(T a);
32

```

```

33 g(foo{}); // ERROR: Fails requirement A.
34 g(bar{}); // ERROR: Fails requirement B.
35 g(baz{}); // PASS.

```

- Compound requirements — an expression in braces followed by a trailing return type or type constraint.

```

1  template <typename T>
2  concept C = requires(T x) {
3      {*x} -> std::convertible_to<typename T::inner>; // the
               type of the expression '*x' is convertible to 'T::inner'
4      {x + 1} -> std::same_as<int>; // the expression 'x + 1'
               satisfies 'std::same_as<decltype((x + 1))>'
5      {x * 1} -> std::convertible_to<T>; // the type of the
               expression 'x * 1' is convertible to 'T'
6  };

```

- Nested requirements — denoted by the `requires` keyword, specify additional constraints (such as those on local parameter arguments).

```

1  template <typename T>
2  concept C = requires(T x) {
3      requires std::same_as<sizeof(x), size_t>;
4  };

```

See also §1.2.8 (concepts library).

three-way comparison

C++20 introduces the spaceship operator (`<=>`) as a way to write comparison functions that reduce boilerplate and help define clearer comparison semantics. Defining a three-way comparison operator can autogenerate the other comparison operators (`==`, `!=`, `<`, etc.).

Three orderings are introduced:

- `std::strong_ordering`: distinguishes items being equal (identical and interchangeable). Provides `less`, `greater`, `equivalent`, and `equal`. Examples: integers, case-sensitive strings.
- `std::weak_ordering`: distinguishes items being equivalent (not identical, but interchangeable for comparison). Provides `less`, `greater`, and `equivalent`. Examples: case-insensitive strings.
- `std::partial_ordering`: like weak ordering but includes the case when an ordering isn't possible. Provides `less`, `greater`, `equivalent`, and `unordered`. Examples: floating-point values (e.g. NaN).

A defaulted three-way comparison operator does a member-wise comparison:

```

1 struct foo {
2     int a;
3     bool b;
4     char c;
5
6     // Compare 'a' first, then 'b', then 'c' ...
7     friend auto operator<=>(const foo&) const = default;
8 };
9
10 foo f1{0, false, 'a'}, f2{0, true, 'b'};
11 f1 < f2; // == true
12 f1 == f2; // == false
13 f1 >= f2; // == false

```

You can also define your own comparisons:

```

1 struct foo {
2     int x;
3     bool b;
4     char c;
5
6     friend std::strong_ordering operator<=>(const foo& other) const
7     {
8         return x <=> other.x;
9     }
10 };
11
12 foo f1{0, false, 'a'}, f2{0, true, 'b'};
13 f1 < f2; // == false
14 f1 == f2; // == true
15 f1 >= f2; // == true

```

designated initializers

C-style designated initializer syntax. Any member fields that are not explicitly listed in the designated initializer list are default-initialized.

```

1 struct A {
2     int x;
3     int y;
4     int z = 123;
5 };
6
7 A a {.x = 1, .z = 2}; // a.x == 1, a.y == 0, a.z == 2

```

template syntax for lambdas

Use familiar template syntax in lambda expressions.

```

1 auto f = []<typename T>(std::vector<T> v) {
2     // ...

```

```
3 };
```

range-based for loop with initializer

This feature simplifies common code patterns, helps keep scopes tight, and offers an elegant solution to a common lifetime problem.

```
1 for (auto v = std::vector{1, 2, 3}; auto& e : v) {  
2     std::cout << e;  
3 }  
4 // prints "123"
```

[[likely]] and [[unlikely]] attributes

Provides a hint to the optimizer that the labelled statement has a high probability of being executed.

```
1 switch (n) {  
2 case 1:  
3     // ...  
4     break;  
5  
6 [[likely]] case 2: // n == 2 is considered to be arbitrarily  
7     // ...           // likely than any other value of n  
8     break;  
9 }
```

If one of the likely/unlikely attributes appears after the right parenthesis of an if-statement, it indicates that the branch is likely/unlikely to have its substatement (body) executed.

```
1 int random = get_random_number_between_x_and_y(0, 3);  
2 if (random > 0) [[likely]] {  
3     // body of if statement  
4     // ...  
5 }
```

It can also be applied to the substatement (body) of an iteration statement.

```
1 while (unlikely_truthy_condition) [[unlikely]] {  
2     // body of while statement  
3     // ...  
4 }
```

deprecate implicit capture of this

Implicitly capturing this in a lambda capture using [=] is now deprecated; prefer capturing explicitly using [=, this] or [=, *this].

```
1 struct int_value {  
2     int n = 0;
```

```

3     auto getter_fn() {
4         // BAD:
5         // return [=]() { return n; };
6
7         // GOOD:
8         return [=, *this]() { return n; };
9     }
10 };

```

class types in non-type template parameters

Classes can now be used in non-type template parameters. Objects passed in as template arguments have the type `const T`, where `T` is the type of the object, and has static storage duration.

```

1 struct foo {
2     foo() = default;
3     constexpr foo(int) {}
4 };
5
6 template <foo f>
7 auto get_foo() {
8     return f;
9 }
10
11 get_foo(); // uses implicit constructor
12 get_foo<foo{123}>();

```

constexpr virtual functions

Virtual functions can now be `constexpr` and evaluated at compile-time. `constexpr` virtual functions can override non-`constexpr` virtual functions and vice-versa.

```

1 struct X1 {
2     virtual int f() const = 0;
3 };
4
5 struct X2: public X1 {
6     constexpr virtual int f() const { return 2; }
7 };
8
9 struct X3: public X2 {
10     virtual int f() const { return 3; }
11 };
12
13 struct X4: public X3 {
14     constexpr virtual int f() const { return 4; }
15 };
16
17 constexpr X4 x4;

```

```
18 x4.f(); // == 4
```

explicit(bool)

Conditionally select at compile-time whether a constructor is made explicit or not. `explicit(true)` is the same as specifying `explicit`.

```
1 struct foo {
2     // Specify non-integral types (strings, floats, etc.) require
   explicit construction.
3     template <typename T>
4     explicit(!std::is_integral_v<T>) foo(T) {}
5 };
6
7 foo a = 123; // OK
8 foo b = "123"; // ERROR: explicit constructor is not a candidate
   (explicit specifier evaluates to true)
9 foo c {"123"}; // OK
```

immediate functions

Similar to `constexpr` functions, but functions with a `constexpr` specifier must produce a constant. These are called immediate functions.

```
1 constexpr int sqr(int n) {
2     return n * n;
3 }
4
5 constexpr int r = sqr(100); // OK
6 int x = 100;
7 int r2 = sqr(x); // ERROR: the value of 'x' is not usable in a
   constant expression
8 // OK if 'sqr' were a 'constexpr' function
```

using enum

Bring an enum's members into scope to improve readability. Before:

```
1 enum class rgba_color_channel { red, green, blue, alpha };
2
3 std::string_view to_string(rgba_color_channel channel) {
4     switch (channel) {
5         case rgba_color_channel::red:     return "red";
6         case rgba_color_channel::green:   return "green";
7         case rgba_color_channel::blue:    return "blue";
8         case rgba_color_channel::alpha:   return "alpha";
9     }
10 }
```

After:


```

1 enum class rgba_color_channel { red, green, blue, alpha };
2
3 std::string_view to_string(rgba_color_channel my_channel) {
4     switch (my_channel) {
5         using enum rgba_color_channel;
6         case red:     return "red";
7         case green:   return "green";
8         case blue:    return "blue";
9         case alpha:   return "alpha";
10    }
11 }

```

lambda capture of parameter pack

Capture parameter packs by value:

```

1 template <typename... Args>
2 auto f(Args&&... args){
3     // BY VALUE:
4     return [...args = std::forward<Args>(args)] {
5         // ...
6     };
7 }

```

Capture parameter packs by reference:

```

1 template <typename... Args>
2 auto f(Args&&... args){
3     // BY REFERENCE:
4     return [&...args = std::forward<Args>(args)] {
5         // ...
6     };
7 }

```

char8_t

Provides a standard type for representing UTF-8 strings.

```

1 char8_t utf8_str[] = u8"\u0123";

```

constexpr

The constexpr specifier requires that a variable must be initialized at compile-time.

```

1 const char* g() { return "dynamic initialization"; }
2 constexpr const char* f(bool p) { return p ? "constant
   initializer" : g(); }
3
4 constexpr const char* c = f(true); // OK
5 constexpr const char* d = f(false); // ERROR: 'g' is not
   constexpr, so 'd' cannot be evaluated at compile-time.

```

`__VA_OPT__`

Helps support variadic macros by evaluating to the given argument if the variadic macro is non-empty.

```
1 #define F(...) f(0 __VA_OPT__(,) __VA_ARGS__)
2 F(a, b, c) // replaced by f(0, a, b, c)
3 F()       // replaced by f(0)
```

1.2.8 C++20 new library features

Following a summary of the new library features introduced with C++20.

concepts library

Concepts are also provided by the standard library for building more complicated concepts. Some of these include:

Core language concepts:

- `same_as` — specifies two types are the same.
- `derived_from` — specifies that a type is derived from another type.
- `convertible_to` — specifies that a type is implicitly convertible to another type.
- `common_with` — specifies that two types share a common type.
- `integral` — specifies that a type is an integral type.
- `default_constructible` — specifies that an object of a type can be default-constructed.

Comparison concepts:

- `boolean` — specifies that a type can be used in Boolean contexts.
- `equality_comparable` — specifies that `operator==` is an equivalence relation.

Object concepts:

- `movable` — specifies that an object of a type can be moved and swapped.
- `copyable` — specifies that an object of a type can be copied, moved, and swapped.
- `semiregular` — specifies that an object of a type can be copied, moved, swapped, and default constructed.
- `regular` — specifies that a type is regular, that is, it is both `semiregular` and `equality_comparable`.

Callable concepts:

- `invocable` — specifies that a callable type can be invoked with a given set of argument types.
- `predicate` — specifies that a callable type is a Boolean predicate.

See also §1.2.7 (concepts).

synchronized buffered outputstream

Buffers output operations for the wrapped output stream ensuring synchronization (i.e. no interleaving of output).

```
1 std::osyncstream{std::cout} << "The value of x is:" << x << std::endl;
```

std::span

A span is a view (i.e. non-owning) of a container providing bounds-checked access to a contiguous group of elements. Since views do not own their elements they are cheap to construct and copy – a simplified way to think about views is they are holding references to their data. As opposed to maintaining a pointer/iterator and length field, a span wraps both of those up in a single object.

Spans can be dynamically-sized or fixed-sized (known as their extent). Fixed-sized spans benefit from bounds-checking.

Span doesn't propagate const so to construct a read-only span use `std::span<const T>`.

Example: using a dynamically-sized span to print integers from various containers.

```
1 void print_ints(std::span<const int> ints) {
2     for (const auto n : ints) {
3         std::cout << n << std::endl;
4     }
5 }
6
7 print_ints(std::vector{ 1, 2, 3 });
8 print_ints(std::array<int, 5>{ 1, 2, 3, 4, 5 });
9
10 int a[10] = { 0 };
11 print_ints(a);
12 // etc.
```

Example: a statically-sized span will fail to compile for containers that don't match the extent of the span.

```
1 void print_three_ints(std::span<const int, 3> ints) {
2     for (const auto n : ints) {
3         std::cout << n << std::endl;
4     }
5 }
6
7 print_three_ints(std::vector{ 1, 2, 3 }); // ERROR
8 print_three_ints(std::array<int, 5>{ 1, 2, 3, 4, 5 }); // ERROR
9 int a[10] = { 0 };
10 print_three_ints(a); // ERROR
11
12 std::array<int, 3> b = { 1, 2, 3 };
13 print_three_ints(b); // OK
14
15 // You can construct a span manually if required:
```

```

16 std::vector c{ 1, 2, 3 };
17 print_three_ints(std::span<const int, 3>{ c.data(), 3 }); // OK:
    set pointer and length field.
18 print_three_ints(std::span<const int, 3>{ c.cbegin(), c.cend() })
    ; // OK: use iterator pairs.

```

bit operations

C++20 provides a new <bit> header which provides some bit operations including popcount.

```

1 std::popcount(0u); // 0
2 std::popcount(1u); // 1
3 std::popcount(0b1111'0000u); // 4

```

math constants

Mathematical constants including PI, Euler's number, etc. defined in the <numbers> header.

```

1 std::numbers::pi; // 3.14159...
2 std::numbers::e; // 2.71828...
3
4 \subsubsection{\href{https://github.com/AnthonyCalandra/modern-
    cpp-features/tree/master/#std::is_constant_evaluated}
5 {std::is\char' _constant\char' _evaluated}}
6 Predicate function which is truthy when it is called in a compile
    -time context.
7
8 \begin{lstlisting}[language=C++]
9 constexpr bool is_compile_time() {
10     return std::is_constant_evaluated();
11 }
12
13 constexpr bool a = is_compile_time(); // true
14 bool b = is_compile_time(); // false

```

std::make_shared supports arrays

```

1 auto p = std::make_shared<int[]>(5); // pointer to 'int[5]'
2 // OR
3 auto p = std::make_shared<int[5]>(); // pointer to 'int[5]'

```

starts_with and ends_with on strings

Strings (and string views) now have the starts_with and ends_with member functions to check if a string starts or ends with the given string.

```

1 std::string str = "foobar";
2 str.starts_with("foo"); // true
3 str.ends_with("baz"); // false

```

check if associative container has element

Associative containers such as sets and maps have a `contains` member function, which can be used instead of the "find and check end of iterator" idiom.

```

1 std::map<int, char> map {{1, 'a'}, {2, 'b'}};
2 map.contains(2); // true
3 map.contains(123); // false
4
5 std::set<int> set {1, 2, 3};
6 set.contains(2); // true

```

std::bit_cast

A safer way to reinterpret an object from one type to another.

```

1 float f = 123.0;
2 int i = std::bit_cast<int>(f);

```

std::midpoint

Calculate the midpoint of two integers safely (without overflow).

```

1 std::midpoint(1, 3); // == 2

```

std::to_array

Converts the given array/"array-like" object to a `std::array`.

```

1 std::to_array("foo"); // returns 'std::array<char, 4>'
2 std::to_array<int>({1, 2, 3}); // returns 'std::array<int, 3>'
3
4 int a[] = {1, 2, 3};
5 std::to_array(a); // returns 'std::array<int, 3>'

```

text formatting

Provides a compile-time checked string formatting library via `std::format`. Runtime formatting uses `std::vformat` and related helpers.

```

1 std::format("{} ", 123); // OK -- returns "123 "
2 std::format("{} {}", 123); // ERROR -- not enough arguments
3 std::format("{} {} ", "Here's a number:", 123); // OK

```

Formatting a string based on a formatter created at runtime:

```

1 std::string fmt = "{} {}";
2 fmt += "{}{}";
3 std::vformat(fmt, std::make_format_args("Here's a number:", 1, 2,
4 // OK -- returns "Here's a number: 123"
3));

```

When formatting fails (such as an invalid format string), `std::vformat` throws `std::format_error`.
To format custom types:

```

1 struct fraction {
2     int numerator;
3     int denominator;
4 };
5
6 template <>
7 struct std::formatter<fraction> {
8     constexpr auto parse(std::format_parse_context& ctx) {
9         return ctx.begin();
10    }
11
12    auto format(const fraction& f, std::format_context& ctx) const
13    {
14        return std::format_to(ctx.out(), "{0:d}/{1:d}", f.numerator,
15        f.denominator);
16    }
17 };
18
19 fraction f{1, 2};
20 std::format("{} ", f); // == "1/2"

```

std::bind_front

Binds the first N arguments to a given free function, lambda, or member function.

```

1 const auto f = [](int a, int b, int c) { return a + b + c; };
2 const auto g = std::bind_front(f, 1, 1);
3 g(1); // == 3

```

uniform container erasure

Provides `std::erase` and/or `std::erase_if` for STL containers such as string, list, vector, map, etc. Both return the number of erased elements.

```

1 std::vector v{0, 1, 0, 2, 0, 3};
2 std::erase(v, 0); // v == {1, 2, 3}
3 std::erase_if(v, [](int n) { return n == 0; }); // v == {1, 2, 3}

```

three-way comparison helpers

Helper functions for naming comparison results:

```

1 std::is_eq(0 <=> 0); // == true
2 std::is_lteq(0 <=> 1); // == true
3 std::is_gt(0 <=> 1); // == false

```

std::lexicographical_compare_three_way

Lexicographically compares two ranges using three-way comparison and produces a result of the strongest applicable comparison category type.

```

1 std::vector a{0, 0, 0}, b{0, 0, 0}, c{1, 1, 1};
2
3 auto cmp_ab = std::lexicographical_compare_three_way(
4     a.begin(), a.end(), b.begin(), b.end());
5 std::is_eq(cmp_ab); // == true
6
7 auto cmp_ac = std::lexicographical_compare_three_way(
8     a.begin(), a.end(), c.begin(), c.end());
9 std::is_lt(cmp_ac); // == true

```

std::jthread

A thread of execution (like `std::thread`) that joins on destruction and can be signaled to stop. Unlike `std::thread`, you need not check `joinable()` before joining — the destructor joins automatically. Request a stop via `request_stop` or the thread's `stop_source`:

```

1 std::jthread t{
2     [](std::stop_token token) {
3         while (!token.stop_requested()) {
4             std::this_thread::sleep_for(1s);
5         }
6     }
7 };
8
9 t.request_stop();
10 // OR, through the stop source:
11 std::stop_source stopSource = t.get_stop_source();
12 stopSource.request_stop();

```

safe integral comparisons

Compare integers, including those of distinct types, without the dangers of integer conversion.

```

1 -1 > 0U; // == true
2 std::cmp_greater(-1, 0U); // == false
3
4 std::cmp_equal(0U, 0); // == true
5 std::cmp_less_equal(-1, 1U); // == true
6

```

```
7 std::in_range<unsigned>(-1); // == false
8 std::in_range<char>(999999); // == false
```

1.2.9 C++23 (to expand)

C++23 material will be added selectively later (e.g. `std::expected`, `std::print`, deducing this, flat containers / `mdspan` as relevant). Until then, use the upstream cheat-sheet and `cppreference`. C++26 waits until the standard is stable.

C++ Bibliography

- [1] *The Most Vexing Parse: How to Spot It and Fix It Quickly*. [Fluent C++](#).
- [2] R. Martinho Fernandes. *Rule of Zero*. [ISOC++ Blog](#). 2012.
- [3] Scott Meyers. *A Concern about the Rule of Zero*. [The View from Aristeia - Scoot Meyers' Activities and Interests](#). 2014.
- [4] *A polymorphic class should suppress public copy/move*. [Standard Cpp Foundation Github](#).
- [5] *If you define or =delete any copy, move, or destructor function, define or =delete them all*. [Standard Cpp Foundation Github](#).
- [6] *References*. [ISOC++ Wiki](#).
- [7] *Most vexing parse*. [WikiPedia](#).
- [8] *Core C++ - lvalues and rvalues*. [Just Software Solutions](#). 2016.
- [9] *Rvalue References and Perfect Forwarding*. [Just Software Solutions](#). 2008.
- [10] Peter Dimov, Howard E. Hinnant, and Dave Abraham. *The Forwarding Problem: Arguments*. [open-std.org - N1385=02-0043](#).
- [11] Meyers, Sutter, and Alexandrescu. *Session Announcement: Adventures in Perfect Forwarding*. [C++ and Beyond](#). 2011.
- [12] *What are the main purposes of std::forward and which problems does it solve?* [StackOverflow Thread](#).
- [13] *What is the move semantic?* [StackOverflow Thread](#).
- [14] Howard Hinnant. *Everything you wanted to know about Move Semantics*. [Slideshare](#). 2014.
- [15] *If your function must not throw declare it noexcept*. [Cpp Core Guidelines](#).
- [16] *What is the meaning of auto keyword*. [StackOverflow Thread](#).
- [17] Herb Sutter. *auto variables, Part 1*. [Guru of the week - 92](#).
- [18] Herb Sutter. *auto variables, Part 2*. [Guru of the week - 93](#).
- [19] Herb Sutter. *AAA style (Almost Always auto)*. [Guru of the week - 94](#).
- [20] *What's In a Class? - The Interface Principle*. [Guru of the week - C++ Report, 10\(3\), March 1998](#).
- [21] *Declaring non-type template arguments with auto*. [Programming Language C++, Evolution Working Group - P0127R1](#). 2016.
- [22] *Declaring non-type template parameters with auto*. [Programming Language C++, Evolution Working Group - P0127R2](#). 2016.
- [23] *Pros and Cons of Alternative Function Syntax in C++*. [Petr Zemek's Blog of a Software Engineer](#).

- [24] *ADL - Argument-dependent lookup*. [Cppreference](#).
- [25] *Lambda Expressions*. [Cppreference](#).
- [26] Crowl Larvi Freeman. *Lambda Expressions and Closures: Wording for Monomorphic Lambdas (Revision 4)*. [open-std.org](#). 2008.
- [27] Vandevoorde. *New wording for C++0x Lambdas*. [open-std.org](#). 2009.
- [28] *Closure (Computer Programming)*. [WikiPedia](#).
- [29] *Abbreviated function template*. [Cppreference](#).
- [30] *decltype* specifier. [Cppreference](#).
- [31] *Member Function Pointers and the Fastest Possible C++ Delegates*. [codeproject.com](#).
- [32] Stroustrup and Dos Reis. *Initializer list (Rev. 3)*. [open-std.org](#). 2007.
- [33] Merrill and Vandevoorde. *Initializer Lists - Alternative Mechanism and Rationale (v.2)*. [Initializer Lists — Alternative Mechanism and Rationale \(v. 2\)](#). 2008.
- [34] Thorsten Ottosen. *Wording for range-based for-loop (revision 2)*. [open-std.org](#). 2007.
- [35] *Range-based for statements and ADL*. [open-std.org](#).
- [36] Svoboda. *Delete operators default to noexcept*. [open-std.org](#). 2010.
- [37] Mauer. *Deducing noexcept for destructors*. [open-std.org](#). 2010.
- [38] Abrahams, Sharoni, and Gregor. *Allowing Move Constructors to Throw (Rev. 1)*. [open-std.org](#). 2010.
- [39] Bjarne Stroustrup. *Bjarne Stroustrup's C++ Style and Technique FAQ*. [stroustrup.com](#).
- [40] Bjarne Stroustrup. *Exception Safety: Concepts and Techniques*. [stroustrup.com](#).
- [41] *Static Initialization Order Fiasco*. [cppreference.com](#).
- [42] *Empty base optimization*. [cppreference.com](#).
- [43] *Apache C++ Standard Template Library*. [apache.org](#).
- [44] *Lapack: Linear Algebra Subroutine Library*. [siam.org](#).
- [45] *Netlib*. [netlib.org](#).
- [46] *Available C++ Libraries FAQ maintained by Nikki Locke*. [trumphurst.com](#).
- [47] *Mathtools - A Resource for Technical Computing Community*. [mathtools.net](#).
- [48] *boost Library*. [boost.org](#).
- [49] *Most Vexing Parse*. [Wiki Most vexing parse](#).
- [50] *Const Correctness*. [ISOC++ Wiki](#).

Idioms

2.1 C++ Idioms

An idiom can be considered as the most natural way to express something or to solve a problem. More specifically, an idiom is a group of code fragments sharing an equivalent semantic role, which recurs frequently in one or more programming languages or libraries.

A comprehensive list of C++ Idioms can be found at [51]. See also [74].

2.1.1 PImpl Idiom

See [53].

"Pointer to implementation" or "pImpl" is a C++ programming technique that **removes implementation details of a class from its object representation** by placing them in a separate class, accessed **through an opaque pointer**.

This idiom is also known as **Compiler Firewall Idiom**. See also [52], [58], [59].

Problematic Scenario

Whenever a class definition changes in a header, all users of that class must be recompiled. The reason is the textual inclusion on which the C++ build model is based. The user is assumed to know two features which could be affected by private members: **size and layout** and **functions**.

Solution

```
1 // Pimpl idiom - basic idea
2 class widget {
3     // :::
4 private:
5     struct impl;           // things to be hidden go here
6     impl* pimpl_;         // opaque pointer to forward-declared
                             class
7 };
```

2.1.2 Fast PImpl Idiom

See [57].

Classic PImpl pays for an extra heap allocation and an extra indirection. The **Fast PImpl** idea is to keep the opaque implementation, but store it in a fixed-size buffer **inside** the object (or otherwise avoid the heap) so that construction stays cheap while the header still hides the real layout.

Trade-off: you must pick a buffer size/alignment large enough for impl, and you typically still need out-of-line destructor / special members because impl is incomplete in the header.

```
1 #include <cstddef>
2 #include <new>
3
4 class Widget {
5 public:
6     Widget();
7     ~Widget();
8     Widget(const Widget&) = delete;
9     Widget& operator=(const Widget&) = delete;
10
11     void draw();
12
13 private:
14     struct Impl;
15     static constexpr std::size_t buf_size = 64;
16     alignas(std::max_align_t) unsigned char buf_[buf_size];
17
18     Impl* pimpl() { return reinterpret_cast<Impl*>(buf_); }
19     const Impl* pimpl() const { return reinterpret_cast<const
20     Impl*>(buf_); }
21 };
22
23 // widget.cpp
24 struct Widget::Impl {
25     int x = 0;
26     void draw() { /* ... */ }
27 };
```

```

28 Widget::Widget() { static_assert(sizeof(Impl) <= buf_size); new (
    buf_) Impl{}; }
29 Widget::~~Widget() { pimpl()->~Impl(); }
30 void Widget::draw() { pimpl()->draw(); }

```

2.1.3 Handle/Object Idiom

See [71].

Separate a thin public **handle** (what clients use) from a heavy **body/object** (where the real state and work live). The handle usually holds a pointer/reference to the body and forwards operations.

This is closely related to PImpl / Bridge: clients depend on a stable handle interface; the body can change, be shared, or be replaced without rewriting client code.

```

1  class StringBody;                // heavy representation
2
3  class String {                   // handle
4  public:
5      String(const char* s);
6      String(const String& other);
7      String& operator=(String other); // often with copy-and-swap
8      ~String();
9
10     char operator[](std::size_t i) const;
11     void append(const char* s);
12
13 private:
14     StringBody* body_;
15 };

```

2.1.4 Copy and Swap Idiom

See [61], [62].

This idiom aims at creating a strong exception-safe implementation of the overloaded assignment operator.

This idiom makes use of the **RAII** idiom in order to acquire the new resource before it forfeits its current resource. If the acquisition is successful, the old resource is released; if it throws, the object is left unchanged (**strong exception guarantee**).

```

1  class Buffer {
2  public:
3      Buffer(std::size_t n);
4      Buffer(const Buffer& other);
5      ~Buffer();
6
7      friend void swap(Buffer& a, Buffer& b) noexcept {
8          using std::swap;
9          swap(a.data_, b.data_);

```

```

10     swap(a.size_, b.size_);
11 }
12
13 Buffer& operator=(Buffer other) noexcept { // pass by value =
14     copy/move
15     swap(*this, other);
16     return *this;
17 }
18 private:
19     char* data_;
20     std::size_t size_;
21 };

```

2.1.5 RAI - Resource Acquisition is Initialization

See [72].

RAII ties a resource's lifetime to an object's lifetime: acquire in the constructor, release in the destructor. Stack unwinding then releases resources automatically on both success and exception paths.

Typical uses: locks, file handles, memory, sockets. Standard library examples include `std::unique_ptr`, `std::lock_guard`, `std::fstream`.

```

1  class File {
2  public:
3      explicit File(const char* path) : f_{std::fopen(path, "rb")}
4      {
5          if (!f_) throw std::runtime_error("open failed");
6      }
7      ~File() { if (f_) std::fclose(f_); }
8
9      File(const File&) = delete;
10     File& operator=(const File&) = delete;
11
12     std::size_t read(void* buf, std::size_t n) {
13         return std::fread(buf, 1, n, f_);
14     }
15 private:
16     std::FILE* f_;
17 };
18
19 void use() {
20     File f("data.bin");           // acquire
21     // ...                       // always released when f goes
22     out of scope
23 }

```

2.1.6 DBDII - Dynamic Binding During Initialization Idiom

See [54].

Calling a virtual function from a constructor/destructor does **not** dispatch to the most-derived override: the dynamic type is still under construction. DBDII avoids that trap by finishing construction first, then invoking a virtual “post-constructor” (often a factory create() that returns a smart pointer and then calls init()).

```
1  class Base {
2  protected:
3      Base() = default;
4  public:
5      virtual ~Base() = default;
6      virtual void post_construct() = 0;    // safe: object fully
                                           constructed
7
8      template <class T, class... Args>
9      static std::unique_ptr<T> create(Args&&... args) {
10         auto p = std::unique_ptr<T>(new T(std::forward<Args>(args)
11         ...));
12         p->post_construct();              // dynamic binding OK
13         here
14         return p;
15     }
16 };
17
18 class Derived : public Base {
19     friend class Base;
20     Derived() = default;
21 public:
22     void post_construct() override { /* use virtuals safely */ }
23 };
24
25 auto d = Base::create<Derived>();
```

2.1.7 Named Constructor Idiom

See [64].

Make constructors private/protected and expose clearer static factory functions with meaningful names. This documents intent, can enforce invariants, and can return by value / smart pointer without overloading ambiguity.

```
1  class Point {
2  public:
3      static Point cartesian(double x, double y) { return Point{x,
4      y}; }
5      static Point polar(double r, double theta) {
6          return Point{r * std::cos(theta), r * std::sin(theta)};
7      }
8  }
```

```

8 private:
9     Point(double x, double y) : x_{x}, y_{y} {}
10    double x_, y_;
11 };
12
13 auto p = Point::polar(1.0, 3.14159 / 4);

```

2.1.8 Construct On First Use Idiom

See [65].

Avoid the static initialization order fiasco: instead of a namespace-scope object whose constructor may run before another translation unit is ready, wrap the object in a function-local static so it is constructed on first call (and destroyed at program exit in a well-defined way for that object).

```

1 // Bad across TUs: relative init order of a and b is undefined
2 // extern Log log;
3 // Log& get_log() { return log; }
4
5 Log& get_log() {
6     static Log log;    // constructed on first call
7     return log;
8 }
9
10 void f() { get_log().write("hi"); }

```

2.1.9 Nifty Counter Idiom

See [66], [67].

Used by the standard streams (std::cout & co.): each translation unit that includes the iostream header gets a static object whose constructor/destructor bump a shared counter. The first construction initializes the stream objects; the last destruction tears them down. That way streams are usable during dynamic initialization of other statics in TUs that included the header.

```

1 // Simplified sketch of the idea (not the real library code)
2 class IostreamInit {
3 public:
4     IostreamInit() {
5         if (count_++ == 0) { /* construct cin/cout/cerr */ }
6     }
7     ~IostreamInit() {
8         if (--count_ == 0) { /* destroy cin/cout/cerr */ }
9     }
10 private:
11     static int count_;
12 };
13

```



```

14 static IostreamInit iostream_init; // in each TU that includes
    the header

```

2.1.10 Named Parameter Idiom

See [68].

Long positional constructors hide which argument is which (`Game(1920, 1080, true, "Demo")` — which is width?). The named-parameter idiom emulates named/defaulted arguments by returning `*this` from chainable setters (a lightweight builder). Call sites read as a sequence of named options instead of a long positional list.

When: configuration objects, optional knobs, APIs where defaults cover most callers and a few fields need overriding. **Not when:** every call site sets every field anyway — then a plain constructor (or aggregate init) is clearer and cheaper to write.

Noise removed: positional argument mix-ups and “what does the third `bool` mean?” comments. **Cost:** more surface area (one setter per option), fluent chaining that can feel like builder boilerplate, and no compile-time check that a required option was set unless you add a final `build()`/validation step.

Interview tip: say *readability for optional config*, not “builders are always better” — contrast with a positional ctor when the arity is small and fixed.

```

1  class Game {
2  public:
3      Game& windowSize(int w, int h) { w_ = w; h_ = h; return *this; }
4      Game& fullscreen(bool v) { full_ = v; return *this; }
5      Game& title(std::string t) { title_ = std::move(t); return *this; }
6
7  private:
8      int w_ = 800, h_ = 600;
9      bool full_ = false;
10     std::string title_ = "game";
11 };
12
13 Game g;
14 g.windowSize(1920, 1080).fullscreen(true).title("Demo");

```

2.1.11 Virtual Friend Function Idiom

See [69].

Friends are not virtual. To get polymorphic behavior for operators like `operator<<`, provide a public non-virtual friend that forwards to a private/protected virtual member.

```

1  class Shape {
2  public:
3      virtual ~Shape() = default;
4      friend std::ostream& operator<< (std::ostream& os, const Shape
    & s) {

```

```

5         return s.print(os);                // virtual dispatch
6     }
7 protected:
8     virtual std::ostream& print(std::ostream& os) const = 0;
9 };
10
11 class Circle : public Shape {
12 protected:
13     std::ostream& print(std::ostream& os) const override {
14         return os << "Circle";
15     }
16 };
17
18 void f(const Shape& s) { std::cout << s << '\n'; }

```

2.1.12 CRTP - Curiously recurring template pattern

See [55], [56].

Virtual dispatch solves “call the right override through a base pointer,” but pays for an indirect call and often blocks inlining on the hot path. CRTP is the static alternative: a class `Derived` inherits from `Base<Derived>`. The base can `static_cast` to `Derived` and call derived members **without virtual functions** (static polymorphism). Common uses: mixins, compile-time interfaces, `enable_shared_from_this`-style helpers.

Noise removed: vtable indirection and the “cannot inline through virtual” wall when the concrete type is known at compile time. **Cost:** each `Base<D>` is a distinct type — no heterogeneous `vector<Base*>` of mixed deriveds; template instantiation can grow binary size; you cannot swap implementations at runtime without another mechanism.

When virtuals win: truly open hierarchies, plugins loaded later, or containers of mixed types behind one interface (then prefer virtuals or type erasure, Section 2.1.13).

Interview tip: “CRTP = static polymorphism, no vtable; virtuals when you need runtime heterogeneity.”

```

1 template <typename D>
2 struct Comparable {
3     bool operator==(const D& other) const {
4         return static_cast<const D*>(this)->value()
5             == other.value();
6     }
7     bool operator!=(const D& other) const { return !(*this ==
8         other); }
9 };
10 struct Point : Comparable<Point> {
11     int x, y;
12     int value() const { return x ^ y; } // used by Comparable
13 };
14
15 Point a{1, 2}, b{1, 2};

```

```
16 bool same = (a == b); // resolved at compile time
```

2.1.13 Type Erasure

See [73].

Templates keep the concrete type everywhere (fast, monomorphic). CRTP (Section 2.1.12) avoids vtables but still cannot put unrelated deriveds in one container. Type erasure hides a concrete type behind a uniform handle while retaining behaviour (virtuals, function pointers, or a small-buffer-optimized wrapper). Clients depend on the erased handle; the real type is known only at the construction site. `std::function`, `std::any`, and polymorphic allocators are standard examples.

Noise removed: forcing a shared inheritance hierarchy just to stuff heterogeneous objects into one vector, or exploding APIs into templates at every call site. **Cost:** typically a heap (or SBO) control block, an indirect call, and possible allocation on construction/-copy — real money on a hot path (Section 15.4). Prefer open templates or CRTP when the set of types is closed and known at compile time.

Interview tip: pick the axis — *heterogeneous container* → erasure or virtuals; *max speed, known types* → templates/CRTP; name the control-block/alloc cost of `std::function`-style erasure.

```
1  class Drawable { // type-erased handle
2  public:
3      template <class T>
4      Drawable(T x) : self_{std::make_shared<Model<T>>(std::move(x))} {}
5
6      void draw() const { self_->draw(); }
7
8  private:
9      struct Concept {
10         virtual ~Concept() = default;
11         virtual void draw() const = 0;
12     };
13     template <class T>
14     struct Model : Concept {
15         explicit Model(T x) : data_{std::move(x)} {}
16         void draw() const override { data_.draw(); }
17         T data_;
18     };
19     std::shared_ptr<const Concept> self_;
20 };
21
22 struct Circle { void draw() const { /* ... */ } };
23 struct Square { void draw() const { /* ... */ } };
24
25 std::vector<Drawable> scene{Circle{}, Square{}};
26 for (const auto& d : scene) d.draw();
```

Idioms Bibliography

- [51] *More C++ Idioms*. [Wikibooks](#).
- [52] *Pimpl Idiom*. [Wiki Wiki Web](#).
- [53] *PImpl Idiom*. [cppreference](#).
- [54] *Calling Virtuals from Constructors: Dynamic Binding During Initialization Idiom*. [ISOC++ Wiki](#).
- [55] *CRTP - Curiously Recurring Template Pattern*. [Wiki Wiki Web](#).
- [56] *CRTP - Curiously Recurring Template Pattern*. [Cppreference](#).
- [57] Herb Sutter. *The Fast Pimpl Idiom*. [Guru of the Week - 28](#).
- [58] Herb Sutter. *Compilation Firewalls*. [Guru of the Week - 24](#).
- [59] Herb Sutter. *Compilation Firewalls (C++11-updated version of GotW #24)*. [Guru of the Week - 100](#).
- [60] Herb Sutter. *The Joy of Pimpls (or more about the Compiler-Firewall Idiom)*. [More About Compiler Firewalls](#).
- [61] *What is the copy and swap idiom?* [Stackoverflow](#).
- [62] Herb Sutter. *Exception-Safe Class Design, Part I: Copy Assignment*. [Guru of the Week - 59](#).
- [63] Herb Sutter. *Exception-Safe Class Design, Part II: Inheritance*. [Guru of the Week - 60](#).
- [64] *What is the Named Constructor Idiom?* [ISOC++ Wiki](#).
- [65] *Construct On First Use Idiom*. [ISOC++ Wiki](#).
- [66] *Nifty Counter Idiom*. [ISOC++ Wiki](#).
- [67] *Nifty Counter Idiom*. [Wikibook](#).
- [68] *Named Parameter Idiom*. [ISOC++ Wiki](#).
- [69] *Virtual Friend Function Idiom*. [ISOC++ Wiki](#).
- [70] *C++ Core Guidelines*. [Standard Cpp Foundation Github](#).
- [71] Anton Eliens. *The Handle/Body Idiom*. [Vrije Universiteit Amsterdam](#).
- [72] *Resource acquisition is initialization (RAII)*. [Wikipedia](#).
- [73] *Type Erasure*. [Wikibooks](#).
- [74] Andrii Polyanskyy. *C++ idioms everyone should know*. [YouTube — Engineering Community](#).

Memory Management

3.1 Memory Management

Allocation means to assign, allot, distribute or "set apart for a particular purpose". Programs manage their memory partitioning or dividing it into different units performing specific tasks.

In C++, there are two ways to create objects and this aspect reflects on the existence of two different units used to allocate the memory, the **stack and heap**, which manage the program's used/unused memory in two different ways.

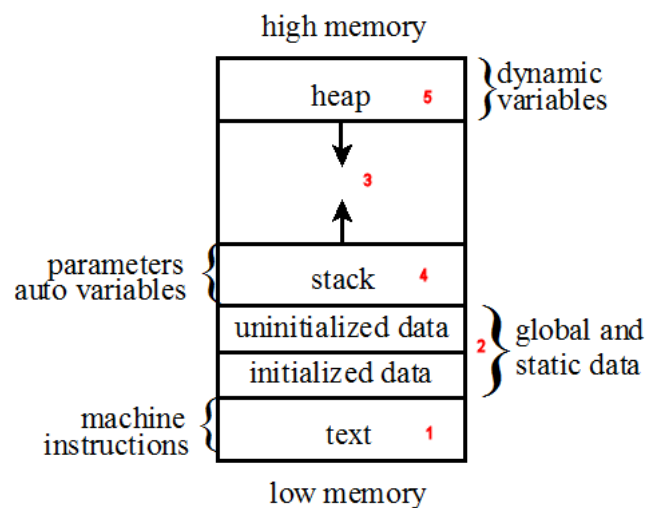


Figure 3.1: Functional memory organization of a running program

Text

This area contains machine instructions (i.e. the executable code). **Global** and **static** variables are typically allocated at program startup. **Stack** contains all the automatic variables. **Heap** is meant to allocate the memory for the dynamic variables through the **new** and **delete** operators. The space in between is allocated by the operating system but not used, and is meant to let both the heap and stack grow.

Stack

This is a simple **LIFO** (last in, first out) data structure which must support at least the **push and pop operations**. Even though a stack only allows accesses at the top, it is easy to implement and fast to execute push and pop operations.

Heap

This is a **more flexible data structure**, which allows programs to allocate or deallocate memory anywhere and in any convenient order. This might cause **holes** in the heap, i.e. unallocated memory surrounded by allocated memory.

The only restriction for the heap is that **memory can be allocated in a contiguous block of memory large enough to satisfy the request with a single chunk of memory**. This restriction has **two effects**: it requires the operating system **to scan the heap** until a contiguous block is found, and **to coalesce adjacent blocks of memory** returned by the program.

Side effect is that the operation on the heap are slower than the operations on the stack.

3.1.1 Memory Allocation and Deallocation in C

Manual heap ownership is a pairing problem: every successful allocation must eventually meet exactly one matching free, on a pointer that still refers to that block. Miss the pair and you leak; free twice, free a non-heap pointer, or touch after free and you have undefined behaviour — often silent until production.

Noise removed by a correct malloc/free discipline: unbounded growth from leaks and the “mystery crash after free” class of bugs. **Cost / latency:** the C allocator is a shared, lock-aware heap — contention, coalescing, and page faults inject jitter (see the heap discussion above and Section 15.4). Prefer pools/arenas on hot paths; keep C allocators for cold setup or C-interop boundaries.

Interview tip: name the pairing invariant first (alloc ↔ free), then the UB cases (double-free, use-after-free, wrong pointer), then why HFT code avoids the global heap on the tick path.

<pre>void* malloc(std::size_t size)</pre>	Allocates size bytes of uninitialized storage. On success, returns the pointer to the beginning of newly allocated memory. To avoid a memory leak, the returned pointer must be deallocated. On failure, returns a null pointer.
<pre>void free(void* ptr);</pre>	Deallocates the space previously allocated • If ptr is a null pointer, the function does nothing. • The behavior is undefined if the value of ptr does not equal a value returned by an earlier allocation the memory area referred to by ptr has already been deallocated, after std::free returns, an access is made through the pointer ptr.

In the previous table, allocation is meant to be done by one of the following:

`std::malloc`, `std::calloc`, `std::aligned_alloc` (since C++17), or `std::realloc`.

Deallocation is meant to be done by one of the following:

`std::free()` or `std::realloc()`

3.1.2 Memory Allocation and Deallocation in C++

A new-expression does two jobs: obtain storage, then construct an object in it. operator new is only the allocation half (raw bytes); delete destroys then deallocates. Mixing levels — free on a new pointer, delete on malloc, or skipping the destructor after placement construction — is UB.

Noise removed by keeping expression-level and operator-level APIs straight: accidental double-destroy / wrong deallocator bugs. **Latency link:** every allocating `new` is a trip into the global heap unless you replace `operator new` or feed placement `new` from a pool (Sections 3.1.2, 3.1.2, and 15.4). On a hot path, prefer preallocated storage over “just `new` another object.”

Interview tip: “`new-expression` = `operator new` + constructor; placement `new` = constructor only” — then point at pools when they ask how to cut alloc jitter.

<code>new expression</code>	Creates and initializes objects with dynamic storage duration, that is, objects whose lifetime is not necessarily limited by the scope in which they were created.
<code>delete expression</code>	Destroys object(s) previously allocated by the <code>new-expression</code> and releases obtained memory area.
<code>operator new</code>	Allocates raw storage only (no constructor). Used by the allocating <code>new-expression</code> ; overridable for custom heaps/pools.
<code>operator delete</code>	Releases storage previously obtained from <code>operator new</code> (no destructor). Paired with the <code>delete-expression</code> after destruction.

placement new

See [82].

A normal `new-expression` does two things: it **allocates** storage (via `operator new`) and then **constructs** an object in that storage.

Placement new skips allocation: you supply a suitably sized and aligned address, and it only runs the constructor there. This is the basis of object pools, arena allocators, and constructing into a preallocated buffer.

```

1 #include <new>      // placement new
2
3 alignas(T) unsigned char buf[sizeof(T)];
4
5 T* p = new (buf) T{/* args */};    // construct T in buf
6 p->do_work();
7 p->~T();                          // destroy manually
8 // do NOT delete p: buf was not allocated by operator new

```

Rules of thumb:

- Storage must be at least `sizeof(T)` and properly aligned (e.g. `alignas(T)`, `std::aligned_storage` or memory from an allocator that guarantees alignment).
- Lifetime ends with an explicit destructor call `p->~T()`, or by reusing/releasing the storage only after destruction.

- Pairing: if memory came from `malloc/operator new/a pool`, free it with the matching deallocator — never delete a pointer whose storage was not obtained from a matching allocating `new`.
- Arrays: `placement new[]` is rarely what you want; construct elements one by one (or use an allocator/`uninitialized_*` helpers).

Typical uses: memory pools, `std::vector` reallocation/growth, optional/variant storage, shared-memory objects, and Fast PImpl-style embedded buffers (see §2.1.2).

Memory pools

See [86], [87].

A **memory pool** preallocates a block (or several blocks) of storage and hands out pieces of it on demand, instead of calling the global heap allocator for every object. Clients typically **construct with placement new** (Section 3.1.2) and **destroy explicitly** before returning the slot to the pool.

Why use one:

- Fewer (or zero) calls into `malloc/operator new` on the hot path.
- More predictable latency — less lock contention and fewer page faults after warm-up.
- Better locality when objects of the same size live in one contiguous arena.

Fixed-size free-list pool. All slots have the same size (here: one `T`). Free slots are linked in a singly linked list. `acquire` pops a slot and constructs; `release` destroys and pushes the slot back.

```

1  #include <cstddef>
2  #include <cstdint>
3  #include <new>
4  #include <utility>
5
6  template <class T, std::size_t N>
7  class FixedPool {
8      union Slot {
9          alignas(T) unsigned char storage[sizeof(T)];
10         Slot* next;
11     };
12
13     Slot slots_[N];
14     Slot* free_ = nullptr;
15
16 public:
17     FixedPool() {
18         for (std::size_t i = 0; i + 1 < N; ++i)
19             slots_[i].next = &slots_[i + 1];
20         slots_[N - 1].next = nullptr;

```

```

21     free_ = &slots_[0];
22 }
23
24 template <class... Args>
25 T* acquire(Args&&... args) {
26     if (!free_) return nullptr;           // pool exhausted
27     Slot* s = free_;
28     free_ = s->next;
29     return new (s->storage) T(std::forward<Args>(args)...);
30 }
31
32 void release(T* p) {
33     if (!p) return;
34     p->~T();
35     auto* s = reinterpret_cast<Slot*>(p);
36     s->next = free_;
37     free_ = s;
38 }
39 };
40
41 // usage
42 FixedPool<Message, 1024> pool;
43 Message* m = pool.acquire(/* ... */);
44 // ... use m ...
45 pool.release(m);

```

Bump / arena allocator (sketch). An arena allocates by advancing a cursor in a big buffer (**bump pointer**). Individual frees are often omitted: you reset or destroy the whole arena at once. Ideal for many short-lived objects with the same lifetime (e.g. one request / one tick).

```

1 class Arena {
2     char* begin_;
3     char* curr_;
4     char* end_;
5 public:
6     Arena(char* buf, std::size_t n)
7         : begin_{buf}, curr_{buf}, end_{buf + n} {}
8
9     void* allocate(std::size_t n, std::size_t align) {
10         auto mis = reinterpret_cast<std::uintptr_t>(curr_) %
align;
11         if (mis) curr_ += align - mis;
12         if (curr_ + n > end_) return nullptr;
13         void* p = curr_;
14         curr_ += n;
15         return p;
16     }
17
18     void reset() { curr_ = begin_; } // bulk free
19 };

```

```

20
21 // T* p = new (arena.allocate(sizeof(T), alignof(T))) T{...};

```

Pitfalls.

- **Lifetime / lifetime:** always destroy before recycling storage; never delete a pool pointer.
- **Exhaustion:** decide whether acquire returns nullptr, throws, or falls back to the heap.
- **Threading:** a shared free list needs synchronization (or thread-local pools). Standard PMR offers `std::pmr::unsynchronized_pool_resource` (single-threaded) and `std::pmr::synchronized_pool_resource`.
- **Fragmentation of sizes:** fixed-size pools waste space if object sizes vary; use size classes or an arena instead.

For low-latency hot paths, preallocate pools at startup and recycle on the critical path (see Section 15.4).

Custom allocators (STL and PMR)

Pools and arenas allocate raw bytes. A [C++ allocator](#) is the adaptor that lets standard containers obtain that storage through a uniform interface. See [88], [89].

Minimal STL-style allocator

Modern containers mainly need `value_type`, `allocate`, and `deallocate` (plus equality). Sketch — backs a `std::vector` with `::operator new` (replace the body with a pool/arena in real code):

```

1  #include <cstddef>
2  #include <memory>
3  #include <new>
4  #include <vector>
5
6  template <class T>
7  struct Mallocator {
8      using value_type = T;
9
10     Mallocator() = default;
11     template <class U> Mallocator(const Mallocator<U>&) noexcept {}
12
13     T* allocate(std::size_t n) {
14         if (n > static_cast<std::size_t>(-1) / sizeof(T))
15             throw std::bad_array_new_length{};
16         return static_cast<T*>(::operator new(n * sizeof(T)));
17     }

```

```

18
19     void deallocate(T* p, std::size_t) noexcept {
20         ::operator delete(p);
21     }
22 };
23
24 template <class T, class U>
25 bool operator==(const Mallocator<T>&, const Mallocator<U>&) {
26     return true; }
27 template <class T, class U>
28 bool operator!=(const Mallocator<T>&, const Mallocator<U>&) {
29     return false; }
30
31 // std::vector<int, Mallocator<int>> v;

```

PMR (polymorphic memory resources)

C++17 `std::pmr` separates *where* memory comes from (`std::pmr::memory_resource`: monotonic buffer, pool, new/delete) from *which* container type you write (`std::pmr::vector`, `polymorphic_allocator`). Same container spelling; swap the resource behind it (e.g. a per-tick arena).

```

1 #include <memory_resource>
2 #include <vector>
3
4 char buf[64 * 1024];
5 std::pmr::monotonic_buffer_resource arena{buf, sizeof(buf)};
6 std::pmr::vector<int> v{&arena}; // grows inside buf until
    exhausted

```

Interview tip: pool/arena = storage strategy; allocator/PMR = how you **plumb** that strategy into `std::vector` and friends. On HFT hot paths prefer preallocated pools; use allocators so cold/library code can share them without rewriting containers.

Memory Management Bibliography

- [75] Herb Sutter. *Leak-Freedom in C++ ... By Default* — CppCon 2016. [CppCon YouTube Video](#). 2016.
- [76] *Memory Management: Stack and Heap*. [Weber CS1410 notes](#).
- [77] *malloc*. [Cppreference](#).
- [78] *calloc*. [Cppreference](#).
- [79] *aligned alloc*. [Cppreference](#).
- [80] *realloc*. [Cppreference](#).
- [81] *free*. [Cppreference](#).
- [82] *new expression*. [Cppreference](#).
- [83] *delete expression*. [Cppreference](#).
- [84] *operator new*. [Cppreference](#).
- [85] *operator delete*. [Cppreference](#).
- [86] *std::pmr::unsynchronized_pool_resource*. [Cppreference](#).
- [87] *Object pool pattern*. [Wikipedia](#).
- [88] *AllocatorAwareContainer / std::allocator*. [Cppreference](#) — [Allocator](#).
- [89] *std::pmr (polymorphic memory resources)*. [Cppreference](#) — [memory_resource](#).

4

CHAPTER

Smart Pointers

4.1 Smart Pointers in C++

See [90].

Smart pointers enable *automatic, exception-safe, object lifetime management*.

Generally speaking, a smart pointer is a *data type whose value can be used like a pointer*, with the *additional feature of memory management*, that *frees the memory when it is no longer used*. A smart pointer should therefore be used any time the ownership of memory needs to be tracked.

Which pointer?

Ownership is the question; the type records the answer.

- Prefer `std::unique_ptr` — exclusive ownership, zero overhead vs a raw pointer + delete. Default choice whenever one owner is enough.
- Use `std::shared_ptr` only when ownership is genuinely shared. Cost: a separate control block plus atomic reference-count traffic on copy/destroy — real noise under contention and on the hot path (Section 15.4). Prefer `std::make_shared` when you do need sharing (one allocation for object + control block).
- Use `std::weak_ptr` to observe a `shared_ptr`-managed object without extending its lifetime (break cycles, caches).

- Do not use `std::auto_ptr` (removed from the language): historical exclusive-ownership attempt that *copied by moving* — the source became null on copy, a silent footgun in containers and by-value APIs. Replaced by `unique_ptr` + move semantics.

Interview tip: “unique by default; shared only with a reason; name the atomic control block; weak breaks cycles; `auto_ptr` is the move-on-copy trap.”

4.1.1 `std::unique_ptr`

See [91].

`std::unique_ptr` is a smart pointer that *owns and manages* another object through a pointer and *disposes of* that object *when* the `std::unique_ptr` goes *out of scope*.

The object is disposed of, using the associated deleter when either of the following happens:

- the managing `std::unique_ptr` object is *destroyed*
- the managing `std::unique_ptr` object is *assigned another pointer* via `operator=` or `reset()`.

Notes

Only a **non-const** `std::unique_ptr` can *transfer the ownership* of the managed object to **another** `std::unique_ptr` (via `move`).

If an object's lifetime is managed by a **const** `std::unique_ptr`, it is *limited to the scope* in which the pointer was created.

`std::unique_ptr` is commonly used to *manage* the *lifetime of objects*, including:

- providing *exception safety* to classes and functions that handle objects with dynamic lifetime, by *guaranteeing deletion* on both normal exit and exit through exception
- *passing ownership* of uniquely-owned objects with dynamic lifetime into functions
- *acquiring ownership* of uniquely-owned objects with dynamic lifetime from functions
- as the element type in move-aware containers, such as `std::vector`, which hold pointers to dynamically-allocated objects (e.g. if *polymorphic behavior* is desired).

`std::unique_ptr` may be constructed for an *incomplete type* `T`, such as to facilitate the use as a handle in the *PImpl idiom* (Chapter 2).

```
1 #include <memory>
2
3 struct Widget { void draw(); };
4
5 std::unique_ptr<Widget> p = std::make_unique<Widget>();
6 p->draw();
7
8 std::unique_ptr<Widget> q = std::move(p); // ownership
   transferred
9 // p == nullptr
```

`std::make_unique`

Prefer `std::make_unique` (C++14) over `new` (see [95]; also §1.2.4): no naked `new`, less repetition of the type name, and better exception safety when building function arguments.

```
1 foo(std::make_unique<T>(), function_that_throws(), std::
    make_unique<T>());
```

Arrays and custom deleters

Array form uses `delete[]`. A custom deleter is useful for C APIs, pools, and `FILE*`.

```
1 // array form: uses delete[]
2 auto buf = std::make_unique<int[]>(1024);
3
4 // custom deleter (e.g. C API / pool / FILE*)
5 std::unique_ptr<FILE, decltype(&std::fclose)> f{
6     std::fopen("data.bin", "rb"), &std::fclose};
```

Member types

pointer	Must satisfy <code>NullablePtr</code>
element_type	T, the type of the object managed by this <code>std::unique_ptr</code>
deleter_type	Deleter, the function object or lvalue reference to function or to function object, to be called from the destructor

Member functions

constructor	constructs a new <code>unique_ptr</code>
destructor	destructs the managed object if such is present
operator=	assigns the <code>unique_ptr</code> (moves ownership)

Modifiers

release	releases ownership and returns the raw pointer (caller must delete)
reset	replaces the managed object (deletes the old one if any)
swap	swaps the managed objects with another <code>unique_ptr</code>

Observers

get	returns a pointer to the managed object
get_deleter	returns the deleter that is used for destruction of the managed object
operator bool	checks if there is an associated managed object

Single-object version, `unique_ptr<T>`

<code>operator*</code> <code>operator-></code>	dereferences pointer to the managed object
--	--

4.1.2 `std::shared_ptr`

See [92].

This is a smart pointer that **retains shared ownership** of an object through a pointer.

Several `shared_ptr` objects may own the same object.

The object is destroyed and its memory deallocated when either of the following happens:

- the **last remaining** `shared_ptr` owning the object **is destroyed**;
- the **last remaining** `shared_ptr` owning the object is **assigned another pointer** via `operator=` or `reset()`.

A `std::shared_ptr` can share ownership of an object while storing a pointer to another object (aliasing constructor).

A `std::shared_ptr` may also own no objects, in which case it is called empty (an empty `std::shared_ptr` may have a non-null stored pointer if the aliasing constructor was used to create it).

`std::make_shared`

Prefer `std::make_shared` (see [96]; also §1.2.2): exception safety, and typically **one allocation** for control block + object.

```
1 auto a = std::make_shared<Widget>();  
2 auto b = a; // shared ownership; use_count() == 2  
3 a.reset(); // b still owns the Widget
```

Control-block bookkeeping is thread-safe; concurrent mutation of the *managed object* is not.

Member functions

constructor	constructs a new <code>shared_ptr</code>
destructor	decrements the reference count; destroys the object if it reaches zero
operator=	assigns the <code>shared_ptr</code>

Modifiers

reset	replaces the managed object
swap	swaps the managed objects

Observers

<code>get</code>	returns the stored pointer
<code>operator*</code> <code>operator-></code>	dereferences the stored pointer
<code>operator[]</code>	provides indexed access to the stored array (C++17)
<code>use_count</code>	returns the number of <code>shared_ptr</code> objects referring to the same managed object
<code>unique</code>	checks if <code>use_count() == 1</code> (deprecated in C++17, removed in C++20)
<code>operator bool</code>	checks if the stored pointer is not null
<code>owner_before</code>	provides owner-based ordering of shared pointers

4.1.3 std::weak_ptr

See [93].

std::weak_ptr is a smart pointer that holds a non-owning (“weak”) reference to an object that is managed by std::shared_ptr. It must be converted to std::shared_ptr (via lock()) in order to access the referenced object.

std::weak_ptr models **temporary / non-owning** observation: it does not keep the object alive, but can detect whether it still exists (expired()).

Breaking cycles

If two objects each hold a shared_ptr to the other, the reference count never reaches zero. Store one side as weak_ptr.

```
1 struct Node {
2     std::shared_ptr<Node> next;    // owning
3     std::weak_ptr<Node> prev;    // non-owning: breaks the cycle
4 };
5
6 auto a = std::make_shared<Node>();
7 auto b = std::make_shared<Node>();
8 a->next = b;
9 b->prev = a;
10
11 if (auto p = b->prev.lock()) {    // promote to shared_ptr if
12     still alive
13     // use *p
14 }
```

Related: std::enable_shared_from_this<T> (see [97]) lets a member function obtain a shared_ptr to *this safely when the object is already owned by a shared_ptr.

Member functions

constructor	creates a new weak_ptr
destructor	destroys a weak_ptr (does not destroy the managed object)
operator=	assigns a weak_ptr

Modifiers

reset	releases the weak reference
swap	swaps with another weak_ptr

Observers

<code>use_count</code>	returns the number of <code>shared_ptr</code> objects that manage the object
<code>expired</code>	checks whether the referenced object was already deleted
<code>lock</code>	creates a <code>shared_ptr</code> that manages the referenced object (or empty if expired)
<code>owner_before</code>	provides owner-based ordering of weak pointers

4.1.4 `std::auto_ptr`

See [94].

`std::auto_ptr` was an early exclusive-ownership smart pointer. Copying it **transferred** ownership (surprising copy semantics), which made it unsafe in standard containers and easy to misuse.

It was **deprecated in C++11** and **removed in C++17**. Use `std::unique_ptr` (move-only) instead.

Smart Pointers Bibliography

- [90] *Dynamic Memory Management Library*. [Cppreference](#).
- [91] *unique pointer*. [Cppreference](#).
- [92] *shared pointer*. [Cppreference](#).
- [93] *weak pointer*. [Cppreference](#).
- [94] *auto pointer*. [Cppreference](#).
- [95] *std::make_unique*. [Cppreference](#).
- [96] *std::make_shared*. [Cppreference](#).
- [97] *std::enable_shared_from_this*. [Cppreference](#).

5

CHAPTER

Containers

See [98].

The Containers library is a *generic collection of class templates and algorithms* that allow programmers *to easily implement common data structures* like queues, lists and stacks.

The container classes are:

- *sequence containers* — data structures that can be accessed sequentially.
- *associative containers* — searchable in $O(\log n)$ (typically tree-based, ordered by key).
- *unordered associative containers* — searchable in $O(1)$ on average and $O(n)$ in the worst case (hash-based).
- *container adaptors* — provide a restricted interface on top of an underlying container (stack, queue, `priority_queue`, and the C++23 `flat_*` family).

Decision Charts

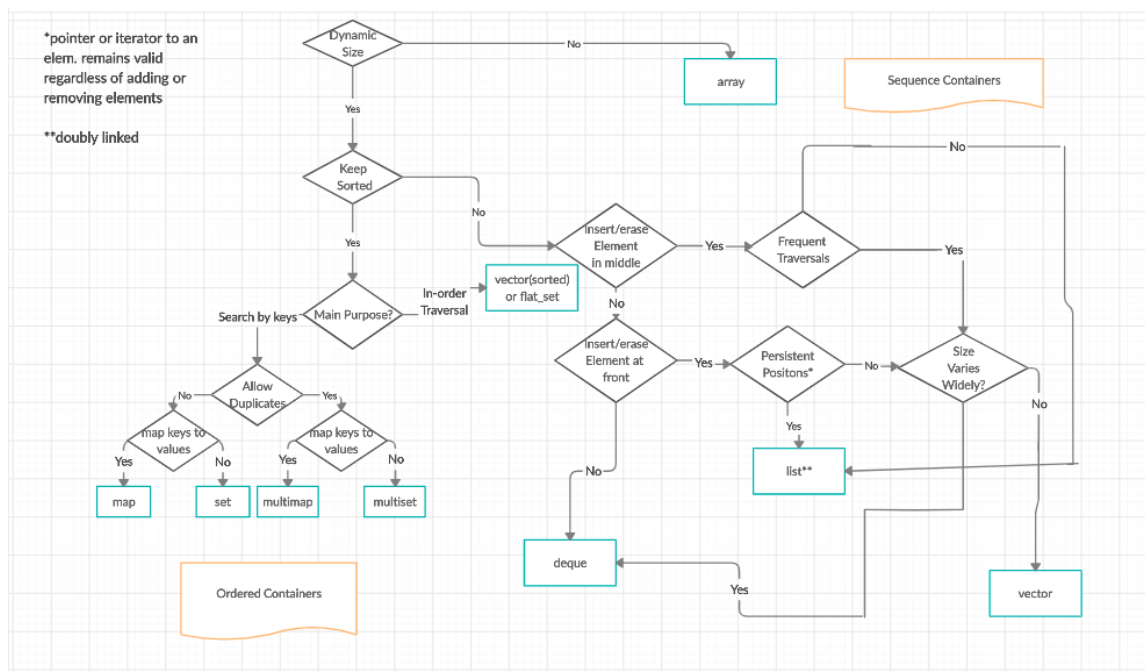


Figure 5.1: Decision Chart: Sequence And Ordered Containers

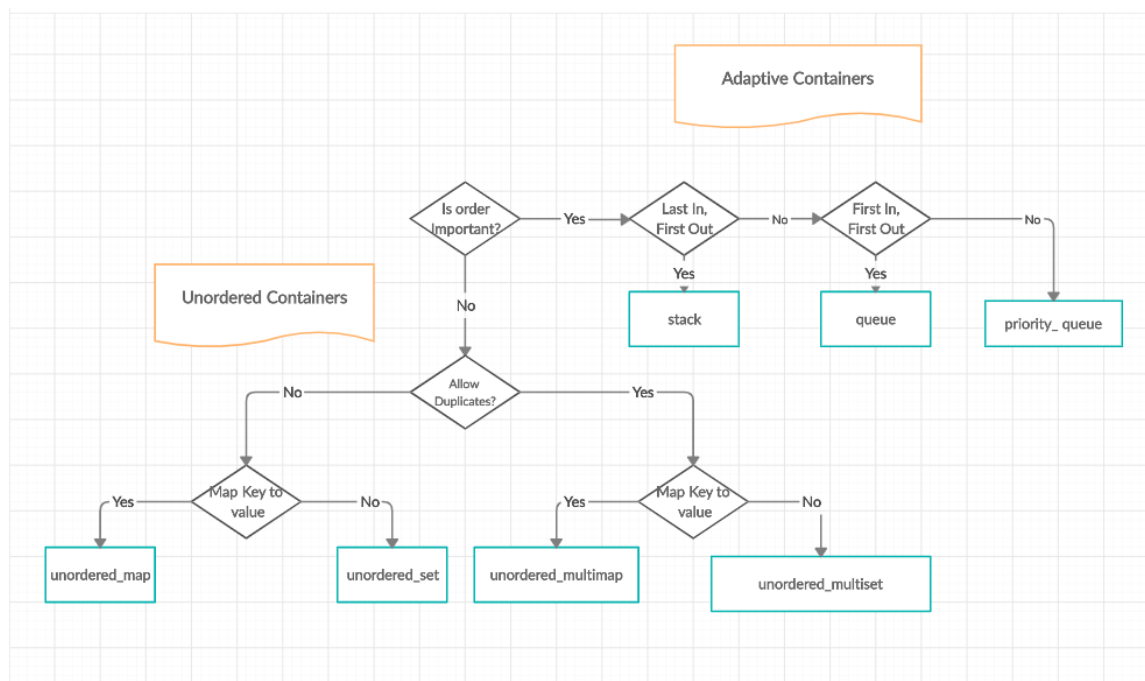


Figure 5.2: Decision Chart: Adaptive And Unordered Containers

5.1 Container catalog

Quick reference to the standard containers (with cppreference links in the tables). Choice guidance and a few idioms follow each family.

5.1.1 Sequence Containers

array	<i>static contiguous</i> array
vector	<i>dynamic contiguous</i> array
deque	<i>double-ended</i> queue
forward_list	<i>singly-linked</i> list
list	<i>doubly-linked</i> list

When to use which

- Default to `std::vector` — contiguous storage, best cache behaviour, random access $O(1)$.
- Use `std::array` for a fixed size known at compile time (no heap).
- Use `std::deque` when you need efficient push/pop at **both** ends (not truly one contiguous block).
- Use `std::list` / `forward_list` when you need stable iterators under insert/erase in the middle, or splicing — at the cost of locality.

vector capacity

Growth may reallocate: all iterators/references are invalidated when capacity changes. Prefer `reserve` when the final size is known (fewer allocations; ties to Chapter 3 and Section 15.4).

`push_back` is *amortized* $O(1)$: a single call may copy the whole buffer ($\Theta(n)$), but n pushes with geometric growth cost $O(n)$ total. That is **not** the same as average-case hashing — see Section 17.1.1.

```
1 std::vector<int> v;  
2 v.reserve(1024);           // capacity >= 1024, size still 0  
3 for (int x : data)  
4     v.push_back(x);        // no reallocation if data.size() <=  
5                             1024  
6 v.shrink_to_fit();          // non-binding request to release  
                             unused capacity
```

5.1.2 Associative Containers

set	collection of <i>unique keys, sorted by keys</i>
map	collection of <i>key-value pairs, sorted by keys, keys are unique</i>
multiset	collection of <i>keys, sorted by keys</i>
multimap	collection of <i>key-value pairs, sorted by keys</i>

When to use which

- Need ordered iteration or predecessor/successor by key → map/set.
- Multiple equivalent keys → multi*.
- Lookup/insert typically $O(\log n)$; iterators stay valid except for erased elements.

```
1 std::map<std::string, int> ages{{"Ann", 30}, {"Bob", 25}};  
2 ages["Ann"] = 31; // insert or assign  
3 if (auto it = ages.find("Bob"); it != ages.end())  
4     ++it->second;
```

5.1.3 Unordered Associative Containers

unordered_set	collection of <i>unique keys, hashed by keys</i>
unordered_map	collection of <i>key-value pairs, hashed by keys, keys are unique</i>
unordered_multiset	collection of <i>keys, hashed by keys</i>
unordered_multimap	collection of <i>key-value pairs, hashed by keys</i>

When to use which

- Prefer when you care about average $O(1)$ lookup and do **not** need order.
- Provide a good hash and equality for custom keys; watch load factor / rehash (rehash invalidates iterators — see Section 5.2).
- Worst case $O(n)$ (pathological hashing / attacks); ordered trees are more predictable.

```
1 std::unordered_map<std::string, int> hist;  
2 ++hist["buy"];  
3 hist.reserve(1024); // buckets; can avoid rehash during fill
```

5.1.4 Ordered vs unordered associative

Same API family, different engines. Applies to both `map/unordered_map` and `set/unordered_set` (`multi*` analogues exist either way).

	Ordered (<code>map/set</code>)	Unordered (<code>unordered_*</code>)
Typical structure	Balanced tree (red-black)	Hash table (buckets)
Lookup / insert / erase	$O(\log n)$ worst-case	Avg $O(1)$; worst $O(n)$
Iteration order	Sorted by key	Arbitrary (bucket order)
Needs from the key	Strict weak ordering ($<$ / comparator)	Hash + equality
Iterator stability	Only erased nodes invalidated	Rehash invalidates iterators
Prefer when	Need order, range queries, predictable latency	Need fast exact lookup, no order

Interview tip: “I need sorted keys / `lower_bound` $\rightarrow$ `map/set`. I only need exact lookup by key $\rightarrow$ `unordered_map/unordered_set`.” Mention worst-case and rehash cost for hash tables.

5.1.5 How a hash table works (sketch)

An `unordered_map/unordered_set` is a [hash table](#):

1. Compute `hash(key)  $\rightarrow$  integer`.
2. Map that integer to a [bucket](#) index (`hash % bucket_count`).
3. Store the element in that bucket. If two keys land in the same bucket ([collision](#)), the implementation chains or otherwise searches within the bucket (still correct via `key_eq`).
4. [Load factor](#) $\approx$ `size / bucket_count`. When it grows too high, the table [rehashes](#): more buckets, elements move — **iterators invalidated**, one-time $O(n)$ cost.

Good average $O(1)$ needs a decent hash and not-too-high load factor. `reserve/rehash` up front when you know the size. Pathological hashes (or adversarial keys) can degrade toward $O(n)$ — that is why ordered trees stay attractive for predictable latency.

`unordered_map::find` complexity

`find / insert / erase`:

- **Average / expected:** $O(1)$ — hash to a bucket, then scan a short chain when the load factor is healthy.
- **Worst case:** $O(n)$ — many keys collide in one bucket (or a rehash is forced elsewhere in the API).

So “hash map lookup is $O(1)$ ” in interview talk means [expected](#) $O(1)$, not a hard guarantee. Contrast with `map::find`: always $O(\log n)$. Vocabulary: Section 17.1.1.

5.1.6 Complexity cheat sheet

Amortized / typical complexities interviewers expect (unique-key associative; multi* similar). “Random access” means operator `[]` / iterator $+ k$. Average-case (hashing) $\neq$ amortized (rare realloc / rebalance) — Section 17.1.1.

Container	Access	Insert	Erase	Find
vector / array	$O(1)$	end amort. $O(1)$; middle $O(n)$	end $O(1)$; middle $O(n)$	$O(n)$ linear
deque	$O(1)$	ends amort. $O(1)$; middle $O(n)$	ends $O(1)$; middle $O(n)$	$O(n)$
list / forward_list	$O(n)$	$O(1)$ at known pos	$O(1)$ at known pos	$O(n)$
set / map	—	$O(\log n)$	$O(\log n)$	$O(\log n)$
unordered_set/map	—	avg $O(1)$	avg $O(1)$	avg $O(1)$
stack/queue (adaptors)	top/front $O(1)$	$O(1)$	$O(1)$	—
priority_queue	top $O(1)$	$O(\log n)$	pop $O(\log n)$	—

Iterate whole container: $O(n)$ for all of the above. `unordered_*` find/insert/erase worst case $O(n)$ (see above).

5.1.7 Container Adaptors

Adaptors do not invent storage; they wrap an underlying container and expose a narrow policy (LIFO, FIFO, heap-ordered). The performance story is mostly the underlying container's — choose deque vs vector for the adaptor's access pattern, not the adaptor name alone.

stack	adapts a container to provide stack (<i>LIFO data structure</i>)
queue	adapts a container to provide queue (<i>FIFO data structure</i>)
priority_queue	adapts a container to provide <i>priority queue</i>

Default underlying containers: deque for stack/queue, vector for priority_queue. They expose a narrow interface (push/pop/top or front), not full random access.

Noise removed: accidental misuse of a full container API when you only needed stack-/queue semantics. **Cost:** no magic — a priority_queue is still $O(\log n)$ per push/pop on its underlying random-access container.

Interview tip: name the policy (LIFO/FIFO/heap) and the default underlying type in one breath.

```
1 std::priority_queue<int> pq;
2 pq.push(3); pq.push(10); pq.push(1);
3 int best = pq.top();    // 10
4 pq.pop();
```

C++23 flat associative adaptors

See [116], [115].

Node-based map/set chase pointers (poor locality, allocator traffic per node). Flat adaptors store keys (and values) in contiguous sorted containers — excellent locality and cheap iteration; inserts/erases in the middle shift elements and are $O(n)$.

When flat_map beats map: small-to-medium n , lookup-heavy / iterate-heavy workloads, or when you build once (sorted bulk load) and query often. **When map wins:** large n with frequent mid-container insert/erase, or when pointer/reference stability to elements matters.

Noise removed: pointer-chasing misses and per-node allocations on read-mostly tables. **Cost:** $O(n)$ insert/erase shift; invalidation rules closer to vector than to node-based maps (see Section 5.2).

flat_set	adapts a container to provide a collection of <i>unique keys, sorted by keys</i>
flat_map	adapts two containers to provide a collection of <i>key-value pairs, sorted by unique keys</i>
flat_multiset	adapts a container to provide a collection of <i>keys, sorted by keys</i>
flat_multimap	adapts two containers to provide a collection of <i>key-value pairs, sorted by keys</i>

5.1.8 Views

See [119], [120].

Views are **non-owning**: they refer to existing contiguous data (or a string) without allocating. Prefer them for function parameters that should not force a copy. (Also covered in Contemporary C++: `std::string_view`, `std::span`.)

Why non-owning: the caller (or a ring buffer / file mapping) keeps the bytes; the callee only needs a window. That is the same ownership discipline as zero-copy market-data parsing (Section 14.3): view the buffer, decode, do not extend the view past reclaim.

Noise removed: mandatory `std::string/vector` copies at API boundaries. **Cost / bug class**: dangling — a `span/string_view` must not outlive the storage it points at (temporaries, locals, reallocated vector buffers).

Interview tip: “non-owning window + lifetime bound to the owner” — same story as ring-buffer zero-copy.

<code>span</code>	non-owning view over a contiguous sequence (C++20)
<code>string_view</code>	non-owning view over a character sequence (C++17)

```
1 void sum(std::span<const int> s);           // works for array, vector,
   C array, ...
2
3 std::vector<int> v{1, 2, 3, 4};
4 sum(v);
5 sum(std::span{v}.subspan(1, 2));           // view into the middle
6
7 void print(std::string_view sv);           // no std::string
   allocation at the boundary
8 print("literal");
9 print(std::string{"temp"});               // OK while the temporary
   lives for the full call
```

Caution: do not return a `string_view/span` that refers to a temporary or locals that are about to die.

5.2 Invalidation of Iterators and References

Iterator invalidation is a use-after-free cousin: the iterator still *looks* like a pointer into the container, but the container moved, rehashed, or erased the element underneath. Classic bugs: keep an iterator across a vector reallocation; insert into an `unordered_map` while iterating and trigger a rehash; erase and then dereference the old iterator.

Noise removed by respecting the rules below: Heisenbugs that only show under growth or load-factor edges. **Cost of ignoring them:** UB — not a clean exception.

Interview tip: pick one container and narrate *when* iterators die (e.g. “vector reallocation invalidates everything; `unordered_*` rehash invalidates iterators”).

Rules of thumb:

- vector: reallocation invalidates **everything**; without reallocation, insert/erase invalidates at/after the modified point.
- deque: inserting/erasing at ends keeps references to other elements (iterators may still be invalidated — treat carefully); modifying the middle is more destructive.
- list / forward_list: insert/erase does not invalidate other iterators/references.
- Associative (ordered): only erased elements are invalidated.
- Unordered: **rehash** invalidates iterators (references to elements typically remain); without rehash, only erased iterators are invalidated.

Category	Container	Valid after insertion		Valid after erasure		Conditionality
		iterators	reference	iterators	reference	
Sequence Containers	array	N/A		N/A		
	vector	No		N/A		Insertion changed capacity
		Yes		Yes		Before modified element(s) (for insertion only if capacity didn't change)
		No		No		At or after modified element(s)
	deque	No	Yes	Yes, but erased one(s)		Modified first or last element
			No	No		Modified middle only
	list	Yes		Yes, but erased one(s)		
	forward_list	Yes		Yes, but erased one(s)		
Associative Containers	set multiset map multimap	Yes		Yes, but erased one(s)		
Unordered associative Containers	unordered_set unordered_multiset	No	Yes	N/A		Insertion caused rehash
	unordered_map unordered_multimap	Yes		Yes, but erased one(s)		No rehash

Containers Bibliography

- [98] *Containers Library*. [Cplusplusreference](#).
- [99] *array*. [Cplusplusreference](#).
- [100] *vector*. [Cplusplusreference](#).
- [101] *deque*. [Cplusplusreference](#).
- [102] *forward list*. [Cplusplusreference](#).
- [103] *list*. [Cplusplusreference](#).
- [104] *set*. [Cplusplusreference](#).
- [105] *map*. [Cplusplusreference](#).
- [106] *multiset*. [Cplusplusreference](#).
- [107] *multimap*. [Cplusplusreference](#).
- [108] *unordered set*. [Cplusplusreference](#).
- [109] *unordered map*. [Cplusplusreference](#).
- [110] *unordered multiset*. [Cplusplusreference](#).
- [111] *unordered multimap*. [Cplusplusreference](#).
- [112] *stack*. [Cplusplusreference](#).
- [113] *queue*. [Cplusplusreference](#).
- [114] *priority queue*. [Cplusplusreference](#).
- [115] *flat set*. [Cplusplusreference](#).
- [116] *flat map*. [Cplusplusreference](#).
- [117] *flat multiset*. [Cplusplusreference](#).
- [118] *flat multimap*. [Cplusplusreference](#).
- [119] *std::span*. [Cplusplusreference](#).
- [120] *std::basic_string_view*. [Cplusplusreference](#).

6

CHAPTER

Concurrency

See [126], [127].

C++ includes built-in support for threads, atomic operations, mutual exclusion, condition variables, and futures. Deeper formal memory-model material lives in Chapter 8; this chapter is the library map plus usage notes.

6.1 Threads

A `std::thread` is the C++ handle for an OS thread of execution. Threads in the same process [share the address space](#): that is why they communicate cheaply through memory, and why unsynchronized shared writes are data races (Chapter 8).

<code>thread</code>	manages a separate thread
<code>jthread</code>	thread with support for auto-joining and cooperative stop (C++20)

Process vs thread (C++)

- A **process** is an OS instance of a program with its own address space.
- A **thread** is a unit of execution *inside* a process; threads of the same process share memory (hence data races if unsynchronized).

- `std::thread` must be `join()`ed or `detach()`ed before destruction, or `std::terminate` is called — a common production footgun.
- Prefer `std::jthread` when you can: it joins on destruction and can receive a stop token (C++20).

Noise / failure modes: leaking joinable threads at shutdown; oversubscribing cores (context-switch jitter) when a thread pool would do; using threads when a single-threaded reactor + queues is simpler for I/O.

Interview tip: join-or-detach before destruction; prefer `jthread` when you can explain the stop token.

```

1  #include <iostream>
2  #include <thread>
3  #include <string>
4
5  void task1(std::string msg) {
6      std::cout << "task1 says: " << msg << '\n';
7  }
8
9  int main() {
10     std::thread t1(task1, "Hello"); // starts running
11     // ... other work on this thread ...
12     t1.join();                      // wait for completion
13 }
```

6.1.1 Functions managing the current thread

These APIs act on the *current* thread. On a latency path, sleeping and yielding are how you **donate the core** back to the scheduler — useful for fairness, harmful if you needed busy-poll determinism.

<code>yield</code>	hints the implementation to reschedule; may be a no-op; not a lock substitute
<code>get_id</code>	returns the id of the current thread
<code>sleep_for</code>	blocks the current thread for at least the given duration (wake-up jitter)
<code>sleep_until</code>	blocks until a time point

Noise removed by *not* sleeping on the hot path: wake-up delay and unpredictable reschedule time. Prefer condition variables / queues when you must wait; prefer busy poll only on isolated cores (Chapter 15).

Interview tip: `yield` is a hint, not a lock; `sleep_for` is wake-up jitter you chose.

6.2 Atomicity

6.2.1 Atomic operations

Fine-grained concurrent updates without a mutex. Each atomic operation is indivisible with respect to other atomic operations on the *same* object. Atomic objects are **free of**

data races on that object — that is the race/UB noise they remove compared to a plain `int` shared by two threads.

Atomics do *not* automatically make multi-word invariants safe; you still choose a `memory_order` (Section 6.2.14) so publishes become visible in the order you intend.

Interview tip: atomic on one word $\neq$ atomic on a struct of several fields — that still needs a mutex or a carefully designed protocol.

```
1 std::atomic<int> counter{0};  
2 // concurrent increments are data-race-free:  
3 counter.fetch_add(1, std::memory_order_relaxed);
```

6.2.2 Atomic types

`std::atomic<T>` wraps a value so loads/stores/RMWs are atomic. Prefer it for flags, counters, and single-word handoffs. `atomic_ref` overlays atomic ops on an existing non-atomic object (lifetime/alignment rules apply) — useful when you cannot change the storage type.

<code>atomic</code>	atomic class template and specializations for <code>bool</code> , integral, and pointer types
<code>atomic_ref</code>	provides atomic operations on non-atomic objects

Check `is_lock_free` when you care: some wide types fall back to internal locks, which reintroduces contention noise you thought you avoided.

Interview tip: ask whether the atomic is lock-free on the target ABI before claiming a lock-free design.

6.2.3 Operations on atomic types

<code>atomic_is_lock_free</code>	checks if the atomic type's operations are lock-free
<code>atomic_store</code> <code>atomic_store_explicit</code>	atomically replaces the value of the atomic object with a non-atomic argument
<code>atomic_load</code> <code>atomic_load_explicit</code>	atomically obtains the value stored in an atomic object
<code>atomic_exchange</code> <code>atomic_exchange_explicit</code>	atomically replaces the value of the atomic object with non-atomic argument and returns the old value of the atomic
<code>atomic_compare_exchange_weak</code> <code>atomic_compare_exchange_weak_explicit</code> <code>atomic_compare_exchange_strong</code> <code>atomic_compare_exchange_strong_explicit</code>	atomically compares the value of the atomic object with non-atomic argument and performs atomic exchange if equal or atomic load if not
<code>atomic_fetch_add</code> <code>atomic_fetch_add_explicit</code>	adds a non-atomic value to an atomic object and obtains the previous value of the atomic

<code>atomic_fetch_sub</code> <code>atomic_fetch_sub_explicit</code>	subtracts a non-atomic value from an atomic object and obtains the previous value of the atomic
<code>atomic_fetch_and</code> <code>atomic_fetch_and_explicit</code>	replaces the atomic object with the result of bitwise AND with a non-atomic argument and obtains the previous value of the atomic
<code>atomic_fetch_or</code> <code>atomic_fetch_or_explicit</code>	replaces the atomic object with the result of bitwise OR with a non-atomic argument and obtains the previous value of the atomic
<code>atomic_fetch_xor</code> <code>atomic_fetch_xor_explicit</code>	replaces the atomic object with the result of bitwise XOR with a non-atomic argument and obtains the previous value of the atomic
<code>atomic_wait</code> <code>atomic_wait_explicit</code>	blocks the thread until notified and the atomic value changes
<code>atomic_notify_one</code>	notifies a thread blocked in <code>atomic_wait</code>
<code>atomic_notify_all</code>	notifies all threads blocked in <code>atomic_wait</code>

6.2.4 Flag type and operations

<code>atomic_flag</code>	the lock-free boolean atomic type
<code>atomic_flag_test_and_set</code> <code>atomic_flag_test_and_set_explicit</code>	atomically sets the flag to true and returns its previous value
<code>atomic_flag_clear</code> <code>atomic_flag_clear_explicit</code>	atomically sets the value of the flag to false
<code>atomic_flag_test</code> <code>atomic_flag_test_explicit</code>	atomically returns the value of the flag
<code>atomic_flag_wait</code> <code>atomic_flag_wait_explicit</code>	blocks the thread until notified and the flag changes
<code>atomic_flag_notify_one</code>	notifies a thread blocked in <code>atomic_flag_wait</code>
<code>atomic_flag_notify_all</code>	notifies all threads blocked in <code>atomic_flag_wait</code>

6.2.5 Initialization

<code>atomic_init</code>	non-atomic initialization of a default-constructed atomic object
<code>ATOMIC_VAR_INIT</code>	constant initialization of an atomic variable of static storage duration
<code>ATOMIC_FLAG_INIT</code>	initializes an <code>std::atomic_flag</code> to false

6.2.6 Memory synchronization ordering

Atomicity alone does not decide *when* other threads see surrounding writes. `memory_order` and fences are how you publish a payload (release) and consume it (acquire), or take the blunt `seq_cst` hammer. Formal happens-before material: Chapter 8; choice guide: Section 6.2.14.

<code>memory_order</code>	defines memory ordering constraints for the given atomic operation
<code>kill_dependency</code>	removes the specified object from the <code>memory_order_consume</code> dependency tree
<code>atomic_thread_fence</code>	generic memory order-dependent fence synchronization primitive
<code>atomic_signal_fence</code>	fence between a thread and a signal handler executed in the same thread

Noise removed by correct ordering: seeing a flag “ready” while the payload stores are still invisible (or the opposite reordering bugs on weakly ordered CPUs).

Interview tip: release publishes the payload; acquire consumes it — say that before naming fence APIs.

6.2.7 Mutual exclusion

See [128].

A mutex serializes a critical section: only one owner at a time. That removes data races on multi-word invariants atomics alone cannot express — at the cost of possible blocking, priority inversion, and cache-line ping-pong under contention.

<code>mutex</code>	basic exclusive lock; default choice
<code>timed_mutex</code>	exclusive lock with try-lock until a timeout (detect / bound wait; do not use as a design default)
<code>recursive_mutex</code>	same thread may lock repeatedly; usually a design smell (refactor call graphs instead when possible)
<code>recursive_timed_mutex</code>	recursive + timeout
<code>shared_mutex</code>	many readers or one writer; see below
<code>shared_timed_mutex</code>	shared mutex with timeouts

On a latency-critical core, prefer shrinking the critical section or using a lock-free SPSC handoff over adding `recursive_mutex` / heavy reader–writer machinery.

Interview tip: default to `mutex`; justify `recursive_mutex` or `shared_mutex` only with a concrete contention story.

6.2.8 Generic mutex management

Always lock with `RAII`. `RawLock()` / `GetLock()` pairs leak the lock on exceptions and early returns — that is the deadlock / stuck-mutex class these wrappers remove.

<code>lock_guard</code>	simplest scope-based exclusive ownership
<code>scoped_lock</code>	RAII for one or many mutexes; avoids lock-order deadlocks when locking several
<code>unique_lock</code>	movable exclusive ownership; required by <code>condition_variable::wait</code>
<code>shared_lock</code>	movable shared ownership for <code>shared_mutex</code>
<code>defer_lock_t</code> <code>try_to_lock_t</code> <code>adopt_lock_t</code>	tag type used to specify locking strategy
<code>defer_lock</code> <code>try_to_lock</code> <code>adopt_lock</code>	tag constants used to specify locking strategy

Interview tip: `lock_guard` by default; `unique_lock` when you need wait; `scoped_lock` when locking more than one mutex.

6.2.9 Generic locking management

<code>try_lock</code>	attempts to obtain ownership of mutexes via repeated calls to <code>try_lock</code>
<code>lock</code>	locks specified mutexes, blocks if any are unavailable

Mutex vs lock vs semaphore (C++)

- A **mutex** (`std::mutex, ...`) is the synchronization object threads lock/unlock.
- A **lock wrapper** (`lock_guard, unique_lock, scoped_lock`) is RAII around a mutex — prefer these over raw `lock()/unlock()`.
- `std::mutex` is *not* an OS-wide inter-process lock; for that you need OS primitives / shared-memory named mutexes.
- A **counting semaphore** (`std::counting_semaphore, C++20`) allows up to N concurrent holders; a binary semaphore is similar to a mutex in spirit but with different API/ownership rules.
- Reader/writer access: `shared_mutex + shared_lock / unique_lock`.

```
1 std::mutex m;  
2 int shared = 0;  
3  
4 void inc() {  
5     std::lock_guard<std::mutex> lock(m); // unlocks at end of  
6     scope  
7     ++shared;  
8 }
```

Shared mutex (readers / writers)

See [129].

`std::shared_mutex` allows **many concurrent readers** or **one writer**, not both at once:

- Readers take `std::shared_lock` (shared ownership).
- Writers take `std::unique_lock` (exclusive ownership).

Use when reads dominate and the critical section is non-trivial. Skip it when writers are frequent or the work under the lock is tiny — writer starvation and lock overhead can make a plain `std::mutex` faster.

```
1 #include <mutex>  
2 #include <shared_mutex>  
3  
4 std::shared_mutex rw;  
5 int config = 0; // read often, written rarely  
6  
7 int read_config() {
```

```

8     std::shared_lock lock(rw);    // many readers OK
9     return config;
10 }
11
12 void set_config(int v) {
13     std::unique_lock lock(rw);    // exclusive
14     config = v;
15 }

```

Interview tip: say “shared lock for readers, unique lock for writers”; then name the trade-off (writer latency / contention vs parallel reads).

6.2.10 Call once

Lazy one-time initialization across threads without hand-rolling a double-checked lock. `call_once` runs the callable at most once; concurrent callers block until it finishes (or observe the completed init). Prefer this over static locals when you need an explicit flag, or when initialization must not run at dynamic init time.

<code>once_flag</code>	helper object to ensure that <code>call_once</code> invokes the function only once
<code>call_once</code>	invokes a function only once even if called from multiple threads

```

1  std::once_flag flag;
2  void init() { /* runs at most once across all threads */ }
3
4  void use() {
5      std::call_once(flag, init);
6  }

```

Interview tip: prefer `call_once` / function-local static over a hand-rolled double-checked lock.

6.2.11 Condition variables

See [130].

A `std::condition_variable` lets a thread [sleep until a condition may be true](#), instead of spinning. It is always paired with a mutex and a [predicate](#) (the actual boolean you care about).

<code>condition_variable</code>	provides a condition variable associated with a <code>std::unique_lock</code>
<code>condition_variable_any</code>	provides a condition variable associated with any lock type
<code>notify_all_at_thread_exit</code>	schedules a call to <code>notify_all</code> to be invoked when this thread is completely finished
<code>cv_status</code>	lists the possible results of timed waits on condition variables

Why the wait needs the mutex

wait atomically unlocks the mutex and blocks; on wake it re-locks before returning. That way the predicate is always checked while holding the mutex — no lost wakeups between “see false” and “go to sleep.”

Predicate loop and spurious wakeups

The OS / library may wake a waiter **without** a matching notify (**spurious wakeup**). Also, another thread may change the shared state again before you run. Therefore:

- Always re-check the predicate after wake.
- Prefer `cv.wait(lock, predicate)` (loops until the predicate is true) over bare `wait(lock)`.

```
1  std::mutex m;
2  std::condition_variable cv;
3  bool ready = false;
4
5  void consumer() {
6      std::unique_lock<std::mutex> lock(m);
7      // Equivalent loop form:
8      // while (!ready) cv.wait(lock);
9      cv.wait(lock, []{ return ready; }); // wait until predicate
    is true
10     // ... use data while still holding lock if needed ...
11 }
12
13 void producer() {
14     {
15         std::lock_guard<std::mutex> lock(m);
16         ready = true; // mutate under the mutex
17     } // unlock before notify is fine (
    and common)
18     cv.notify_one();
19 }
```

Interview tip: “mutex + predicate + wait with predicate (or while); notify after updating state.” Spurious wakeups $\Rightarrow$ never assume the condition is true just because you woke up.

6.2.12 Futures

<code>promise</code>	stores a value for asynchronous retrieval
<code>packaged_task</code>	packages a function to store its return value for asynchronous retrieval
<code>future</code>	waits for a value that is set asynchronously
<code>shared_future</code>	waits for a value (possibly referenced by other futures) that is set asynchronously
<code>async</code>	runs a function asynchronously (potentially in a new thread) and returns a <code>std::future</code> that will hold the result
<code>launch</code>	specifies the launch policy for <code>std::async</code>
<code>future_status</code>	specifies the results of timed waits performed on <code>std::future</code> and <code>std::shared_future</code>

```
1 auto fut = std::async(std::launch::async, []{ return 42; });
2 int v = fut.get();    // waits if needed; retrieves the value (or
                       // rethrows)
```

6.2.13 Future errors

Futures fail loudly when misused: get twice, abandon a shared state, break the promise/future pairing. These types name that error category — useful in logs, not something to “catch and ignore” on a hot path.

<code>future_error</code>	reports an error related to futures or promises
<code>future_category</code>	identifies the future error category
<code>future_errc</code>	identifies the future error codes

6.2.14 Memory Order Semantics

See [127], [123], [122].

For `std::memory_order` API listings, see Section 6.2.6 above.

The default for many atomic ops is `seq_cst`: safest, and often the **expensive** choice on hot paths (extra fences / store buffers on some CPUs). Weaker orders are correct only when you understand the publish/consume pattern; x86's strong hardware model can hide bugs that appear on ARM.

Guidance:

- `memory_order_relaxed` — counters and statistics; atomicity only, no ordering with other memory.
- `memory_order_acquire` — consumer side of a handoff; no reads/writes may re-order before the load.
- `memory_order_release` — producer side of a handoff; prior writes must be visible after the store.
- `memory_order_acq_rel` — RMW that both publishes and consumes (e.g. unlock / flag CAS).
- `memory_order_seq_cst` — safest default when in doubt; single total order across all `seq_cst` ops.

Noise removed by weakening carefully: unnecessary barriers on every increment. **Noise introduced by weakening wrongly:** readers seeing torn or stale payloads.

Interview tip: name the pair — release publish, acquire consume; use `acq_rel` on RMWs that do both; default `seq_cst` only when you cannot justify weaker.

For formal happens-before and sequential consistency, see Section 8.1.4 in Chapter 8. Also [121], [124], [125].

Concurrency Bibliography

- [121] Crowl McKenney Bohem. *C++ Data-Dependency Ordering: Atomics and Memory Model*. [ISO/IEC JTC1 SC22 WG21 N2556](#). 2008.
- [122] *Memory model synchronization modes*. [GNU gcc Wiki](#).
- [123] *What do each memory_order mean?* [StackOverflow Thread](#).
- [124] *Atomic's memory orders, what for?* [YouTube](#).
- [125] *std::atomic memory_orders compared*. [YouTube](#).
- [126] *Standard library header <thread>*. [Cppreference](#).
- [127] *std::memory_order*. [Cppreference](#).
- [128] *std::mutex*. [Cppreference](#).
- [129] *std::shared_mutex*. [Cppreference](#).
- [130] *std::condition_variable*. [Cppreference](#).



PART

Systems and Low Latency

Linux Processes and Threads

From a C++ program, concurrency starts with `std::thread` (Chapter 6). Underneath, the operating system still sees [processes](#) and [threads](#): separate schedulable entities with PIDs, address spaces, and CPU time. This chapter connects the C++ view to the Linux view, which matters for latency tuning (affinity, scheduling classes, syscall cost).

7.1 Processes vs Threads

7.1.1 Processes

A [program](#) is an executable file on disk. A [process](#) is that program *loaded into memory and running*: the OS gives it a [PID](#) (*process identifier*), private resources, and its own virtual address space (Section 7.2).

Creating a new process (e.g. via `fork(2)` [131]) duplicates the address space: the child gets a new PID and a copy-on-write view of memory. Multiprocess designs (separate feed handler and strategy engine) communicate via IPC (pipes, shared memory — Chapter 13), not by sharing heap pointers.

Every C++ program starts as [one process](#). `main()` runs on the process's [initial thread](#) (also called the [main thread](#)).

7.1.2 Threads

See Chapter 6 for the C++ thread API (`std::thread`, `std::jthread`, and related types). Here we focus on what the [kernel](#) sees.

A **thread** is a unit of execution *inside* a process. Threads in the same process **share** the virtual address space (heap, globals, code); each thread has its own stack, registers, and program counter. The kernel schedules threads (Linux treats each thread as a **task** with its own TID).

On Linux, `std::thread` is typically implemented with **pthread**s. Pthread creation ultimately uses the `clone(2)` [132] system call with flags that share memory with the parent — a new thread, not a new process.

You do **not** need to learn the full pthread API (`pthread_create`, `void*`, attributes). Use `std::thread` for structure and lifetime; use `native_handle()` only for Linux-specific tuning (affinity, real-time priority) in Section 7.5.

```
1 #include <iostream>
2 #include <thread>
3 #include <unistd.h> // getpid()
4
5 int main()
6 {
7     const pid_t pid = getpid();
8     std::cout << "Process PID: " << pid << "\n";
9     std::cout << "Main std::thread::id: " << std::this_thread::
    get_id() << "\n";
10
11     std::thread worker([pid] {
12         std::cout << "Worker std::thread::id: "
13                 << std::this_thread::get_id() << "\n";
14         std::cout << "Worker sees same PID: " << getpid()
15                 << " (expected " << pid << ")\n";
16     });
17     worker.join();
18 }
```

`std::thread::get_id()` is a C++ identifier; `getpid()` [133] returns the OS process ID. All threads print the **same PID** because they belong to one process.

7.2 Address Space

Each process has its own **virtual address space**: a range of virtual addresses mapped by the MMU to physical RAM (or swap). The layout usually includes:

- **Code** (`.text`) — machine instructions, often read-only.
- **Data** — globals and statics; **heap** (dynamic allocation); **stack** per thread (function frames, local variables).
- **Kernel space** — not directly accessible from user code; entered on syscalls.

Memory is managed in **pages** (commonly 4 KiB on x86-64). **All threads of a process share the same virtual address space** for heap and globals; each thread's stack grows in the same address space but on separate stacks.

Shared VA is a double-edged sword. Threads can pass pointers and share a ring or book in place — cheap IPC compared to pipes or sockets — but any unsynchronized write is a **data race**: the hardware will not protect you (Chapter 8). Multiprocess designs avoid that class of bug by construction and pay for IPC instead (shared memory maps — Chapter 13).

Noise removed by careful layout: TLB walks and page-fault jitter from a sprawling working set. Hot structures should be compact and page-stable on the critical path; sparse heaps and cold first-touch inject μ s-scale faults into what looks like ordinary loads. See Section 9.6 for translation cost; Section 13.3.4 for demand paging.

Interview tip: same address space $\Rightarrow$ shared heap/globals $\Rightarrow$ races unless synchronized; separate processes need explicit IPC but isolate faults and memory.

7.3 User Mode vs Kernel Mode

The CPU runs in **user mode** when executing your C++ code. Sensitive operations (I/O, scheduling, memory mapping, locking in the kernel) require a **system call**: the CPU switches to **kernel mode**, runs the kernel handler, and returns to user mode.

Examples that trap into the kernel from a typical trading stack:

- read/recv on a socket (Chapter 11).
- mmap for shared memory (Chapter 13).
- pthread_setaffinity_np / sched_setaffinity to pin threads (Section 7.5.4).
- Blocking mutex or I/O wait — the thread may be descheduled inside the kernel.

Each user/kernel transition has **latency cost** (flush pipeline, TLB effects, cache coldness). Low-latency paths minimize syscalls and blocking on hot paths; kernel bypass (Chapter 14) exists partly to avoid this stack for networking.

7.4 Linux Scheduler

The **scheduler** decides **which thread runs on which CPU core** and for how long. You do not pick this in portable C++; the kernel does, unless you influence it with affinity, scheduling policy, or isolation (Sections 7.5 and 15.1).

7.4.1 Context Switch

A **context switch** occurs when the CPU stops running one thread and runs another:

- Save the current thread's registers and program counter into its **task struct**.
- Restore another thread's saved state.
- Optionally migrate execution to a different core (**cache cold start** — Chapter 9).

The direct cost is already on the order of `microseconds`, not nanoseconds — expensive versus a function call. The secondary cost is often worse for latency budgets: after a switch (especially a core hop), L1/L2 are cold and the TLB is colder, so the next few hundred instructions pay DRAM/coherence traffic that the hot path did not budget for. **Noise removed** by pinning and isolation: involuntary deschedule and migration so the strategy thread keeps a warm private cache. See Section 7.5 and Section 15.1.

Interview tip: quote both halves — “ μ s of switch + cache/TLB cold start” — not just “context switches are slow.”

7.4.2 Scheduling Classes

Linux exposes several scheduling policies [139]:

- `SCHED_OTHER` (default) — CFS (*Completely Fair Scheduler*). General-purpose, time-sharing; good throughput, **not** hard latency guarantees.
- `SCHED_BATCH`, `SCHED_IDLE` — lower priority background work.
- `SCHED_FIFO`, `SCHED_RR` — real-time classes (Section 7.4.3).

Most C++ threads run under CFS unless you explicitly request a real-time policy with `sched_setscheduler(2)` [137].

CFS optimizes for `fair sharing`: runnable tasks take turns so nobody monopolizes the CPU. On a latency path that “fairness” is noise — your pinned strategy thread can still be preempted for an unrelated CFS task that looked entitled to a slice. Real-time classes invert the priority story: they run ahead of CFS, which removes that jitter *and* introduces starvation risk if an RT thread never blocks (Section 7.4.3).

Noise removed by moving the hot thread off CFS onto RT on an isolated core: fair-share preemption from housekeeping and background work. Cost: you must leave room for the OS somewhere else — typically housekeeping CPUs that stay on CFS.

7.4.3 Real-Time Scheduling

`SCHED_FIFO` and `SCHED_RR` run at higher priority than normal threads. `SCHED_FIFO` runs until it blocks or yields; `SCHED_RR` adds a time quantum among equal-priority RT threads.

Use with care: a busy RT thread can `starve` the rest of the system (including housekeeping and logging). Typical HFT setups combine `RT policy on dedicated cores` with `isolcpus` (Section 15.1) rather than marking every thread RT.

To raise priority on an existing `std::thread`, use `pthread_setschedparam` [138] on `native_handle()` (requires appropriate privileges):

```
1 #include <pthread.h>
2 #include <sched.h>
3
4 void set_fifo_priority(std::thread& t, int priority)
5 {
```



```

6     sched_param param{};
7     param.sched_priority = priority; // 1..99 for SCHED_FIFO on
    Linux
8     pthread_setschedparam(t.native_handle(), SCHED_FIFO, &param);
9 }

```

Interview tip: CFS = fair sharing (throughput, jitter); RT = priority over CFS (lower jitter, starvation risk) — pair RT with isolated cores, not with “make everything FIFO.”

7.5 Thread Affinity

7.5.1 CPU Affinity and Thread Pinning

CPU affinity restricts **which cores** a thread may run on. **Thread pinning** means setting affinity to a **single core** (or a small fixed set) so the thread stays cache-hot and avoids scheduler migration.

By default, Linux may **migrate** threads across cores to balance load. That causes:

- **Cache loss** — data warmed in L1/L2 on the old core is not reused on the new one (Chapter 9).
- **Extra context switches** — bad for microsecond-scale budgets (Section 7.4.1).

Why it matters for HFT:

- **Less jitter** — fewer context switches and core hops.
- **Warmer caches** — L1/L2 data stays valid on one core.
- **Predictable layout** — dedicate cores to feed, strategy, and gateway roles.

Avoid pinning hot paths to core 0 on typical Linux servers: core 0 often handles many hardware interrupts (IRQs), timers, and kernel housekeeping, which adds **non-deterministic jitter**. Prefer cores reserved for the application (Section 15.1).

Hyper-threading (HT): two logical CPUs may share one physical core (siblings). For maximum isolation, pin latency-critical threads to **one logical CPU per physical core** (or disable HT in BIOS on dedicated trading boxes). Inspect topology with `lscpu -e` (Section 7.5.2).

Pinning APIs are in the next subsection. For NUMA node placement and `libnuma`, see Section 10.3.2. For OS-level core isolation (`isolcpus`), see Section 15.1. For moving NIC interrupts off trading cores, see Section 7.5.3.

7.5.2 Machine Topology and Core Selection

Before pinning, inspect the machine. Useful commands:

- `lscpu -e` — table of logical CPUs with `NODE`, `SOCKET`, `CORE`.
- `lstopo (hwloc)` — visual map of cores, caches, and NUMA nodes.

- `lspci | grep -i ether` — locate the **NIC** (*Network Interface Card*, the Ethernet hardware).

On a single-socket laptop, all CPUs often share `NODE 0`; multi-socket servers need the thread and its memory on the **same NUMA node** as the NIC (Chapter 10).

Example workflow for a PCI Ethernet device at `05:00.0`:

1. Read NUMA node: `cat /sys/bus/pci/devices/0000:05:00.0/numa_node`.
2. If the value is `-1`, the kernel reports no specific node (common on consumer hardware); treat as node 0 on single-node systems.
3. List CPUs on that node: `cat /sys/devices/system/node/node0/cpulist` (may show ranges such as `0-15`).
4. Pin trading threads to cores in that list, usually **avoiding core 0** and HT siblings.

Typical role split on a tuned server: one core for **network IRQ / RX**, separate cores for **strategy logic**, with generic OS work kept off isolated cores.

7.5.3 Network Interrupts

When a packet arrives, the NIC raises an **interrupt (IRQ)**. The kernel runs an interrupt handler, often on a core that may **preempt** your trading thread — another source of jitter.

Inspect distribution with `cat /proc/interrupts` (look for your driver name, e.g. `eth0`). Columns are per-CPU interrupt counts. If core 0 handles most NIC interrupts, consider moving IRQ affinity (`/proc/irq/<N>/smp_affinity_list`) to a **dedicated non-trading core**. This requires root and careful validation; combine with `isolcpus` (Section 15.1) and busy polling (Section 15.2) where appropriate.

7.5.4 Pinning with `pthread_setaffinity_np` and `sched_setaffinity`

There are two common Linux APIs for the same idea:

- **`pthread_setaffinity_np(3)`** [134] — pthread layer; natural fit with `std::thread::native_handle`
- **`sched_setaffinity(2)`** [135] — scheduler syscall; takes a kernel **TID** (*thread ID*). On Linux, `pthread_setaffinity_np` typically wraps this call.

Both use `cpu_set_t` and macros `CPU_ZERO`, `CPU_SET`, etc. Compile with `#define _GNU_SOURCE` before includes for the pthread extension.

`cpu_set_t` is a **bitmask**: bit `n` set means the thread **may** run on logical CPU `n`. `CPU_ZERO` clears all bits; each `CPU_SET(core, &cpuset)` **sets** one bit (calling it in a loop **accumulates** allowed cores — it does not replace the whole set). `pthread_setaffinity_np` tells the kernel the allowed mask for that thread.

```

1 #define _GNU_SOURCE
2 #include <pthread.h>
3 #include <sched.h>
4
5 #include <vector>
6
7 bool pin_current_thread_to_cpus(const std::vector<int>& cores)
8 {
9     cpu_set_t cpuset;
10    CPU_ZERO(&cpuset);
11    for (int core : cores) {
12        CPU_SET(core, &cpuset);
13    }
14    return pthread_setaffinity_np(pthread_self(), sizeof(
15    cpu_set_t), &cpuset) == 0;
16 }
17
18 // Example: allow cores 4-7 only (avoid core 0 on a 16-logical-
19 // CPU box)
20 // pin_current_thread_to_cpus({4, 5, 6, 7});

```

```

1 #define _GNU_SOURCE
2 #include <pthread.h>
3 #include <sched.h>
4
5 #include <cerrno>
6 #include <cstring>
7 #include <iostream>
8 #include <thread>
9
10 // Pin the calling pthread to a single CPU (0-based index).
11 bool pin_current_thread_to_cpu(int cpu)
12 {
13     cpu_set_t cpuset;
14     CPU_ZERO(&cpuset);
15     CPU_SET(cpu, &cpuset);
16
17     const int rc = pthread_setaffinity_np(
18         pthread_self(), sizeof(cpu_set_t), &cpuset);
19     if (rc != 0) {
20         std::cerr << "pthread_setaffinity_np: " << std::strerror(
21         rc) << "\n";
22         return false;
23     }
24     return true;
25 }
26
27 // Pin an existing std::thread (call after thread is running).
28 bool pin_std_thread_to_cpu(std::thread& t, int cpu)
29 {
30     cpu_set_t cpuset;

```

```

30     CPU_ZERO(&cpuset);
31     CPU_SET(cpu, &cpuset);
32
33     const int rc = pthread_setaffinity_np(
34         t.native_handle(), sizeof(cpu_set_t), &cpuset);
35     if (rc != 0) {
36         std::cerr << "pthread_setaffinity_np: " << std::strerror(
37             rc) << "\n";
38         return false;
39     }
40     return true;
41 }
42
43 int main()
44 {
45     std::thread worker([] {
46         if (!pin_current_thread_to_cpu(2)) {
47             return;
48         }
49         // Hot loop: strategy or feed handler work on core 2...
50     });
51
52     worker.join();
53 }

```

The same pinning via the syscall directly (useful when you have a TID or no pthread handle):

```

1  #include <sched.h>
2  #include <sys/syscall.h>
3  #include <unistd.h>
4
5  #include <cstdint>
6
7  pid_t linux_tid()
8  {
9      return static_cast<pid_t>(syscall(SYS_gettid));
10 }
11
12 bool pin_current_thread_via_sched(int cpu)
13 {
14     cpu_set_t cpuset;
15     CPU_ZERO(&cpuset);
16     CPU_SET(cpu, &cpuset);
17     return sched_setaffinity(linux_tid(), sizeof(cpu_set_t), &
18                             cpuset) == 0;
19 }

```

Verify affinity with `sched_getaffinity(2)` [136] or by reading `/proc/self/task/<tid>/status` (`Cpus_allowed_list`). In production, pinning is combined with [dedicated cores](#) (`isolcpus`, Section 15.1) and [NUMA-local](#) memory (Chapter 10) so the pinned thread's data stays on the same socket.

Linux Processes and Threads Bibliography

- [131] *fork(2)*. [Linux man-pages](#).
- [132] *clone(2)*. [Linux man-pages](#).
- [133] *getpid(2)*. [Linux man-pages](#).
- [134] *pthread_setaffinity_np(3)*. [Linux man-pages](#).
- [135] *sched_setaffinity(2)*. [Linux man-pages](#).
- [136] *sched_getaffinity(2)*. [Linux man-pages](#).
- [137] *sched_setscheduler(2)*. [Linux man-pages](#).
- [138] *pthread_setschedparam(3)*. [Linux man-pages](#).
- [139] *sched(7)*. [Linux man-pages](#).

8

CHAPTER

Memory Model and Synchronization

Chapter 6 lists the C++ APIs (`std::atomic`, `mutexes`, `memory_order` enumerators). This chapter explains the **rules**: when two threads may see each other's writes, which orders are enough for a handoff, and how to build a small **lock-free SPSC** queue used on HFT hot paths, then two classic CAS structures (Treiber stack, Michael–Scott queue) that interviewers often expect next.

8.1 C++ Memory Model

The C++ memory model defines which program executions are allowed when threads share memory. Compilers and CPUs may **reorder** loads and stores unless synchronisation forbids it. Portable concurrent code relies on atomics, mutexes, or other synchronising operations — not on “it works on my machine.”

8.1.1 Data Race

A **data race** occurs when two threads access the **same memory location**, at least one access is a **write**, and the accesses are **not ordered** by the memory model (no mutex, no atomic with adequate ordering, no other happens-before edge).

In C++, a data race is **undefined behavior**. Fix it with atomics, locks, or by giving each thread private data.

```
1 #include <thread>
2
3 int g = 0; // shared, non-atomic
```

```

4
5 void bad()
6 {
7     std::thread a([] { g = 1; });
8     std::thread b([] { g = 2; }); // data race with a
9     a.join();
10    b.join();
11 }

```

8.1.2 Race Condition

A **race condition** is a broader engineering term: the program's correctness depends on **timing** or interleaving. It may or may not be a formal data race.

Example: both threads use a mutex (no data race) but the business logic still breaks if updates run in the wrong order.

Interview tip: say **data race** when you mean UB; say **race condition** for logic/timing bugs.

8.1.3 Happens-Before

Happens-before is the relation that makes writes in one thread **visible** to another. If A happens-before B, then B is guaranteed to see the side effects of A (for the relevant memory).

Typical ways to create happens-before between threads:

- Unlock of a mutex happens-before a later lock of the **same** mutex.
- Atomic release store happens-before an atomic acquire load that **reads the value written** by that store (on the same atomic object).
- Thread construction / join edges with the thread's entry and exit.

Without a happens-before edge, another thread may see stale or torn values.

8.1.4 Sequential Consistency

See Chapter 6, Section 6.2.14 for `memory_order_seq_cst`. This subsection covers the formal model only.

Under **sequential consistency**, there is a single total order of all `seq_cst` atomic operations that all threads agree on, and it is consistent with each thread's program order. It is the **easiest mental model** and the default for many atomic operations — but often **stronger (and costlier)** than a producer/consumer handoff needs.

8.2 Memory Orders

8.2.1 C++ Reference

This subsection is a pointer, not a second API catalog. Chapter 6 owns the `memory_order` / fence [surface](#) (Section 6.2.6) and the when-to-pick bullets (Section 6.2.14).

Here the goal is [theory](#): happens-before, release-acquire pairing, and why weakening is a trade between mental cost and hardware barriers. Read the Concurrency tables for names; stay in this chapter for proof sketches and lock-free structures.

Interview tip: say “API in Concurrency, happens-before here” — do not recite every enumerator twice.

8.2.2 Release-Acquire Pairing

The standard pattern for publishing data from a producer to a consumer:

1. Producer writes the payload (non-atomic or relaxed atomics).
2. Producer performs a [release](#) store on a flag / index (“data is ready”).
3. Consumer performs an [acquire](#) load on that flag / index.
4. If the consumer sees the producer’s value, it may safely read the payload.

The release-acquire pair creates happens-before: all writes [before](#) the release become visible to the consumer [after](#) the matching acquire.

```
1  #include <atomic>
2  #include <thread>
3
4  int payload = 0;
5  std::atomic<bool> ready{false};
6
7  void producer()
8  {
9      payload = 42;                                // (1)
10     ready.store(true, std::memory_order_release); // (2)
11 }
12
13 void consumer()
14 {
15     while (!ready.load(std::memory_order_acquire)) { // (3)
16         // spin or wait
17     }
18     // (4) safe to read payload: sees 42
19     int x = payload;
20 }
```

`memory_order_acq_rel` is for read-modify-write that both acquire and release (e.g. some lock implementations). `memory_order_relaxed` only guarantees atomicity of that location — [no](#) synchronisation with other memory.

8.2.3 When to Weaken Ordering

Default atomics are often `seq_cst`: one total order that is easy to reason about and **expensive** on weakly ordered CPUs (extra fences / store-buffer drains). Weakening is how you remove that barrier noise — only after the publish/consume story is clear.

- **seq_cst** — default / when unsure; simplest reasoning; may cost more on weak HW.
- **release / acquire** — message passing, SPSC indices, “publish flag” patterns. **Noise removed**: full-system `seq_cst` fences you did not need for a two-thread handoff.
- **relaxed** — pure counters, statistics, when no other data depends on the atomic. **Noise removed**: acquire/release barriers on increments that never publish a payload.
- Prefer **correct first**; measure before weakening further on x86 (often strong in practice, but portable code must still use the right orders — ARM will expose the bug).

Interview tip: weaken only with a named pattern (counter vs publish/consume); never “relaxed everywhere because x86.”

8.3 Lock-Free Ring Buffer (SPSC)

A **single-producer single-consumer** ring buffer moves messages between two threads without a mutex. It is a standard HFT building block (feed thread → strategy thread).

Noise removed versus a mutex queue: no kernel futex sleep/wake on the common path, no lock-word ping-pong, and a fixed preallocated buffer (no new per message). **Cost**: exactly one writer and one reader forever; overflow/empty must be handled explicitly (drop, block on a colder path, or size for burst).

8.3.1 SPSC Model

Exactly **one** thread pushes, exactly **one** thread pops. That restriction allows a simple design with two indices and no CAS loops for the common path: each index has a single writer, so updates need atomics for visibility, not contention resolution.

Multi-producer or multi-consumer queues need different algorithms (Sections 8.4–8.5) and are harder to get right. If your design needs MPMC on the hot path, revisit ownership first — many HFT stacks shard into several SPSC rings instead.

8.3.2 Head and Tail Indices

Use a fixed-capacity array. The producer advances a **tail** (write position); the consumer advances a **head** (read position). Indices are `std::atomic` (or indexed modulo capacity).

Only the producer writes `tail`; only the consumer writes `head`. Each thread may **read**

the other's index to decide full/empty. That single-writer discipline is why SPSC avoids CAS on the steady path.

8.3.3 Producer and Consumer Ordering

Ordering is the same publish/consume story as Section 8.2.2:

- Producer: write slot, then `tail.store(..., release)`.
- Consumer: head-side acquire load of `tail` (or acquire load when observing new data), then read slot.

Release-acquire on the indices publishes the slot contents to the consumer. **Noise removed by getting this right:** seeing a new index while the payload store is still invisible (or reading a slot after a relaxed bump that never synchronized).

8.3.4 Full and Empty Conditions

With capacity N (often power of two):

- **Empty** when `head == tail` (consumer has caught up).
- **Full** when `tail - head == N` (no free slot) — exact formula depends on whether you leave one slot empty or use an explicit size counter.

Never let producer and consumer update the **same** index. Leaving one slot empty is a common trick so full and empty are distinguishable without an extra atomic size.

8.3.5 Cache-Line Layout

See Section 9.3.4 for padding and alignment as a general technique. This subsection covers SPSC-specific head/tail placement only.

Put head and tail on **separate cache lines** (`alignas(64)`; see Section 9.3.4 for the portable constant) so the producer's stores to `tail` do not invalidate the consumer's line for `head` (false sharing).

8.3.6 Implementation

Sketch (power-of-two capacity, one empty slot to distinguish full/empty):

```
1 #include <atomic>
2 #include <cstdint>
3 #include <vector>
4
5 template <typename T, std::size_t N>
6 class SpscRing {
7     static_assert((N & (N - 1)) == 0, "N power of two");
8     static_assert(N >= 2);
9 }
```

```

10     alignas(64) std::atomic<std::size_t> head_{0}; // consumer
11     alignas(64) std::atomic<std::size_t> tail_{0}; // producer
12     T buf_[N];
13
14 public:
15     bool try_push(const T& v)
16     {
17         const std::size_t t = tail_.load(std::
memory_order_relaxed);
18         const std::size_t next = (t + 1) & (N - 1);
19         if (next == head_.load(std::memory_order_acquire)) {
20             return false; // full
21         }
22         buf_[t] = v;
23         tail_.store(next, std::memory_order_release);
24         return true;
25     }
26
27     bool try_pop(T& out)
28     {
29         const std::size_t h = head_.load(std::
memory_order_relaxed);
30         if (h == tail_.load(std::memory_order_acquire)) {
31             return false; // empty
32         }
33         out = buf_[h];
34         head_.store((h + 1) & (N - 1), std::memory_order_release)
;
35         return true;
36     }
37 };

```

Interview tip: SPSC only; release/acquire on indices; cache-line separation; preallocate `buf_` (no new on the hot path).

8.4 Lock-Free Stack (Treiber)

See [142]. The [Treiber stack](#) is the usual first multi-producer / multi-consumer lock-free structure after SPSC: a linked list with a single atomic head, updated with CAS.

Any thread may push or pop. Progress of one operation does not need a mutex, but a slow thread can still starve if CAS keeps failing (lock-free $\neq$ wait-free).

8.4.1 Push and Pop

Both operations race on a single atomic head. Progress is lock-free because some CAS eventually wins; an individual thread may still spin under contention (lock-free $\neq$ wait-free, and $\neq$ “fast under load”).

- **push:** allocate a node, set node->next = head, compare_exchange head from old head to node; retry on failure.
- **pop:** load head; if null, empty; else CAS head from old to old->next; retry on failure; then read the value out of old.

Ordering sketch: release on successful push publish; acquire on pop's load/CAS so the node fields are visible.

Noise removed versus a mutex stack: no futex; failed CAS retries in user space. **Noise introduced:** allocator traffic per node, ABA, and reclamation — why HFT hot paths usually prefer preallocated rings over unbounded Treiber stacks.

8.4.2 Implementation Sketch

```

1  #include <atomic>
2  #include <utility>
3
4  template <typename T>
5  class TreiberStack {
6      struct Node {
7          T value;
8          Node* next;
9      };
10
11     std::atomic<Node*> head_{nullptr};
12
13 public:
14     void push(T v)
15     {
16         Node* n = new Node{std::move(v), nullptr};
17         Node* old = head_.load(std::memory_order_relaxed);
18         do {
19             n->next = old;
20         } while (!head_.compare_exchange_weak(
21             old, n,
22             std::memory_order_release,
23             std::memory_order_relaxed));
24     }
25
26     bool try_pop(T& out)
27     {
28         Node* old = head_.load(std::memory_order_acquire);
29         do {
30             if (old == nullptr) {
31                 return false;
32             }
33         } while (!head_.compare_exchange_weak(
34             old, old->next,
35             std::memory_order_acquire,
36             std::memory_order_acquire));

```

```

37
38     out = std::move(old->value);
39     // delete old; // unsafe without a reclamation scheme (
    see below)
40     return true;
41 }
42 };

```

8.4.3 ABA and Memory Reclamation

Two hard parts interviewers probe:

- **ABA**: thread A reads `head == X`; another thread pops X, pushes other nodes, then pushes a recycled X; A's CAS succeeds even though the list changed. Mitigations: tagged pointers (version + pointer in a wide CAS), or never reuse addresses while a lagged reader might still CAS them.
- **Safe reclamation**: you cannot delete a popped node while another thread might still be reading `old->next`. Real systems use hazard pointers, epoch-based reclamation, RCU-style schemes, or a freelist with enough tagging — not raw delete on the hot path.

Interview tip: draw the CAS loop on `head`; name **ABA** and **reclamation** as the reasons production stacks are more than ten lines. HFT often avoids unbounded Treiber stacks on the hot path (allocation + reclamation) and prefers preallocated SPSC/MPMC rings.

8.5 Lock-Free Queue (Michael–Scott)

See [143], [144]. The **Michael–Scott** queue is the classic lock-free **MPMC FIFO**: atomic `head` and `tail`, a dummy node so `enqueue` and `dequeue` touch different ends, and CAS on `next` / `head` / `tail`.

Harder than Treiber; worth knowing the story even if you ship a bounded ring in production.

8.5.1 Structure

Treiber is a stack (LIFO) with one CAS site. Michael–Scott is a FIFO, so `enqueue` and `dequeue` must touch **different** ends without locking — that is why the dummy node exists.

- Each node holds a value and an atomic `next` (initially null).
- The queue always contains at least a **dummy** node. `head` points at the dummy (dequeue end); `tail` points at the last node (enqueue end).
- **enqueue** swings `tail->next`, then helps advance `tail`.

- **dequeue** advances head over the dummy to the first real node, returns that value, and makes the old first node the new dummy.

Noise removed by the dummy: enqueue and dequeue rarely contend on the same pointer in the empty/steady cases; without it, empty-queue races on a single head/tail pair become much nastier.

8.5.2 Enqueue / Dequeue (logic)

Helping is the other core idea: if tail lags behind a linked next, any thread may CAS tail forward. That keeps the algorithm lock-free when a producer stalls mid-enqueue.

Enqueue of value v :

1. Allocate node n with value v , `next = nullptr`.
2. Loop: read `tail`; read `tail->next`.
3. If `tail` is still the observed tail:
 - If `tail->next` is null, CAS it to n (link). On success, try to CAS `tail` to n (may fail — another thread helps).
 - Else (tail lagging), CAS `tail` forward to `tail->next` (help) and retry.

Dequeue:

1. Loop: read `head`, `tail`, and `head->next` (first real).
2. If `head` is still current:
 - If `head == tail` and `head->next` is null $\Rightarrow$ empty.
 - If `head == tail` and `head->next` is non-null $\Rightarrow$ help advance `tail`, retry.
 - Else CAS `head` from old `head` to `head->next`; on success the value in the new `head` (former first real node) is yours; retire the old dummy.

Helping keeps the algorithm lock-free: a stalled enqueuer does not permanently block dequeuers.

8.5.3 Implementation Sketch

Didactic sketch (same ABA / reclamation caveats as Treiber; not production-ready):

```

1 #include <atomic>
2 #include <optional>
3 #include <utility>
4
5 template <typename T>
6 class MichaelScottQueue {
7     struct Node {

```

```

8         std::optional<T> value;                                // dummy starts
empty
9         std::atomic<Node*> next{nullptr};
10        explicit Node(std::optional<T> v = std::nullopt)
11            : value(std::move(v)) {}
12    };
13
14    std::atomic<Node*> head_;
15    std::atomic<Node*> tail_;
16
17 public:
18     MichaelScottQueue()
19     {
20         Node* dummy = new Node{};                            // no value
21         head_.store(dummy, std::memory_order_relaxed);
22         tail_.store(dummy, std::memory_order_relaxed);
23     }
24
25     void enqueue(T v)
26     {
27         Node* n = new Node{std::move(v)};
28         for (;;) {
29             Node* t = tail_.load(std::memory_order_acquire);
30             Node* next = t->next.load(std::memory_order_acquire);
31             if (t != tail_.load(std::memory_order_acquire)) {
32                 continue;                                     // stale tail
33             }
34             if (next == nullptr) {
35                 if (t->next.compare_exchange_weak(
36                     next, n,
37                     std::memory_order_release,
38                     std::memory_order_relaxed)) {
39                     tail_.compare_exchange_strong(
40                         t, n,
41                         std::memory_order_release,
42                         std::memory_order_relaxed); // help OK
43
44                     if fails
45                         return;
46                 }
47             } else {
48                 tail_.compare_exchange_weak(
49                     t, next,
50                     std::memory_order_release,
51                     std::memory_order_relaxed); // help advance
52                 tail
53             }
54         }
55     }
56
57     bool try_dequeue(T& out)
58     {

```



```

56     for (;;) {
57         Node* h = head_.load(std::memory_order_acquire);
58         Node* t = tail_.load(std::memory_order_acquire);
59         Node* next = h->next.load(std::memory_order_acquire);
60         if (h != head_.load(std::memory_order_acquire)) {
61             continue;
62         }
63         if (h == t) {
64             if (next == nullptr) {
65                 return false; // empty
66             }
67             tail_.compare_exchange_weak(
68                 t, next,
69                 std::memory_order_release,
70                 std::memory_order_relaxed);
71             continue;
72         }
73         // next is the first real node; claim it by swinging
74         head
75         if (head_.compare_exchange_weak(
76             h, next,
77             std::memory_order_release,
78             std::memory_order_relaxed)) {
79             out = std::move(*next->value);
80             // retire h (old dummy) via a reclamation scheme
81             return true;
82         }
83     }
84 };

```

8.5.4 HFT Takeaway

- Michael–Scott teaches [CAS + helping](#) and why MPMC FIFO is subtle.
- On latency-critical paths, shops often prefer [bounded](#) rings (SPSC, or a known MPMC ring with preallocated slots) over unbounded linked queues: no new on the hot path, simpler failure mode (“full”), friendlier to caches.
- Still: if an interviewer asks for “a lock-free queue,” outlining Michael–Scott (dummy node, CAS next, help tail, ABA/reclamation) is the expected answer — then say what you would ship instead and why.

Interview tip: SPSC = indices + release/acquire; Treiber = CAS head; Michael–Scott = dummy + CAS next/head/tail + helping. Always mention [ABA](#) and [memory reclamation](#).

8.6 Synchronization

8.6.1 C++ Reference

Again a deliberate split: Chapter 6, Section 6.2.7 lists `std::mutex`, `std::shared_mutex`, RAII wrappers, and condition variables. This chapter asks **when** those primitives hurt — contention, convoys, and when a spinlock or SPSC handoff is the better latency story.

Interview tip: name the Concurrency type, then say the performance failure mode from the subsections below.

8.6.2 Spinlock

A **spinlock** waits by spinning on an atomic flag instead of sleeping in the kernel. On a **dedicated core** with very short critical sections, it can beat a mutex (no context switch). On a busy machine, spinning wastes CPU and can hurt latency elsewhere.

Rough rule: mutex for most code; spinlock only when you control the core and the critical section is tiny. See also busy polling (Section 15.2).

```
1  #include <atomic>
2
3  struct Spinlock {
4      std::atomic_flag flag = ATOMIC_FLAG_INIT;
5
6      void lock()
7      {
8          while (flag.test_and_set(std::memory_order_acquire)) {
9              // spin
10             }
11         }
12
13         void unlock()
14         {
15             flag.clear(std::memory_order_release);
16         }
17     };
```

8.7 Performance Considerations

8.7.1 Lock Contention

Contention means many threads fight for the same lock. Hold times grow, cores wait, and caches bounce the lock word (coherence traffic — Chapter 9).

Noise removed by shrinking the critical section or removing the share: wait time, futex wake storms, and lock-word ping-pong. Fixes: finer-grained locks, shard data, or remove sharing (SPSC / thread-local). On a latency path, “add another mutex” is rarely the first answer.

Interview tip: contention is a [sharing](#) problem; name what data is shared before naming lock types.

8.7.2 Lock Convoy

A [lock convoy](#) happens when threads repeatedly wake, find the lock taken, and sleep again — often with priority or scheduling effects that serialize progress. Symptom: high context-switch rate around one mutex without useful work between switches.

Noise: wake → try → sleep loops that burn scheduler time while the critical section stays long. Prefer shorter critical sections, fewer waiters, and less sharing — not a higher priority on every waiter.

8.7.3 Synchronization Scalability

Ask: does adding cores increase useful work, or only increase waiting on one lock? Designs that scale typically [partition](#) work (per-core queues, sharding) instead of one global mutex.

Noise removed by partitioning: serialising the whole machine on one lock word. HFT systems often accept [single-threaded](#) hot paths plus SPSC handoffs rather than fine-grained locking everywhere — scalability by *not* sharing the hot data structure.

Interview tip: “more threads” is not scalability if they all queue on one mutex; say how work is partitioned.

Memory Model and Synchronization Bibliography

- [140] *std::memory_order*. [Cplusplusreference](#).
- [141] *std::atomic*. [Cplusplusreference](#).
- [142] *Treiber stack*. [Wikipedia](#).
- [143] *Michael–Scott queue (Non-blocking linked list)*. [Wikipedia](#).
- [144] Maged M. Michael and Michael L. Scott. “Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms”. In: *PoPP*. 1996.

Cache Coherence and Performance

Modern CPUs are fast; **DRAM** is slow. Caches hide that gap.

Interview tip: explain why two threads touching **nearby** memory can destroy performance (**false sharing**), how coherence protocols move ownership of cache lines, and how to **measure** misses with `perf`.

9.1 CPU Cache Hierarchy

Caches store recently used data and instructions in small, fast SRAM. Capacity grows and latency worsens as you go down the hierarchy. Typical orders of magnitude on a server CPU (rough; learn the story, not exact ns):

- L1: ~1–4 cycles, tens of KiB per core.
- L2: ~10+ cycles, hundreds of KiB per core.
- L3: tens of cycles, shared across cores on a socket, MBs.
- DRAM: hundreds of cycles.

A **cache line** is the transfer unit (commonly 64 bytes on current x86-64 CPUs). The hardware loads/invalidates whole lines, not single ints.

9.1.1 L1

L1 is split into instruction (L1i) and data (L1d) on most cores. It is private to the core, tiny, and the first place a hot trading loop should keep its working set. A miss in L1 often

hits L2 next.

Noise removed when the hot working set fits in L1: DRAM-scale stalls on every load. Cost of missing: a few to tens of cycles per access — still “local,” but enough to matter when the loop is already nanosecond-sensitive. Instruction-cache misses (cold code paths, giant binaries) add the same class of jitter on branches you rarely take until a rare event.

9.1.2 L2

L2 is larger and still typically **per-core** (or lightly shared). It absorbs what does not fit in L1. Pinning a thread to one core (Chapter 7) helps keep L1/L2 **warm**.

Noise removed by keeping the mid-size working set in L2 after L1 overflows: repeated trips to the shared LLC or DRAM. Cost of missing L2: typically tens of cycles plus contention if the line must come from another core or L3. Migration undoes L1/L2 warmth — that is why pinning is a cache story, not only a scheduler story.

9.1.3 L3

L3 (LLC) is usually **shared** by cores on the same socket. It helps when threads on the same socket share read-mostly data. It does **not** remove false sharing: two cores writing the same line still bounce ownership through the coherence protocol.

Noise removed when shared read-mostly state stays in L3 instead of DRAM: hundreds of cycles per miss and interconnect pressure. Cost: other cores on the socket compete for the same LLC capacity — noisy neighbors thrash your lines even when your thread is pinned. Remote DRAM (Chapter 10) is the next cliff after L3.

Interview tip: frame each level by *working set that fits* and *what you pay when it does not* — L1 (cycles), L2 (tens), L3 (tens–hundreds), DRAM (hundreds+).

9.2 Cache Coherence

When multiple cores cache the same physical line, they must agree on its contents. **Cache coherence** protocols maintain that agreement. Without them, core A could keep a stale copy while core B wrote a new value.

9.2.1 MESI

See [146].

MESI is a classic write-invalidate protocol. Each cached line is in one of:

- **M (Modified)** — this core has the only copy; it is dirty (differs from DRAM).
- **E (Exclusive)** — this core has the only copy; clean.
- **S (Shared)** — several cores may hold a read-only copy.
- **I (Invalid)** — not present / must not be used.

A write typically requires **Exclusive** or **Modified** ownership: other cores' copies are invalidated first.

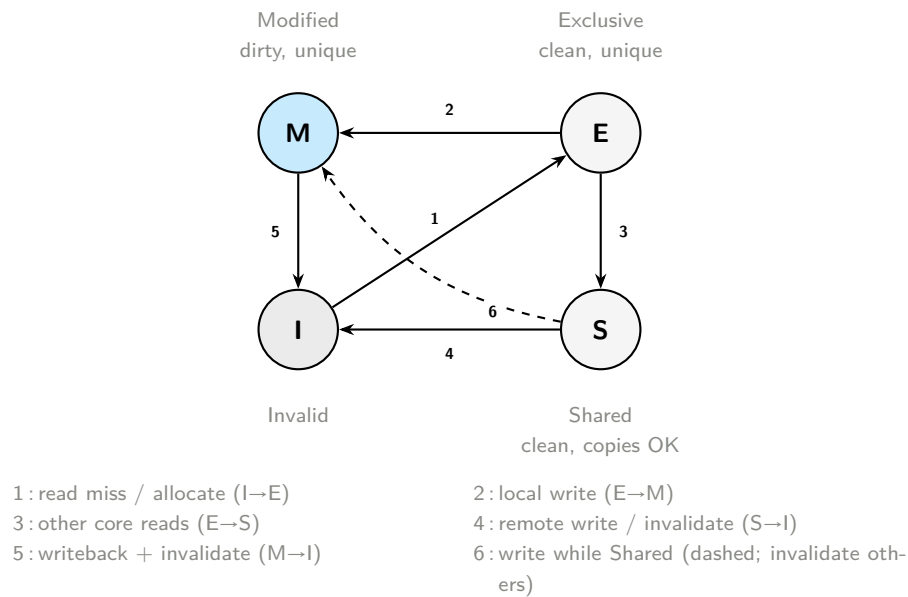


Figure 9.1: MESI states (sketch). Arrows are common transitions, not the full protocol.

Walkthrough (two cores, one line)

1. Both start **I**. Core A reads $\Rightarrow$ line arrives in **E** on A.
2. Core B reads the same address $\Rightarrow$ both hold **S** (A drops from **E** to **S**).
3. Core A writes $\Rightarrow$ it must **invalidate** B's copy ($B \rightarrow I$); A's line becomes **M**.
4. Core B reads again $\Rightarrow$ miss; it may force A to write back, then both can end in **S** (or B gets a fresh copy and A is demoted). Exact bus messages vary by CPU; the interview point is **invalidation + refetch cost**.

That bounce is what you feel as **coherence traffic**. False sharing (Section 9.3) is the same mechanism when the “shared” line holds unrelated variables.

Cache Invalidation

When core A writes a line held as Shared elsewhere, it sends **invalidations**. Other cores mark their copies Invalid. Their next access **misses** and must refetch. Frequent invalidations across cores = **coherence traffic** and latency spikes.

Interview tip: name the four letters, then walk one write that turns Shared copies into Invalid — that is the link to false sharing.

9.3 False Sharing

Interview tip: false sharing is one of the highest-yield topics in this chapter.

9.3.1 What It Is

False sharing happens when two threads update **different variables** that live on the **same cache line**. Logically there is no shared data race on one variable; physically the line is shared and bounced between cores.

```
1  #include <atomic>
2  #include <thread>
3
4  struct SharedCounters {
5      std::atomic<long> a{0};
6      std::atomic<long> b{0}; // may share a cache line with a
7  };
8
9  void false_sharing_demo(SharedCounters& c)
10 {
11     std::thread t1([&] {
12         for (int i = 0; i < 100000000; ++i) c.a.fetch_add(1, std
13             ::memory_order_relaxed);
14     });
15     std::thread t2([&] {
16         for (int i = 0; i < 100000000; ++i) c.b.fetch_add(1, std
17             ::memory_order_relaxed);
18     });
19     t1.join();
20     t2.join();
21 }
```

9.3.2 Why It Slows Things Down

Each store may force the other core to **invalidate** and reload the line. You pay coherence traffic even though a and b are independent. Symptoms: poor scaling when adding threads, high cache-miss / remote-hit counters, time spent in what looks like “simple atomics.”

9.3.3 Cache-Line Ping-Pong

The line oscillates (“**ping-pong**”) between cores: M on core 0, then M on core 1, and so on. Hot SPSC indices without padding show the same pathology (Section 8.3.5).

9.3.4 Solutions

- **Pad / align** so hot fields sit on different lines. Examples below use `alignas(64)` because 64 bytes is the usual line size on x86-64 and keeps listings readable. Prefer the portable C++17 constant `std::hardware_destructive_interference_size` (`<new>`) in production code — it is the destructive interference size (padding to **avoid** sharing a line). The related `std::hardware_constructive_interference_size` is for packing data that **should** share a line.


```

1 #include <atomic>
2
3 struct PaddedCounters {
4     alignas(64) std::atomic<long> a{0};
5     alignas(64) std::atomic<long> b{0};
6 };

```

- Give each thread its **own** counter (or shard), then reduce infrequently.
- Keep writers on the **same core** if they must share a line (often worse than padding).
- Verify with perf (Section 9.4).

9.4 Profiling

Guessing is expensive; **measure**. On Linux, perf is the standard first tool.

Decision narrative: start wide (perf stat), decide whether the bottleneck looks cache-like, then zoom to code (record/annotate), then classify the miss pattern (sharing vs capacity vs NUMA). Do not jump to padding or huge pages until the counters and samples point there.

9.4.1 perf stat

High-level counters for a whole run:

```

1 perf stat -e cycles,instructions,cache-misses,cache-references ./
  my_bench

```

Look at cache-misses relative to cache-references, and instructions per cycle (IPC). A sudden miss rate jump after a “harmless” structural change often points at layout / false sharing.

Noise removed by this step: debating micro-optimizations while the miss rate was never the problem — or while IPC was already healthy and the stall was I/O or locks.

9.4.2 perf record and perf annotate

```

1 perf record -e cache-misses -g ./my_bench
2 perf report
3 perf annotate

```

record samples where misses occur; annotate maps them to instructions / source. Use this when stat says misses are high but you do not know **which** line of code.

If samples cluster on two atomics a few bytes apart, suspect false sharing (Section 9.3). If they fan out across a large array scan, suspect capacity / layout. If they spike only under multi-socket load, open the NUMA chapter.

9.4.3 Cache Miss Events

Useful event names vary by CPU; common ones include `cache-misses`, `cache-references`, and more specific L1/LLC miss events from `perf list`. On Intel, also watch for remote DRAM / cross-socket accesses when NUMA is in play (Chapter 10).

Prefer the most specific event that answers your question: L1d misses for a tight loop, LLC misses for working-set overflow, remote-hit / DRAM events when pinning or allocation locality is in doubt. Generic `cache-misses` alone does not tell you *which* level failed.

9.4.4 Interpreting Results

- High misses + poor scaling with thread count → suspect sharing / false sharing.
- High misses in a single-threaded scan → suspect layout, random access, or working set > cache.
- Good IPC and low misses → look elsewhere (syscalls, locks, I/O).

Interview tip: say the sequence — `stat` to see it, `record` to find it, event type / scaling behavior to name the cause — not just “we ran `perf`.”

9.4.5 Detecting False Sharing

Practical recipe:

1. Benchmark the contended structure; note throughput / `perf stat`.
2. Apply `alignas(64)` (or `pad`) between hot fields; see the note in Section 9.3.4 on `std::hardware_destructive_interference_size`.
3. Re-run: large improvement with the same logic ⇒ false sharing was likely.

Some toolchains expose `perf c2c` (cache-to-cache) analysis on supported platforms — useful on real servers when available.

9.5 Cache Locality

Locality is why contiguous, predictable access patterns beat pointer-chasing.

9.5.1 Spatial Locality

Spatial locality: using address X makes nearby addresses ($X + 64, \dots$) cheap because they may already be in the same line or prefetched. Prefer **arrays of structs** laid out for the hot scan order; avoid scattering hot fields across the heap.

9.5.2 Temporal Locality

Temporal locality: reusing the same data soon keeps it in L1/L2. Tight loops over a small working set win; thrashing a huge working set forces DRAM traffic. Pinning (Chapter 7) protects temporal locality from migration.

9.6 TLB

See Section 7.2 for virtual address space and pages.

Caches hide DRAM latency for *data*; the **TLB** hides page-table walk latency for *addresses*. A hot path can look cache-friendly in L1/L2 counters and still stall on translations if it touches too many pages.

9.6.1 Translation Lookaside Buffer

The **TLB** caches virtual→physical translations. Every load/store needs a translation; a TLB hit is cheap compared to a page-table walk.

Noise removed by a warm TLB: multi-level page-table walks (and the cache misses those walks themselves cause) on every new page the hot loop touches.

9.6.2 TLB Misses

A **TLB miss** walks page tables (and may incur further cache misses). Large, sparsely accessed working sets or many small pages increase TLB pressure. With 4 KiB pages, a few dozen MiB of randomly touched memory already needs thousands of TLB entries — more than a typical L1 TLB holds — so misses become steady noise, not a one-time startup cost.

Noise removed by **huge pages** (2 MiB / 1 GiB where supported): far fewer translations for the same physical span, so the TLB covers a much larger working set. Cost: less flexible allocation, setup/`mmap` flags or transparent huge pages to get right, and wasted RAM if you oversize. Still pair with compact layout — huge pages do not fix pointer-chasing across a sparse heap. Page faults and demand paging are covered in Section 13.3.4.

Interview tip: TLB miss $\neq$ cache miss; huge pages buy fewer translations, not faster DRAM — measure both when the working set is large.

Cache Coherence and Performance Bibliography

- [145] *perf: Linux profiling with performance counters*. [perf Wiki](#).
- [146] *Cache coherence (MESI / MESIF overview)*. [Wikipedia — MESI protocol](#).

10

CHAPTER

NUMA Architectures

On a laptop with one CPU socket, almost all RAM is “local.” On multi-socket servers, each socket has its own memory controllers: accessing [local](#) RAM is faster than crossing to another socket’s RAM. That design is [NUMA](#) (*Non-Uniform Memory Access*). HFT boxes are often multi-socket.

Interview tip: keep [threads, memory, and NICs](#) on the same node when it matters.

10.1 NUMA Fundamentals

A [NUMA node](#) is typically one CPU socket plus the DRAM attached to it. Cores on that socket reach local DRAM with lower latency (and often higher bandwidth) than DRAM attached to another socket. Inter-socket links (e.g. Intel UPI / older QPI) carry remote traffic and add delay.

10.1.1 Local Memory

[Local memory](#) is RAM attached to the same node as the core executing the load/store. Prefer allocating hot structures (order books, ring buffers, decode scratch) on the node where the owning thread runs.

Noise removed: the inter-socket interconnect round-trip on every hot load/store. Local DRAM is still hundreds of cycles versus cache, but it is the best DRAM you can get on that box — and it keeps bandwidth on the local memory controllers.

10.1.2 Remote Memory

Remote memory is RAM on another node. Every remote access pays the interconnect tax and can increase jitter. Cross-node false sharing / coherence traffic is especially costly (Chapter 9).

Noise removed by staying local: UPI/QPI (or equivalent) hops and remote memory-controller contention that show up as fat tails even when median latency looks fine. Cost of fixing it: explicit binding and allocation discipline — the OS will not guess your topology for you.

Interview tip: local = same node as the core; remote = another node's DRAM — name both latency and the coherence tax when lines bounce across sockets.

10.2 NUMA Performance

10.2.1 Memory Access Latency

Interview tip: remote DRAM can be on the order of $\sim 1.5\times - 2\times +$ local latency (platform-dependent; measure on the box). Bandwidth can suffer too when many cores hammer the interconnect.

Classic failure mode: thread pinned on node 0, but `malloc/new` first-touched pages on node 1 (or the OS placed them poorly) $\Rightarrow$ every hot access is remote.

That split is easy to miss in code review: affinity looks correct in taskset, yet profiles show remote DRAM hits because allocation happened on a different node (or before pinning). Another variant: NIC on node 0, strategy on node 1 — packets cross the interconnect twice (DMA placement + CPU access) unless buffers and threads share a node.

Noise removed by first-touch / `membind` after pinning on the NIC's node: systematic remote DRAM on the hot path. See Section 10.3.1.

10.3 NUMA-Aware Programming

Goals:

1. Run the thread on the right node (CPU binding).
2. Allocate its hot memory on that same node (allocation locality).
3. Prefer the NIC that sits on that node (PCI proximity; Chapter 7 topology notes).

10.3.1 Allocation Locality

Linux often uses **first-touch**: the node of the CPU that first writes a page owns that page. Patterns:

- Pin the thread first, then allocate / initialise large buffers on that thread.
- Or allocate explicitly with `libnuma` / `numactl`.

Command-line control (process-wide):

```
1 # Run on CPUs of node 0; prefer allocating on node 0
2 numactl --cpunodebind=0 --membind=0 ./trading_engine
3
4 # Show topology
5 numactl --hardware
6 lscpu | grep NUMA
```

Library sketch (link with `-lnuma`; package `libnuma-dev` on Debian/Ubuntu):

```
1 #include <numa.h>
2
3 void* alloc_on_node(std::size_t bytes, int node)
4 {
5     if (numa_available() < 0) {
6         return nullptr; // not a NUMA system / lib unavailable
7     }
8     return numa_alloc_onnode(bytes, node);
9 }
10
11 void free_numa(void* p, std::size_t bytes)
12 {
13     numa_free(p, bytes);
14 }
```

On a [single-node](#) machine (common laptops), `numactl -hardware` shows one node; NUMA tuning still matters less than pinning and IRQ isolation, but the vocabulary remains interview-relevant for servers.

10.3.2 NUMA Thread and CPU Binding

For core-level pinning with `sched_setaffinity()` / `pthread_setaffinity_np`, see Section 7.5.1. This section covers placing threads on CPUs [within a NUMA node](#) so they stay next to their data (and often next to the NIC).

Pinning a thread without placing its memory (and ideally its NIC) on the same node only removes migration noise — remote DRAM remains. The workflow below closes that gap.

Workflow:

1. Find the NIC's node: `/sys/bus/pci/devices/<addr>/numa_node` (-1 often means “unspecified,” treat as node 0 on single-node hosts).
2. List CPUs on that node: `/sys/devices/system/node/nodeN/cpulist`.
3. Pin trading threads to those CPUs (avoid core 0 when IRQs land there).
4. Allocate hot memory on the same node (`membind` / `first-touch` / `numa_alloc_onnode`).

Noise removed: remote memory accesses and cross-node packet paths that survive “we pinned the thread” alone. Cost: less OS freedom to balance load; you own placement.

libnuma helpers such as `numa_run_on_node(node)` restrict the calling thread to that node's CPUs; combine with explicit affinity when you need a [specific](#) core.

Interview tip: [pin compute next to memory next to the NIC](#) — same NUMA node — unless you have measured that another layout wins.

NUMA Architectures Bibliography

- [147] *numa(3) / numa(7)*. [Linux man-pages](#).
- [148] *numactl(8)*. [Linux man-pages](#).

Linux Networking Fundamentals

From user space, networking starts with the [Berkeley sockets](#) API. HFT stacks care about [which](#) protocol carries market data versus orders, and whether a call can [block](#) the hot thread (unpredictable wake-ups). Multiplexing with `epoll` is Chapter 12; kernel bypass is Chapter 14.

Do [not](#) memorize every flag and `errno`. Use `man 2 <call>` and `man 7 socket / man 7 tcp`.

Interview tip: know which calls block, how non-blocking I/O fails with `EAGAIN`, and latency knobs such as `TCP_NODELAY`.

11.1 TCP vs UDP

- [TCP](#) — connection-oriented, reliable, ordered byte stream. Handshake, retransmission, congestion control. Common for order/execution sessions where loss is unacceptable. Cost: more kernel work and latency variance.
- [UDP](#) — datagrams, no connection, no delivery guarantee. Common for [market data](#) (including multicast feeds): the app handles gaps/seq numbers. Lower overhead; you own reliability.

The MD vs OE split is the interview framing. Market-data firehoses optimize for [rate and latency](#): a dropped datagram is a gap to detect/recover, not a reason to stall the wire with retransmit storms. Order/execution sessions optimize for [correctness of the session](#): loss, reordering, or silent truncation is a trading bug — TCP's reliability machinery (and its

jitter) is the usual trade.

Noise removed by UDP on MD: connection state, ordered-byte machinery, and retransmit-driven latency spikes on the receive path. **Noise removed** by TCP on OE: application-level reinventing of reliable delivery under loss. Cost either way: UDP pushes gap handling into your code; TCP pushes variance into the kernel stack.

Interview tip: pick TCP when you need a reliable session; UDP when you need firehose throughput and can tolerate/detect loss yourself — say MD vs OE explicitly.

11.2 Which Calls Block?

By default a new socket is **blocking**. `O_NONBLOCK` (via `fcntl` or `SOCK_NONBLOCK` / `accept4`) changes that.

On a latency path, “blocking” is not just “slower”: the thread **sleeps** in the kernel until the condition clears, then wakes at an unpredictable time — classic wake-up jitter (Section 7.4.1, Chapter 15). The table below marks which calls can introduce that sleep by default.

Call	Default	Notes
<code>socket</code>	non-blocking	Local kernel alloc only.
<code>bind</code>	non-blocking	Local name binding.
<code>listen</code>	non-blocking	Marks socket passive; sets backlog.
<code>accept</code>	blocking	Sleeps if no pending connection; non-blocking → <code>EAGAIN</code> .
<code>connect</code>	blocking	Waits for TCP handshake; non-blocking → <code>EINPROGRESS</code> .
<code>send</code>	blocking	Sleeps if send buffer full; may send partially .
<code>recv</code>	blocking	Sleeps if no data; return 0 = peer closed cleanly.
<code>close</code>	usually not	Returns quickly; kernel finishes teardown async. Can block with <code>SO_LINGER</code> .

Noise removed by non-blocking + `epoll`/busy-poll: sleep/wake on the hot `recv/send/accept` path. Cost: you must handle `EAGAIN` / `EINPROGRESS` and drive readiness yourself (Section 11.4).

Interview tip: sleep = jitter; name the calls that sleep by default and what non-blocking returns instead.

11.3 Socket Lifecycle

A socket is an endpoint (an `fd`). Typical TCP server path: `socket` → `setsockopt` → `bind` → `listen` → `accept` loop → `recv/send` → `close`. Client path: `socket` → `connect` → I/O → `close`. UDP often skips `listen/accept` and uses `sendto/recvfrom` (or `connect` on UDP to fix a peer).

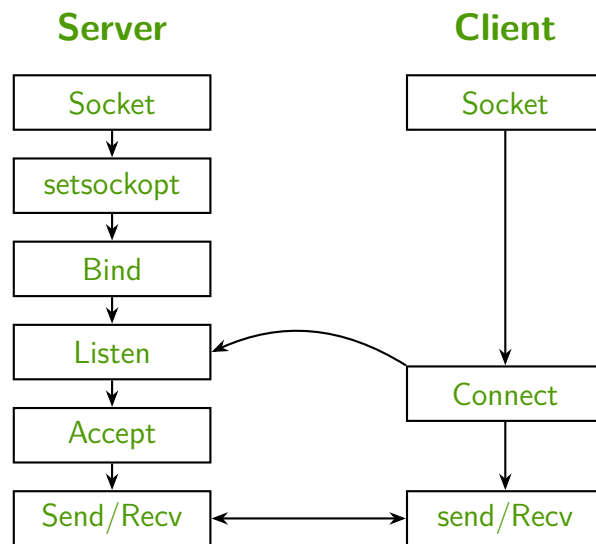


Figure 11.1: TCP socket lifecycle: server setup versus client connect, then bidirectional I/O.

11.3.1 socket()

Creates an endpoint. Example: `socket(AF_INET, SOCK_STREAM, 0)` for TCP IPv4; `SOCK_DGRAM` for UDP. Returns a file descriptor or `-1` on error (e.g. `EMFILE` if the process is out of fds). Not a blocking wait on the network.

11.3.2 bind()

Assigns a local address (IP + port). Servers bind before listening; `INADDR_ANY` means “all interfaces.” Not a network wait.

EADDRINUSE / TIME_WAIT: after a restart, bind often fails because the port is still reserved while TCP finishes delayed segments. Set `SO_REUSEADDR` with `setsockopt` before bind so the address can be reused.

Interview tip: `EADDRINUSE` after restart is a classic pitfall — set `SO_REUSEADDR` before bind.

11.3.3 listen()

Marks a TCP socket as passive. The backlog limits the queue of connections that finished the handshake but are not yet taken by `accept`. If the queue is full, clients may see `ECONNREFUSED`. System cap: `/proc/sys/net/core/somaxconn`.

11.3.4 accept()

Dequeues a completed connection and returns a new fd for that client. Blocking `accept` sleeps until a client arrives. Non-blocking: `-1` with `EAGAIN/ETIMEDOUT`. Optional Linux `accept4` can set `SOCK_NONBLOCK` / `SOCK_CLOEXEC` atomically on the new fd.

11.3.5 connect()

TCP client: starts the three-way handshake. Blocking connect may wait a long time on failure/timeout. Non-blocking: returns `-1 / EINPROGRESS`; completion is detected by polling for `writability` (`epoll/poll`), then checking socket error via `getsockopt(SO_ERROR)`. If the client did not bind, the kernel picks an ephemeral local port.

11.3.6 send()

Copies bytes into the kernel send buffer (TCP stream or connected UDP). May accept `fewer` bytes than requested — loop until all data is queued. Blocking sleeps when the buffer is full; non-blocking yields `EAGAIN`.

MSG_NOSIGNAL: writing to a closed peer can raise `SIGPIPE` and `kill` the process. Pass `MSG_NOSIGNAL` so failure becomes `-1 / EPIPE` instead — critical for trading processes that must not die on a dropped session.

11.3.7 recv()

- `> 0` — bytes read (may be partial).
- `0` — peer closed cleanly (`close/shutdown`).
- `-1` — error (`EAGAIN` if non-blocking and empty; `ECONNRESET` if peer aborted).

11.3.8 close()

Drops one reference on the fd. TCP teardown usually continues in the kernel without blocking your thread. With `SO_LINGER`, `close` can block until data is flushed or a timeout expires. After fork, the connection stays up until `all` processes that hold the fd call `close`; use `shutdown` when you need to signal end-of-stream regardless of refcount.

```
1 #include <sys/socket.h>
2 #include <netinet/in.h>
3 #include <netinet/tcp.h>
4 #include <unistd.h>
5
6 // Minimal TCP listen socket (error checks omitted for brevity)
7 int make_tcp_listener(uint16_t port)
8 {
9     int fd = socket(AF_INET, SOCK_STREAM, 0);
10    int yes = 1;
11    setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));
12
13    sockaddr_in addr{};
14    addr.sin_family = AF_INET;
15    addr.sin_addr.s_addr = htonl(INADDR_ANY);
16    addr.sin_port = htons(port);
17
18    bind(fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));
```

```

19     listen(fd, 128);
20     return fd;
21 }

```

11.4 Blocking vs Non-Blocking I/O

This is the HFT-critical distinction in this chapter.

11.4.1 Blocking Sockets

By default, `recv/accept/connect/send` may [sleep](#) in the kernel. The thread is de-scheduled; wake-up latency is unpredictable (Chapter 7). Fine for tools; dangerous on a pinned strategy core.

11.4.2 Non-Blocking Sockets

Operations return immediately. If they cannot complete, you get `EAGAIN / EWOULDBLOCK` (or `EINPROGRESS` for `connect`) and try later — typically after `epoll` says the `fd` is ready (Chapter 12), or in a busy-poll loop (Chapter 15).

11.4.3 `O_NONBLOCK`

Set the file status flag so all I/O on that `fd` is non-blocking:

```

1  #include <fcntl.h>
2
3  bool set_nonblocking(int fd)
4  {
5      int flags = fcntl(fd, F_GETFL, 0);
6      if (flags < 0) return false;
7      return fcntl(fd, F_SETFL, flags | O_NONBLOCK) == 0;
8  }

```

11.4.4 `MSG_DONTWAIT`

Per-call non-blocking for `recv/send` without changing the socket's default mode: pass `MSG_DONTWAIT` in the flags argument. Useful when only some reads must not block.

11.5 Latency Knobs

11.5.1 `TCP_NODELAY`

By default TCP may use [Nagle's algorithm](#): small writes can be delayed to coalesce packets (better bandwidth, worse latency). In HFT that delay is unacceptable.

```
1 int one = 1;
2 setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &one, sizeof(one));
```

Interview tip: disabling Nagle sends ASAP at the cost of more packets — a classic question.

11.5.2 MSG_NOSIGNAL

See send above: avoid SIGPIPE killing the process; prefer handling EPIPE.

Default UNIX behavior on a write to a closed peer is often SIGPIPE. An unhandled signal [terminates the process](#) — every session, every book, every in-flight decision dies with it. That failure mode is worse than a single broken connection: EPIPE is a recoverable error you can log, close, and reconnect; process death is an outage.

Noise removed: unexpected whole-process termination on a dropped peer. Cost: you must check send return values and treat EPIPE as session teardown, not as “ignore errors.” SO_NOSIGPIPE (where available) or a process-wide SIGPIPE ignore are alternatives; MSG_NOSIGNAL keeps the fix local to the call.

Interview tip: process death >> EPIPE — say why trading processes pass MSG_NOSIGNAL (or equivalent) on every hot write.

11.6 Common Errors

Always check return values; use perror / strerror(errno) while debugging. Know these by [name and meaning](#), not numeric codes. Each errno below usually points at a [design choice](#) (blocking vs non-blocking, reuse flags, peer state), not a random kernel quirk.

11.6.1 EAGAIN / EWOULDBLOCK

“Try again” on a non-blocking socket (no data / no send buffer space). On Linux they are the same value for sockets. Normal in event-driven loops — not a fatal error.

Design link: you chose non-blocking I/O; readiness must come from epoll/poll/busy-poll instead of sleeping inside recv/send.

11.6.2 EINPROGRESS

Non-blocking connect started; wait for completion via I/O multiplexing.

Design link: same non-blocking contract for connection setup — poll for writability, then getsockopt(SO_ERROR), rather than blocking the thread on the handshake.

11.6.3 EADDRINUSE

Address/port still busy (often TIME_WAIT). Use SO_REUSEADDR before bind.

Design link: restart / bind policy — without reuse, rapid process restarts collide with

TCP's delayed-segment protection.

11.6.4 ECONNREFUSED

Peer actively refused (nothing listening, or listen backlog full).

Design link: peer not up, wrong endpoint, or your (or their) listen backlog / accept rate cannot keep up — capacity and deployment, not a transient “try again.”

11.6.5 ETIMEDOUT

No timely response (path/firewall/server down) — different from an active refuse.

Design link: path health and timeouts — treat as reachability failure; do not conflate with ECONNREFUSED when diagnosing exchange connectivity.

11.6.6 EPIPE / ECONNRESET

Peer gone. With MSG_NOSIGNAL, broken writes surface as EPIPE instead of SIGPIPE. ECONNRESET often means an abrupt abort.

Design link: session teardown vs process lifetime — handle the errno and close cleanly; without MSG_NOSIGNAL, the same event can kill the process (Section 11.5.2).

Interview tip: map errno → design choice (non-blocking, reuse, backlog, peer death), not just a dictionary definition.

Linux Networking Fundamentals Bibliography

- [149] *socket(2)*. [Linux man-pages](#).
- [150] *fcntl(2)*. [Linux man-pages](#).
- [151] *tcp(7)*. [Linux man-pages](#).
- [152] *socket(7)*. [Linux man-pages](#).
- [153] Brian “Beej” Hall. *Beej’s Guide to Network Programming*. [beej.us](#).

12

Event Driven I/O

Non-blocking sockets (Chapter 11) return immediately with `EAGAIN` when there is nothing to do. That alone is not enough: you still need a way to `wait until an fd is ready` without spinning forever or blocking on one socket at a time.

`I/O multiplexing` answers “which of my fds can I read/write now?” Classic APIs are `select` and `poll`; Linux’s scalable answer is `epoll`. Busy-polling and kernel bypass come later (Chapters 15 and 14).

12.1 `select()`

`select(2)` [154] is the oldest portable multiplexer.

Interview tip: people still ask about `select` to contrast with `epoll`.

12.1.1 How It Works

You pass three bitmasks (`fd_set`): readable, writable, and exceptional conditions, plus an optional timeout. The kernel blocks (or returns after the timeout) and `rewrites` the sets so only ready fds remain set.

```
1 #include <sys/select.h>
2
3 fd_set rd;
4 FD_ZERO(&rd);
5 FD_SET(listen_fd, &rd);
6 int nfds = listen_fd + 1;
```

```

7
8  timeval tv{};
9  tv.tv_sec = 0;
10 tv.tv_usec = 1000; // 1 ms
11
12 int n = select(nfds, &rd, nullptr, nullptr, &tv);
13 if (n > 0 && FD_ISSET(listen_fd, &rd)) {
14     // accept / recv is safe to attempt
15 }

```

You must **rebuild** the `fd_set` before every call (the kernel mutates it). `nfds` is one past the highest fd number you care about.

12.1.2 Limits

- **FD_SETSIZE** — typically 1024. Fds above that cannot be tracked without rebuilding libc / using non-portable tricks.
- **Linear cost** — the kernel walks your interest set (cost linear in the number of fds); user space also copies and scans bitmasks. Cost grows with the **number of monitored fds**, not only with ready ones.
- **Stateful rebuild** — every wait starts from scratch; no persistent interest list in the kernel.

Fine for tiny tools; the wrong model for thousands of connections.

12.2 poll()

`poll(2)` [155] replaces bitmasks with an array of `pollfd` (fd, events, revents).

12.2.1 Improvements over select()

- No `FD_SETSIZE` hard limit (array length is yours).
- Clearer event flags (`POLLIN`, `POLLOUT`, `POLLERR`, ...).
- Input events and output revents are separate — you do not rebuild interest from scratch the same awkward way as `select`.

```

1  #include <poll.h>
2
3  pollfd pfd{};
4  pfd.fd = sock;
5  pfd.events = POLLIN | POLLOUT;
6
7  int n = poll(&pfd, 1, /* timeout_ms */ 1);
8  if (n > 0 && (pfd.revents & POLLIN)) {
9      // readable
10 }

```

12.2.2 Still Linear

`poll` still scales linearly in the number of fds: each wait copies/scans the whole array. Better than `select` for large fd numbers, but not the HFT/server default on Linux.

12.3 `epoll()`

`epoll` [156] is Linux-specific and built for many fds. Interest is registered **once** and stored in the kernel; waits return only **ready** events.

Three syscalls:

- `epoll_create1` — create an `epoll` instance (returns an `epoll` fd).
- `epoll_ctl` — add/mod/del interest for a target fd.
- `epoll_wait` — block until events (or timeout).

```
1 #include <sys/epoll.h>
2 #include <unistd.h>
3
4 int epfd = epoll_create1(0);
5 epoll_event ev{};
6 ev.events = EPOLLIN;
7 ev.data.fd = sock;
8 epoll_ctl(epfd, EPOLL_CTL_ADD, sock, &ev);
9
10 epoll_event out[64];
11 int n = epoll_wait(epfd, out, 64, /* timeout_ms */ 1);
12 for (int i = 0; i < n; ++i) {
13     if (out[i].events & EPOLLIN) {
14         // handle out[i].data.fd
15     }
16 }
```

`ev.data` is a union (fd, pointer, u64, ...) — store whatever your event loop needs to dispatch without another lookup.

12.3.1 Level Triggered

Default mode (no `EPOLLET`). While the condition remains true (e.g. data still in the receive buffer), `epoll_wait` keeps reporting the fd.

Pros: easier to use; missing a byte once does not permanently silence the fd. Cons: if you do not drain carefully, you can get woken repeatedly for the same readiness (extra wake-ups). Pair with non-blocking I/O and read until `EAGAIN`.

12.3.2 Edge Triggered

Add `EPOLLET`: you are notified on a **state transition** (e.g. went from empty to readable), not continuously while ready.

Interview tip: with edge-triggered `epoll`, expect these rules:

- Fds **must** be non-blocking.
- On notification, **drain** until `EAGAIN/EOULDBLOCK` (read/write loops). Stopping early can leave data buffered with **no further edge** until more arrives — classic ET bug.
- Often combined with `EPOLLONESHOT` (re-arm explicitly after handling) in multi-threaded reactors.

LT is safer for learning; ET is common in high-performance loops once the drain discipline is solid.

12.3.3 Epoll Scalability

- Interest list lives in the kernel — `epoll_ctl` is roughly constant time per registration change; not rebuilt every wait.
- `epoll_wait` cost tracks **ready** fds (plus your userspace handling), not the full interest set size.
- One thread can monitor tens of thousands of sockets; the limit becomes your processing, not the wait API.

That is why Linux network servers and many trading gateways use `epoll` (or go further with kernel bypass). `select/poll` remain useful vocabulary and for tiny portability niches.

12.4 Event Loop Architecture

Blocking I/O on many sockets either wastes threads (one per connection) or serializes behind sleeps — both inject wake-up jitter and poor scaling. A **reactor** multiplexes readiness on one (or a few) threads: wait for events, then do non-blocking work.

Noise removed: per-connection blocking sleeps and thread storms. Cost: all readiness handling shares the loop — a slow callback stalls everyone on that thread.

A typical reactor loop:

1. Create `epfd`; register listening and connected sockets (non-blocking).
2. `epoll_wait` (timeout 0 for pure poll, small ms, or -1 to sleep).
3. For each ready event: accept new clients, or `recv/send` on existing ones; on ET, loop until `EAGAIN`.
4. Handle `EPOLLERR / EPOLLHUP`: close and `EPOLL_CTL_DEL`.
5. Repeat.

Design notes for low-latency stacks:

- Keep the hot path on a **pinned** core (Chapter 7); avoid blocking syscalls inside the loop.
- Prefer **one** epoll thread owning the sockets, or shard by connection with clear ownership — shared epoll across many threads needs oneshot/re-arm discipline.
- Timeout 0 + spinning is busy-poll territory (Chapter 15); a small positive timeout trades a bit of latency for less wasted power when idle.
- Still kernel-mediated I/O: packet path goes through the networking stack. Cutting that cost is Chapter 14.

```
1 // Sketch: single-threaded LT epoll loop (errors omitted)
2 void run_loop(int epfd)
3 {
4     epoll_event out[64];
5     for (;;) {
6         const int n = epoll_wait(epfd, out, 64, 1);
7         for (int i = 0; i < n; ++i) {
8             const int fd = out[i].data.fd;
9             if (out[i].events & (EPOLLERR | EPOLLHUP)) {
10                 epoll_ctl(epfd, EPOLL_CTL_DEL, fd, nullptr);
11                 close(fd);
12                 continue;
13             }
14             if (out[i].events & EPOLLIN) {
15                 // read until EAGAIN if using EPOLLET; else one
16                 // or more reads
17                 handle_readable(fd);
18             }
19             if (out[i].events & EPOLLOUT) {
20                 handle_writable(fd);
21             }
22         }
23     }
```

Interview tip: select/poll scan all interest (linear in fds); epoll keeps interest in-kernel and returns ready set. Prefer LT until you need ET; with ET always use non-blocking fds and drain to EAGAIN.

Event Driven I/O Bibliography

- [154] *select(2)*. [Linux man-pages](#).
- [155] *poll(2)*. [Linux man-pages](#).
- [156] *epoll(7)*. [Linux man-pages](#).
- [157] *epoll_ctl(2)*. [Linux man-pages](#).

13

CHAPTER

Memory Mapping and Shared Memory

Processes do not share heap pointers across address spaces (Chapter 7). They still need fast ways to move bytes: classic read/write, or `map` pages into the virtual address space with `mmap`. Mapped regions matter for IPC, large files, and low-copy data paths; page behavior ties to the TLB (Section 9.6) and NUMA first-touch (Chapter 10).

13.1 File I/O

The traditional path copies between kernel buffers and user buffers on every call. Fine for configuration and logs; often too heavy for hot market-data or shared state.

Each read/write is a `syscall` plus at least one `copy` across the user/kernel boundary (and possibly disk/page-cache work). That is the noise `mmap` and shared-memory designs try to remove from the hot path (Section 13.2): fewer crossings, touch pages in place.

Noise removed by mapping instead of looping read: per-call copy and syscall overhead on steady-state access. Cost of staying on classic I/O: simpler error codes and streaming semantics — often the right tool for cold paths.

13.1.1 `open()`

`open(2)` [158] returns a file descriptor for a path. Flags select access (`O_RDONLY`, `O_RDWR`, ...) and behavior (`O_CREAT`, `O_CLOEXEC`, ...). Always check for `-1 / errno`.

Setup cost only; not the steady-state copy tax — but a missing `O_CLOEXEC` can leak fds

across exec and become an ops footgun.

13.1.2 read()

Copies up to count bytes from the fd into a user buffer. May return fewer bytes than requested (partial read). Return 0 means end-of-file for regular files.

Syscall + copy into your buffer every call; blocking read can also sleep on the device — wake-up jitter if this sits on a latency thread.

13.1.3 write()

Copies from a user buffer toward the fd. May be partial; loop until all intended bytes are queued (same discipline as send in Chapter 11).

Same copy/syscall framing as read; partial writes are a correctness requirement, not an edge case.

13.1.4 close()

Drops the fd. After the last close, kernel resources for that open file description are released. Mapped pages that still reference the file can outlive the fd until munmap.

13.1.5 lseek()

Moves the file offset used by subsequent read/write. Irrelevant for mmap access (you index the mapped pointer instead), but still used when mixing mapped and unmapped I/O on the same file.

Extra syscall noise when you could have used a mapped pointer or pread/pwrite; keep seeking off the hot path when possible.

```
1 #include <fcntl.h>
2 #include <unistd.h>
3
4 int fd = open("feed.bin", O_RDONLY);
5 if (fd < 0) { /* handle error */ }
6 char buf[4096];
7 ssize_t n = read(fd, buf, sizeof(buf));
8 close(fd);
```

13.2 Memory Mapping

mmap maps a file (or anonymous memory) into the calling process's virtual address space. After success you touch memory with ordinary loads/stores; the kernel fills pages on demand.

13.2.1 mmap()

mmap(2) [159]:

```
1 #include <sys/mman.h>
2 #include <fcntl.h>
3 #include <unistd.h>
4
5 int fd = open("book.bin", O_RDWR);
6 void* p = mmap(/* addr */ nullptr,
7               /* length */ 4096,
8               PROT_READ | PROT_WRITE,
9               MAP_SHARED,
10              fd,
11              /* offset */ 0);
12 if (p == MAP_FAILED) { /* handle error */ }
13 // use p as a typed pointer into the file
14 close(fd); // mapping can outlive the fd
```

Important flags:

- **PROT_READ / PROT_WRITE / PROT_EXEC** — allowed access; mismatches fault.
- **MAP_SHARED** — writes are visible to other mappers of the same object and (for files) can be written back to the file.
- **MAP_PRIVATE** — copy-on-write private view; writes do not update the underlying file or other processes.
- **MAP_ANONYMOUS** — no file; pages backed by RAM/swap (fd is typically -1). Used for large private arenas or, with care, shared anonymous maps via inheritance after fork.

Offset and length should be page-aligned where the man page requires it. Prefer **MAP_FAILED** checks — never compare only to **nullptr**.

13.2.2 munmap()

munmap(2) [160] removes the mapping. Accessing the old pointer afterward is undefined (likely **SIGSEGV**). For file-backed **MAP_SHARED** data you need on disk, **msync** can flush dirty pages before unmap if durability matters.

13.3 Concepts

See Section 7.2 for virtual address space and pages.

13.3.1 Memory-Mapped Files

A file-backed map lets you treat file contents as a byte array (or struct overlay) in memory. Benefits versus repeated read:

- Fewer explicit syscalls on the hot path (page faults still enter the kernel when cold).
- OS page cache and your process share physical frames for the same file pages.
- Natural fit for large read-mostly datasets (reference data, sometimes capture files).

Costs: harder error handling (faults instead of read return codes), alignment/struct-layout care, and accidental large working sets that thrash the TLB/cache.

13.3.2 Shared Memory

To share a region between unrelated processes, map the [same](#) object with MAP_SHARED:

- **POSIX shared memory:** `shm_open` [161] creates/opens a name under `/dev/shm`; then `ftruncate + mmap(..., MAP_SHARED, ...)`.
- **File-backed:** both processes `mmap` the same path with MAP_SHARED.

```

1 #include <sys/mman.h>
2 #include <sys/stat.h>
3 #include <fcntl.h>
4 #include <unistd.h>
5
6 int fd = shm_open("/hft_ring", O_CREAT | O_RDWR, 0600);
7 ftruncate(fd, 4096);
8 void* p = mmap(nullptr, 4096, PROT_READ | PROT_WRITE, MAP_SHARED,
9                 fd, 0);

```

Readers and writers still need a [protocol](#) (atomics, seqlocks, SPSC rings — Chapter 8). Mapping alone does not make concurrent access safe.

HFT use: publisher maps a ring or book snapshot region; consumers map read-only or shared and poll without socket copies. Pin producers/consumers with NUMA in mind (Chapter 10).

13.3.3 Lazy Loading

`mmap` usually does [not](#) pull the whole file into RAM up front. The kernel establishes virtual mappings; physical pages appear when first touched ([demand paging](#)). That keeps startup cheap and RAM use proportional to the working set — until you touch everything and fault it all in.

Cold vs warm matters for latency. A first touch on the trading path can be a [major page fault](#) (disk / uncached) — milliseconds of jitter dressed as a normal load. After pages are resident, subsequent touches are cheap until the OS reclaims them under memory pressure.

Noise removed by an explicit warm-up: walk (or `mlock`/`advise`) hot pages at startup so first-touch faults happen before the market opens. Cost: longer startup, higher RSS, and you must size the warm set honestly — warming “everything” defeats lazy loading’s point. See Section 13.3.4.

Interview tip: `mmap` is lazy; cold first-touch on the hot path is a fault, not a free load — warm before you trade.

13.3.4 Page Faults

A **page fault** occurs when the CPU accesses a virtual address that is not currently resident (or lacks permission). The kernel may:

- Load the page from file/swap (major fault — slow),
- Map an already-cached page (minor fault — cheaper),
- Or deliver SIGSEGV on illegal access.

On a latency-critical path, unexpected major faults are jitter. Warm important maps (touch pages at startup), size for the working set, and watch TLB pressure (Section 9.6).

13.3.5 Copy-on-Write

With `MAP_PRIVATE`, the first write to a shared-looking page allocates a **private** copy for that process; other mappers keep the old contents. `fork` uses the same idea for the child's address space: pages stay shared read-only until either side writes.

Interview tip: `MAP_SHARED` shares updates; `MAP_PRIVATE` is COW and does not update the file. Shared memory still needs synchronization on top of the map.

Memory Mapping and Shared Memory Bibliography

- [158] *open(2)*. [Linux man-pages](#).
- [159] *mmap(2)*. [Linux man-pages](#).
- [160] *munmap(2)*. [Linux man-pages](#).
- [161] *shm_open(3)*. [Linux man-pages](#).

14

Kernel Bypass and Low Latency Networking

Chapters 11–12 stay inside the [kernel networking stack](#): sockets, skbs, interrupts, and syscalls. That path is correct and portable — and often too slow or too jittery for the hottest market-data and order paths. [Kernel bypass](#) moves packet I/O into user space (or a thin driver path) so the trading process touches buffers with far fewer kernel transitions.

This chapter is conceptual.

Interview tip: focus on [why](#) bypass exists and [when](#) it is justified, not a full DPDK tutorial.

14.1 Why Kernel Bypass

14.1.1 Kernel Networking Overhead

A typical UDP/TCP receive on Linux roughly does:

1. NIC DMA into kernel memory; interrupt or NAPI softirq runs.
2. Protocol processing (Ethernet/IP/UDP or TCP) and socket buffer enqueue.
3. Your thread wakes (or polls) and `recv` [copies](#) into a user buffer (unless you use specialized zero-copy APIs).

Each stage adds latency and [variance](#): scheduling, cache misses, lock contention in the stack, interrupt coalescing, and noisy neighbors on the same core (Chapter 7). Throughput can still be high; [tail latency](#) is what HFT cares about.

14.1.2 System Call Overhead

See Section 7.3 for user mode vs kernel mode. Here, syscall overhead is framed as a networking latency cost.

Every `recv/send/epoll_wait` crosses into the kernel: mode switch, validation, possible sleep/wake. Even a non-blocking `recv` that returns `EAGAIN` paid for the round trip. Busy-polling sockets (Chapter 15) reduces wake-ups but still uses the kernel path. Bypass aims to **remove** those syscalls from the packet hot path: the app polls NIC rings (or a user-space stack) directly.

14.1.3 Context Switches

See Section 7.4.1 for the general OS definition. Here, context switches are a latency cost of kernel networking and syscall-heavy I/O paths.

Interrupt handlers and softirqs can preempt your hot thread; blocking I/O forces a full deschedule. Isolating CPUs and pinning (Chapter 15, Chapter 7) help, but the cleanest fix for NIC noise is often: dedicated cores, IRQ affinity away from the strategy core, and a bypass stack that does not bounce through the kernel for every packet.

14.2 Technologies

Vendors and open-source stacks differ in API and hardware support. The shared idea: **user-space packet I/O** + poll mode, often with huge pages and pinned memory.

14.2.1 DPDK

The **Data Plane Development Kit** [162] is a widely cited open stack for poll-mode drivers (PMDs), memory pools (mbufs), and run-to-completion loops on dedicated cores. The application owns RX/TX rings; the kernel is largely out of the data path for those NICs/-ports.

Noise removed: kernel softirq/socket path, per-packet interrupts on the hot core, and `recv` copies into user buffers. Ops cost: you own huge pages, NUMA pools, core dedication, and often the interface itself (no casual `tcpdump` / `iptables` story on a fully bound PMD port).

Interview tip: poll mode (no interrupt per packet on the hot path), huge pages, NUMA-aware pools (Chapter 10), and the operational cost (you become the network stack for that interface — routing, filtering, and safety are your problem).

14.2.2 Solarflare OpenOnload

OpenOnload (Solarflare / Xilinx / AMD lineage) [163] accelerates **existing** Berkeley sockets via an `LD_PRELOAD` user-space stack for supported NICs. Many apps keep `socket/recv/send` while the library short-circuits into user space when possible.

Appeal for trading firms: less rewrite than a full DPDK port; still hardware-specific and

ops-heavy. Exact product names evolve.

Noise removed: much of the kernel stack crossing while keeping the sockets API. Ops cost: NIC/vendor lock-in, preload/environment discipline, and debugging a hybrid path when some calls still fall through to the kernel.

Interview tip: the idea is [sockets API + user-space acceleration](#), not memorizing version strings.

14.2.3 AF_XDP

AF_XDP [164] is a Linux socket family for high-performance packet I/O into user-space [UMEM](#) rings, with optional zero-copy paths and XDP programs in the driver/NIC. It sits between “full kernel stack” and “leave Linux entirely”: you still integrate with the kernel’s XDP/AF_XDP machinery, but the hot path can avoid `sk_buff` copies and heavy protocol processing.

Noise removed: `sk_buff`-heavy receive and extra copies on the packet path, without abandoning Linux entirely. Ops cost: UMEM/ring setup, driver/XDP feature matrix, and still writing a packet-oriented app (not a drop-in for arbitrary TCP sessions).

Useful vocabulary: XDP for early drop/redirect in-kernel; AF_XDP when a user process must consume raw/fast packet streams with lower overhead than classic sockets.

14.3 Zero-Copy Market-Data Parsing

Packets on a bypass path typically land in a [user-space ring](#) (DPDK mbuf, Onload buffer, AF_XDP UMEM slot). The descriptor owns that memory until you release it back to the ring. [Zero-copy](#) parsing means decoding the wire message *in place* — no `malloc`, no `memcpy` into an owned `std::vector` on the hot path (Section 15.4).

`std::span` is a non-owning contiguous view (pointer + length); API details live in Section 5.1.8. Here it is only the handle you pass into the parser while the ring still owns the bytes.

Noise removed: an extra copy (and often an allocation) between NIC buffer and typed fields — plus the cache pollution of touching the payload twice. **Hard part:** lifetime and reclaim — a `span` must not outlive the descriptor; decode, hand off what you need (or copy only the small fields you will keep), then return the slot.

Safer loads of possibly unaligned / endian-swapped wire bytes use `memcpy` or explicit byte loads into a host struct. `reinterpret_cast` is shown below because interviews ask for it; it is correct only when alignment and layout match the wire, and the view’s lifetime is still bound to the ring slot.

```
1 #include <cstdint>
2 #include <stdint>
3 #include <span>
4
5 #pragma pack(push, 1)
6 struct WireHeader {
7     std::uint8_t msg_type;
```

```

8      std::uint16_t length;    // real feeds: convert from network
                                endian
9  };
10 #pragma pack(pop)
11
12 // pkt views bytes still owned by the RX ring / mbuf --- do not
    store this pointer.
13 const WireHeader* header_view(std::span<const std::byte> pkt)
14 {
15     if (pkt.size() < sizeof(WireHeader)) {
16         return nullptr;
17     }
18     return reinterpret_cast<const WireHeader*>(pkt.data());
19 }
20
21 void on_rx(std::span<const std::byte> pkt)
22 {
23     const WireHeader* h = header_view(pkt);
24     if (!h) {
25         return; // truncated --- drop / count
26     }
27     // use h->msg_type / payload subspan; then release the ring
    slot
28     auto body = pkt.subspan(sizeof(WireHeader));
29     (void)body;
30 }

```

Interview tip: zero-copy is ownership discipline, not a magic type — span views the ring; reclaim is when the view dies.

14.4 When To Use Them

Bypass is not free: hardware lock-in, harder debugging, security/ops burden, and often a dedicated team. Use it when measurements say the kernel path dominates your latency budget.

14.4.1 Trading

- **Market data** — UDP/multicast firehose where microseconds and jitter matter; classic recv + softirqs show up in profiles.
- **Order entry** — sometimes still kernel TCP with careful tuning (TCP_NODELAY, pinned cores); bypass when the tick-to-trade path is NIC-bound and proven.
- Prefer proving the bottleneck (Chapter 9 profiling mindset) before rewriting the I/O layer.

Decision tree (short): if profiles show kernel networking / softirq / syscall dominate *and* the SLA is tick-to-trade, consider bypass on that NIC path. If the bottleneck is book

logic, allocation, locks, or NUMA, fix those first — bypass will not help.

When *not* to bypass: control-plane / admin sockets, low message rates, teams without NIC/ops ownership, environments that need standard Linux tooling on the same port, or when tuned kernel sockets already meet the budget. Bypass is a last-mile I/O move, not a substitute for isolation, pinning, and allocation discipline (Chapter 15).

Interview tip: say when you would stay on kernel sockets — most paths — and when measured NIC-path latency forces DPDK / OpenOnload / AF_XDP.

14.4.2 HPC

High-performance computing and storage fabrics use similar ideas (RDMA, user-space drivers, poll mode) for message rates and latency between nodes. Same trade-off: performance vs complexity and portability.

14.4.3 Ultra-Low Latency Systems

When the SLA is “kernel networking is already too slow even when tuned,” combine:

- Bypass or accelerated sockets on the NIC path,
- CPU isolation and busy poll (Chapter 15),
- Careful NUMA placement of threads and buffers (Chapter 10),
- Preallocated rings and no hot-path allocations (Chapter 8).

Interview tip: kernel sockets copy and syscall; bypass polls user-space rings for lower median and tail latency at the cost of complexity. Name DPDK (PMD), OpenOnload-style socket acceleration, and AF_XDP as the Linux-native middle ground.

Kernel Bypass and Low Latency Networking Bibliography

- [162] *DPDK Documentation*. doc.dpdk.org.
- [163] *OpenOnload / TCPDirect Documentation*. [AMD/Xilinx Onload](#).
- [164] *AF_XDP*. [Linux kernel docs](#).

15

CHAPTER

Low Latency Engineering

Earlier chapters give the building blocks: pinning and scheduling (Chapter 7), caches and false sharing (Chapter 9), NUMA (Chapter 10), non-blocking I/O and `epoll` (Chapters 11 and 12), and kernel bypass when the stack itself is the limit (Chapter 14). This chapter is the [ops checklist](#): isolate cores, poll instead of sleep, keep the hot path cache-friendly and allocation-free, and optimize for latency even when that hurts bulk throughput.

15.1 CPU Isolation

Assumes per-thread pinning from Section 7.5.4. This section covers kernel boot parameters and IRQ routing that isolate cores from general scheduler and interrupt noise. Pinning alone is not enough if the kernel still steals your core for timers, RCU callbacks, hard IRQs, or unrelated threads. Isolation makes a core [mostly yours](#): each knob below removes a different class of jitter. Use them as a [set](#), not as independent toggles.

15.1.1 `isolcpus`: scheduling exclusion

By default the Linux scheduler treats every online CPU as a shared worker: the CFS load balancer can migrate runnable threads onto any core that looks idle. `isolcpus=` [\[165\]](#) removes listed CPUs from that regular balancing pool (exact flags and semantics depend on kernel version). Those cores no longer absorb random housekeeping or application threads unless you explicitly place work there with `sched_setaffinity`.

Noise removed: the balancer treating your core as free capacity for unrelated tasks.

Typical pattern: housekeeping (IRQs, OS daemons, logging) on CPUs 0– N ; strategy / market-data threads pinned onto isolated CPUs $N+1$ onward. `isolcpus` alone is incomplete — timer ticks, RCU callbacks, and hard IRQs can still hit the core (sections below).

15.1.2 `nohz_full`: adaptive-ticks / tickless

The kernel normally arms a [periodic timer interrupt](#) on each CPU (the scheduling “tick”, historically every few milliseconds) for accounting, timers, and preemption decisions. Even a single pinned busy-loop thread gets interrupted by that tick — pure jitter on a latency path.

`nohz_full`= [166] enables [adaptive-ticks](#) (also called tickless / full dynticks) on the listed CPUs: when only one runnable task is present, the kernel can stop the periodic tick and wake the CPU mainly for real work. Needs a kernel built with full dynticks support; validate on your distro.

Noise removed: the periodic scheduling-clock interrupt that would otherwise preempt your pinned thread even when nothing else is runnable.

Pair with `isolcpus` and *one* pinned busy-loop or poll thread per core; multiple runnable tasks on the same CPU defeat the point.

15.1.3 `rcu_nocbs`: RCU callback offload

RCU (Read-Copy-Update) is a Linux synchronization style for read-heavy shared data: readers proceed almost without locking; writers publish a new version and *defer* freeing the old one until every concurrent reader is done. That deferred cleanup runs as [RCU callbacks](#) (often via `softirq` / dedicated kthreads).

By default, callback work can run on the same CPUs that queued it — including an “isolated” core — and inject `softirq` latency into your hot path. `rcu_nocbs`= [167] (and related `rcu_nocb_poll`) marks listed CPUs as *no-callback* CPUs and moves that processing onto housekeeping cores.

Noise removed: deferred RCU cleanup / `softirq` work on the cores you wanted quiet.

Together with the other knobs: `isolcpus` + `nohz_full` + `rcu_nocbs` + IRQ affinity + pinned threads.

15.1.4 `irqaffinity` / `smp_affinity`: hard-IRQ routing

A [hard IRQ](#) is a device interrupt (NIC queue, timer, disk controller, ...) delivered by hardware to some CPU. Unlike a voluntary context switch, it *forces* that CPU into kernel interrupt context — disastrous for tick-to-trade tails if the NIC’s IRQs land on your strategy core.

`isolcpus` does not automatically move IRQs. You must steer them onto housekeeping CPUs [168]:

- `irqaffinity= (boot)`: default CPU mask for IRQs that do not set their own affinity — keep that default on housekeeping cores.
- `/proc/irq/<n>/smp_affinity` (or `smp_affinity_list`): per-IRQ mask; pin each NIC queue IRQ explicitly away from isolated strategy cores.
- `irqbalance`: userspace daemon that reshuffles IRQ affinity at runtime for throughput fairness. On latency boxes, disable it or constrain it — otherwise it undoes your masks.

Interview tip: say the full set — scheduler exclusion, tickless, RCU offload, hard-IRQ routing, then pinned threads — not just “we set `isolcpus`”.

15.2 Busy Polling

Builds on non-blocking sockets (Section 11.4) and `epoll` (Chapter 12).

On a blocking path, `epoll_wait / recv` puts the thread to sleep until the kernel wakes it. That wake-up crosses a syscall boundary, may involve a context switch, and lands at an unpredictable time relative to the packet — classic [wake-up jitter](#) on a latency path.

Busy polling trades a dedicated core for lower wake-up delay: the thread stays runnable and keeps asking “is data ready?” instead of sleeping.

- `epoll_wait` with timeout 0 in a tight loop (user-space spin until ready).
- Socket [busy poll](#) options (`SO_BUSY_POLL` and related sysctls) so `recv/poll` spin briefly *inside* the kernel networking stack before sleeping [169].
- User-space poll loops on bypass rings (Chapter 14) — the same idea with the kernel path removed.

Noise removed: scheduler sleep/wake latency on the critical receive path.

Cost: the core sits near 100% for as long as you spin, and you burn power / capacity you could give to other work. Only do this on *isolated* CPUs dedicated to that job (Section 15.1); spinning on a shared core just fights the balancer.

Interview tip: sleep = jitter; busy poll = latency bought with watts and a dedicated core.

15.3 Cache Optimization

See Chapter 9 for hierarchy, MESI, false sharing, and `perf`. Here the focus is the [ops checklist](#) on a tick-to-trade path: every unnecessary miss or line bounce is jitter you own.

Noise removed by the practices below: remote cache-line traffic, cold I-cache/D-cache on the first messages of the day, and NUMA remote DRAM on the hot thread.

- Keep hot structures within few cache lines; align and pad writers that share a struct so they do not false-share.

- Prefer sequential access and preallocated contiguous buffers (rings, arenas) — hardware prefetch and TLB like dense layouts.
- Pin the thread *and* allocate its memory on the same NUMA node as the NIC (Chapter 10); first-touch after pinning, not before.
- Warm critical code and data at startup (touch pages, run a dry path) so the first live tick is not a major-fault / cold-cache surprise (Chapter 13).
- Measure with perf — do not guess which miss dominates the path.

Interview tip: layout and placement beat micro-optimizing a cold path you have not profiled.

15.4 Avoiding Allocations

See Section 8.3 for pre-allocated SPSC rings; Section 3.1.2 for pools.

new/malloc, growing std::string/vector, and hidden temporaries are not “just slower”: they inject **unbounded** latency (allocator locks, syscalls, page faults), scramble caches/TLBs, and create load-dependent jitter that quiet unit tests never show.

Noise removed when the hot thread stops allocating: allocator contention, unexpected page faults, and allocator-induced cache pollution.

- Preallocate pools and rings at startup; recycle objects on the critical path.
- Fixed-size messages; no unbounded buffering on the hot thread (spill to a colder queue if you must).
- Defer logging, formatting, and allocations to a colder thread via a lock-free queue.
- Prefer stack/static storage or arenas with LIFO free for short-lived data.

Interview tip: “no alloc on the hot path” is a latency invariant, not a style preference.

15.5 Latency vs Throughput

These goals pull knobs in opposite directions. Name which axis you are on before tuning.

Goal	Optimize for	Often sacrifices
Latency	Median and tail response time; predictable wake-ups; short paths	CPU utilization, batching efficiency, multi-tenant density
Throughput	Messages/sec or bandwidth; batching, coalescing, deep queues	Per-message delay; tail latency under load

Latency-first knobs (typical HFT market-data / order path): busy poll, TCP_NODELAY, shallow queues, one connection per hot thread, CPU isolation, IRQ steering off the strategy core, no interrupt coalescing on that NIC queue.

Throughput-first knobs (bulk transfer, analytics ingest): large buffers, interrupt coalescing, Nagle left on, sleep when idle, shared noisy cores are acceptable.

Mixing the two without noticing is a common interview failure: techniques that maximize packets/sec often **hurt** tick-to-trade, and latency tricks waste capacity when you only care about bulk bandwidth.

Interview tip: state the axis, then name two knobs that move it and one that would move the *other* axis the wrong way.

Low Latency Engineering Bibliography

- [165] *CPU isolation (isolcpus)*. [kernel.org admin-guide](#).
- [166] *NO_HZ: Reducing Scheduling-Clock Ticks*. [kernel.org](#).
- [167] *RCU and rcu_nocbs*. [kernel.org RCU](#).
- [168] *IRQ affinity (smp_affinity / irqaffinity)*. [kernel.org IRQ affinity](#).
- [169] *busy_poll (socket busy polling)*. [socket\(7\)](#).

16

CHAPTER

Quant Basics for Developers

See [170], [171], [172].

This chapter assumes you know [math and programming](#), but *not* finance. It builds ideas in order — like a physics course where each section uses only what came before. Formulas are there for precision; the prose is what you should read first. Order-book *data structures* live in Chapter 17; latency engineering in Part II (Low Latency, NUMA, Cache).

How to read this chapter

Two tracks share the same chapter. Pick the path that matches the interview, then skim the other once.

Track A — Markets and trading systems (quant-dev / HFT core)

1. **Role and lifecycle** (Sections 16.1–16.2): what you build, and the path from intent to P&L.
2. **Wire protocols** (Section 16.3): FIX vs native binary vs crypto APIs — what the gateway and MD handler actually speak.
3. **Market map** (Sections 16.4–16.8): who trades where; instruments; spot vs futures infra; crypto; bid/ask/order types. Rehearse the interview Q&A at the end of the chapter.

4. **Optional depth:** Complex derivatives systems notes (Section 16.23) if the role touches options risk or swaps.
5. **Close:** Interview cheat sheet (Section 16.26) — say each bullet aloud.

Track B — Pricing and risk (options / QR-flavoured loops)

1. **Plain language** (Sections 16.9–16.11): stock, option, rates, vol.
2. **Model core** (Sections 16.12–16.16): GBM intuition through Black–Scholes and greeks.
3. **Desk practice** (Sections 16.17–16.25): P&L explain, smile, code, Americans / large universes, production.
4. **Close:** Cheat sheet + pricing Q&A (Sections 16.26–16.27).

First pass: Track A *or* Track B in one sitting. Second pass: the other track plus any gap that showed up in a mock interview.

16.1 What a Quant Developer Does

At systematic trading firms, a **Senior Quant Developer** typically:

- Writes C++ (or similar) to **price** options and compute **greeks**.
- Makes libraries **fast, deterministic, and tested** on paths that affect live **P&L** (profit and loss).
- Connects **market data** (prices, rates, vol surfaces) to those libraries.
- Explains daily P&L: “how much was stock move vs vol move vs time decay?”

The job is less “invent a new model” and more *keep the production path honest*.

Noise removed in live systems: stale market data (marks and hedges from yesterday’s book), wrong units (vol in percent vs decimal, rates vs day-count), and untested greeks (analytic formulas that never met a bump check, so risk and P&L explain silently lie).

A **pure quant researcher** owns the model story — hypothesis, fit, paper P&L. A quant developer owns the path from that model into wires, books, risk, and regression tests: same economics, but the failure modes are desync, units, and latency, not only statistical edge.

Interview tip: understand *both* the model and *where production breaks* (wrong units, stale data, smile not flat, etc.).

16.2 Trade Lifecycle: From Intent to P&L

An interview classic: walk an order from the moment a strategy wants to trade until the firm updates position and P&L. This is the **control-plane + data-plane** story of a trading system — complementary to tick-to-trade latency (Section 16.27.3), which zooms in on the hot path only.

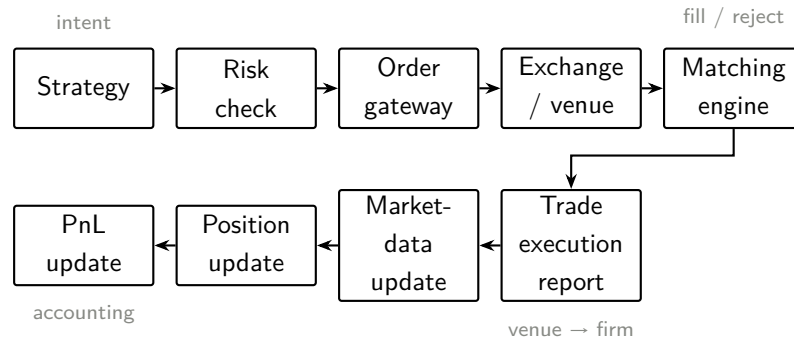


Figure 16.1: Trade lifecycle (simplified). Outbound intent on top; venue feedback and firm state below.

Strategy

Signal or model decides to buy/sell (or adjust a quote). Output is an **order intent**: instrument, side, quantity, order type, and constraints (e.g. limit price, time-in-force). No money has moved yet.

Risk check

Pre-trade gates: position / exposure limits, fat-finger checks, forbidden instruments, credit, kill-switches. Fail here $\Rightarrow$ no order leaves the firm. In HFT this must be **cheap and deterministic** (Chapter 15).

Order gateway

Firm-side component that owns the venue session: encodes the native protocol, assigns client order ids, handles heartbeats/reconnects, and is often the last process before the wire. Think of it as the **door to the exchange**.

Exchange / venue

Your order is now their problem: throttling, fee logic, and admission into the matching engine. For OTC there may be no central book — a dealer or platform stands in the middle (Section 16.4.3).

Matching engine

Applies venue rules — typically **price-time priority** (Chapter 17) — and either **fills** (full/partial), **rests** the order in the book, or **rejects** it. This is where “liquidity” becomes concrete: who was available at which price.

Trade execution (report)

Fill notices (and cancel/reject acks) travel back to the firm. Your gateway normalizes them into internal events. Until this lands, strategy state and risk may still be optimistic — design for that.

Market-data update

Public (or private) feeds update trades and/or book changes caused by your fill and everyone else's. Your local book and marks move (Section 16.27). Sequence numbers matter: gaps force snapshot rebuilds.

Position update

Inventory and open orders change: filled qty, average price, remaining working orders. Downstream risk and strategy both read this.

P&L update

Mark-to-market and/or realized P&L refresh (Section 16.17 for the options greeks decomposition). Ops and desks care that this matches the venue's view within known lags.

Interview tip: say the pipeline in order, then pick **one** stage you owned (e.g. gateway, book, or risk) and name the failure mode (gap, partial fill, risk reject, clock jump). Interviewers want system thinking, not a memorized list.

16.3 Market Wire Protocols

The order gateway and market-data handler do not speak “equities” or “FX”: they speak a **venue (or broker) protocol**. This section is vocabulary for interviews and onboarding — not a protocol encyclopedia. Networking mechanics (TCP, UDP multicast, kernel bypass) live in Part II; here the question is *what rides on the wire*. See [179], [180].

16.3.1 Two planes

- **Market data (MD)** — venue → firm: trades, book updates, sometimes reference data. Often **UDP multicast** on exchange campuses; sequence numbers and gap recovery matter (Section 16.27).
- **Order entry (OE)** — firm → venue: new order, cancel, replace; venue → firm: acks, rejects, fills. Usually **sessionful** (TCP or a reliable session on top), with heartbeats and recovery.

One firm may use **different** protocols for MD and OE to the same venue (e.g. binary MD + FIX OE, or two native binaries).

16.3.2 FIX

FIX (Financial Information eXchange) is the industry workhorse for many buy-side / sell-side / broker links: a mostly **text** tag=value encoding delimited by SOH (0x01).

Shape of a message

Fields are numbered tags, e.g. 35=D (MsgType = New Order Single), 55=MSFT (Symbol), 44=178.50 (Price). A message has a header (including MsgType and sequence number), a body, and a trailer (checksum). Humans can read a capture; parsers still must be careful (repeating groups, optional fields, version drift).

Session vs application

- **Session layer** — Logon / Logout, Heartbeat / Test Request, sequence numbers, Resend Request when gaps appear. If the session dies, you **must** recover state before trusting live flow again.
- **Application layer** — business messages. Classics interviewers expect: **NewOrderSingle** (MsgType D), **ExecutionReport** (MsgType 8: ack, fill, reject, cancel confirm, ...), cancel / cancel-replace.

“FIX” in conversation often means both the wire encoding and the session behaviour. Variants exist (FIXML, FIX over different transports); the mental model above is enough for most quant-dev interviews.

Latency reality

Classic text FIX is **not** what colocated HFT typically uses against the matching engine. It is still everywhere for broker connectivity, drop copies, slower venues, and ops tooling. Saying “we speak FIX” without naming *which* hop (broker vs exchange native) is imprecise.

Interview tip: walk one happy path — Logon → NewOrderSingle → ExecutionReport(s) — then one failure (sequence gap / disconnect → resend / replay).

16.3.3 Native binary protocols

Exchange hot paths usually use **venue-specific binary** layouts: fixed or schema-described structs, integer prices (ticks), compact enums, and explicit sequence spaces. Examples of the *pattern* (names vary by venue and era):

- **Market data** — Nasdaq TotalView-**ITCH**-style feeds: typed binary messages (add/order/execute/delete/...) on multicast; rebuild L3 from the event stream (plus snapshot/recovery channels).
- **Order entry** — **OUCH**-style (and peers): binary enter/replace/cancel and terse acks — designed for speed and simple state machines, not human readability.
- **Futures / others** — each major venue ships its own MD/OE pair (e.g. CME-style MDP binaries). Learn the *local* spec when you join; the engineering themes repeat.

Common engineering chores: map wire symbols → internal ids at session start, validate sequences, handle snapshots after gaps, and never allocate on the hot decode path (Chapter 15).

16.3.4 SBE and other compact encodings

SBE (Simple Binary Encoding) is a binary message schema used in some low-latency stacks (including FIX-world tooling). Think: generated codecs, fixed layouts, fewer parsing surprises than free-form text — still not a substitute for knowing the venue’s actual MD/OE contract. Other shops roll their own IDL → C++ structs. The interview point: **schema + codegen** beats ad-hoc memcpy folklore when messages evolve.

16.3.5 Crypto and modern HTTP APIs

Crypto venues (Section 16.7) often expose:

- **REST** — slow path: account, config, sometimes order entry.
- **WebSocket** — streaming MD and/or OE over a persistent connection; JSON or compact payloads depending on the venue.

Useful and ubiquitous, but **not** the same latency class as colocated UDP/binary exchange feeds. Design for reconnects, snapshot-after-gap, and rate limits; do not assume exchange-grade sequence semantics unless the docs promise them.

16.3.6 Orientation table

Family	Typical use	Engineering flavour
FIX (text)	Brokers, many OE/drop-copy links	Tags, sessions, resend; readable; heavier parse
Native binary MD	Exchange firehose (often multicast)	Structs, seq gaps, book rebuild
Native binary OE	Colocated / low-latency entry	Tight state machine, fast encode
SBE / schema binary	Low-latency messaging	Codegen codecs, versioned layouts
REST / WebSocket	Crypto and some retail APIs	JSON, reconnect, rate limits

Interview tip: name **MD vs OE**, then one protocol you actually shipped (or studied). Prefer “binary ITCH-like MD + session OE” over buzzword lists. Cross-ref: gateway role in Section 16.2; kernel/multicast path in Chapters 11–14.

16.4 Market Structure

Before pricing models: *who* trades, *where*, and what “a market” actually does. Keep this vocabulary ready for desk and systems interviews.

16.4.1 What is a financial market?

A **financial market** is a venue (or network of venues) where buyers and sellers agree prices for assets or contracts. Two jobs matter for engineers:

- **Liquidity** — how easily you can trade size near a fair price without moving the market much. Thin books $\Rightarrow$ larger slippage and worse fills.
- **Price discovery** — the process by which publicly observed prices (and books) incorporate information. Matching engines and continuous quoting are the machinery behind that.

No liquidity $\Rightarrow$ your strategy cannot exit; broken price discovery $\Rightarrow$ marks and risk are fiction.

16.4.2 Market participants

- **Retail** — individuals; usually small size, often via brokers' apps; rarely colocated.
- **Institutional investors** — asset managers, pensions, insurers; large tickets, care about cost of trading and benchmarks.
- **Hedge funds / prop firms** — absolute-return or firm-capital strategies; many hire quant developers for research *and* production systems (your ProMarket / HFT lane).
- **Market makers** — continuously post bid and ask, earn spread, manage inventory risk; often the "other side" of flow.
- **Exchanges / venues** — run matching, publish market data, set rules (fees, tick size, sessions).
- **Brokers** — route client orders to venues (or internalize); provide access, clearing introductions, and sometimes algo execution.

Interview tip: say which participant *you* built for (e.g. prop desk vs exchange) — incentives differ.

16.4.3 Exchange vs OTC

- **Exchange-traded:** standardized contracts, central matching (order book), clearing house, transparent feeds. Examples: listed equities, futures, many listed options.
- **OTC (over-the-counter):** bilaterally negotiated (or via a dealer platform), customized terms, **counterparty risk** unless cleared. Classic: many forwards, bespoke swaps, a large share of FX.

Same economic exposure can be exchange *or* OTC (e.g. forward vs future). Ops and eng change: protocols, credit, confirmation, and how you get a "book."

16.4.4 Major venues (orientation only)

Names interviewers expect you to place on the map — not memorize every product:

- **NYSE** — US cash equities; hybrid auction + continuous trading heritage; listed stocks.
- **NASDAQ** — US cash equities; electronic matching; many tech names; related venues/feeds in the US equity complex.
- **CME** (Chicago Mercantile Exchange Group) — futures and options on futures (rates, equity indices, FX, commodities, ...); sessionized products and rolls matter for data (Section 16.6).
- **Crypto exchanges** (e.g. Binance, Coinbase, Deribit for options) — often 24/7, venue-specific derivatives (perps); API and custody models differ from traditional CME/NYSE stacks (Section 16.7).

Your stack always binds to a **specific market-data and order-entry protocol** for a venue — “equities” or “FX” alone is not an interface (Section 16.3).

Interview tip: contrast **central limit order book** venues with **dealer / OTC** FX: connectivity and “what is the book?” change.

16.5 Asset Classes and Instruments

A map of *what* is traded. Pricing depth for equities options continues in Section 16.9; wires, rolls, and implied books are in Section 16.6.

16.5.1 Asset classes (tour sheet)

- **Stocks (equities)** — ownership shares; cash market + listed options/futures on single names or indices. Systems: exchange sessions and auctions; listed L2/L3 MD; corporate-action calendars matter for marks.
- **Bonds** — debt; price moves mainly with rates and credit; huge OTC component. Systems: dealer/RFQ more than a central book; reference data and curves dominate over nanosecond book updates.
- **ETFs** — exchange-traded funds; trade like stocks, hold a basket or strategy exposure. Systems: equity-like sessions and feeds, plus NAV / creation–redemption clocks that research bars must not ignore.
- **Commodities** — oil, metals, ags, ...; often futures-first. Systems: futures sessions, rolls, and delivery months drive symbol maps and MD shape (Section 16.6).
- **FX (foreign exchange)** — currency pairs; enormous OTC spot and forwards, plus exchange-traded FX futures. Systems: long/24×5 hours, ECN or dealer quotes rather than one CLOB; MD is quote/trade streams more than equity L3.

- **Crypto** — digital assets; spot, futures, perpetuals, options; 24/7 venues (Section 16.7). Systems: no equity close; WebSocket/REST reconnect and funding schedules are first-class product logic.

Interview tip: name the asset class, then one ops fact (session hours, OTC vs listed, or MD shape) — that is what the stack actually binds to.

16.5.2 Spot

A **spot** trade is for (near) **immediate** delivery of the asset at the agreed price — “cash” market, not a dated forward.

Settlement lag: many cash equities settle **T+1** or **T+2** (trade date + N business days) even though the price is “spot.” The lag is operational (clearing/settlement), not a forward contract by design.

Crypto spot is often effectively **T+0** on the venue (balance updates as soon as the trade matches) — still venue- and custody-dependent.

Interview tip: “spot” = economic immediacy; always ask what the *settlement calendar* is for that market.

16.5.3 Forward contracts

A **forward** locks in a price today for delivery at a future date T .

- Usually **OTC** and **customized** (size, date, underlying).
- **Counterparty risk:** you care whether the other side pays; may be mitigated by collateral or clearing.
- No exchange mark-to-market by default (unlike listed futures).

Fair forward level ties to spot, rates, and carry (see Section 16.10 for the equity-style $F = Se^{(r-q)(T-t)}$ intuition).

16.5.4 Futures

A **future** is the **exchange-traded**, standardized cousin of a forward: fixed contract specs, central clearing, and a public order book.

- **Margin** — post collateral, not the full notional.
- **Leverage** — small margin controls large notional (risk cuts both ways).
- **Expiration** — contracts die; desks **roll** into later months (data and strategy must know the front month).

Initial margin, maintenance margin, mark-to-market

Initial margin	Collateral required to <i>open</i> a futures position.
Maintenance margin	Floor; if your account equity falls below this after losses, you get a margin call (post more or reduce).
Mark-to-market (MtM)	Daily (or intraday) revaluation: gains/losses settled in cash against the clearing house. Keeps counterparty risk manageable.

MtM is why listed futures feel different from a quiet OTC forward: P&L hits the account continuously, not only at expiry.

16.5.5 Options (light intro)

An [option](#) is the right (not obligation) to buy ([call](#)) or sell ([put](#)) an underlying at a [strike](#) K by/at an [expiration](#). You pay a [premium](#) up front.

Listed options are exchange-traded with a chain of strikes and expiries; OTC options can be bespoke. Full pricing path (European call/put, Black–Scholes, greeks) starts in Section 16.9.

16.5.6 Swaps (panorama)

A [swap](#) exchanges cash-flow streams. Know the names; deep pricing is a rates/credit book, not this chapter's core:

- [Interest rate swap \(IRS\)](#) — typically fixed vs floating rate payments on a notional.
- [Currency / FX swap](#) — exchange principals and/or interest in two currencies (related to FX forwards).
- [Credit default swap \(CDS\)](#) — protection against a credit event; premium vs contingent payoff (systems notes in Section 16.23).

Mostly OTC historically; much is now cleared. Eng impact: confirmations, curves, and lifecycle events — not a simple L2 equity book.

16.5.7 Why derivatives exist

- [Hedging](#) — transfer unwanted risk (e.g. airline hedges jet fuel).
- [Speculation](#) — express a view with defined payoff / leverage.
- [Leverage](#) — futures/options embed gearing via margin or premium.
- [Price discovery](#) — futures and options markets reveal forwards and implied vol used across the industry.

16.5.8 Spot vs futures — interview staple

Why trade futures instead of spot?

- **Leverage and capital efficiency** via margin vs full notional cash.
- **Shorting / expressing views** without locating stock or holding cash FX balances the same way.
- **Standardization and liquidity** on major contracts; easy roll of exposure.
- **Clearing** reduces bilateral counterparty risk vs many OTC forwards.
- Operational: no (or different) physical/cash settlement until expiry; daily MtM.

Trade-offs: basis vs spot, roll cost, expiry pin risk, and session/roll data complexity (Section 16.6). FX spot shops (e.g. prop FX) often stay in continuous OTC/ECN spot because that *is* the product they trade — futures are an alternative listing, not a universal upgrade.

Interview tip: answer with **capital, shorting, clearing, and basis** — then mention roll/-expiry if they push on operations.

16.6 Spot vs Futures Infrastructure

Section 16.5 compared the *contracts*. Here the question is what you **build and operate**: sessions, settlement, rolls, and (for futures) **implied** liquidity that the exchange synthesizes across contract months.

16.6.1 Spot workflows

Cash / FX spot systems emphasize **continuous connectivity** and fast path from quote to fill to position:

- **Trading** — stream prices (ECN / dealer / venue), decide, send orders; often 24/5 or long hours for FX.
- **Clearing / confirmation** — trade confirmations with counterparties or the platform; breaks show up as ops tickets, not only as strategy bugs.
- **Settlement** — move cash/currency on the settlement calendar (T+0 / T+1 / T+2). Engineering still cares: failed settlement is a P&L and credit event even if matching was correct.

FX spot / prop angle: at a firm like ProMarket the product *is* continuous FX spot (and related OTC). The hot path is market data + routing + risk on always-on sessions — not roll calendars. Latency budgets, sequence gaps, and failover (e.g. Binary Star-style redundancy) dominate over “which expiry is front.”

16.6.2 Futures workflows

Listed futures add calendar structure your code must own:

- **Expiration** — each contract has a last trade / delivery date; after that the symbol is dead.
- **Rolling** — keep exposure by closing the near contract and opening a further one (calendar spread). Strategies and risk need a **roll schedule**; naive “always trade the ticker string” breaks at roll.
- **Sessionized data** — many futures markets have defined open, close, auctions, and halt rules. Timestamps, volume profiles, and “session bars” depend on the venue calendar (CME Globex vs pit heritage, etc.). Replay and research pipelines must use the *same* session definition as production.

Interview tip: if you say you built futures MD, expect “how do you handle roll?” and “what is your session clock?”

16.6.3 Implied orders and synthetic liquidity

On many futures complexes the exchange does not only show outright bids/asks per month. It also publishes **implied** (synthetic) liquidity: a bid or offer in one contract that is *derived* by combining resting orders in related contracts (e.g. calendar spreads + another outright). Traders see deeper effective books; the matching engine may fill you against a combination of legs.

For a market-data handler this means:

- Messages may carry **outright** levels and **implied** levels (sometimes separate channels or flags).
- Your local book must decide: show implieds to strategy, keep them in a side structure, or ignore them for BBO-only strategies — but **never confuse** implied qty with firm resting qty if risk cares.
- Book updates can touch **multiple instruments** from one economic event; update order and atomicity matter.

Parsing efficiently in C++

Handbook sketch (venue-specific details vary):

- Decode once into a flat event (`instrument_id`, `side`, `px`, `qty`, `flags`: outright vs implied, `seq`). Avoid stringly-typed months on the hot path — map `symbol` → `int` at session start.
- Keep per-instrument books contiguous (tick ladder or flat map); implied overlays as a second layer or tagged levels so a depth walk can skip them.

- Prefer `preallocated` pools; no `std::string` per tick (Chapters 8, 15).
- If one packet updates several legs, apply in venue-defined order under a single sequence check — gap $\Rightarrow$ snapshot, not silent desync.

Interview tip: “implies are exchange-generated synthetic liquidity across months; my book tags them so strategy and risk know what is real resting interest.”

16.7 Crypto Markets (Compact)

Same microstructure ideas (books, makers, latency) — different ops envelope. This is orientation for interviews, not a trading manual.

- **Spot** — buy/sell the coin vs quote currency (or stablecoin); balances update on the venue, often effectively T+0 (Section 16.5.2). Custody and withdrawal rules are part of risk.
- **Futures** — dated contracts (more like traditional futures); expiry and delivery/settlement rules are venue-specific.
- **Perpetual futures (“perps”)** — no expiry; exposure stays open until you close. To keep the perp near spot, venues use a `funding rate`: longs and shorts periodically pay each other depending on mark vs spot. Funding is a first-class P&L line, not a footnote.
- **Options** — calls/puts on crypto underlyings (e.g. Deribit); chains can be large; pricing still uses vol surfaces, but underlying trades 24/7.
- **Liquidations** — leveraged positions forced closed when margin is exhausted. Cascades move spot and perps together; MD and risk must handle bursty cancel/fill storms.
- **24/7 trading** — no neat equity “close.” Sessions for research bars are *your* convention; maintenance windows and venue incidents replace the closing auction.

Eng takeaways: reconnect/gap logic never sleeps; clocks and funding schedules are product logic; perps $\neq$ CME-style dated futures (Section 16.6).

Interview tip: explain funding in one sentence and that liquidations are a mechanical flow your book will see in the tape.

16.8 Microstructure Essentials

Interview vocabulary for the book and the order. Data-structure depth: Chapter 17. Feed reconstruction drill: Section 16.27.

16.8.1 Bid, ask, mid, spread, liquidity

- **Bid** — highest price buyers will pay (best bid = top of buy side).
- **Ask** (offer) — lowest price sellers will take.
- **Mid** — often $(\text{best bid} + \text{best ask}) / 2$; a mark when you need a single number (not a tradable price).
- **Spread** — best ask – best bid; cost of crossing / tightness of the touch.
- **Liquidity** — size available near a fair price without moving the market much (depth + resilience). Thin book $\Rightarrow$ slippage.

Healthy continuous books usually have best bid < best ask. Crossed/locked books are a feed, race, or auction-state smell (Chapter 17).

16.8.2 L1 / L2 / L3 and matching

- **L1** — top of book / BBO (+ often last trade).
- **L2** — depth by price (aggregated size per level).
- **L3** — each resting order id (full queue).

“L3 feed” and “L3 storage” are not the same bill. The feed is bandwidth and decode CPU on every add/cancel/execute; the in-memory book is pointers, queues, and by_id maps that grow with resting interest. L1 is a few fields per instrument — cheap to store and update. L3 pays more CPU per message and far more memory/cache pressure to keep every order id and FIFO queue intact.

Matching on most equity/futures books is **price-time priority**: better price first; at the same price, earlier resting orders first. Your L3 model must preserve FIFO at a level if you care about queue position — shuffle that order and fill-probability math (and any queue-join strategy) is fiction.

Interview tip: say what you store (BBO vs depth vs order ids), then what it costs on the tick path vs what price-time forces you to preserve.

16.8.3 Order types and time-in-force

Before the menu: **make** vs **take**. A resting limit that adds liquidity is a make (you join the book); an order that crosses the spread and hits someone else’s resting interest is a take. Venues usually charge different **fees** (or pay rebates) for each — so the same economic trade can be cheap or expensive depending on whether you rest or cross. Time-in-force decides whether leftover size may rest (GTC) or must die immediately (IOC/FOK).

Market	Take liquidity now; walk the book; price uncertain.
Limit	Rest (or take if marketable) at a limit price; you control worst price.
Stop	Becomes a market order when a trigger price trades/quotes through.
Stop-limit	Becomes a <i>limit</i> order at a limit price once triggered.
IOC (immediate-or-cancel)	Fill what you can now; cancel the rest — no rest.
FOK (fill-or-kill)	Full size now or cancel entirely — all-or-nothing.
GTC (good-till-cancel)	Rest until filled or explicitly canceled (venue rules / session ends may still cancel).

A marketable limit or market order is a take *before* anything rests; a non-marketable limit is a make *after* it lands in the book. IOC/FOK never become makers on the remainder. Venue names and exact stop semantics differ — say what *your* gateway encoded.

Interview tip: “make = rest / rebate side; take = cross / fee side; TIF decides whether leftovers can rest.”

16.8.4 Market-data shapes (recap)

Trades vs quotes; snapshot then incremental updates; sequence numbers for gaps. Full Q&A in Section 16.27.

The failure modes are mechanical. Miss a snapshot and start mid-stream $\Rightarrow$ ghost levels. Apply incrementals after a sequence gap without rebuild $\Rightarrow$ a book that looks live but is wrong. Drop trades while keeping quotes (or the reverse) $\Rightarrow$ marks and markouts diverge from the tape. Core-engineer interviews often jump from here straight into: “show me your book update path” and “what do you do on a gap?”

Interview tip: define BBO/spread in one breath, then name the order types you actually sent in production (e.g. limit + IOC), not the whole menu.

16.9 Stocks and Options in Plain Language

Instrument taxonomy is in Section 16.5; this section deepens the equity option story used for Black–Scholes.

16.9.1 Stock

A stock (share) is a piece of ownership in a company. Its spot price S is what the market pays *right now* for one share. S moves over time — that randomness is what makes derivatives non-trivial.

16.9.2 Derivative / option

A derivative is a contract whose payoff depends on something else (the underlying, usually the stock). The simplest running example is a European call option:

- Today you agree on a **strike** K and an **expiry** date T .
- At T , if the stock is above K , the call pays $(S_T - K)$; otherwise it pays 0.
- In symbols: $\text{payoff} = \max(S_T - K, 0)$.

A **European put** pays $\max(K - S_T, 0)$ — you gain if the stock *falls* below K . “European” means you only settle at T , not before.

16.9.3 Why anyone buys an option

- **Insurance:** a put limits losses if you own stock and the market crashes.
- **Leverage:** a call gives exposure to upside with limited upfront cost (the **premium**, i.e. the option’s price today).
- **Trading / hedging:** desks replicate payoffs, trade volatility, or hedge other books — your job as quant dev is to **price** and **risk-manage** these contracts in code.

Key idea: the option has a **value today**, $V(S, t)$, even though the payoff is uncertain. Pricing = finding a fair V given S , K , T , and model assumptions.

16.10 Interest Rates, Dividends, and Forward Price

Desks rarely trade “spot plus a story about interest”: they quote a **forward** — the fair price to deliver the stock at T agreed today. The rate r is the cost of financing cash; the yield q is what you give up by not holding the stock; together they pin that forward so cash-and-carry arguments close. The symbols below are just that bookkeeping made precise.

- r — **risk-free rate** (annualized). Money in the bank grows as e^{rt} ; discounting future cash by $e^{-r(T-t)}$ is the continuous analogue of “\$1 today vs \$1 in six months.”
- q — **dividend yield**. Shareholders receive dividends; holding stock is slightly less attractive than holding cash at r , so the *effective* drift of S in no-arbitrage pricing uses $(r - q)$, not r alone.
- **Forward price** (simplified): $F = S e^{(r-q)(T-t)}$ is the fair locked-in price to deliver stock at T agreed today. Many desks think in forwards rather than spots.

Interview tip: you do not need fixed-income expertise for a first pass — just know that e^{-rT} discounts and q shifts the fair drift.

16.11 Volatility

Volatility σ measures how much S fluctuates. In the models below, σ is the coefficient on the random noise (like a diffusion constant). It is quoted as a **decimal**: $\sigma = 0.20$ means “20% per year” in the standard annualized sense.

Higher $\sigma \Rightarrow$ more uncertainty $\Rightarrow$ an option (which pays off in the tails) is usually **worth more**. That is the economic content of **vega** (Section 16.16).

16.12 Modelling the Stock: Random Walk and GBM

Physics analogy: treat S_t like a stochastic process with drift and noise.

16.12.1 Discrete warm-up

Over a small time step Δt , a crude model is:

$$\frac{\Delta S}{S} \approx \mu \Delta t + \sigma \sqrt{\Delta t} \xi, \quad \xi \sim \mathcal{N}(0, 1).$$

μ is average relative return; σ controls the size of shocks. Taking $\Delta t \rightarrow 0$ gives **geometric Brownian motion** (GBM):

$$dS_t = \mu S_t dt + \sigma S_t dW_t, \quad (16.1)$$

with W_t standard Brownian motion. Because the noise multiplies S_t , S stays **positive** and is **log-normal** at fixed time (not Gaussian).

16.12.2 Risk-neutral measure (pricing only)

For **pricing**, we do not use your real-world μ (which is hard to estimate). No-arbitrage implies we can price as if, under a special probability measure $\mathbb{Q}$,

$$dS_t = (r - q) S_t dt + \sigma S_t dW_t^{\mathbb{Q}}. \quad (16.2)$$

In words: in the pricing world, the *expected* growth rate of S (adjusted for dividends) equals the risk-free rate. The discounted option value $e^{-rt} V_t$ is then a **martingale** — expected future payoff, discounted, equals today’s price. This is the finance version of “choose coordinates so the equation simplifies.”

16.13 Black–Scholes: Assumptions and the Big Idea

Black–Scholes (BS) [173] is the baseline model. It assumes:

- S follows GBM with **constant** σ .
- You can trade S continuously, no fees, borrow/lend at rate r .
- No jumps, no smile built in (vol same for all strikes).

The big idea (replication): if you hold the option and continuously adjust a position in the stock, you can **cancel the leading risk** of dS . The remaining portfolio must earn the risk-free rate (else arbitrage). That balance *forces* a PDE on $V(S, t)$ — you are not guessing a price; you are solving for the unique price consistent with no arbitrage + hedging.

16.14 The Black–Scholes PDE

Let $V(S, t)$ be the option value. **Itô’s lemma** (chain rule with dW) gives how dV depends on dS and dt . A portfolio “long 1 option, short Δ shares” can be made insensitive to dS to first order by choosing $\Delta = \partial V / \partial S$. No-arbitrage then yields:

$$\frac{\partial V}{\partial t} + (r - q) S \frac{\partial V}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - r V = 0. \quad (16.3)$$

Read it like a physics PDE: time derivative + drift in S + diffusion in S ($\frac{1}{2} \sigma^2 S^2 \partial_{SS}$) + discount term $-rV = 0$. **Terminal condition** at T : $V(S, T) = \max(S - K, 0)$ for a call. Solve backward in time from T to today.

16.15 Closed-Form Price (European Call and Put)

BS gives an explicit solution (no need to solve the PDE numerically for Europeans). Define:

$$d_1 = \frac{\ln(S/K) + (r - q + \frac{1}{2} \sigma^2)(T - t)}{\sigma \sqrt{T - t}}, \quad d_2 = d_1 - \sigma \sqrt{T - t}. \quad (16.4)$$

$N(x)$ = standard normal CDF, $\phi(x)$ = PDF. Then:

$$C = S e^{-q(T-t)} N(d_1) - K e^{-r(T-t)} N(d_2) \quad (\text{call}), \quad (16.5)$$

$$P = K e^{-r(T-t)} N(-d_2) - S e^{-q(T-t)} N(-d_1) \quad (\text{put}). \quad (16.6)$$

Rough intuition:

- $N(d_2)$ is (roughly) the risk-neutral probability the call finishes in the money.
- $N(d_1)$ weights the stock leg of the replicating portfolio (the Δ hedge).
- $e^{-r(T-t)}$ and $e^{-q(T-t)}$ discount strike and spot contributions.

Interview tip: you do not need to derive d_1, d_2 — know they encode **moneyiness** (S/K), **time to expiry**, **rates**, and **vol**.

16.15.1 Put–Call Parity

For the same K, T [175]:

$$C - P = S e^{-q(T-t)} - K e^{-r(T-t)}. \quad (16.7)$$

In words: “long call + short put” replicates a forward on the stock. If this fails in your pricer, you have a bug or inconsistent inputs.

16.16 The Greeks: Sensitivities of the Price

Greeks [174] are partial derivatives of V — exactly like sensitivity of an observable to parameters in physics.

Name	Symbol	Meaning in one sentence
Delta	$\Delta = \partial V / \partial S$	How much option value moves when S moves \$1 (hedge ratio in shares).
Gamma	$\Gamma = \partial^2 V / \partial S^2$	How fast Δ changes when S moves (curvature / convexity).
Vega	$\nu = \partial V / \partial \sigma$	Sensitivity to vol; long options usually have $\nu > 0$.
Theta	$\Theta = \partial V / \partial t$	Time decay; long vanilla usually loses value as T approaches.
Rho	$\rho = \partial V / \partial r$	Sensitivity to interest rates (often smaller for short-dated equity options).

For a European **call** under BS:

$$\Delta = e^{-q(T-t)} N(d_1), \quad \Gamma = \frac{e^{-q(T-t)} \phi(d_1)}{S \sigma \sqrt{T-t}}, \quad (16.8)$$

$$\nu = S e^{-q(T-t)} \phi(d_1) \sqrt{T-t}. \quad (16.9)$$

Units trap: $\sigma = 0.20$ is 20%; “vega per vol *point*” often means $\nu/100$ (per 1 percentage point).

16.16.1 Delta hedging

If you are **short** one call (you sold it), you are exposed to S going up. Hold Δ shares long: for small moves, option loss $\approx$ stock gain. **Delta-neutral** = net Δ of book is zero. You re-hedge when S moves (discrete hedging) — real P&L differs from the idealized continuous model.

16.16.2 Gamma and theta (long option)

Long a call: $\Gamma > 0$ (convexity — you gain more from up moves than you lose from down moves, to second order), but $\Theta < 0$ (you *pay* for that convexity via time decay). **Short** option: opposite signs — you collect theta but suffer if S moves a lot (gamma risk). Near expiry at-the-money, Γ spikes — hedging is hardest there.

16.17 P&L Explain: Accounting for Daily Profit and Loss

Desks decompose one day’s P&L using a Taylor expansion — same logic as “ $\delta(\text{observable}) \approx (\partial O / \partial p) \delta p$ ”:

$$\Delta \text{PnL} \approx \Delta \delta S + \frac{1}{2} \Gamma (\delta S)^2 + \nu \delta \sigma + \Theta \delta t + \rho \delta r + \text{residual}. \quad (16.10)$$

Concrete reading:

- $\Delta \delta S$ — P&L from the stock moving (delta explain).
- $\frac{1}{2} \Gamma (\delta S)^2$ — correction when the move was large (gamma explain).
- $\nu \delta \sigma$ — P&L from implied vol changing (vol explain).
- $\Theta \delta t$ — P&L from one day passing (time decay).
- **Residual** — anything left: bad data, model mismatch, cross terms, discrete hedging, dividends not in model, etc.

Interview tip: “We lost \$2M; \$1.6M is delta on a -3% move, \$300k gamma, \$100k vega when skew steepened; \$50k unexplained — we should check surface build.”

16.18 Implied Volatility and the Smile

BS uses one number σ for all strikes. **Markets do not:** if you take each traded option price and ask “what σ makes BS match?”, you get **implied volatility** $\sigma_{\text{imp}}(K, T)$ — typically a **smile/skew** [176] (equity puts often have higher implied vol than calls at same expiry).

In plain terms: BS is a wrong model, but σ_{imp} is a **useful coordinate** for quoting options (like using action angle variables when the true Hamiltonian is messy). Production stores a **vol surface**, not one scalar σ .

16.19 Beyond Vanilla (Short Glossary)

Read these once; depth can come later. Interviewers use the names to probe whether you know *why* the vanilla BS story is incomplete.

- **Realized vol** — measured from past returns; **implied vol** — inferred from option prices. Shows up to test that you do not confuse a historical statistic with a market quote coordinate.
- **Exotic** — non-vanilla payoff (barrier, Asian average, digital); often priced by Monte Carlo or PDE on a grid. Shows up when they ask how you price path dependence once closed forms die.
- **American** — can exercise before T ; worth at least European; no vanilla BS closed form — see Section 16.23.2. Classic trap: “just use BS” fails; early exercise needs a free boundary.
- **Vanna / volga** — cross and second derivatives w.r.t. S and σ ; matter on large joint moves. Desk language for higher-order risk when spot and vol move together.

- **Bid-ask / order book** — where theoretical price meets tradable price (Part II networking chapters). Reminds you that a theo mid is not a fill; microstructure sits between.

Interview tip: one sentence per term is enough — name the gap in the vanilla story each term plugs.

16.20 Computing Greeks in Code

Two approaches:

- **Analytic:** plug into closed forms above (fast for BS).
- **Bump:** reprice with $S \rightarrow S + h$, $\sigma \rightarrow \sigma + \varepsilon$; finite difference (works for any pricer):

$$\Delta \approx \frac{V(S+h) - V(S-h)}{2h}, \quad v \approx \frac{V(\sigma + \varepsilon) - V(\sigma - \varepsilon)}{2\varepsilon}. \quad (16.11)$$

Choose h like any numerical derivative: not too small (roundoff) or too large (truncation).

Minimal C++ pricer sketch

```

1  #include <cmath>
2
3  struct BsParams { double S, K, r, q, sigma, T; };
4
5  inline double norm_cdf(double x) {
6      return 0.5 * std::erfc(-x / std::sqrt(2.0));
7  }
8
9  double bs_call(const BsParams& p) {
10     const double sqT = std::sqrt(p.T);
11     const double d1 = (std::log(p.S / p.K) + (p.r - p.q + 0.5 * p
        .sigma * p.sigma) * p.T)
12         / (p.sigma * sqT);
13     const double d2 = d1 - p.sigma * sqT;
14     return p.S * std::exp(-p.q * p.T) * norm_cdf(d1)
15         - p.K * std::exp(-p.r * p.T) * norm_cdf(d2);
16 }
17
18 double bump_delta(const BsParams& p, double h = 1e-4 * p.S) {
19     BsParams up = p; up.S += h;
20     BsParams dn = p; dn.S -= h;
21     return (bs_call(up) - bs_call(dn)) / (2.0 * h);
22 }
```

Use `std::erfc` [178] for stable $N(x)$ in the tails.

16.21 Implementation Pitfalls

These are not style nits — they are how silent P&L and CI failures are born.

- **Near expiry ATM:** greeks blow up ($\Gamma, \nu \rightarrow \text{large as } T \rightarrow 0$ at the money). Failure story: risk shows enormous hedges overnight; traders over-hedge; morning P&L explain is nonsense. Use limits, switch to intrinsic, or freeze greeks in a guarded near-expiry regime.
- **Units:** vol decimal vs percent; rates; day-count conventions. Failure story: feed stores 20 for “20%”, pricer expects 0.20 — prices look “a bit off” until the desk loses real money. Same class of bug for ACT/365 vs ACT/360 on short rates.
- **Put–call parity:** automated test on every build. Failure story: a refactor breaks the put formula; calls still match BS tables; CI stays green if you only golden-test calls — parity catches the silent half.
- **Determinism:** Monte Carlo greeks need fixed seeds for regression tests. Failure story: unseeded MC makes bump greeks wander; CI flakes red/green; worse, a NaN from a bad path ($S \leq 0$, log of junk) propagates into risk as quiet NaN marks until someone notices unexplained residual.

Interview tip: pick one pitfall and narrate the *desk* symptom (wrong hedge, red CI, NaN mark) — not only the math edge case.

16.22 Monte Carlo and PDE (When BS Is Not Enough)

Monte Carlo [177]: simulate many paths of S , average discounted payoffs — works for path-dependent exotics; error $\propto 1/\sqrt{N}$.

PDE / finite differences: discretize Eq. (16.3) on a grid in (S, t) ; needed for Americans and some barriers. As quant dev you choose method + validate against known cases (BS, parity). American early-exercise intuition: Section 16.23.2.

16.23 Complex Derivatives: Systems Notes

Pricing theory for vanilla Europeans is above. Here: what changes when the book is swaps, Americans, or *tens of thousands* of listed options — the interview angle at places like Optiver / Citadel Securities style shops.

16.23.1 IRS and CDS (systems view)

Still not a rates textbook — know the *objects* your code touches.

- **Interest rate swap (IRS):** exchange fixed vs floating cash flows on a notional. Valuation needs a **yield curve** (discount factors / forward rates), schedules (payment

dates, day-count), and often a clearing / collateral story. Eng work: curve build from instruments, schedule generators, PV and DV01 (sensitivity to rate bumps), lifecycle (fixings, payments, compressions).

- **Credit default swap (CDS)**: buyer pays a spread; seller pays on a credit event (default, etc.). Needs hazard / credit curves, recovery assumptions, and careful event calendars. Eng work: reference-entity master data, curve bootstrapping, and rare but critical event handling.

Compared with an equity L2 book: more **reference data and schedules**, fewer nanosecond book updates — but bad curve or date bugs still blow P&L.

16.23.2 American options

American contracts can be exercised *any time* up to expiry (or on discrete exercise dates). European BS closed forms do **not** apply in general: the holder chooses optimally, which adds a free-boundary / early-exercise premium.

Common numerical routes (you already named them in Section 16.22):

- **Binomial / tree**: roll backward; at each node take $\max(\text{exercise}, \text{continuation})$.
- **PDE / finite difference**: same BS operator with an inequality / obstacle condition (complementarity) for early exercise.
- Longstaff–Schwartz–style MC for some path-dependent Americans — heavier.

Validate against European ($\text{American} \geq \text{European}$) and known put cases. As quant dev: pick method, freeze seeds/grids, regression-test parity and early-exercise boundary sanity.

16.23.3 Streaming greeks on a large options universe

A single underlying can have **thousands** of strikes $\times$ expiries. Risk may want greeks for 10^4 – 10^5 contracts as underlyings and vols tick. Interviewers care how you avoid melting the cache and the allocator.

- **Instrument table**: dense contiguous array indexed by `option_id` (or (`expiry`, `strike`, `right`) packed key) — not a deep tree of `std::map` per tick.
- **Hot fields together**: store $\Delta, \Gamma, \nu, \Theta$ (and maybe bid/ask theo) in SoA or tight AoS so a full scan touches few cache lines (Chapter 9).
- **Invalidate narrowly**: on an underlying tick, reprice / rebucket only the chain for that underlying (or only near-ATM + large gamma), not the whole firm inventory. Vol-surface bumps invalidate by expiry slice.
- **Two cadences**: ultra-hot path may carry cached greeks + first-order update ($\Delta \delta S$); full recalibration on a slower thread or after N ticks / timer.

- **No alloc on the greek storm:** pre-size buffers; bump or analytic into preallocated output; publish via SPSC to risk (Chapters 8, 6).

Interview tip: “I keep a dense per-underlying chain, SoA greeks, narrow invalidation on *S* and surface slices, and a slow full rebuild — so a spot tick does not thrash 50k independent heap objects.”

16.24 Production Architecture (Lite)

A production stack is a staged pipeline, not a single “pricer” binary. Walk it left to right and name what each stage removes — and what it costs.

- **Market data** — normalize ticks, gaps, and reference data into a firm clock. **Noise removed:** raw venue quirks and stale symbols. **Cost:** decode CPU, sequence recovery, snapshot rebuilds.
- **Book / marks** — L1–L3 (or quote) state the strategy and risk actually read. **Noise removed:** wire-level event soup. **Cost:** memory and cache traffic; wrong book ⇒ wrong signal.
- **Curves / surfaces** — rates, dividends, vol surface snapshots versioned with the MD clock. **Noise removed:** mixing today’s spot with yesterday’s smile. **Cost:** build/-calibration latency; versioning storage so you can replay “what did we think at 10:00?”
- **Signal / pricer** — theo, greeks, or alpha from book + surfaces. **Noise removed:** ad-hoc spreadsheet numerics on the hot path. **Cost:** CPU per tick; must stay deterministic and allocation-thin.
- **Risk** — aggregate exposure, limits, kill-switches before and after intent. **Noise removed:** unchecked size and forbidden instruments. **Cost:** must stay cheap enough not to dominate tick-to-trade.
- **Order entry (OE)** — encode and send; handle acks/rejects/fills. **Noise removed:** strategy talking protocol dialects directly. **Cost:** session state, retries, and partial-fill bookkeeping.

For listed options at scale see Section 16.23.3. See Chapter 15 for hot-path engineering and Chapter 6 for thread-safe shared state.

Interview tip: draw MD → book → signal → risk → OE, then pick *one* stage you owned and its failure mode (gap, stale surface, limit reject).

16.25 Testing and Python/C++ Split

Python wins research: notebooks, plotting, quick surface experiments, and reference pricers you can change in minutes. **C++** wins the hot path: fixed latency, no GIL, and

deterministic regression under load. The split works only if both sides share the *same* numerics contract — thin bindings, not two forever-divergent implementations.

Golden-test what desks notice when wrong: put–call parity, analytic vs bump greeks (within a tight tolerance), and edge cases ($T \rightarrow 0$, deep OTM/ITM, zero vol). Freeze seeds for MC; freeze input snapshots for replay. A green research notebook is not a substitute for a C++ regression that fails the build when parity breaks.

Interview tip: “Python to explore and golden; C++ to ship and guard — same formulas, stricter latency and determinism in prod.”

16.26 Interview Cheat Sheet (Read Last)

After reading the chapter once, rehearse these aloud:

- **Lifecycle:** strategy $\rightarrow$ risk $\rightarrow$ gateway $\rightarrow$ venue/match $\rightarrow$ exec $\rightarrow$ MD $\rightarrow$ position $\rightarrow$ P&L (Section 16.2).
- **Wire protocols:** MD vs OE; FIX session + NewOrderSingle / ExecutionReport; native binary (ITCH/OUCH-style); crypto REST/WebSocket; text FIX $\neq$ colocated hot path (Section 16.3).
- **Market structure:** liquidity + price discovery; participants; exchange vs OTC; name a venue you actually connected to (Section 16.4).
- **Instruments:** spot vs forward vs future vs option vs swap; margins/MtM; why derivatives; why futures instead of spot (Section 16.5).
- **Spot/futures infra:** continuous FX spot vs rolls/sessions; implied liquidity and tagged book updates (Section 16.6).
- **Crypto:** spot / dated futures / perps + funding; liquidations; 24/7 ops (Section 16.7).
- **Microstructure:** bid/ask/mid/spread; L1–L3; price–time; market/limit/stop/IOC/-FOK/GTC (Section 16.8; book layout in Chapter 17).
- **Complex derivatives (systems):** IRS/CDS objects; American numerics; dense chains + narrow greek invalidation (Section 16.23).
- **Option:** right to receive $\max(S_T - K, 0)$ at T ; priced by no-arbitrage + hedging, not by guessing μ .
- **BS:** GBM, constant σ , replicate with Δ shares $\Rightarrow$ PDE $\Rightarrow$ closed form with $N(d_1), N(d_2)$.
- **Greeks:** ∂V w.r.t. S, σ, t, r ; delta hedge; long option = +gamma, –theta usually.
- **Smile:** one σ is wrong; desks use surfaces; implied vol is a quote coordinate.
- **P&L explain:** Taylor in $\delta S, \delta \sigma, \delta t$; residual = bug or model gap.
- **Dev:** test parity, mind units, bump vs analytic, deterministic MC.

16.27 Typical Interview Questions

Short answers you can expand on a whiteboard. For order-book data structures in more depth, see also Chapter 17 (order book section).

16.27.1 Markets and instruments

Exchange-traded vs OTC?

Exchange: standardized, central matching, clearing, public book/feeds. OTC: bilateral or dealer, often customized, counterparty risk unless cleared (Section 16.4.3).

Why would someone trade futures instead of spot?

Capital efficiency (margin), easier short/levered exposure, liquidity on major contracts, clearing — at the cost of basis, rolls, and expiry (Section 16.5.8).

Initial vs maintenance margin? What is mark-to-market?

Initial to open; maintenance is the floor that triggers margin calls; MtM settles daily P&L with the clearing house (Section 16.5.4).

What breaks in futures market data that spot FX often does not have?

Contract **rolls**, **session** calendars, and **implied** (synthetic) book levels across months — tag implieds, map symbols to ids, and define the session clock explicitly (Section 16.6).

What is a perpetual futures contract? What is funding?

A leveraged futures-like product **without expiry**. The **funding rate** is a periodic payment between longs and shorts that pulls the perp mark toward spot (Section 16.7).

16.27.2 Market data and the order book

What is the difference between Level 1, Level 2, and Level 3 market data?

- **Level 1 (L1) / top of book**: best available prices only. Typically best bid price/size, best ask price/size, and often last trade price/volume. You see the **spread** and the last print, but *not* book depth.
- **Level 2 (L2) / market depth**: many price levels with **aggregated** size at each price (not every order id). Example:

Ask		Bid	
101.30	20	101.27	15
101.29	35	101.26	40
101.28	50	101.25	60

From L2 you can read depth, liquidity, bid/ask [imbalance](#), and short-horizon pressure. Many HFT strategies need at least L2; queue-position strategies may need L3.

- **Level 3 (L3) / order-by-order:** individual orders with ids. Instead of 101.25 → 300, you see order 1452 buy 100, 1891 buy 50, 2110 buy 150, and events New/Modify/Cancel/Execution. Full [limit order book](#) reconstruction and price–time queues are possible; heaviest bandwidth and bookkeeping.

Naming varies by venue/vendor; say what *your* feed actually contains.

What information is contained in a market data feed?

Depends on the feed. Common pieces:

- **Order events:** new / modify / cancel / replace (L3) or level updates (L2).
- **Trade events:** execution price and size.
- **Quotes:** resting interest or aggregated BBO (an intention, not a fill).
- **Market state:** halt, auction, open, close.
- **Timestamps:** exchange / matching / send — critical for HFT (often μ s or ns resolution, venue-dependent).
- **Sequence numbers:** gap detection and replay.

Options/futures feeds may add extra fields (e.g. open interest) — venue-dependent.

What is the difference between trades and quotes?

A [quote](#) is an *intention*: e.g. buy 100 @ 100.20 or sell 200 @ 100.25 — resting (or displayed) interest, not necessarily a fill. It may arrive as full orders (L3) or as aggregated BBO/level updates. A [trade](#) is an *execution*: e.g. 50 @ 100.23 — someone bought and someone sold. **Short:** quotes $\approx$ interest in the book (or top-of-book updates); trades $\approx$ prints. Strategies use quotes for shape and trades for markouts.

How would you reconstruct an order book from incremental updates?

Modern feeds rarely blast the full book every time; they send an [event stream](#).

1. Start from an initial [snapshot](#) (or empty book if the protocol allows).
2. Apply sequenced messages locally: Add (add liquidity), Modify (size/price), Cancel (remove), Execution/Trade (reduce or delete the resting order that was hit).
3. Keep invariants: no empty levels; BBO consistent; detect [sequence gaps](#) and re-subscribe/snapshot.

Incremental updates are far cheaper than continuous full snapshots. Never assume cross-feed wall-clock order without sequence checks.

Why would an HFT strategy need Level 2 instead of Level 1?

L1 only shows the touch. Two books can share the [same best ask](#) and look identical on L1 while liquidity behind the touch is completely different:

Ask A		Ask B	
101.00	5	101.00	500
101.01	800	101.01	5
101.02	900	101.02	5

L2 unlocks depth, how far a take walks the book, and signals such as [order-book imbalance](#), liquidity, pressure, support/resistance style levels, and [microprice](#). Many quantitative short-horizon strategies use exactly these.

How would you represent an order book in memory?

State the hot query first (BBO vs deep walk vs cancel-by-id). A common L3 sketch:

```
1 // bids: highest price first; asks: lowest first
2 std::map<Price, PriceLevel, std::greater<>> > bids;
3 std::map<Price, PriceLevel> asks;
4 // PriceLevel: price, total volume, queue of orders (price-time)
5 std::unordered_map<OrderId, Order*> by_id; // O(1)
   modify/cancel
```

Each price level holds the resting orders at that price; `by_id` makes `modify/cancel` fast. In real HFT stacks, generic `std::map` is often avoided when the tick grid or price range is bounded: prefer [cache-friendly](#) layouts (tick-indexed arrays, custom trees, pooled nodes) to cut allocations and improve locality. See Chapter 17.

What is price-time priority?

At the same price, earlier resting orders trade first. Your L3 book must preserve queue order if you model fill probability at the touch.

Market order vs limit order?

[Market](#): take liquidity now at whatever prices are available (walk the book). [Limit](#): rest at a price (or take if marketable). HFT cares about [make](#) vs [take](#), fees, and queue position. See also Section 16.8.3.

What are IOC, FOK, and GTC?

IOC: fill now, cancel remainder (no rest). **FOK**: full size now or nothing. **GTC**: rest until filled/canceled (venue may still expire at session end). Stops / stop-limits trigger into market or limit when a price is hit (Section 16.8.3).

Define mid and spread.

Mid $\approx$ (bid + ask)/2; spread = ask – bid. Liquidity is how much you can trade near mid without moving the market (Section 16.8.1).

16.27.3 Latency and systems (quant-dev)

What is FIX, and when do you use it?

FIX is a [tag=value](#) messaging standard with a session layer (logon, heartbeats, sequence recovery) and application messages such as NewOrderSingle and ExecutionReport (Section 16.3.2). Common for broker and many non-colocated order links. Colocated HFT against an exchange matching engine usually uses [native binary](#) MD/OE instead; say which hop you mean.

ITCH vs OUCH (or “binary MD vs binary OE”) — what is the split?

Same idea as MD vs OE (Section 16.3.1): an [ITCH](#)-style feed is a typed binary *market-data* stream (book/trade events, often multicast); an [OUCH](#)-style protocol is *order entry* (enter/cancel/replace + acks). Venues rename these; the split remains.

How is tick-to-trade latency composed?

Always define the path first. The industry default is [tick-to-trade](#): **market-data packet in** → **order packet out** toward the venue.

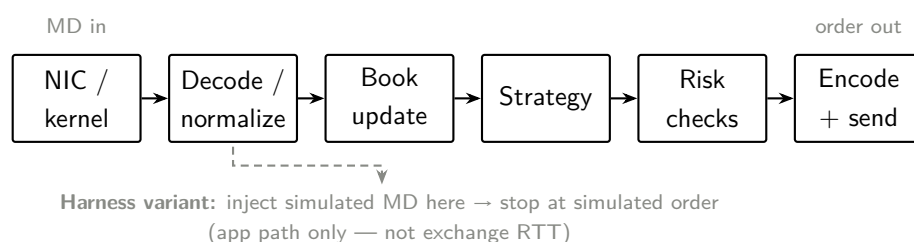


Figure 16.2: Software tick-to-trade path (and harness variant).

Budget buckets (whiteboard version):

Stage	What eats time
NIC / kernel	Interrupt or poll, sk_buff, copies, syscall; shrinks with busy-poll / bypass (Chapters 12, 14).
Decode / normalize	Parse wire format, instrument map, sequence checks.
Book update	Apply L2/L3 deltas; map/array walks; allocator traffic.
Strategy	Signal + decision; can dominate if heavy or blocked on locks.
Risk / pre-trade	Limits, fat-finger, position; must stay allocation-thin.
Encode + send	Serialize order, send/write; Nagle off on hot TCP.
Cross-cutting tax	Queues, context switches, cache misses / false sharing, NUMA remote, page faults, logging — often the <i>tail</i> , not the median.

What usually dominates?

- **Fat app:** book + strategy + risk + hidden allocations.
- **Lean app:** kernel path, copies, cache/NUMA, NIC — optimize the wire side (Part II) once the software path is already thin.

Interview line: give a number only with a path (“local process path ~1–10 μs is plausible; exchange RTT is another story”).

What about inject → simulated order (harness latency)?

Lab metrics often start at **controlled injection** of market data and stop when the stack emits a **simulated order** (no venue ACK). That measures the **application path** under load and is valid if you *name* it — it is *not* full tick-to-trade to the exchange. Use the same harness before/after each optimization (pinning, NUMA split, lock-free queues, lean serialization, monotonic clocks). Production still needs separate clocks on the real MD-in / order-out boundaries.

Median vs tail — why HFT cares about p99?

Median (p50) shows the happy path. Tails (p99/p999) come from interrupts, stealing, allocator locks, cold caches, page faults, disk logs, and noisy neighbors. A strategy can look fine on average and still lose on the slow prints. Optimize for **predictability**, not only average.

Where does latency come from on a tick-to-trade path?

See the composition above (Figure 16.2). Short answer: NIC/kernel → decode → book → strategy → risk → encode/send, plus jitter taxes. Book + strategy + risk often dominate until the app is lean.

How would you measure software latency?

- Define start/end events on a **named path**.
- Steady clock: `CLOCK_MONOTONIC_RAW` (or equivalent) — not `gettimeofday` for intervals.
- Report median *and* tails; same hardware affinity before/after.
- Separate **harness / app path** from **exchange RTT**.

Why pin threads and care about NUMA?

Keep hot threads on dedicated cores; keep memory and NIC on the **same node** as the consumer to avoid remote DRAM and cross-QPI/UPI traffic (Chapters 9, 10).

How do you pass opaque payloads in C++?

Be explicit about the layer: hot-path FFI often uses `void* + size` (or a byte buffer + tag). Library design may use type erasure (`std::any`, `std::function`, or a small virtual/SOP eraser). Ask which layer they mean.

16.27.4 Pricing, risk, and greeks

What is put-call parity?

For European options with the same K, T : $C - P = Se^{-q(T-t)} - Ke^{-r(T-t)}$. Violation $\Rightarrow$ arb or a pricer bug.

What is delta hedging in one sentence?

Hold $-\Delta$ shares per short option (signs flip with position) so small moves in S cancel to first order; rebalance as Δ changes (gamma).

What is implied volatility?

The σ that makes the BS price match the market price. One number per option; across strikes you get a [smile/skew](#), so production uses a surface, not one global σ .

Analytic vs bump greeks?

Analytic: closed-form ∂V under BS — fast, model-tied. Bump: reprice at $S \pm h$ or $\sigma \pm \varepsilon$ — works for any pricer, careful with step size and noise.

How do you explain daily P&L?

Taylor: $\Delta \delta S + \frac{1}{2} \Gamma (\delta S)^2 + \nu \delta \sigma + \Theta \delta t + \dots$ plus residual (data/model/hedge error).

Why is there no Black-Scholes closed form for Americans?

Early exercise is an optimal-control decision; the price solves a free-boundary problem. Use trees or PDE with an exercise obstacle; American $\geq$ European (Section 16.23.2).

How do you stream greeks for tens of thousands of options?

Dense per-underlying tables, contiguous greek storage, [narrow invalidation](#) on spot/vol-slice changes, optional slow full rebuild — avoid per-tick heap churn across the whole universe (Section 16.23.3).

16.27.5 Behavioural / role

Why move from industrial C++ (e.g. semis) back to trading systems?

Tight feedback loop: ship, measure latency/correctness, iterate with the desk. Ownership of a hot path and measurable impact — not month-long silicon cycles.

What would you ask us?

Team size and greenfield vs reuse; what you own on the critical path; how latency is measured in prod; on-call expectations; what success looks like in six months.

Quant Basics for Developers Bibliography

- [170] John C. Hull. *Options, Futures, and Other Derivatives*. Pearson, 2021.
- [171] Steven E. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer, 2004.
- [172] Paul Wilmott. *Paul Wilmott on Quantitative Finance*. Wiley, 2006.
- [173] *Black–Scholes model*. [Wikipedia](#).
- [174] *Greeks (finance)*. [Wikipedia](#).
- [175] *Put–call parity*. [Wikipedia](#).
- [176] *Volatility smile*. [Wikipedia](#).
- [177] *Monte Carlo methods for option pricing*. [Wikipedia](#).
- [178] `std::erfc`. [Cplusplusreference](#).
- [179] *Financial Information eXchange*. [Wikipedia](#).
- [180] *What is FIX?* [FIX Trading Community](#).



PART

Algorithms

17

CHAPTER

Algorithms

17.1 Time Complexity

Time complexity for an algorithm has three different facets

- The *worst-case* complexity of the algorithm is the function defined by the maximum number of steps taken in any instance of size n .
- The *best-case* complexity of the algorithm is the function defined by the minimum number of steps taken in any instance of size n .
- The *average-case* complexity or expected time of the algorithm, which is the function defined by the average number of steps over all instances of size n .

17.1.1 Amortized vs average-case

These two phrases sound alike and are **not** the same.

- **Average-case / expected** (e.g. `unordered_map::find`): the bound is over a *distribution of inputs* (or a random hash). With a good hash, most lookups cost constant time; a bad collision pattern can make one lookup $\Theta(n)$. The *expected* cost per op is $O(1)$; the *worst* single op is $O(n)$. See Chapter 5.
- **Amortized** (e.g. `vector::push_back`): the bound is over a *sequence of operations* on one structure. One `push_back` may reallocate and cost $\Theta(n)$; most cost $\Theta(1)$.

With geometric growth, n pushes cost $O(n)$ *total*, so the **amortized** cost per push is $O(1)$. A single push can still be $\Theta(n)$ worst-case.

Interview tip: say which kind you mean. “find is expected $O(1)$, worst $O(n)$ ” is about hashing; “push_back is amortized $O(1)$ ” is about spreading rare reallocations across many cheap inserts. Monotonic stack / Union-Find “amortized $O(1)$ per index” uses the same idea: each element is charged a constant number of expensive pops over the whole run.

17.1.2 Big O notation

There exist constants $c > 0$ and n_0 such that

$$f(n) = O(g(n)) \quad \text{if} \quad 0 \leq f(n) \leq c g(n) \quad \forall n \geq n_0. \quad (17.1)$$

For all values $n \geq n_0$, $f(n)$ is on or below $c g(n)$.

A constant multiple of $g(n)$ is an *asymptotic upper bound* for $f(n)$.

17.1.3 Omega notation

There exist constants $c > 0$ and n_0 such that

$$f(n) = \Omega(g(n)) \quad \text{if} \quad 0 \leq c g(n) \leq f(n) \quad \forall n \geq n_0. \quad (17.2)$$

For all values $n \geq n_0$, $f(n)$ is on or above $c g(n)$.

A constant multiple of $g(n)$ is an *asymptotic lower bound* for $f(n)$.

17.1.4 Theta notation

There exist constants $c_1, c_2 > 0$ and n_0 such that

$$f(n) = \Theta(g(n)) \quad \text{if} \quad 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall n \geq n_0. \quad (17.3)$$

For all values $n \geq n_0$, $f(n)$ equals $g(n)$ up to constant factors.

$g(n)$ is an *asymptotically tight bound* for $f(n)$.

17.1.5 Meaning of previous notations - Insertion sort case

Big O notation

describes an *upper bound*. When we use it to *bound the worst-case running time* of an algorithm, we have a bound on the running time of the algorithm on *every input*.

Thus, the $O(n^2)$ bound on worst-case running time of *insertion sort* also applies to its running time on every input.

Omega notation

describes a *lower bound*. When we say that the running time (no modifier) of an algorithm is $\Omega(g(n))$, we mean that *no matter what particular input of size n is chosen* for each value of n , the running time on that input is at least a constant times $g(n)$, for sufficiently large n .

Equivalently, we are giving a *lower bound on the best-case running time* of an algorithm. For example, the best-case running time of insertion sort is $\Omega(n)$, which implies that the running time of insertion sort is $\Omega(n)$.

The *running time of insertion sort therefore belongs to both $\Omega(n)$ and $O(n^2)$* , since it falls anywhere between a linear function of n and a quadratic function of n . Moreover, these bounds are asymptotically as tight as possible: for instance, the running time of insertion sort is not $\Omega(n^2)$, since there exists an input for which insertion sort runs in $\Theta(n)$

time (e.g., when the input is already sorted). It is not contradictory, however, to say that the worst-case running time of insertion sort is $\Omega(n^2)$, since there exists an input that causes the algorithm to take $\Omega(n^2)$ time.

Theta notation

describes a *tight bound*. The $\Theta(n^2)$ notation bound on the worst-case running time of insertion sort, however, does not imply a $\Theta(n^2)$ bound on the running time of insertion sort on every input. For example, when the input is already sorted, insertion sort runs in $\Theta(n)$ time. Technically, it is an abuse to say that the running time of insertion sort is $O(n^2)$, since for a given n , the actual running time varies, depending on the particular input of size n . When we say “the running time is $O(n^2)$ ”, we mean that there is a function $f(n)$ that is $O(n^2)$ such that for any value of n , no matter what particular input of size n is chosen, the running time on that input is bounded from above by the value $f(n)$. Equivalently, we mean that the worst-case running time is $O(n^2)$.

17.1.6 Running times of well known algorithms

Algorithm	Best	Average	Worst	Extra space
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ avg stack; $O(n)$ worst
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Timsort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$ aux. (in-place)
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Shell Sort	gap-dep. [†]	gap-dep. [†]	$\leq O(n^2)$ [†]	$O(1)$
Bucket Sort	$O(n + k)^*$	$O(n + k)^*$	$O(n^2)$ [‡]	$O(n + k)$
Radix Sort	$O(d(n + r))$	$O(d(n + r))$	$O(d(n + r))$	$O(n + r)$

*Bucket: roughly uniform keys, k buckets; also depends on per-bucket sort. [‡]Worst when nearly all keys fall in one bucket sorted quadratically. [†]Shellsort: no universal closed form — depends on the gap sequence; poor gaps stay $O(n^2)$. Radix: d = digits/passes, r = radix (digit range).

Verbal description of the previous sorting algorithms:

- **Quicksort**

Efficient general-purpose sorting algorithm. On **randomized** or typical data it is often **faster in practice** than mergesort/heapsort thanks to locality and small constants — that is an empirical claim, not a theorem. Worst-case time is $O(n^2)$ (e.g. adversarial pivots); randomized or median-of-three pivots make that rare.

It selects a **pivot** and **partitions** the other elements into **two sub-arrays** (less / greater than the pivot) — hence **partition-exchange sort**. Sub-arrays are then sorted recursively. Stack / recursion depth is $O(\log n)$ on balanced partitions and $O(n)$ in the degenerate case.

- **Mergesort**

Divide: split the array into two halves and sort each half recursively until runs of length one. **Conquer**: **merge** two already-sorted halves into one sorted run by walking both with two pointers. Stable; guaranteed $O(n \log n)$ time; needs $\Theta(n)$ extra buffer for a standard merge.

- **Timsort**

A hybrid, **stable** sort derived from **mergesort** and **insertion sort**, designed for real-world partially ordered data. It finds already-ordered **runs** and merges them under stack invariants. Implemented by Tim Peters (2002) for CPython; it remains the algorithm behind Python's `list.sort` / `sorted` (including current 3.x).

- **Heapsort**

Comparison-based sort that can be seen as **selection sort on a heap**: grow a sorted

region by repeatedly extracting the max from a max-heap over the unsorted region. Unlike naive selection sort, finding the max is $O(\log n)$ via the heap, not a linear scan. In-place with $O(1)$ auxiliary space; not stable.

- **Bubble Sort**

Repeatedly passes through the list, comparing adjacent elements and swapping if out of order, until a pass makes no swaps (list sorted). Best case $O(n)$ with an early-exit flag; average/worst $O(n^2)$.

- **Insertion Sort**

Grows a sorted prefix: each step takes the next unsorted element and inserts it into the correct place in the sorted prefix (shifting larger elements). Excellent on nearly sorted data and small n ; used inside Timsort for short runs.

- **Selection Sort**

Repeatedly finds the smallest remaining unsorted element and swaps it into the next position of the sorted prefix. Always $\Theta(n^2)$ comparisons; few writes; not stable in the usual swap form.

- **Shell Sort**

Insertion sort on interleaved subarrays with decreasing gap sizes (e.g. $n/2, n/4, \dots, 1$). Early large gaps move elements far; the last gap-1 pass is ordinary insertion sort on a nearly sorted array. Complexity depends on the gap sequence: some sequences give $o(n^2)$ worst case; poor sequences remain $O(n^2)$. No single universal average-case formula — quote the sequence when you quote a bound.

- **Bucket Sort**

Scatter keys into k buckets by range, sort each bucket (often insertion sort or recursion), then concatenate. A distribution sort (related to pigeonhole / radix ideas). Best when keys are roughly uniform; if almost all keys land in one bucket and that bucket is sorted quadratically, time falls to $O(n^2)$. Comparisons appear only inside buckets if you use a comparison sort there — bucket sort itself is not “just” a comparison sort.

- **Radix Sort**

Sort integers (or fixed-length keys) digit by digit (LSD or MSD), typically with a stable counting pass per digit. Time $O(d(n+r))$ for d digits and radix r (e.g. $r = 256$ for bytes). Not a comparison sort; excellent when d is small relative to $\log n$.

17.1.7 Methods for solving recurrences

See [181]. Divide-and-conquer running times often satisfy a recurrence $T(n) = aT(n/b) + f(n)$ (plus a base case). Three classic ways to solve them:

Substitution

Guess a closed form, then prove it by induction (strengthen the induction hypothesis if additive lower-order terms get in the way). Example — binary search: $T(n) = T(\lfloor n/2 \rfloor) + \Theta(1)$. Guess $T(n) \leq c \log n$ for large enough c ; the inductive step is immediate once the base case and c absorb the $\Theta(1)$ work. Result: $T(n) = \Theta(\log n)$.

Recursion tree

Draw the tree of recursive calls; sum the non-recursive cost $f(\cdot)$ at each level, then sum over levels (plus leaves if needed). Example — mergesort: $T(n) = 2T(n/2) + \Theta(n)$. Each level costs $\Theta(n)$ in total; there are $\Theta(\log n)$ levels until size-1 leaves — so $T(n) = \Theta(n \log n)$. Good for building intuition before a formal proof.

Master theorem

For $T(n) = aT(n/b) + f(n)$ with $a \geq 1$, $b > 1$, compare $f(n)$ to $n^{\log_b a}$ (let $c_{\text{crit}} = \log_b a$):

- **Case 1:** $f(n) = O(n^{c_{\text{crit}} - \varepsilon})$ for some $\varepsilon > 0 \Rightarrow T(n) = \Theta(n^{c_{\text{crit}}})$ (leaves dominate).
- **Case 2:** $f(n) = \Theta(n^{c_{\text{crit}}}) \Rightarrow T(n) = \Theta(n^{c_{\text{crit}}} \log n)$ (levels cost about the same; $\Theta(\log n)$ of them).
- **Case 3:** $f(n) = \Omega(n^{c_{\text{crit}} + \varepsilon})$ and a regularity condition on $f \Rightarrow T(n) = \Theta(f(n))$ (root work dominates).

Quick checks: mergesort $a=2$, $b=2$, $f(n)=\Theta(n)$ is Case 2 $\Rightarrow \Theta(n \log n)$; binary search $a=1$, $b=2$, $f(n)=\Theta(1)$ is Case 2 with $c_{\text{crit}}=0 \Rightarrow \Theta(\log n)$. Not every recurrence fits — when a, b vary or f is awkward, fall back to substitution or the tree.

17.1.8 Comparison-sort lower bound

In the algebraic decision-tree / comparison model, any correct comparison-based sort needs $\Omega(n \log n)$ comparisons in the worst case (there are $n!$ permutations; each comparison has two outcomes, so the decision tree height is $\Omega(\log(n!)) = \Omega(n \log n)$ by Stirling). Hence mergesort and heapsort are **asymptotically optimal** among comparison sorts; radix/bucket can beat $n \log n$ only by using the *structure of the keys*, not comparisons alone.

17.2 Space Complexity

Space complexity counts memory used as a function of input size n . Distinguish:

- **Input space** — the array/structure you are given.
- **Auxiliary (extra) space** — stack frames, buffers, heaps beyond the input.
- **Total space** — input + auxiliary.

Examples: in-place heapsort / insertion sort use $O(1)$ auxiliary space; standard merge-sort uses $\Theta(n)$ auxiliary for the merge buffer; quicksort's auxiliary cost is mainly the **call stack** ($O(\log n)$ typical, $O(n)$ worst). For interviews, say whether you mean auxiliary or total, and whether recursion counts.

17.3 Order Book

An **order book** stores resting interest: buy and sell orders waiting to trade. Exchanges and many trading systems publish incremental updates (add / modify / cancel / trade); your process rebuilds a local book and reads **best bid/offer** (BBO) and depth for decisions.

Interview tip: focus on data layout, price-time priority, and hot-path cost — not exchange matching-engine internals.

17.3.1 Price Levels and Market Depth

Orders at the **same price** aggregate into a **price level**: typically price + total quantity (+ optional order count or per-order queue).

Market depth is how many levels you keep on each side (top of book only vs N levels vs full book). In market-data vocabulary that lines up roughly with **L1** (top of book / BBO), **L2** (depth by price: aggregated quantity per level), and **L3** (depth by order: each resting order id). Exact feed labels vary by venue; you can also subscribe to L2/L3 and still **store** only L1 locally. Deeper books cost more memory, more update work, and worse cache behavior; many strategies only need top-of-book or a small depth window.

17.3.2 Bids and Asks

- **Bid** — buy interest. Best bid = **highest** buy price.
- **Ask** (offer) — sell interest. Best ask = **lowest** sell price.

A healthy book has best bid < best ask. The difference is the **spread**; midpoint is often $(\text{best bid} + \text{best ask})/2$. Crossed or locked books ($\text{bid} \geq \text{ask}$) usually mean a bad feed, race, or auction state — worth detecting in production.

17.3.3 Data Structures

See Chapter 5 for container APIs. This subsection compares order-book-specific representations only.

Feed level $\neq$ local storage: you can subscribe to L2 or L3 and still keep only L1 (or a shallow depth window) in memory — store what the strategy reads on the hot path.

You need: fast **find level by price**, fast **BBO**, and (often) walk levels in price order. Common choices:

- **Ordered map** (`std::map` / `flat map`): price $\rightarrow$ level. Logarithmic update; natural ordered iteration. Simple; often fine for whiteboard sketches and moderate update rates.
- **Hash map + sorted structure**: `unordered_map` for price $\rightarrow$ level, plus a heap or side tree for best price. Faster random price lookup; more moving parts to keep consistent.
- **Price-indexed array / sparse ladder**: if the instrument has a discrete tick grid and a bounded price range, index by tick. Best-case $O(1)$ level access; careful with range and memory.
- **Per-level order queue**: if you need individual order ids (modify/cancel by id), each level holds a FIFO list (price-time priority). Aggregate-only books store quantity per price and skip per-order lists.

Pointer chasing vs flat layout

Node-based structures (`std::map`, linked lists, pointer-heavy trees) store each level or order in a separately allocated node. Walking the book then **pointer-chases**: load a node, follow `left/next`, miss cache, repeat. Under a burst of depth updates or a strategy that scans several levels, those misses dominate — not the $O(\log n)$ compare count on the whiteboard.

Noise removed by flat, preallocated contiguous storage: random node hops on the hot path. A **tick-indexed ladder** (or dense vector of levels over a bounded price window) keeps neighbouring prices in neighbouring cache lines so hardware prefetch helps. Pair with pools for L3 order nodes if you still need per-id queues (Section 15.4; Chapter 9).

Cost / when a map is fine: a sparse book over a huge price range wastes RAM on a dense ladder; whiteboard sketches and moderate update rates often keep `std::map`. Choose the layout for the instrument's tick grid and the strategy's read pattern (BBO only vs deep walk), not as a universal rule.

```

1 // Sketch: bids as qty[tick], tick = price / tick_size (bounded
   window).
2 constexpr int kTicks = 4096;
3 alignas(64) int64_t bid_qty[kTicks]{}; // preallocated; index =
   px_to_tick(px)
4 // BBO: track best_tick explicitly on each update --- do not scan
   4096 slots.
```

Interview tip: “pointer chasing vs contiguous ladder” — name the miss, then the bound on price range that makes the array legal.

Aggregate book (L2-style storage)

Quantity (and optional order count) per price — no per-order queues. Matches an L2 feed, or an L3 feed that you collapse locally.

```

1 #include <cstdint>
```

```

2  #include <map>
3
4  struct Level {
5      int64_t qty{0};
6      uint32_t orders{0};
7  };
8
9  // Bids: highest price first. Asks: lowest price first.
10 std::map<int64_t, Level, std::greater<int64_t>> bids;
11 std::map<int64_t, Level, std::less<int64_t>> asks;

```

Order-by-order book (L3-style storage)

Per-level FIFO for price–time priority, plus an id index for $O(1)$ modify/cancel.

```

1  #include <cstdint>
2  #include <deque>
3  #include <map>
4  #include <unordered_map>
5
6  using Price    = int64_t;
7  using OrderId  = uint64_t;
8
9  struct Order {
10     OrderId id{};
11     Price   px{};
12     int64_t qty{0};
13 };
14
15 struct PriceLevel {
16     int64_t total_qty{0};
17     std::deque<Order> queue; // price-time FIFO at this price
18 };
19
20 std::map<Price, PriceLevel, std::greater<>> bids;
21 std::map<Price, PriceLevel> asks;
22 std::unordered_map<OrderId, Order*> by_id;

```

Interview tip: say [what](#) you optimize (BBO vs deep book vs cancel-by-id) before picking the structure. In real HFT stacks, generic `std::map` is often replaced by tick-indexed arrays or pooled trees when the price range is bounded (Chapter 16).

17.3.4 Add, Update, and Remove

The feed does not hand you a finished book — it streams [deltas](#). Your hot path is applying those deltas correctly and cheaply. Wrong apply logic is silent corruption (ghost levels, wrong BBO); expensive apply logic is jitter under burst.

Feed handlers typically apply:

- [Add](#) — new order at a price (create level or increase qty / enqueue order).

- **Modify** / replace — change qty or price (often cancel+add semantics).
- **Cancel** / delete — remove qty or whole order; drop empty levels.
- **Trade** / execute — reduce resting qty when a fill hits the book (depending on whether the feed already netted that into cancel/modify).

Invariants to keep every tick: no empty levels left in the maps; BBO cache invalidated or updated when the top level changes; order-id index consistent if you track individual orders. A missed empty-level cleanup is a classic “phantom liquidity” bug.

Noise removed by treating modify as cancel+add when the venue semantics allow it: one code path, fewer special cases. **Cost:** you must still match the venue’s exact rules (in-place qty change vs new priority).

Sequence numbers / gaps: market-data feeds are often UDP; detect gaps and rebuild from snapshot (Chapter 11). Applying increments after a gap without a resync is how books drift from the venue.

Interview tip: walk one add that creates a new best bid, then a cancel that restores the old BBO — and say which caches you touch.

17.3.5 Best Bid and Offer

BBO = best bid price/qty and best ask price/qty. Hot strategies read this every tick. Keep it as an explicit cached pair updated on each top-of-book change rather than re-scanning the book.

```

1 struct BBO {
2     int64_t bid_px{0}, bid_qty{0};
3     int64_t ask_px{0}, ask_qty{0};
4 };
5
6 BBO top(const std::map<int64_t, Level, std::greater<int64_t>>&
7         bids,
8         const std::map<int64_t, Level, std::less<int64_t>>& asks)
9 {
10     BBO b{};
11     if (!bids.empty()) {
12         b.bid_px = bids.begin()->first;
13         b.bid_qty = bids.begin()->second.qty;
14     }
15     if (!asks.empty()) {
16         b.ask_px = asks.begin()->first;
17         b.ask_qty = asks.begin()->second.qty;
18     }
19     return b;
20 }

```

17.3.6 Hot-Path Design

The book update loop sits on the critical path between packet and decision. Asymptotic “log n map” talk is not enough if the real noise is allocator locks, false sharing, or a mutex inside the decode loop.

Noise removed by the practices below: allocation jitter, cache-line ping-pong on shared book state, and scheduler wake-ups from contended locks.

- **No allocations** on the update path — pre-sized maps/pools, recycled nodes (Section 15.4).
- **Contiguous levels** and stable layout beat pointer-chasing lists when depth walks matter (see pointer chasing vs flat layout above; Chapter 9).
- **Single writer** for a given instrument book when possible (SPSC into the book thread — Section 8.3); avoid locking inside the decode loop.
- **NUMA / pinning**: build the book on the core that consumes it (Chapters 10, 15).
- Measure update latency and cache misses under realistic burst rates; asymptotic map cost is not enough if allocation jitter dominates.

Cost: more upfront sizing and ownership discipline; you give up casual multi-threaded mutation of one book. That trade is usually correct for tick-to-trade.

Interview tip: name the noise first (alloc, false share, lock), then the knob — not a laundry list of buzzwords.

Algorithms Bibliography

- [181] Thomas H. Cormen et al. *Introduction to Algorithms*. 3rd. MIT Press, 2009.
- [182] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley.
- [183] Steven S. Skiena. *The Algorithm Design Manual*. Springer.
- [184] *Quicksort*. [Wikipedia](#).
- [185] *Merge Sort*. [Wikipedia](#).
- [186] *Timsort*. [Wikipedia](#).
- [187] *Heapsort*. [Wikipedia](#).
- [188] *Bubble Sort*. [Wikipedia](#).
- [189] *Insertion Sort*. [Wikipedia](#).
- [190] *Selection Sort*. [Wikipedia](#).
- [191] *Shell Sort*. [Wikipedia](#).
- [192] *Bucket Sort*. [Wikipedia](#).
- [193] *Radix Sort*. [Wikipedia](#).

18

CHAPTER

Algorithmic Problem-Solving Patterns

See [195], [196], [194].

This chapter is a [pattern catalog](#) for coding interviews: recognize the shape, pick a recipe, sketch one clean solution. It complements Chapter 17 (asymptotics and the order book), it does not replace timed practice. Complexity vocabulary (*worst / average-expected / amortized*) is defined in Section 17.1; hash-table find costs are in Section 5.1.5.

How to use

- Learn the **trigger** (“I see intervals → sort + scan”).
- Know the **complexity** of the recipe you claim.
- Code one canonical sketch; variants come from drilling problems.

Track A (quant-dev / HFT screens — learn first) Arrays & hash maps, two pointers, sliding window, binary search, intervals, heaps / top- K , linked-list cycle / reverse, stack (matching).

Track B (full loops — after Track A) Trees, graphs, backtracking, DP.

18.1 Arrays and Hash Maps

Hash maps turn “have I seen this?” and “how often?” into [expected](#) $O(1)$ lookups (worst case per op $O(n)$ — Section 5.1.5). Prefer `unordered_map` / `unordered_set` when or-

der does not matter; use a plain array when the key domain is tiny (ASCII, digits, small enums).

18.1.1 Frequency map

Many problems ask about counts: anagrams, majority element, “unique frequencies”, “characters that appear exactly k times”. The recipe is to build a map from value to occurrence count in one pass, then either answer from the map or make a second pass that consults it.

Trigger: counts, anagrams, majority, unique frequency.

Complexity: expected $O(n)$ time with hashing (or $O(n)$ with a fixed array), $O(k)$ space (k = distinct keys).

```
1 // true iff s and t are anagrams
2 bool isAnagram(const std::string& s, const std::string& t) {
3     if (s.size() != t.size()) return false;
4     std::array<int, 26> freq{};
5     for (char c : s) ++freq[c - 'a'];
6     for (char c : t) if (--freq[c - 'a'] < 0) return false;
7     return true;
8 }
```

18.1.2 Lookup / complement

When you need a pair (or more) that satisfies a relation like “sum to target”, scan once and remember what you have already seen. For each new value x , ask whether the complement ($\text{target} - x$) is in the map. Storing the index lets you return positions, not only existence.

Trigger: Two Sum, “pair with target”, “seen before?”.

Complexity: expected $O(n)$ time, $O(n)$ space (each find/insert expected $O(1)$).

```
1 std::pair<int, int> twoSum(const std::vector<int>& a, int target)
2 {
3     std::unordered_map<int, int> seen; // value -> index
4     for (int i = 0; i < (int)a.size(); ++i) {
5         auto it = seen.find(target - a[i]);
6         if (it != seen.end()) return {it->second, i};
7         seen[a[i]] = i;
8     }
9     return {-1, -1};
10 }
```

18.1.3 Prefix sum

Range sums become $O(1)$ after an $O(n)$ prefix build: $\text{pref}[i]$ holds the sum of the first i elements (often 1-indexed or length $n+1$ with $\text{pref}[0] = 0$). Subarray sum equals k is the same idea plus a hash of prefix values seen so far.

Trigger: range sum queries, subarray sum equals k .

Complexity: $O(n)$ build, $O(1)$ per range query.

```
1  std::vector<long long> buildPref(const std::vector<int>& a) {
2      int n = (int)a.size();
3      std::vector<long long> pref(n + 1);
4      for (int i = 0; i < n; ++i) pref[i + 1] = pref[i] + a[i];
5      return pref;    // sum a[L..R] = pref[R+1] - pref[L]
6  }
7
8  // number of subarrays with sum == k
9  int subarraySum(const std::vector<int>& a, int k) {
10     std::unordered_map<long long, int> seen{{0, 1}};
11     long long pref = 0;
12     int ans = 0;
13     for (int x : a) {
14         pref += x;
15         ans += seen[pref - k];
16         ++seen[pref];
17     }
18     return ans;
19 }
```

18.1.4 Prefix / suffix aggregation

Some answers depend on “everything to the left” and “everything to the right” at each index (product except self, prefix/suffix maxima, best buy/sell around i). Build a left-to-right aggregate, then a right-to-left aggregate, and combine. With care you can overwrite one array in place and keep $O(1)$ extra memory.

Trigger: product except self, rain water, best-so-far from left/right.

Complexity: $O(n)$ time, $O(n)$ or $O(1)$ extra space.

```
1  std::vector<int> productExceptSelf(const std::vector<int>& a) {
2      int n = (int)a.size();
3      std::vector<int> out(n, 1);
4      for (int i = 1; i < n; ++i) out[i] = out[i - 1] * a[i - 1];
5      // left products
6      int right = 1;
7      for (int i = n - 1; i >= 0; --i) {
8          out[i] *= right;
9          right *= a[i];
10     }
11     return out;
12 }
```

18.2 Two Pointers

Two indices cooperate so each element is considered a constant number of times. On **sorted** data, this often reduces a brute-force pair scan from $O(n^2)$ to $O(n)$. If sorting is

required first, the total is usually $O(n \log n)$.

18.2.1 Opposite-direction pointers

Place L at the start and R at the end of a sorted array (or string). Move the side that can improve the invariant: if the sum is too small, advance L ; if too large, retreat R . The same skeleton works for container-with-most-water and two-sided palindrome checks.

Why it works: after sorting, if $a[L] + a[R] < \text{target}$, every pair that still uses $a[L]$ with an index $\leq R$ is also too small, so advancing L is safe. Symmetrically, if the sum is too large, every pair that still uses $a[R]$ with an index $\geq L$ is too large, so R can retreat. Ordering lets you discard whole sets of candidates without testing them one by one.

Trigger: sorted pair sum, container with most water, palindrome check.

Complexity: $O(n)$ after an optional $O(n \log n)$ sort.

```
1 bool hasPairSum(std::vector<int> a, int target) {
2     std::sort(a.begin(), a.end());
3     int l = 0, r = (int)a.size() - 1;
4     while (l < r) {
5         int s = a[l] + a[r];
6         if (s == target) return true;
7         if (s < target) ++l; else --r;
8     }
9     return false;
10 }
```

18.2.2 Same-direction pointers

Both pointers move forward. A common shape is read / write: read scans every element; write marks where the next kept element should go. Use this for in-place duplicate removal, partitioning, and stable “filter” passes without a second buffer.

Trigger: remove duplicates in place, partition, slow write index.

Complexity: $O(n)$ time, $O(1)$ extra space.

```
1 // remove duplicates from sorted array; return new length
2 int removeDuplicates(std::vector<int>& a) {
3     if (a.empty()) return 0;
4     int w = 1;
5     for (int r = 1; r < (int)a.size(); ++r)
6         if (a[r] != a[w - 1]) a[w++] = a[r];
7     return w;
8 }
```

18.2.3 Fast / slow pointers

Advance one pointer by one step and another by two. Meeting detects a cycle; the same speed relationship places slow near the middle when fast reaches the end. Full sketches (middle, Floyd cycle) live in Section 18.9.

Trigger: cycle in list, middle of list, happy number.

Complexity: $O(n)$ time, $O(1)$ space.

18.3 Sliding Window

A window is a contiguous segment $[L, R]$ whose aggregate you maintain incrementally. Expanding and shrinking by one index avoids rebuilding from scratch, so many string/array window problems become $O(n)$.

18.3.1 Fixed window

When every candidate subarray has the same length k , maintain the aggregate of the current block of k elements. Slide by subtracting the element that leaves and adding the element that enters.

Trigger: max/sum of every subarray of length k .

Complexity: $O(n)$ time, $O(1)$ extra space for sum/max of scalars.

```
1 int maxSumFixedWindow(const std::vector<int>& a, int k) {
2     long long sum = 0, best = LLONG_MIN;
3     for (int i = 0; i < (int)a.size(); ++i) {
4         sum += a[i];
5         if (i >= k) sum -= a[i - k];
6         if (i >= k - 1) best = std::max(best, sum);
7     }
8     return (int)best;
9 }
```

18.3.2 Variable window

Grow R freely; whenever the window becomes invalid (too many distinct characters, sum too large, missing a required character), advance L until it is valid again. Track the best length or the best concrete window along the way.

Why it works: each index enters and leaves the window at most once, so the total work is $O(n)$ (plus $O(1)$ or $O(|\Sigma|)$ state updates). You never rebuild the window from scratch.

Trigger: longest substring with $\leq k$ distinct, minimum covering window.

Complexity: $O(n)$ for typical char/int alphabets.

```
1 int longestWithAtMostKDistinct(const std::string& s, int k) {
2     std::unordered_map<char, int> freq;
3     int best = 0;
4     for (int r = 0, l = 0; r < (int)s.size(); ++r) {
5         ++freq[s[r]];
6         while ((int)freq.size() > k) {
7             if (--freq[s[l]] == 0) freq.erase(s[l]);
8             ++l;
9         }
10        best = std::max(best, r - l + 1);
11    }
```

```

11     }
12     return best;
13 }

```

18.3.3 Frequency map + window

Combine a sliding window with a frequency (or deficit) map when the validity condition is about counts: “contains an anagram of p ”, “permutation of need”, “all characters of t covered”. Maintain how many constraints are currently satisfied and update that score as characters enter and leave.

Trigger: anagrams in a string, permutation in string.

Complexity: $O(n)$ over a fixed alphabet.

```

1 // true if s2 contains a permutation of s1
2 bool checkInclusion(const std::string& s1, const std::string& s2)
3 {
4     if (s1.size() > s2.size()) return false;
5     std::array<int, 26> need{}, win{};
6     for (char c : s1) ++need[c - 'a'];
7     int m = (int)s1.size();
8     for (int i = 0; i < (int)s2.size(); ++i) {
9         ++win[s2[i] - 'a'];
10        if (i >= m) --win[s2[i - m] - 'a'];
11        if (i >= m - 1 && win == need) return true;
12    }
13    return false;
14 }

```

18.4 Stack and Monotonic Stack

A stack matches nested structure and, when kept monotonic, answers “next greater/smaller” queries in [amortized](#) $O(1)$ per index (each index is pushed and popped at most once; a single step may pop many, but the total over n indices is $O(n)$ — Section 17.1.1).

18.4.1 Matching / parsing

Push opening tokens; on a closer, pop and verify the match. The same idea simplifies paths ($\dots$), evaluates RPN, and validates nesting. Empty stack at the end (and never popping empty) is the usual success criterion.

Trigger: valid parentheses, path simplify, nested structures.

Complexity: $O(n)$ time and space.

```

1 bool isValid(const std::string& s) {
2     std::stack<char> st;
3     for (char c : s) {
4         if (c == '(' || c == '[' || c == '{') st.push(c);
5         else {

```



```

6         if (st.empty()) return false;
7         char o = st.top(); st.pop();
8         if ((c == ')', && o != '(') ||
9             (c == ']', && o != '[') ||
10            (c == '}', && o != '{')) return false;
11     }
12 }
13 return st.empty();
14 }

```

18.4.2 Next greater / smaller

Scan left to right (or right to left). While the stack's top is “beaten” by the current value, pop it and record the current index/value as its next greater (or smaller). Indices not resolved by the end have no next greater.

Trigger: daily temperatures, next greater element.

Complexity: $O(n)$ total / amortized $O(1)$ per index — each index pushed and popped at most once.

```

1 // ans[i] = days until a warmer temperature, or 0
2 std::vector<int> dailyTemperatures(const std::vector<int>& t) {
3     int n = (int)t.size();
4     std::vector<int> ans(n);
5     std::stack<int> st; // indices, temps strictly decreasing
6     for (int i = 0; i < n; ++i) {
7         while (!st.empty() && t[i] > t[st.top()]) {
8             int j = st.top(); st.pop();
9             ans[j] = i - j;
10        }
11        st.push(i);
12    }
13    return ans;
14 }

```

18.4.3 Monotonic stack

Keep indices whose values are strictly increasing or decreasing. When an index is popped, you know the first barrier to its left/right, which unlocks histogram area, contribution tricks, and range-minimum style answers without a segment tree.

Why it works: the stack invariant (e.g. strictly increasing heights) means the first smaller bar to the right is exactly when an index is popped; with the inequality used in the sketch, the new top identifies the nearest remaining candidate on the left. Amortized linearity follows because each index enters/leaves the stack once.

Trigger: largest rectangle in histogram, contribution of each bar.

Complexity: $O(n)$ total (amortized $O(1)$ per index).

```

1 int largestRectangleArea(std::vector<int> h) {
2     h.push_back(0); // sentinel flush

```

```

3     std::stack<int> st;
4     int best = 0;
5     for (int i = 0; i < (int)h.size(); ++i) {
6         while (!st.empty() && h[i] < h[st.top()]) {
7             int height = h[st.top()]; st.pop();
8             int left = st.empty() ? -1 : st.top();
9             best = std::max(best, height * (i - left - 1));
10        }
11        st.push(i);
12    }
13    return best;
14 }

```

18.5 Queue and Deque

Queues drive BFS. Deques that stay monotonic answer sliding-window extrema in $O(n)$.

18.5.1 BFS

Enqueue the start; while the queue is non-empty, pop front and push unseen neighbours. Mark visited when you *enqueue* (not when you pop) to avoid duplicate work.

Why it works: on an unweighted graph, edges all cost 1, so the queue explores in non-decreasing distance. The first time you reach a node is therefore a shortest path.

Trigger: shortest path in unweighted graph/grid, level-order tree.

Complexity: $O(V + E)$.

```

1  int shortestPathGrid(std::vector<std::vector<int>>& g) {
2      int n = (int)g.size(), m = (int)g[0].size();
3      if (g[0][0] || g[n-1][m-1]) return -1;
4      const int dr[] = {1,-1,0,0}, dc[] = {0,0,1,-1};
5      std::queue<std::pair<int,int>> q;
6      q.push({0, 0});
7      g[0][0] = 1; // mark visited / steps
8      while (!q.empty()) {
9          auto [r, c] = q.front(); q.pop();
10         if (r == n - 1 && c == m - 1) return g[r][c];
11         for (int k = 0; k < 4; ++k) {
12             int nr = r + dr[k], nc = c + dc[k];
13             if (nr < 0 || nc < 0 || nr >= n || nc >= m || g[nr][
14 nc]) continue;
15             g[nr][nc] = g[r][c] + 1;
16             q.push({nr, nc});
17         }
18     }
19     return -1;
20 }

```

18.5.2 Monotonic deque

Store indices in a deque so values are decreasing (for window maximum) or increasing (for minimum). The front is always the current extremum. Before pushing a new index, pop from the back every index that can no longer be the extremum once the new value is present.

Checklist (same algorithm for max or min): for each right endpoint, (1) drop indices that left the window from the front, (2) drop dominated candidates from the back, (3) push R , (4) read $a[dq.front()]$. Prefer this over a naive $O(nk)$ scan of each window.

Trigger: sliding-window maximum/minimum.

Complexity: $O(n)$ — each index enters/leaves the deque at most once (amortized $O(1)$ per index).

```
1  std::vector<int> maxSlidingWindow(const std::vector<int>& a, int
   k) {
2      std::deque<int> dq; // indices, a[] decreasing
3      std::vector<int> out;
4      for (int i = 0; i < (int)a.size(); ++i) {
5          if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
6          while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back()
7          ;
8          dq.push_back(i);
9          if (i >= k - 1) out.push_back(a[dq.front()]);
10     }
11     return out;
12 }
13 // min variant: keep a[] increasing (compare with >= when popping
   back)
14 std::vector<int> minSlidingWindow(const std::vector<int>& a, int
   k) {
15     std::deque<int> dq;
16     std::vector<int> out;
17     for (int i = 0; i < (int)a.size(); ++i) {
18         if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
19         while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back()
20         ;
21         dq.push_back(i);
22         if (i >= k - 1) out.push_back(a[dq.front()]);
23     }
24     return out;
25 }
```

18.6 Binary Search

On a monotonic predicate, binary search finds a boundary in $O(\log n)$ probes. Prefer half-open ranges $[lo, hi)$ and the STL bounds helpers when the predicate is “first position satisfying ...”.

18.6.1 Exact search

Classic search for a key in a sorted array: compare mid, discard half. In interviews, state whether the array may contain duplicates and what you return on a miss.

Trigger: find exact value in sorted data.

Complexity: $O(\log n)$.

```
1 int binarySearch(const std::vector<int>& a, int x) {
2     int lo = 0, hi = (int)a.size(); // [lo, hi)
3     while (lo < hi) {
4         int mid = lo + (hi - lo) / 2;
5         if (a[mid] == x) return mid;
6         if (a[mid] < x) lo = mid + 1;
7         else hi = mid;
8     }
9     return -1;
10 }
```

18.6.2 lower_bound / upper_bound

lower_bound is the first position $\geq x$; upper_bound is the first $> x$. Together they give the half-open equal range and the insert position. Prefer the standard algorithms over hand-rolled off-by-one bugs when allowed.

Trigger: first $\geq x$, first $> x$, insert position.

Complexity: $O(\log n)$.

```
1 int lowerBoundIdx(const std::vector<int>& a, int x) {
2     return (int)(std::lower_bound(a.begin(), a.end(), x) - a.
3     begin());
4 }
5 int upperBoundIdx(const std::vector<int>& a, int x) {
6     return (int)(std::upper_bound(a.begin(), a.end(), x) - a.
7     begin());
8 }
9 // count of value x: upperBoundIdx(a,x) - lowerBoundIdx(a,x)
```

18.6.3 First / last valid element

When duplicates exist, run a binary search for the first index where a predicate becomes true (e.g. $a[i] \geq x$), and optionally another for the last. This is the same lower/upper-bound idea written by hand.

Trigger: first and last position of a target in a sorted array.

Complexity: $O(\log n)$.

```
1 // first index with a[i] >= x, or n if none
2 int firstGe(const std::vector<int>& a, int x) {
3     int lo = 0, hi = (int)a.size();
4     while (lo < hi) {
5         int mid = lo + (hi - lo) / 2;
```

```

6         if (a[mid] < x) lo = mid + 1;
7         else             hi = mid;
8     }
9     return lo;
10 }

```

18.6.4 Binary search on answer

Sometimes the searchable space is not an index but a candidate answer (minimum possible max load, eating speed, split threshold). Binary-search the answer range; for each mid, run a linear feasibility check.

Why it works: correctness needs [monotonicity](#): if capacity mid is feasible, every larger capacity is too (or the symmetric “every smaller” form). That turns the answer domain into a sorted predicate so binary search is valid.

Trigger: minimize max load, koko eating bananas, split-array largest sum.

Complexity: $O(\text{check} \cdot \log \text{range})$.

```

1 // minimum capacity to ship packages within D days
2 int shipWithinDays(const std::vector<int>& w, int days) {
3     auto ok = [&](int cap) {
4         int d = 1, load = 0;
5         for (int x : w) {
6             if (load + x > cap) { ++d; load = 0; }
7             load += x;
8         }
9         return d <= days;
10    };
11    int lo = *std::max_element(w.begin(), w.end());
12    int hi = std::accumulate(w.begin(), w.end(), 0);
13    while (lo < hi) {
14        int mid = lo + (hi - lo) / 2;
15        if (ok(mid)) hi = mid; else lo = mid + 1;
16    }
17    return lo;
18 }

```

18.7 Sorting-Based Patterns

Sorting imposes order so a linear scan, two pointers, or a greedy choice becomes obvious. Pay $O(n \log n)$ once; the rest is usually $O(n)$.

18.7.1 Sort + scan

Sort, then walk once looking at neighbours or running aggregates: merge intervals, consecutive gap checks, unique pairs after sorting. State clearly what the sort key is.

Trigger: merge intervals, unique pairs, consecutive gaps.

Complexity: $O(n \log n)$.

```

1 int maxGapAfterSort(std::vector<int> a) {
2     if (a.size() < 2) return 0;
3     std::sort(a.begin(), a.end());
4     int best = 0;
5     for (int i = 1; i < (int)a.size(); ++i)
6         best = std::max(best, a[i] - a[i - 1]);
7     return best;
8 }

```

18.7.2 Sort + two pointers

After sorting, fix one index and run opposite pointers on the remainder (3Sum, 3Sum closest). Skip duplicates deliberately if the problem wants unique triplets.

Trigger: 3Sum, pair sums after sorting.

Complexity: $O(n^2)$ typical for 3Sum.

```

1 std::vector<std::vector<int>> threeSum(std::vector<int> a) {
2     std::sort(a.begin(), a.end());
3     std::vector<std::vector<int>> out;
4     int n = (int)a.size();
5     for (int i = 0; i < n; ++i) {
6         if (i && a[i] == a[i - 1]) continue;
7         int l = i + 1, r = n - 1;
8         while (l < r) {
9             int s = a[i] + a[l] + a[r];
10            if (s == 0) {
11                out.push_back({a[i], a[l], a[r]});
12                while (l < r && a[l] == a[l + 1]) ++l;
13                while (l < r && a[r] == a[r - 1]) --r;
14                ++l; --r;
15            } else if (s < 0) ++l;
16            else --r;
17        }
18    }
19    return out;
20 }

```

18.7.3 Custom comparator

Pass a lambda (or functor) to `std::sort` when the order is not the natural key order: sort by end time, by ratio, by concatenated string value. The comparator must define a strict weak ordering (irreflexive, transitive asymmetry) or you risk undefined behaviour.

Trigger: sort by second field, largest number arrangement, ratio order.

Complexity: $O(n \log n)$.

```

1 // arrange integers to form the largest number (as string)
2 std::string largestNumber(std::vector<int> nums) {
3     std::vector<std::string> s;

```

```

4     for (int x : nums) s.push_back(std::to_string(x));
5     std::sort(s.begin(), s.end(), [](const std::string& a, const
std::string& b) {
6         return a + b > b + a;
7     });
8     if (s[0] == "0") return "0";
9     std::string out;
10    for (auto& t : s) out += t;
11    return out;
12 }

```

18.7.4 Partial sorting / nth_element

When you only need the k -th order statistic (or the k largest unordered), full sort is wasteful. `std::nth_element` rearranges the range so the element at the requested position is in place, with all preceding elements $\leq$ it and all following elements $\geq$ it, average $O(n)$. A size- k heap is the alternative when streaming.

Trigger: k -th largest without full sort.

Complexity: average $O(n)$ for `nth_element`.

```

1 int findKthLargest(std::vector<int> a, int k) {
2     std::nth_element(a.begin(), a.end() - k, a.end());
3     return a[a.size() - k];
4 }

```

18.8 Intervals

Interval problems almost always start with sorting by start or by end. After that, a linear scan or a sweep over events finishes the job.

18.8.1 Merge intervals

Sort by start. Keep a running merged interval; if the next starts after the current end, append a new one; otherwise extend the end. This is the prototype “sort + scan” on ranges.

Trigger: merge overlapping ranges.

Complexity: $O(n \log n)$.

```

1 std::vector<std::pair<int,int>> merge(
2     std::vector<std::pair<int,int>> v) {
3     std::sort(v.begin(), v.end());
4     std::vector<std::pair<int,int>> out;
5     for (auto [s, e] : v) {
6         if (out.empty() || s > out.back().second) out.push_back({
s, e});
7         else out.back().second = std::max(out.back().second, e);
8     }

```

```

9     return out;
10 }

```

18.8.2 Overlap detection

To test whether any two intervals conflict (meeting rooms, resource exclusive use), sort by start and check each consecutive pair: overlap iff `next.start < prev.end` (adjust for closed/open conventions).

Trigger: can attend all meetings?

Complexity: $O(n \log n)$.

```

1 bool canAttendMeetings(std::vector<std::pair<int,int>> v) {
2     std::sort(v.begin(), v.end());
3     for (int i = 1; i < (int)v.size(); ++i)
4         if (v[i].first < v[i - 1].second) return false;
5     return true;
6 }

```

18.8.3 Interval scheduling

Maximum number of non-overlapping intervals: greedy by earliest finishing time. Sort by end; repeatedly take the next interval that starts at or after the last chosen end. This is optimal for the classic unweighted scheduling problem.

Trigger: max number of non-overlapping intervals.

Complexity: $O(n \log n)$.

```

1 int eraseOverlapCount(std::vector<std::pair<int,int>> v) {
2     // returns how many to remove to eliminate overlaps
3     if (v.empty()) return 0;
4     std::sort(v.begin(), v.end(),
5         [](auto& a, auto& b) { return a.second < b.second;
6     });
7     int kept = 1, end = v[0].second;
8     for (int i = 1; i < (int)v.size(); ++i) {
9         if (v[i].first >= end) { ++kept; end = v[i].second; }
10    }
11    return (int)v.size() - kept;

```

18.8.4 Sweep line

Turn intervals into events: +1 at start, -1 at end (process ends before starts at the same coordinate if you need open/closed care). Sort events and scan with a running counter for “how many active now”; max overlap, skyline variants follow the same skeleton.

Trigger: skyline, maximum overlapping intervals.

Complexity: $O(n \log n)$.


```

1 int maxOverlap(const std::vector<std::pair<int,int>>& v) {
2     std::vector<std::pair<int,int>> ev; // {time, delta}
3     for (auto [s, e] : v) {
4         ev.push_back({s, +1});
5         ev.push_back({e, -1}); // treat end as exclusive
6     }
7     std::sort(ev.begin(), ev.end());
8     int cur = 0, best = 0;
9     for (auto [t, d] : ev) {
10         cur += d;
11         best = std::max(best, cur);
12     }
13     return best;
14 }

```

18.9 Linked Lists

Prefer iterative pointer rewiring: fewer allocations, clearer $O(1)$ space. Draw prev/cur/next before coding.

18.9.1 Reverse

Three pointers: save next, point cur back to prev, advance. Return the new head (prev after the loop). The same idea reverses a sublist between two positions.

Trigger: reverse whole list or a segment.

Complexity: $O(n)$ time, $O(1)$ space.

```

1 struct ListNode { int val; ListNode* next; };
2
3 ListNode* reverseList(ListNode* head) {
4     ListNode* prev = nullptr;
5     while (head) {
6         ListNode* nxt = head->next;
7         head->next = prev;
8         prev = head;
9         head = nxt;
10    }
11    return prev;
12 }

```

18.9.2 Fast / slow

Start both at the head (or slow at head and fast one step ahead, depending on even/odd conventions). When fast cannot move twice, slow is at the middle — useful to split for merge sort on lists or palindrome checks.

Trigger: middle node, split list in half.

Complexity: $O(n)$ time, $O(1)$ space.

```

1 ListNode* middleNode(ListNode* head) {
2     ListNode *slow = head, *fast = head;
3     while (fast && fast->next) {
4         slow = slow->next;
5         fast = fast->next->next;
6     }
7     return slow;
8 }

```

18.9.3 Cycle detection

Floyd's algorithm: if slow and fast meet, a cycle exists. Optional second phase: reset one pointer to head and advance both by one; they meet at the cycle entry.

Trigger: linked-list cycle, find cycle entrance.

Complexity: $O(n)$ time, $O(1)$ space.

```

1 bool hasCycle(ListNode* head) {
2     ListNode *slow = head, *fast = head;
3     while (fast && fast->next) {
4         slow = slow->next;
5         fast = fast->next->next;
6         if (slow == fast) return true;
7     }
8     return false;
9 }

```

18.9.4 Merge

Merge two sorted lists with a dummy head and a tail pointer: always attach the smaller of the two fronts. For k lists, push current heads into a min-heap (see K-way merge).

Trigger: merge two / k sorted lists.

Complexity: $O(n)$ for two lists; $O(N \log k)$ for k lists.

```

1 ListNode* mergeTwoLists(ListNode* a, ListNode* b) {
2     ListNode dummy{0, nullptr}, *tail = &dummy;
3     while (a && b) {
4         if (a->val < b->val) { tail->next = a; a = a->next; }
5         else { tail->next = b; b = b->next; }
6         tail = tail->next;
7     }
8     tail->next = a ? a : b;
9     return dummy.next;
10 }

```

18.10 Trees

A useful first heuristic: path/aggregate questions often suggest DFS; level or unweighted-distance questions often suggest BFS. Always clarify null-child handling.

18.10.1 DFS

Recurse (or use an explicit stack) in preorder, inorder, or postorder. Preorder suits serialize/clone; inorder suits BST; postorder suits delete and “compute child answers then combine”.

Trigger: depth, path sums, serialize, subtree properties.

Complexity: $O(n)$ time, $O(h)$ stack.

```
1 struct TreeNode { int val; TreeNode *left, *right; };
2
3 int maxDepth(TreeNode* root) {
4     if (!root) return 0;
5     return 1 + std::max(maxDepth(root->left), maxDepth(root->
6     right));
7 }
```

18.10.2 BFS

Queue the root; process level by level by recording the queue size at the start of each round. Natural for level averages, zigzag order, and “right side view”.

Trigger: level-order, per-level aggregates.

Complexity: $O(n)$ time and space.

```
1 std::vector<std::vector<int>> levelOrder(TreeNode* root) {
2     std::vector<std::vector<int>> out;
3     if (!root) return out;
4     std::queue<TreeNode*> q;
5     q.push(root);
6     while (!q.empty()) {
7         int sz = (int)q.size();
8         out.emplace_back();
9         for (int i = 0; i < sz; ++i) {
10             TreeNode* u = q.front(); q.pop();
11             out.back().push_back(u->val);
12             if (u->left) q.push(u->left);
13             if (u->right) q.push(u->right);
14         }
15     }
16     return out;
17 }
```

18.10.3 BST

Search / insert follow the key: left if smaller, right if larger. Inorder traversal yields sorted keys — use that for validation (“strictly increasing”) and for k -th smallest.

Trigger: search, insert, validate BST, k -th smallest.

Complexity: $O(h)$ per op (h height; balanced $\log n$).

```
1  TreeNode* searchBST(TreeNode* root, int val) {
2      while (root && root->val != val)
3          root = val < root->val ? root->left : root->right;
4      return root;
5  }
```

18.10.4 LCA

In a binary tree, recurse: if both targets lie in different subtrees of the current node, that node is the LCA. In a BST, walk from the root using key order until the path splits.

Trigger: lowest common ancestor.

Complexity: $O(n)$ general tree; $O(h)$ BST.

```
1  TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p,
2      TreeNode* q) {
3      if (!root || root == p || root == q) return root;
4      TreeNode* L = lowestCommonAncestor(root->left, p, q);
5      TreeNode* R = lowestCommonAncestor(root->right, p, q);
6      if (L && R) return root;
7      return L ? L : R;
8  }
```

18.11 Graphs

Represent with adjacency lists unless the grid itself is the graph. Always discuss directed vs undirected and whether weights are present before picking BFS, Dijkstra, or Union-Find.

18.11.1 DFS / BFS

Both visit every reachable vertex once. BFS gives unweighted distances; DFS is often simpler for flood-fill, path existence, and “explore whole component”.

Trigger: traversal, reachability, grid flood fill.

Complexity: $O(V + E)$.

```
1  void dfs(int u, const std::vector<std::vector<int>>& g,
2      std::vector<char>& seen) {
3      seen[u] = 1;
4      for (int v : g[u]) if (!seen[v]) dfs(v, g, seen);
5  }
```

18.11.2 Connected components

Count how many times you start a new DFS/BFS from an unvisited vertex, or run Union-Find over all edges and count roots. On a grid, each unvisited land cell starts a flood.

Trigger: number of provinces / islands.

Complexity: $O(V + E)$.

```
1 int numIslands(std::vector<std::vector<char>>& grid) {
2     int n = (int)grid.size(), m = (int)grid[0].size(), count = 0;
3     std::function<void(int,int)> flood = [&](int r, int c) {
4         if (r < 0 || c < 0 || r >= n || c >= m || grid[r][c] != '1') return;
5         grid[r][c] = '0';
6         flood(r+1,c); flood(r-1,c); flood(r,c+1); flood(r,c-1);
7     };
8     for (int i = 0; i < n; ++i)
9         for (int j = 0; j < m; ++j)
10            if (grid[i][j] == '1') { ++count; flood(i, j); }
11     return count;
12 }
```

18.11.3 Cycle detection

Undirected: DFS with parent tracking, or Union-Find on edges (unite fails $\Rightarrow$ cycle).
Directed: three-colour DFS (GRAY = on stack) or fail Kahn's algorithm when leftover nodes remain.

Trigger: detect cycle in undirected / directed graphs.

Complexity: $O(V + E)$.

```
1 bool hasCycleUndirected(const std::vector<std::vector<int>>& g) {
2     int n = (int)g.size();
3     std::vector<char> seen(n);
4     std::function<bool(int,int)> dfs = [&](int u, int parent) {
5         seen[u] = 1;
6         for (int v : g[u]) {
7             if (v == parent) continue;
8             if (seen[v] || dfs(v, u)) return true;
9         }
10        return false;
11    };
12    for (int i = 0; i < n; ++i)
13        if (!seen[i] && dfs(i, -1)) return true;
14    return false;
15 }
```

18.11.4 Topological sort

Only on DAGs. Kahn: compute indegrees, queue zeros, peel layers. DFS: push nodes after exploring outbound edges, then reverse. If you cannot order all vertices, a cycle

exists.

Trigger: course schedule, build order, task dependencies.

Complexity: $O(V + E)$.

```
1  std::vector<int> topoKahn(int n, const std::vector<std::pair<int,
    int>>& edges) {
2      std::vector<std::vector<int>> g(n);
3      std::vector<int> indeg(n);
4      for (auto [u, v] : edges) { g[u].push_back(v); ++indeg[v]; }
5      std::queue<int> q;
6      for (int i = 0; i < n; ++i) if (!indeg[i]) q.push(i);
7      std::vector<int> order;
8      while (!q.empty()) {
9          int u = q.front(); q.pop();
10         order.push_back(u);
11         for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
12     }
13     if ((int)order.size() != n) return {}; // cycle
14     return order;
15 }
```

18.11.5 Union-Find (DSU)

Maintain disjoint sets with path compression and union by rank. Ideal for dynamic connectivity, redundant edges, and Kruskal's MST.

Why it works (amortized): path compression flattens trees on find; union by rank keeps them shallow. Over a sequence of ops the *amortized* cost is effectively constant (inverse-Ackermann); interview speak: “almost $O(1)$ amortized per op” (Section 17.1.1).

Trigger: dynamic connectivity, redundant connection, Kruskal.

Complexity: almost $O(1)$ per op amortized.

```
1  struct DSU {
2      std::vector<int> p, r;
3      DSU(int n) : p(n), r(n, 0) { std::iota(p.begin(), p.end(), 0)
    ; }
4      int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
5      bool unite(int a, int b) {
6          a = find(a); b = find(b);
7          if (a == b) return false;
8          if (r[a] < r[b]) std::swap(a, b);
9          p[b] = a;
10         if (r[a] == r[b]) ++r[a];
11         return true;
12     }
13 };
```

18.11.6 Shortest paths

Pick by edge weights: unweighted $\rightarrow$ BFS; non-negative weights $\rightarrow$ Dijkstra with a binary heap (**Dijkstra requires non-negative edge weights**); negative edges $\rightarrow$ Bellman–Ford (and say why Dijkstra would be wrong). Interview sketches usually stop at Dijkstra + BFS.

Trigger: distances with / without weights.

Complexity: BFS $O(V + E)$; Dijkstra $O((V + E) \log V)$ with a binary heap.

```
1  std::vector<long long> dijkstra(int src,
2      const std::vector<std::vector<std::pair<int,int>>>& g) {
3      const long long INF = (1LL << 62);
4      std::vector<long long> dist(g.size(), INF);
5      using Node = std::pair<long long,int>;
6      std::priority_queue<Node, std::vector<Node>, std::greater<>>
pq;
7      dist[src] = 0;
8      pq.push({0, src});
9      while (!pq.empty()) {
10         auto [d, u] = pq.top(); pq.pop();
11         if (d != dist[u]) continue;
12         for (auto [v, w] : g[u]) {
13             if (dist[v] > d + w) {
14                 dist[v] = d + w;
15                 pq.push({dist[v], v});
16             }
17         }
18     }
19     return dist;
20 }
```

18.12 Heaps

`std::priority_queue` is a max-heap by default; use `greater<>` for a min-heap. Heaps shine when you repeatedly need the current best among a changing candidate set.

18.12.1 Top- K

Keep a size- k min-heap of the best candidates so far (or max-heap of size k for the smallest k). Frequency top- K first counts, then heapifies the map.

Why it works: the heap root is the weakest member of the current top- K ; if a new value beats it, replace. You never need more than k candidates, so each of n items costs $O(\log k)$.

Trigger: k largest / most frequent.

Complexity: $O(n \log k)$.

```
1  std::vector<int> topKFrequent(const std::vector<int>& a, int k) {
2      std::unordered_map<int, int> freq;
```

```

3     for (int x : a) ++freq[x];
4     using P = std::pair<int,int>; // freq, value
5     std::priority_queue<P, std::vector<P>, std::greater<>> pq;
6     for (auto [val, f] : freq) {
7         pq.push({f, val});
8         if ((int)pq.size() > k) pq.pop();
9     }
10    std::vector<int> out;
11    while (!pq.empty()) { out.push_back(pq.top().second); pq.pop(); }
12    return out;
13 }

```

18.12.2 K-way merge

Merge k sorted sequences by storing the current head of each in a min-heap keyed by value (plus list id / index). Pop the global minimum, push that list's next element. Total $O(N \log k)$ for N output elements.

Trigger: merge k sorted lists / arrays.

Complexity: $O(N \log k)$.

```

1 std::vector<int> mergeKSorted(const std::vector<std::vector<int>
2                               >>& lists) {
3     using T = std::tuple<int,int,int>; // value, list, index
4     std::priority_queue<T, std::vector<T>, std::greater<>> pq;
5     for (int i = 0; i < (int)lists.size(); ++i)
6         if (!lists[i].empty()) pq.push({lists[i][0], i, 0});
7     std::vector<int> out;
8     while (!pq.empty()) {
9         auto [val, li, idx] = pq.top(); pq.pop();
10        out.push_back(val);
11        if (idx + 1 < (int)lists[li].size())
12            pq.push({lists[li][idx + 1], li, idx + 1});
13    }
14    return out;
15 }

```

18.12.3 Running median

Two heaps: max-heap for the lower half, min-heap for the upper half. Rebalance so sizes differ by at most one; the median is the top of the larger half (or the average of both tops).

Trigger: median from data stream.

Complexity: $O(\log n)$ per insertion.

```

1 struct MedianFinder {
2     std::priority_queue<int> low; // max-heap
3     std::priority_queue<int, std::vector<int>, std::greater<>>
4     high;
5 };

```



```

4     void addNum(int x) {
5         if (low.empty() || x <= low.top()) low.push(x); else high
        .push(x);
6         if ((int)low.size() > (int)high.size() + 1) {
7             high.push(low.top()); low.pop();
8         } else if ((int)high.size() > (int)low.size()) {
9             low.push(high.top()); high.pop();
10        }
11    }
12    double findMedian() const {
13        if (low.size() > high.size()) return low.top();
14        return 0.5 * (low.top() + high.top());
15    }
16 };

```

18.12.4 Scheduling

When events compete for a resource (meeting rooms, CPU tasks), a heap of end times or remaining work lets you always pick the earliest-free room or highest-priority job. Sort arrivals first; the heap tracks active state.

Trigger: meeting rooms II, task scheduler with cooldown.

Complexity: typically $O(n \log n)$.

```

1  int minMeetingRooms(std::vector<std::pair<int,int>> v) {
2      std::sort(v.begin(), v.end());
3      std::priority_queue<int, std::vector<int>, std::greater<>>
        end; // end times
4      for (auto [s, e] : v) {
5          // Reuse the room that becomes free earliest, if
        available.
6          if (!end.empty() && end.top() <= s) end.pop();
7          end.push(e);
8      }
9      return (int)end.size();
10 }

```

18.13 Backtracking

The skeleton is always: choose → explore → unchoose. Prune as soon as a partial solution cannot lead to a valid complete one.

18.13.1 Subsets

For each index, either skip or take the element (then pop). Equivalent bit masks iterate $0..2^n - 1$. Output size dominates the complexity.

Trigger: power set, subset-sum variants.

Complexity: $O(2^n \cdot n)$ output-sensitive.

```

1 void subsetsDfs(int i, std::vector<int>& cur, const std::vector<
    int>& a,
2                 std::vector<std::vector<int>>& out) {
3     if (i == (int)a.size()) { out.push_back(cur); return; }
4     subsetsDfs(i + 1, cur, a, out);           // skip
5     cur.push_back(a[i]);
6     subsetsDfs(i + 1, cur, a, out);           // take
7     cur.pop_back();
8 }

```

18.13.2 Permutations

Swap the current position with each candidate to the right, recurse, then swap back. A used[] array with a path vector is equivalent and often clearer when duplicates need skipping.

Trigger: all orderings of a collection.

Complexity: $O(n \cdot n!)$.

```

1 void permuteDfs(int i, std::vector<int>& a,
2                 std::vector<std::vector<int>>& out) {
3     if (i == (int)a.size()) { out.push_back(a); return; }
4     for (int j = i; j < (int)a.size(); ++j) {
5         std::swap(a[i], a[j]);
6         permuteDfs(i + 1, a, out);
7         std::swap(a[i], a[j]);
8     }
9 }

```

18.13.3 Combinations

Choose k of n : recurse with a start index so order inside a combination does not duplicate. Same prune opportunities as subset-sum when a target bound exists.

Trigger: choose k elements, combination sum.

Complexity: $O(\binom{n}{k} \cdot k)$ output-sensitive.

```

1 void combineDfs(int start, int k, std::vector<int>& cur,
2                 int n, std::vector<std::vector<int>>& out) {
3     if ((int)cur.size() == k) { out.push_back(cur); return; }
4     for (int x = start; x <= n; ++x) {
5         cur.push_back(x);
6         combineDfs(x + 1, k, cur, n, out);
7         cur.pop_back();
8     }
9 }

```

18.13.4 Pruning

Before recursing, abandon branches that cannot succeed: partial sum already exceeds target, remaining slots cannot fill the deficit, or a placement attacks an existing queen. Pruning does not change the worst-case class but dominates practical runtime.

Trigger: combination sum, N-queens, sudoku.

Complexity: still exponential worst case; much less work with tight cuts.

```
1 void comboSum(int start, int remain, std::vector<int>& cur,
2               const std::vector<int>& a,
3               std::vector<std::vector<int>>& out) {
4     if (remain == 0) { out.push_back(cur); return; }
5     for (int i = start; i < (int)a.size(); ++i) {
6         if (a[i] > remain) break;           // prune (a sorted)
7         cur.push_back(a[i]);
8         comboSum(i, remain - a[i], cur, a, out); // reuse
9         cur.pop_back();
10    }
11 }
```

18.14 Dynamic Programming

Name the [state](#), the [transition](#), and the [base cases](#) before writing loops. Memoization and tabulation solve the same DAG of subproblems.

18.14.1 Memoization

Top-down: recurse on the natural state; cache results in a map or array keyed by arguments. Easiest when the recurrence is obvious and many states are unused.

Trigger: overlapping subproblems stated recursively.

Complexity: number of states $\times$ work per transition.

```
1 int fib(int n, std::vector<int>& memo) {
2     if (n <= 1) return n;
3     if (memo[n] != -1) return memo[n];
4     return memo[n] = fib(n - 1, memo) + fib(n - 2, memo);
5 }
```

18.14.2 Tabulation

Bottom-up: allocate the DP table and fill in an order that respects dependencies. Usually less recursion overhead and clearer $O(1)$ rolling-array optimisations.

Trigger: same problems as memo, when you want iterative control.

Complexity: same as memo on the full state space.

```
1 int fibTab(int n) {
2     if (n <= 1) return n;
```

```

3     int a = 0, b = 1;
4     for (int i = 2; i <= n; ++i) {
5         int c = a + b;
6         a = b; b = c;
7     }
8     return b;
9 }

```

18.14.3 1D / 2D DP

- **1D:** state is an index or capacity — climbing stairs, house robber, coin change (unbounded knapsack flavour).
- **2D:** state is a pair (i, j) — grid paths, LCS, edit distance.

Interview tip: speak state → transition → base cases, then code.

```

1 // 1D: fewest coins for amount (unbounded)
2 int coinChange(const std::vector<int>& coins, int amount) {
3     std::vector<int> dp(amount + 1, amount + 1);
4     dp[0] = 0;
5     for (int x = 1; x <= amount; ++x)
6         for (int c : coins)
7             if (c <= x) dp[x] = std::min(dp[x], dp[x - c] + 1);
8     return dp[amount] > amount ? -1 : dp[amount];
9 }
10
11 // 2D: unique paths in an m x n grid (move right/down only)
12 int uniquePaths(int m, int n) {
13     std::vector<std::vector<int>> dp(m, std::vector<int>(n, 1));
14     for (int i = 1; i < m; ++i)
15         for (int j = 1; j < n; ++j)
16             dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
17     return dp[m - 1][n - 1];
18 }

```

18.15 Where C++ / systems interview topics live

This chapter is algorithms only. C++ and systems screen topics sit elsewhere:

- RAIL, Rule of 0/5, move, ownership — Chapters 1, 2, 4.
- Containers, iterator invalidation, reserve, string_view, hash find cost — Chapter 5.
- Stack vs heap, allocators, pools — Chapter 3.
- Cache locality, false sharing — Chapter 9.
- Mutex, atomics, memory ordering, CVs, lock-free — Chapters 6, 8.

- Worst / average / amortized — Section 17.1.

18.16 Interview Drill Sheet

Say aloud, then code once without notes:

1. Two Sum (hash complement).
2. Valid parentheses (stack).
3. Merge intervals.
4. Binary search first/last position.
5. Linked-list cycle (Floyd).
6. Sliding-window maximum (mono deque); optionally also longest substring with $\leq k$ distinct.
7. Top- K frequent (heap or `nth_element`).
8. Subsets (backtracking).

Then add tree BFS level-order and one DP (coin change or unique paths).

Reminder: the catalog is not a substitute for a timer. Timed online screens fail from speed and bugs, not from missing a section title.

Algorithmic Patterns Bibliography

- [194] *NeetCode Roadmap*. neetcode.io/roadmap.
- [195] Thomas H. Cormen et al. *Introduction to Algorithms*. 3rd. MIT Press, 2009.
- [196] Steven S. Skiena. *The Algorithm Design Manual*. Springer.



PART

Software Design

19

CHAPTER

Design Patterns

Design Patterns describe how to solve recurring design problems to design flexible and reusable object-oriented software, that is, objects that are easier to implement, change, test, and reuse.

19.1 Creational Patterns

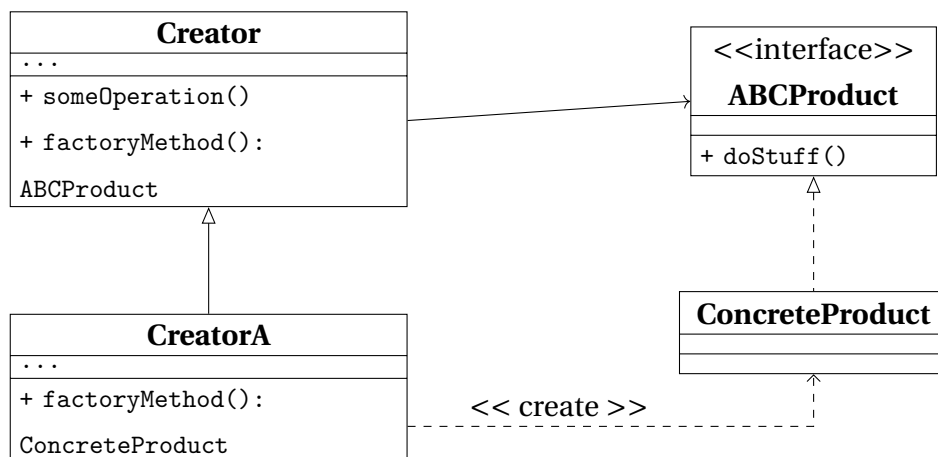
Creational patterns isolate *how* objects come into existence from the code that uses them. They matter when constructors alone couple callers to concrete types, or when creation itself is a design concern (families, multi-step builds, clones, unique instances).

19.1.1 Factory Method

Factory Method lets a creator decide *which* concrete product to build behind a shared product interface. Callers depend on that interface and on the creator's factory method, not on `new ConcreteType(...)`. Subclassing the creator (or overriding a default factory method) is how new product variants plug in.

What for	<i>creating objects</i> without having to specify the exact class of the object that will be created
How to	a <i>factory method</i> is used <i>instead of calling a constructor</i> : it can be either <i>specified in an interface</i> and implemented by child classes, or <i>implemented in a base class</i> and optionally overridden by derived classes
When needed	• exact types and dependencies of the objects are unknown beforehand • internal components of a library or framework need to be extended by users • several creators share one workflow but differ only in which product they construct

Structure



ABCProduct	interface common to all the objects.
ConcreteProduct	different implementations of the product interface.
Creator	base class which declares the factory method: • factory method - abstract or returning a default product; • returned type - must match the interface. It can have some <i>core business logic</i> which can be split from the product creation.
CreatorA, CreatorB	concrete classes that <i>override the base factory method</i> to return the different types of concrete products.

Table 19.1: Factory Method Class Diagram.

Pros and Cons

Pros	• <i>decoupling</i> of the creation of concrete objects from the core business logic of the Creator; • <i>Single Responsibility Principle</i> (creation isolated from use) • <i>Open/Closed Principle</i> (new products via new creators)
Cons	more classes and indirection; overkill for one-off types

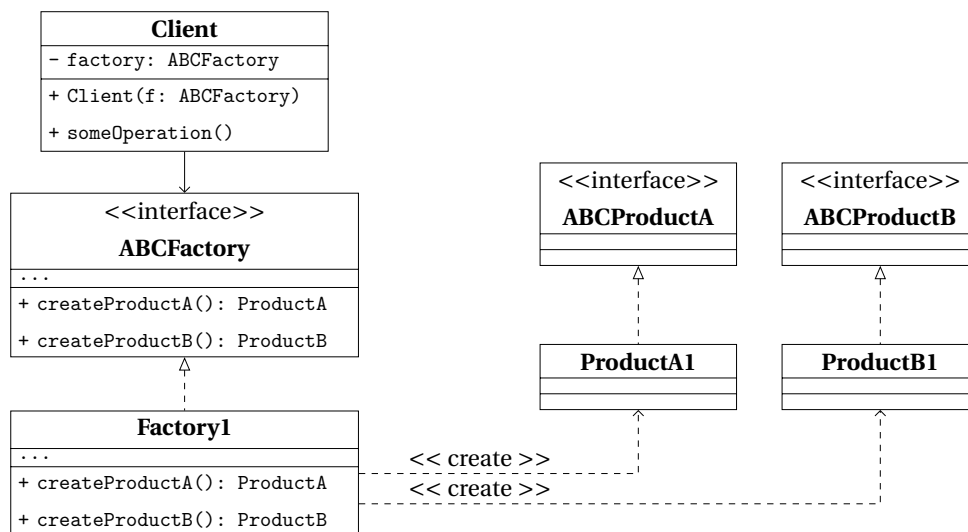
Interview tip: Contrast with Abstract Factory—Factory Method creates *one* product hierarchy; Abstract Factory coordinates *families* of related products.

19.1.2 Abstract Factory

Abstract Factory groups creation methods for a whole product family behind one factory interface. Swap the concrete factory and the client receives a matching set of products that work together, still typed only by abstract product interfaces. Use it when “theme” or platform variants must stay consistent across several object kinds.

What for	<i>producing families of related items</i> without specifying their concrete classes
How to	declare <i>explicit interfaces</i> for each distinct product of the product family, and make all variants of products follow those interfaces; provide a factory interface with one creation method per product kind
When needed	• client needs to work with various families of related objects, without depending on the latter’s concrete classes • a cluster of factory methods is blurring a class’s main responsibility and should be extracted

Structure



ABCProductA, ABCProductB	interfaces for a set or family of related but distinct products
ProductA1, ProductB1	implementations of the concrete variants of abstract products. Each abstract product must be implemented in all given variants.
ABCFactory	interface for the sets of methods needed to create each of the abstract products
Factory1	implements the creation methods of the abstract factory, each creating only the corresponding product variant
Client	can work with any concrete factory as long as it communicates with the objects through their abstract interfaces. Therefore, the signature of concrete-factories' creation-methods must match the corresponding abstract interfaces'.

Table 19.2: *Abstract Factory Class Diagram.*

Pros and Cons

Pros	• <i>interchangeable concrete implementations</i> without changing the code that uses them • <i>no tight coupling</i> between concrete products and client code • <i>Single Responsibility Principle</i> • <i>Open/Closed Principle</i> (new families via new factories)
Cons	• more interfaces and classes up front • adding a new product kind touches every factory in the family

Interview tip: Say when you would pick Abstract Factory over Factory Method: multiple related types must vary together (UI kit, platform bindings), not a single replaceable product.

19.1.3 Builder

Builder moves multi-step construction out of the product (and out of telescoping constructors) into dedicated builder objects. Optional steps can be skipped; different builders can yield different representations from the same step sequence. A Director is optional: it encodes recipes (step order) while the client still chooses which builder to use.

What for	<i>building complex objects step-by-step</i> , producing different types and representations using the same construction steps
How to	<i>object construction code is moved into a Builder</i> class, that builds the object through steps that can be executed in series and only if needed. Several Builders can be implemented when different ways of building make sense. A <i>Director</i> class can manage the order of the building steps.
When needed	• constructors became telescopic because of too many optional parameters and need to be cleaned up • code needs to create different representations of some products • a <i>composite</i> object construction needs to be managed step-by-step

Structure

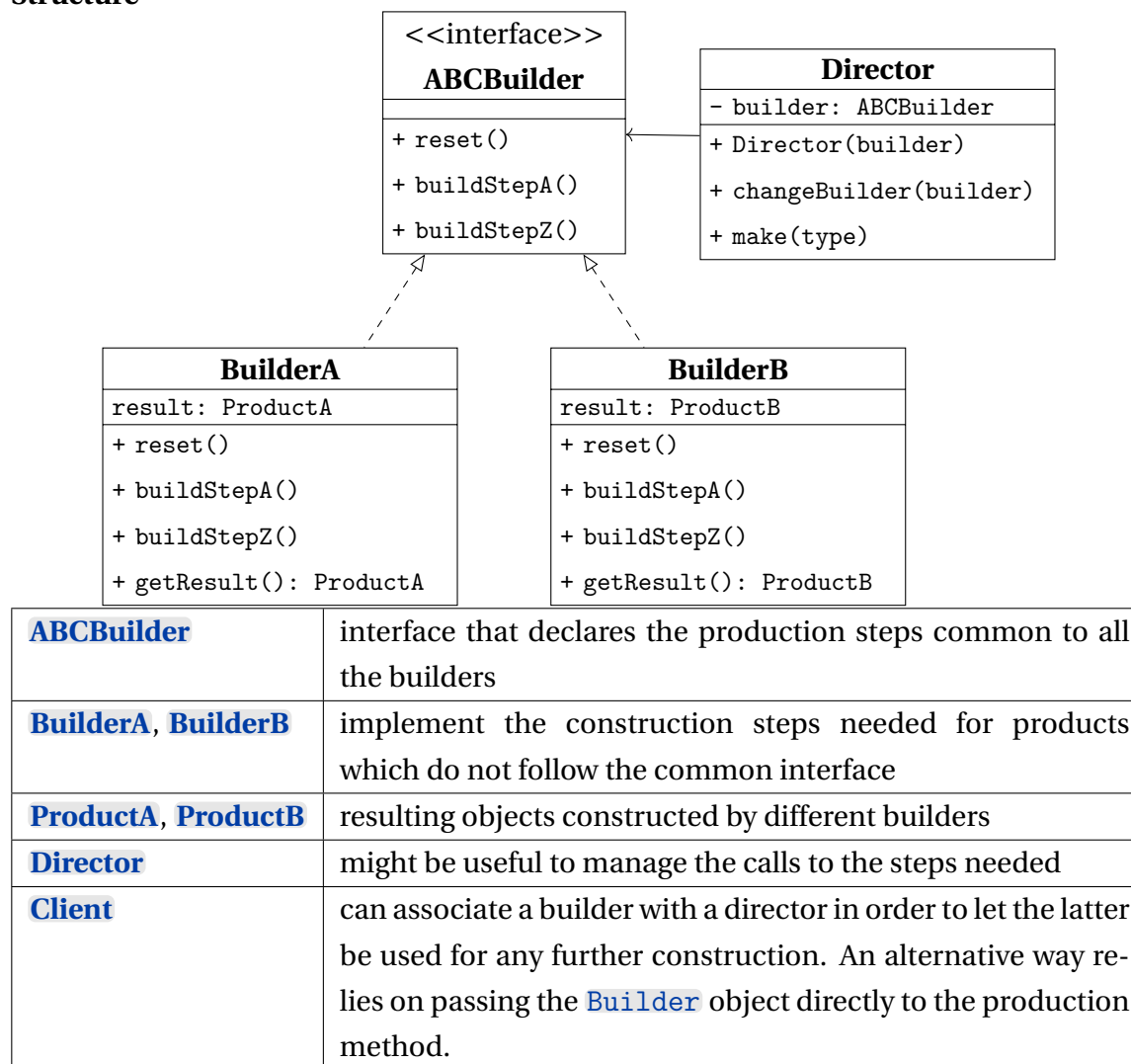


Table 19.3: *Builder Class Diagram.*

Pros and Cons

Pros	• construction goes step-by-step and can be deferred or run recursively • construction steps can be reused across representations • Single Responsibility Principle (build logic separated from product)
Cons	multiple classes required by this pattern can increase the code complexity

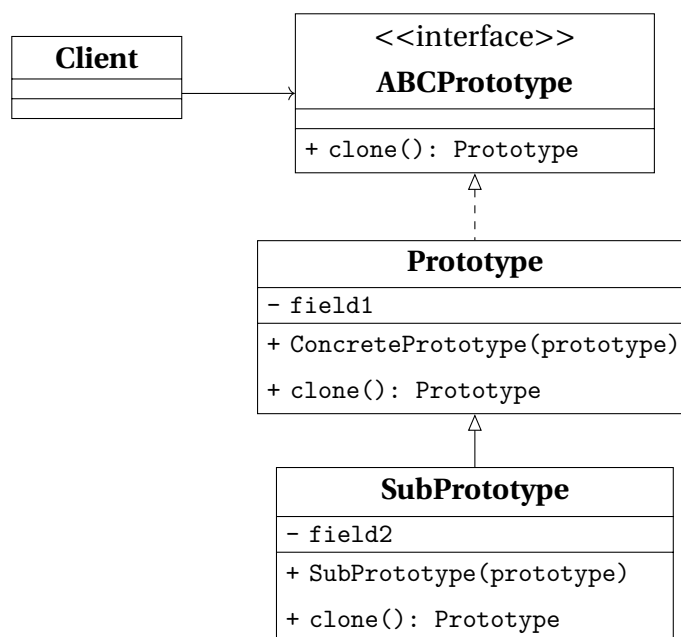
Interview tip: Director is optional—mention fluent builders without a Director when the client owns the step order.

19.1.4 Prototype

Prototype clones an existing instance instead of constructing from scratch through a type-specific API. The clone operation lives on the object (or a shared prototype interface), so private state stays accessible without leaking into client code. It shines when setup is expensive or when many variants differ only in initial configuration.

What for	<i>copying existing objects</i> without implementing code which depends on their classes
How to	a common interface is implemented for all objects that need to be cloned, and the actual cloning procedure is delegated to the objects themselves in order to have access to private and protected details as well
When needed	• code shouldn't depend on the concrete classes of the objects being copied • reducing the number of subclasses that differ only in how they initialize their respective objects

Structure



ABCPrototype	interface that declares the cloning method
Prototype	class that implements the cloning method; it can have some logic needed to manage edge cases
SubPrototype	a class that can be handy to add subsets of prototypes
Client	can produce a copy of any object which follows the prototype interface

Table 19.4: *Prototype Class Diagram.*

Pros and Cons

Pros	• cloning objects without coupling to their concrete classes • repeated initialization can be replaced by cloning pre-built prototypes • easier production of complex objects • alternative to inheritance when configuring complex objects
Cons	deep cloning gets harder with circular references or external resource handles

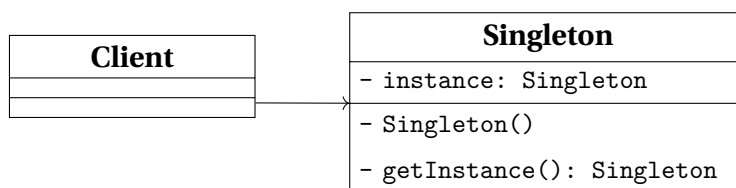
Interview tip: Ask about shallow vs deep copy, and whether the prototype registry is preferable to subclassing for configuration variants.

19.1.5 Singleton

Singleton guarantees at most one instance of a class and exposes a single global access point to it. Hide the constructor and route creation through a static method that returns the cached instance on later calls. Prefer it sparingly: shared mutable global state is hard to test and easy to overuse.

What for	<i>ensuring that a class has one instance only</i> , while providing a global access point to that instance
How to	the <i>constructor is made private</i> to prevent calls from outside the class and a <i>static creation method</i> is used in its place to create the object and return it for successive calls
When needed	• a class should have just a single instance available, e.g. a shared database connection or configuration object • a stricter control over global variables is needed

Structure



Singleton	class declaring a static method that returns the same instance it creates the first time it's called; this class' constructor is made private to hide it from external calls.
------------------	---

Table 19.5: *Singleton Class Diagram.*

Pros and Cons

Pros	• avoiding <i>global namespace pollution</i> from introducing raw global variables • <i>initialization</i> takes place only the first time the object is requested • <i>multiple but limited instances</i> can be allowed by modifying only the static method
Cons	• violation of <i>Single Responsibility Principle</i> • can mask bad design decisions, like too much overlapping between components • multithreaded programs need special care to avoid multiple instance creation • unit tests are harder because mocks cannot easily access the Singleton's constructor

Interview tip: Lead with thread-safety (e.g. Meyers singleton / double-checked locking pitfalls) and why DI often beats a classic Singleton in tests.

19.2 Behavioral Patterns

19.2.1 Chain of Responsibility

Pass a request along a chain of handlers so each can handle it or forward it. The client need not know which handler will process the request, and the chain order can be fixed at build time or rebuilt at runtime.

What for	<i>pass a request along a chain of handlers</i> , each of which can either manage the request or pass it to the next handler in the chain
How to	each particular behavior is implemented into a stand-alone handler, the handlers are linked one to another in a chain and the request, along with the data needed, is passed along the chain
When needed	• different kinds of requests must be processed in different ways and the request type is not known beforehand • the set or order of handlers can change at runtime

Structure

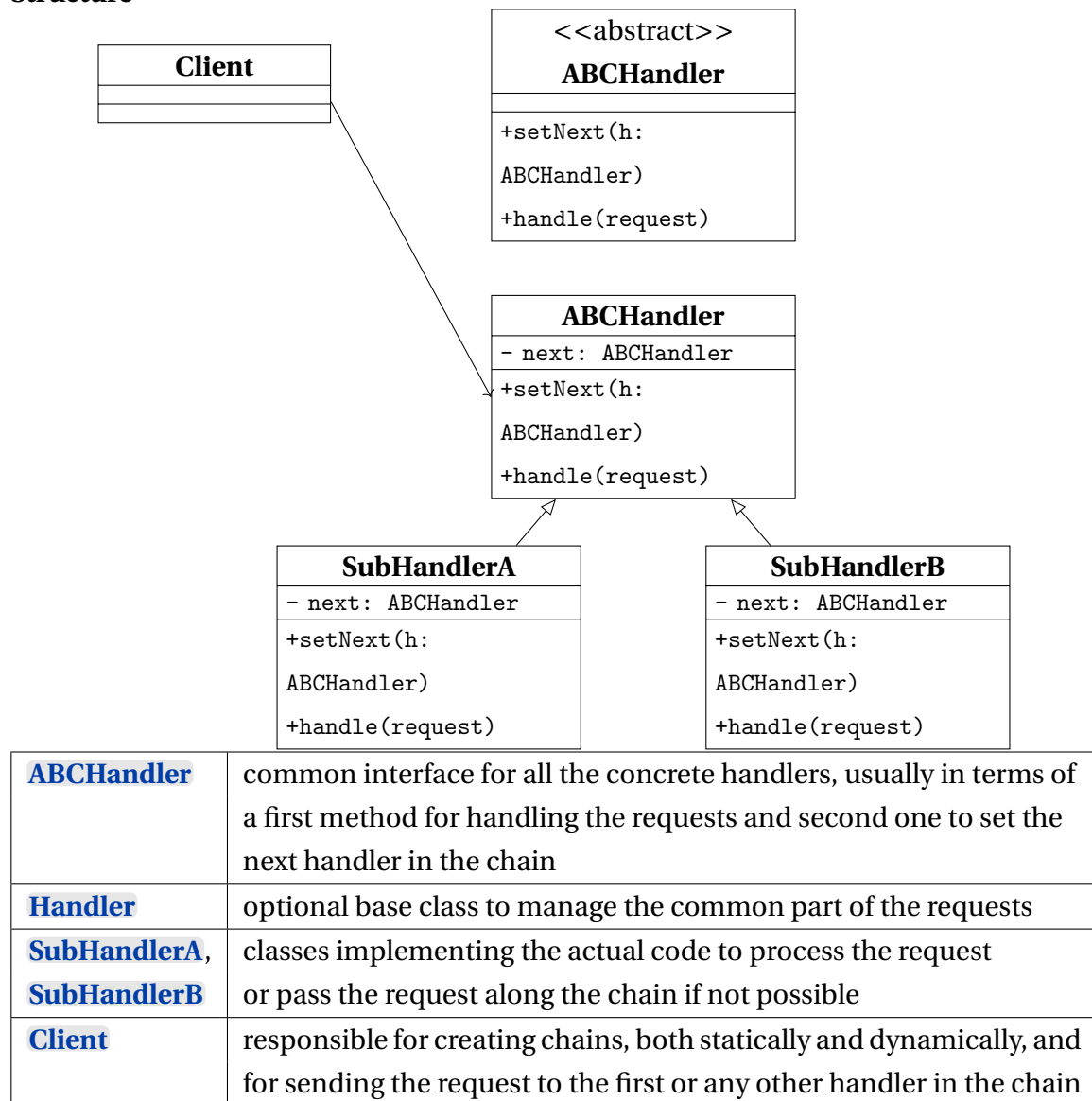


Table 19.6: Chain of Responsibility Class Diagram.

Pros and Cons

Pros	- the order of request handling can be under complete control - flexibility in deciding whether a handler should stop the request or pass it along the chain <i>Single Responsibility Principle</i> <i>Open/Closed Principle</i>
Cons	some requests may end up unhandled

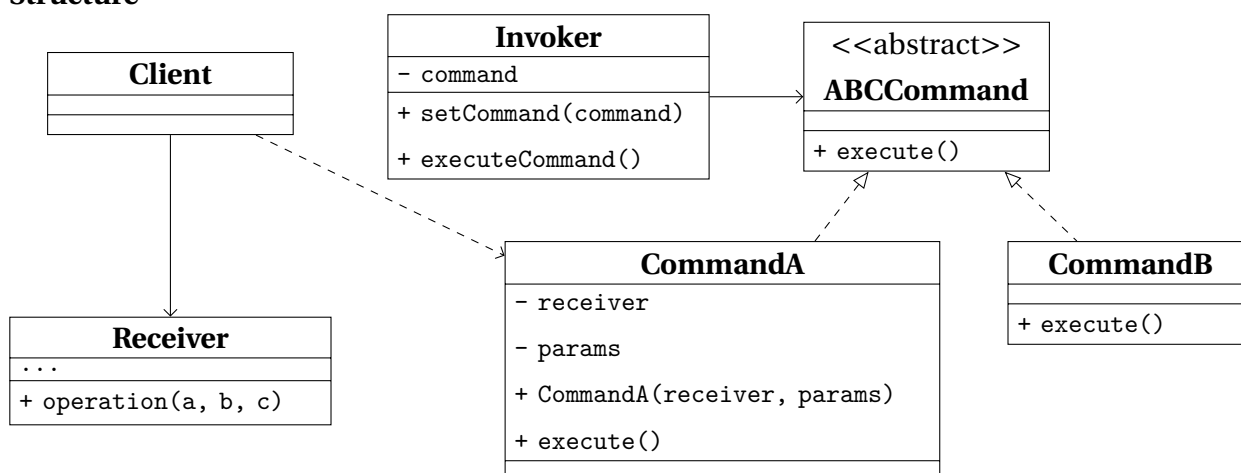
Interview tip: say “who decides next” — the current handler forwarding, not a central dispatcher.

19.2.2 Command

Turn a request into a stand-alone object that stores the receiver and parameters. An invoker triggers `execute()` without knowing how the work is done, so commands can be queued, logged, or undone.

What for	<i>turning a request into a stand-alone object</i> containing all information about it.
How to	a request is turned into a <i>command object</i> that stores the receiver and the operation parameters; an <i>invoker</i> triggers <code>execute()</code> without knowing how the work is done, so commands can be queued, logged, or undone
When needed	• there is the need to parametrize objects with operations • there is the need for reversible operations

Structure



Invoker / Sender	class responsible for creating and sending requests, directly triggering a reference to a command object, usually provided and not created by the sender
ABCCCommand	interface declaring the method to execute the command
CommandA, CommandB	classes implementing the concrete requests, passing the call to one of the business logic objects
Receiver	class that implements the business logic and does the actual work most of the time
Client	class responsible for creating and configuring commands, passing all the parameters needed

Table 19.7: *Command Class Diagram.*

Pros and Cons

Pros	• <i>Single Responsibility Principle</i> • <i>Open/Closed Principle</i> • implementing <i>undo/redo</i> is easier • implementing deferred execution of operation is easier • set of simple commands can be assembled into composites
Cons	programs end up having more layers

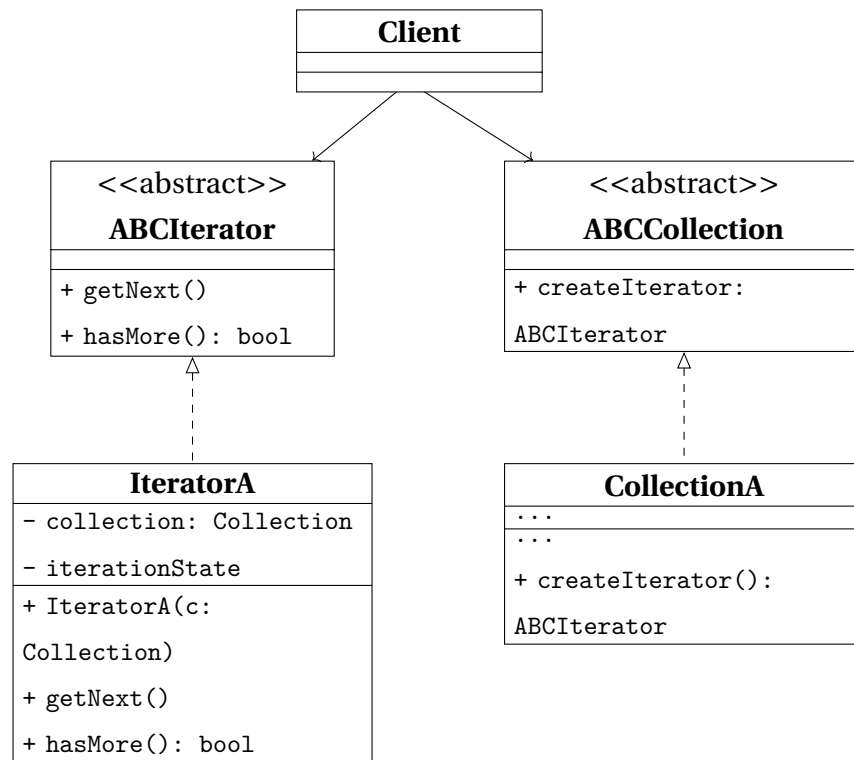
Interview tip: contrast with Strategy: Command packages an action (often with undo/queue); Strategy swaps an algorithm behind one interface.

19.2.3 Iterator

Traverse a collection without exposing its underlying representation. Traversal logic lives in a separate iterator object; the collection only creates iterators that clients use through a common interface.

What for	<i>traversing elements of a collection without exposing its underlying representation</i>
How to	traversal logic is moved into a separate <i>iterator</i> object that exposes next/hasNext (or equivalent); the collection only creates iterators, so clients walk elements without knowing the underlying structure
When needed	• a collection has a complex structure that must stay hidden • reducing duplicated traversal code • traversing structures whose layout is unknown beforehand

Structure



ABCIterator	interface declaring the operations needed to traverse a collection, like fetching the next element, keeping track of the current position and so on
IteratorA	implements specific algorithms, independent of each other so several iterators can run on the same collection
ABCCollection	interface declaring one or multiple methods for getting iterator compatible with the collection. The return type of these methods must match the Iterator interface.
CollectionA	returns instances of a particular concrete iterator class on client's request.
Client	can work with both the collection and the iterator using their interfaces.

Table 19.8: *Iterator Class Diagram.*

Pros and Cons

Pros	• <i>Single Responsibility Principle</i> • <i>Open/Closed Principle</i> • parallel iterations are possible • iteration can be delayed to when it is needed
Cons	• can be an overkill for simple collections • can be less efficient than going directly through the elements of some specialized collection

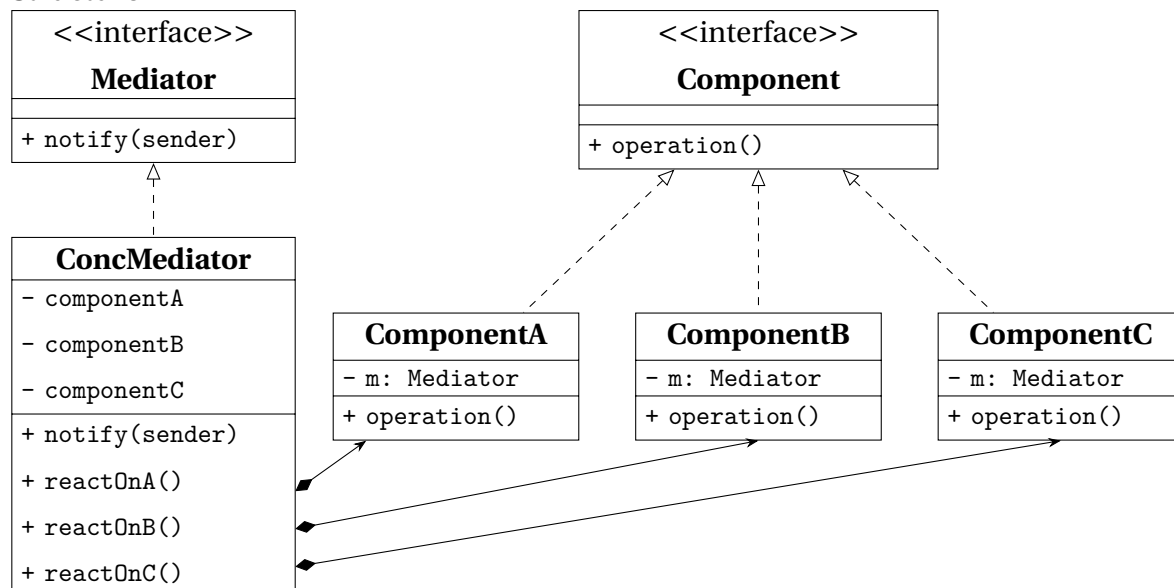
Interview tip: “same collection, many cursors” — each iterator owns its own position.

19.2.4 Mediator

Reduce chaotic dependencies by routing component communication through a mediator. Colleagues talk only to the mediator interface; the concrete mediator encodes interaction rules so adding a component does not grow a new mesh of links.

What for	<i>reducing chaotic and confusing dependencies</i> between objects and <i>restricting direct communications</i> between the objects
How to	components talk only to a <i>mediator</i> interface instead of to each other; the concrete mediator encapsulates interaction rules and routes calls, so adding a component does not wire a new mesh of dependencies
When needed	• changing an already existing class which is strongly coupled • reusing a strongly dependent component in a different program • reusing basic behaviors in completely different contexts

Structure



Component	interface for communication with other Components through its Mediator
ComponentA , ..., ComponentC	implements the Component interface and communicates with other Components through its Mediator
Mediator	interface for communication between Component objects
ConcMediator	implements the mediator interface, coordinating communication between Component objects. It knows the Components and their roles in inter-communication

Table 19.9: Mediator Class Diagram.

Pros and Cons

Pros	• <i>Single Responsibility Principle</i> • <i>Open/Closed Principle</i> • reducing coupling between components • reusing individual components is easier
Cons	mediator can evolve into a <i>God object</i>

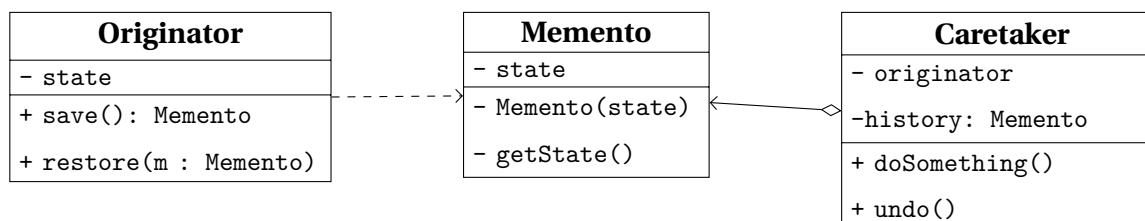
Interview tip: vs Observer: Mediator centralizes many-to-many chatter; Observer is one-to-many publish/subscribe.

19.2.5 Memento

Save and restore an object's state without exposing its internals. The originator writes an opaque memento; a caretaker stores mementos and asks for restore without reading the saved fields.

What for	<i>saving/restoring states</i> of an object without exposing internals
How to	the <i>originator</i> creates an opaque <i>memento</i> that stores its internal state; a <i>caretaker</i> keeps mementos and asks the originator to restore from one, without reading the saved fields itself
When needed	• snapshots of an object's state are needed in order to restore them later • when encapsulation would be violated by directly accessing object's fields

Structure



Originator	generic class that can produce snapshots of its own state, and restore its state from one of them
Memento	class that acts like a snapshot, and could be made immutable
Caretaker	knows the <i>why</i> and <i>when</i> to capture a snapshot or restore a previous one. This class can manage an history of Originator's states in a stack data structure.

Table 19.10: *Memento Class Diagram.*

Often the Memento type is nested inside the Originator so the originator can reach snapshot fields without exposing them to caretakers.

Pros and Cons

Pros	• producing snapshots of the object's state
Cons	• memory consumption increases • to destroy outdated mementos, the originator's lifecycle should be tracked • modern dynamic languages do not guarantee the state within the memento stays untouched

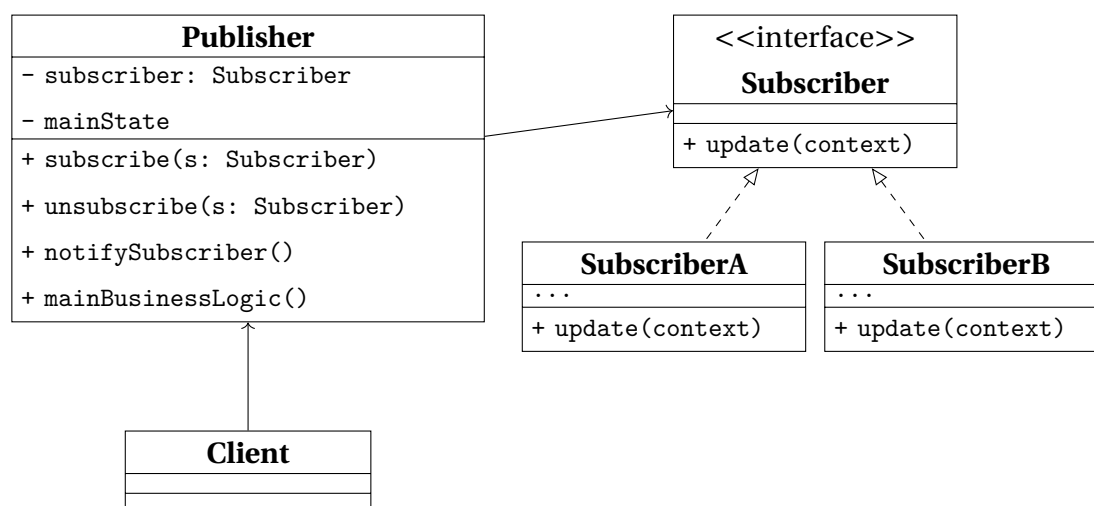
Interview tip: caretaker stores history; only the originator opens the memento — that is the encapsulation win.

19.2.6 Observer

Implement a subscription mechanism so dependents update when a subject changes. A publisher keeps subscribers and notifies them; subscribers register at runtime through a shared update interface.

What for	implementing a <i>subscription mechanism</i>
How to	a <i>publisher</i> keeps a list of <i>subscribers</i> and notifies them on state changes; subscribers implement a common update interface and register/un-register at runtime, so the publisher never hard-codes their types
When needed	• changes to the state of one object may require changing other objects that are unknown beforehand or change dynamically • objects must observe each other, for a limited time and in specific circumstances

Structure



Publisher	class that issues events of interest to other objects. This class contains a subscription infrastructure that lets subscribers both join and leave the subscriber list
Subscriber	interface declares the notification interface, that in most cases is a single update method with some contextual parameters.
SubscriberA, SubscriberB	implement the different logics in response to notifications issued by the publisher.
Client	creates both publisher and subscriber objects and manages the registrations for updates

Table 19.11: Observer Class Diagram.

Pros and Cons

Pros	• Open/Closed Principle • relations between objects can be established at runtime
Cons	• subscribers can be notified in random order

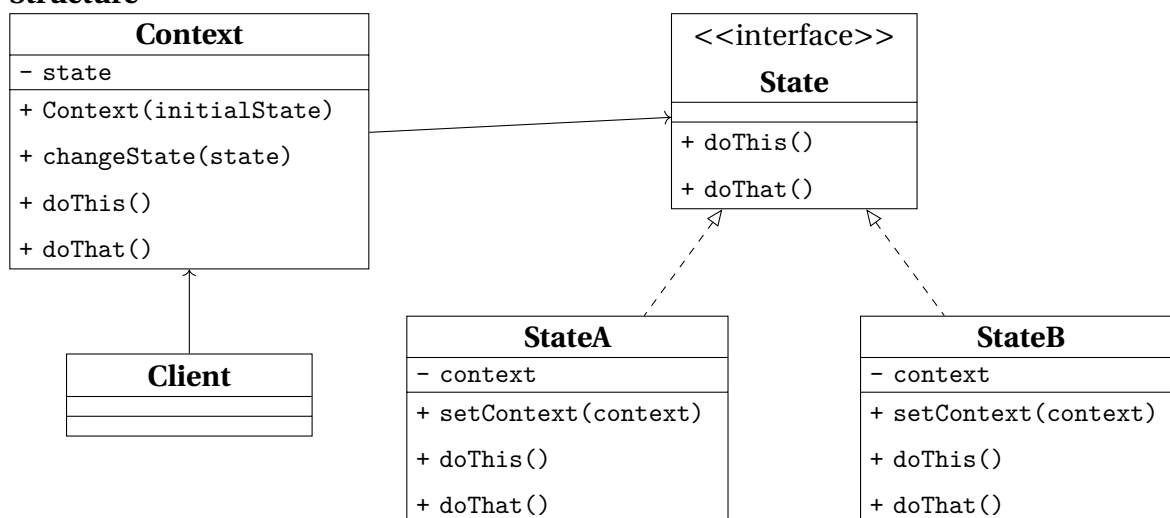
Interview tip: name the push: who calls update, and whether payload is data or “pull from publisher”.

19.2.7 State

Let an object change behavior when its internal state changes, as if its class changed. State-specific behavior lives in separate state classes; the context delegates to the current state and swaps the reference on transitions.

What for	implementing <i>objects whose behavior depends on their internal state</i>
How to	state-specific behavior is extracted into separate <i>state</i> classes that share an interface; the context holds a current state object and delegates calls to it, switching the reference when the state changes
When needed	• implementing an object that behaves differently depending on its current state, the number of states is considerable, and the state-specific code changes frequently • managing better a class that changes its behavior depending on some internal field's value • managing better duplicate code across similar states and transitions of a condition-based state machine

Structure



Context	stores a reference to one of the concrete state objects, to which it delegates all the state-related work. The Context interacts with the state interface, in order to be able to interact with any Concrete State
State	interface declares the state generic methods, that should make sense for all concrete states for a matter of consistency
StateA StateB	implement their own specific methods, and can make use of abstract classes to encapsulate some common behavior

Table 19.12: *State Class Diagram.*

Pros and Cons

Pros	• <i>Single Responsibility Principle</i> • <i>Open/Closed Principle</i> • bulky state-machine code is eliminated
Cons	can be overkill when few states change rarely

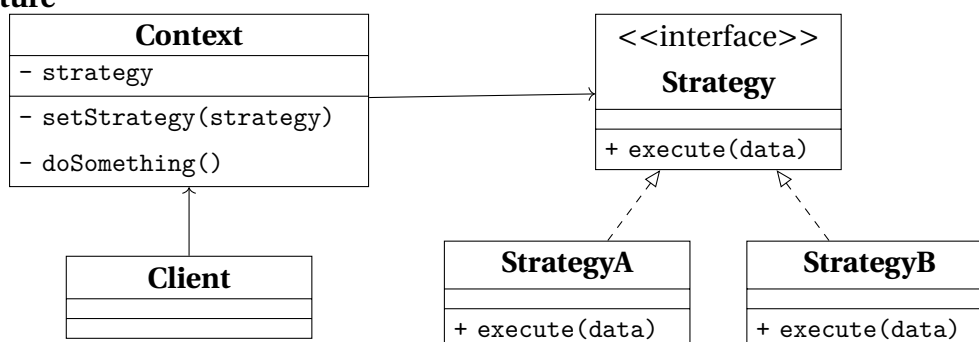
Interview tip: vs Strategy — State often owns the next transition; Strategy is usually chosen by the client.

19.2.8 Strategy

Define a family of interchangeable algorithms, each in its own class. The context holds a strategy reference and delegates the varying step, so the algorithm can be swapped without changing the context.

What for	defining a <i>family of algorithms, each implemented in a separate class</i> , in order to use them interchangeably
How to	each algorithm variant becomes a <i>strategy</i> class behind a shared interface; the context holds a strategy reference and delegates the varying step to it, so the algorithm can be swapped without changing the context
When needed	• different variants of the same algorithm must be used by an external object that can be able to switch between them • managing many similar classes only differing in the way they execute some behavior • isolating business logic from the implementation details • better management of massive conditional statements that switch between variants of the same algorithm

Structure



Context	has a reference to the Strategy Interface, used to reference the Concrete Strategy in use, and methods to replace the strategy at run-time.
Strategy	interface defines the part which is common to all concrete strategies and declares a method used by the Context to execute it.
StrategyA, StrategyB	implement a variety of algorithms needed by the context.
Client	manages the creation of the strategies and passes them to the context.

Table 19.13: Strategy Class Diagram.

Pros and Cons

Pros	• swapping algorithms at runtime • implementation details of an algorithm is separate from the client using it • inheritance is replaced with composition • Open/Closed Principle
Cons	• can be an overkill in case not many algorithms will be used and they will not change much at runtime • clients must know the differences between the algorithms to be able to choose • most of the modern programming languages implement different versions of an algorithm inside a set of anonymous functions and let the client use them in a similar way

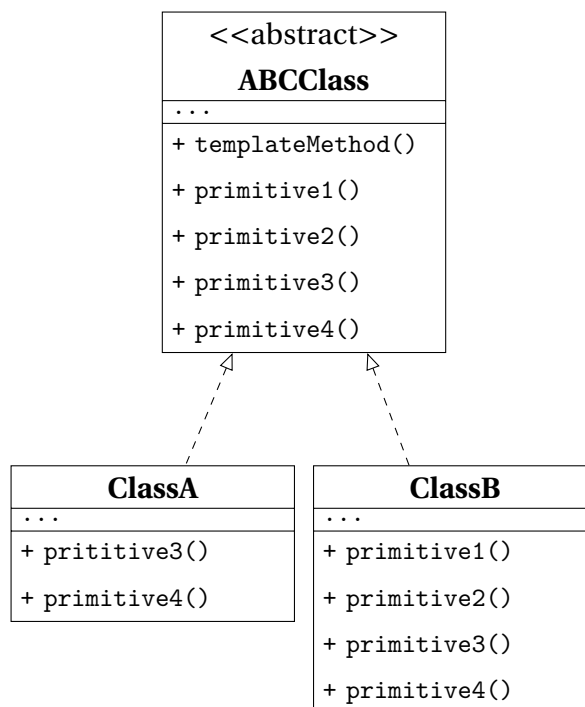
Interview tip: “vary the algorithm, keep the context” — composition over a switch on strategy type.

19.2.9 Template Method

Define an algorithm skeleton in a base class and let subclasses override selected steps. The template method calls a fixed sequence; invariant steps stay in the base class, while primitive or hook steps are filled in via inheritance.

What for	<i>defining the skeleton of an algorithm in a class</i> and letting subclasses override specific steps
How to	a base class defines a <i>template method</i> that calls a fixed sequence of steps; invariant steps stay in the base class, while <i>primitive/hook</i> steps are abstract or virtual so subclasses fill them in via inheritance
When needed	• client needs to extend only specific steps of an algorithm • managing several classes containing almost identical algorithms

Structure



ABCClass	declares the steps of an algorithm and the <i>templateMethod</i> used to call them in a specific order. The steps may both be declared as abstract or implement some actual routines.
ClassA, ClassB	override all of the steps, but not the <i>templateMethod</i> itself.

Table 19.14: *Template Method Class Diagram.*

Pros and Cons

Pros	• overriding parts of an algorithm without affecting the rest • less code duplication
Cons	• limited by the fixed skeleton of the algorithm • risk of breaking the <i>Liskov Substitution Principle</i> • maintenance gets harder as the number of steps grows

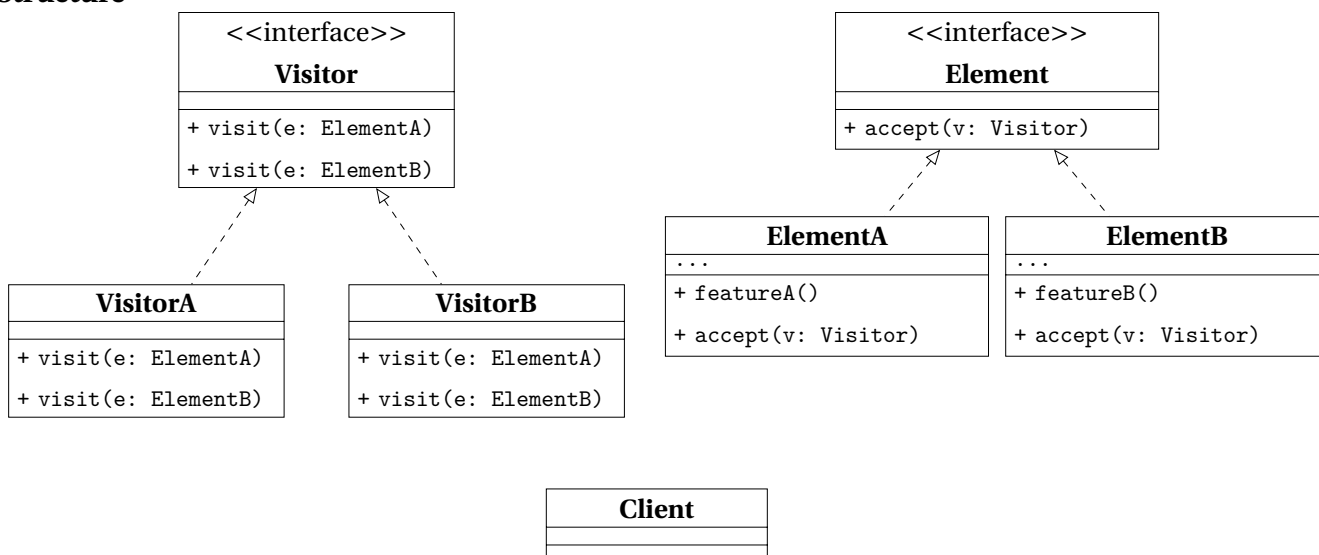
Interview tip: vs Strategy — Template Method uses inheritance for steps; Strategy uses composition for the whole algorithm.

19.2.10 Visitor

Separate algorithms from the object structure they operate on. Elements accept a visitor and call back `visit(this)`; new operations become new visitor classes while the element hierarchy stays closed to that change (double dispatch).

What for	<i>separating algorithms from the objects</i> they operate on
How to	each element class accepts a <i>visitor</i> and calls back <code>visit(this)</code> ; new operations are added as new visitor classes, while the element hierarchy stays closed to that change (double dispatch)
When needed	• performing an operation on all elements of a complex data structure • cleaning up the business logic extracting other behaviors • separating behaviors that make sense only for specific classes

Structure



Visitor	interface declares a set of visiting methods, each with a different Concrete Element taken as argument.
VisitorA, VisitorB	implement the different behaviors for each Concrete Element.
Element	interface declares a method to accept the Visitor interface as argument.
ElementA, ElementB	implements the acceptance method, whose purpose is to redirect the call to the proper visitor method for the current element class. This method must be overridden in any subclass even though it was already developed in the base one.
Client	usually needs to work on a collection or a similar complex object whose type is ignored by the Client, since it works with the abstract interface.

Table 19.15: Visitor Class Diagram.

Pros and Cons

Pros	• <i>Open/Closed Principle</i> • <i>Single Responsibility Principle</i> • can store information while working with different objects
Cons	• adding/removing a class requires all visitors to get updated • visitors may need access to private data/ methods of the elements they visit

Interview tip: open for new operations (new visitors), closed for new element types — adding a class forces every visitor to update.

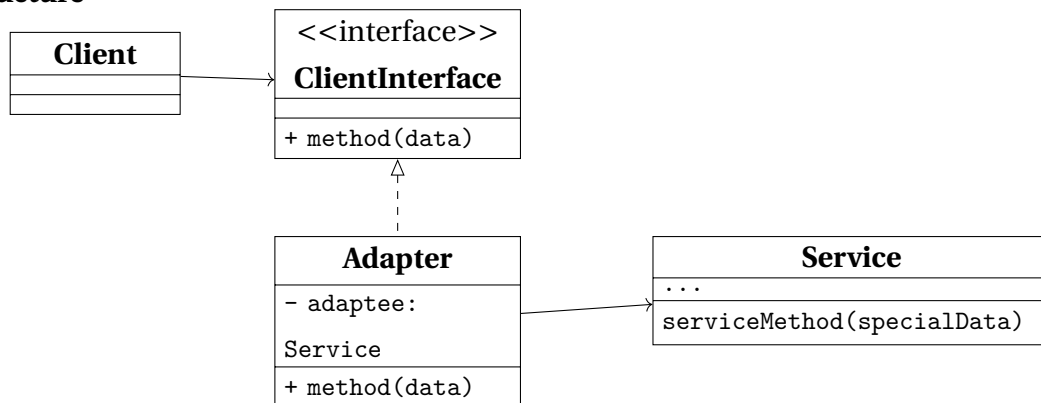
19.3 Structural Patterns

19.3.1 Adapter

An *Adapter* sits between a client that expects one interface and a class that already works but speaks a different API. It implements the target protocol and translates each call into the adaptee's terms, so neither side needs to change. Use it when you must reuse a third-party or legacy type without rewriting callers.

What for	allowing <i>objects with incompatible interfaces to collaborate</i>
How to	an <i>adapter</i> implements the interface clients expect and <i>translates</i> calls to the adaptee's API; clients depend only on the target interface, so the foreign class can be reused without changing either side
When needed	• a class's interface is incompatible with the rest of the code • reusing subclasses that lack some common functionality that cannot be added in the superclass

Structure



Client	the business logic of the program
ClientInterface	the protocol a class must follow to collaborate with the client
Service	usually a third-party or legacy class with an interface incompatible with the Client
Adapter	a class that works with both the client and the service; it receives calls from the client and translates them into calls on a wrapped service object

Table 19.16: *Adapter Class Diagram.*

Pros and Cons

Pros	• <i>Single Responsibility Principle</i> • <i>Open/Closed Principle</i>
Cons	• sometimes it is simpler to change the service class to match than to add a new set of interfaces

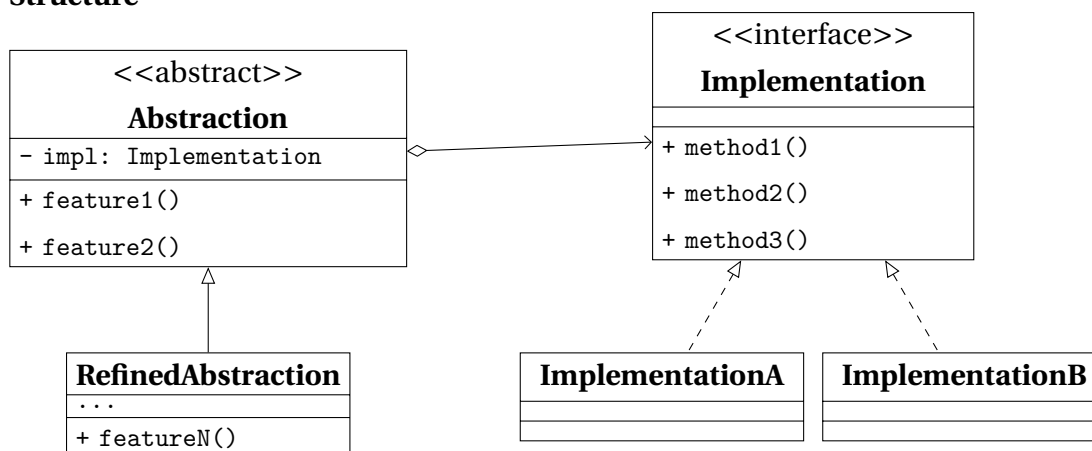
Interview tip: Adapter changes the interface; Proxy keeps the same interface and controls access; Decorator keeps the same interface and adds behavior.

19.3.2 Bridge

Bridge splits a fat hierarchy into two: an abstraction that clients talk to, and an implementation hierarchy that does the low-level work. Composition replaces a combinatorial inheritance tree, so each side can grow on its own. Swap the implementation at runtime when the platform or backend must change without rewriting the abstraction.

What for	<i>splitting a large class or a set of related classes into two hierarchies</i> , an abstraction and an implementation, developed independently from each other
How to	the <i>abstraction</i> hierarchy holds a reference to an <i>implementation</i> interface and delegates low-level work to it; both hierarchies can evolve independently, and the implementation can be swapped at runtime
When needed	- dividing and organizing a monolithic class that has several variants of some functionality - extending a class in several independent dimensions - switching implementation at runtime is a requirement

Structure



Abstraction	high-level interface; relies on Implementation objects for the actual work
RefinedAbstraction	optional variants of control logic
Implementation	interface shared by all concrete implementations; the abstraction builds complex behavior on these primitives
ImplementationA, ImplementationB	implement specific code
Client	interacts with the abstraction after linking it with one of the implementation objects

Table 19.17: Bridge Class Diagram.

Pros and Cons

Pros	• platform-independent classes are easier to create • clients are not exposed to platform details and work with high-level abstractions • Open/Closed Principle • Single Responsibility Principle
Cons	applying Bridge to a highly cohesive class can make the code more complicated

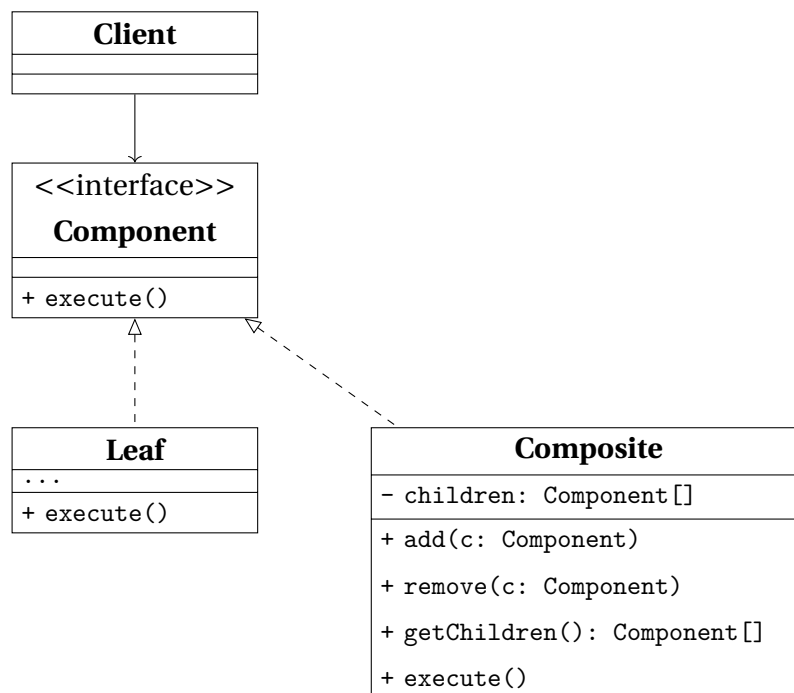
Interview tip: Bridge is planned up front to separate orthogonal axes; Adapter is usually bolted on later to make existing types cooperate.

19.3.3 Composite

Composite builds tree structures where leaves and containers share one component interface. Clients call the same operations on a single node or on a whole subtree; composites forward work to children. Reach for it when the domain is naturally hierarchical (UI widgets, file systems, expression trees).

What for	<i>composing objects into tree structures</i> in order to work with these structures as if they were individual objects
How to	both leaf and composite objects implement the same component interface; leaves act as single entities while composites hold children, add/remove them, and delegate work downward
When needed	• implementing a tree-like data structure • clients need to interact uniformly with simple and complex objects

Structure



Component	interface with the common operations for both simple and complex elements of the tree
Leaf	a basic element of a tree that contains no children; usually the part that ends up doing the real work
Composite	an element that has children (leaves or other composites) and talks to them through the component interface
Client	works with both simple and complex elements through the component interface

Table 19.18: *Composite Class Diagram.*

Pros and Cons

Pros	- polymorphism and recursion make complex tree structures easier to use Open/Closed Principle
Cons	a common interface is hard when leaf and composite behaviors differ too much

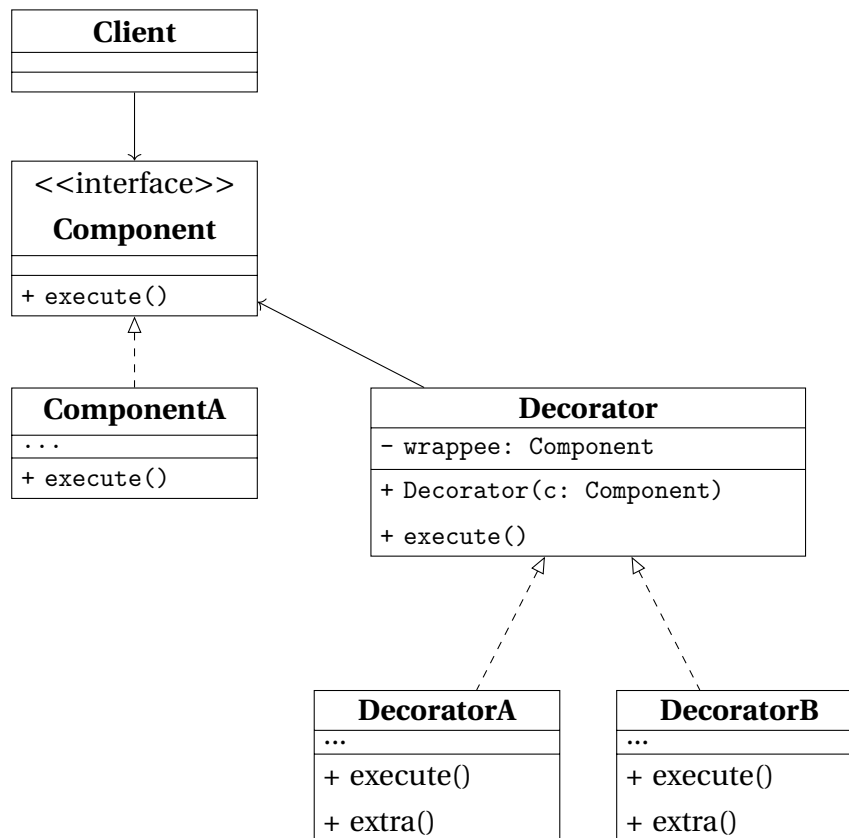
Interview tip: clients should not need instanceof checks — if they do, the shared component interface is probably incomplete or over-stretched.

19.3.4 Decorator

Decorator wraps an object in another object of the same interface to add behavior without subclassing the core type. Wrappers nest: each layer delegates to the inner component and can run extra work before or after. Prefer it when inheritance is blocked (`final`) or when combinations of extras would explode a class hierarchy.

What for	<i>attach new behaviors to objects</i> by placing them inside special wrappers
How to	objects are placed inside special wrapper objects that contain the new behaviors
When needed	• extra behavior needs to be assigned to objects at runtime • extending an object's behavior when inheritance cannot be used (for example when a class is declared as <code>final</code>)

Structure



Component	interface common to both wrapper and wrapped objects
ComponentA	class being wrapped; its basic behavior can be altered by decorators
Decorator	holds a reference to the wrapped object typed as Component; the base decorator delegates all operations to it
DecoratorA , DecoratorB	define behaviors that can be added to Components dynamically; each concrete decorator overrides the base behavior and runs the variant before or after calling the parent method

Table 19.19: *Decorator Class Diagram.*

Pros and Cons

Pros	• no new subclasses of the core type are needed to extend behavior • responsibilities can be added and removed at runtime • Open/Closed Principle
Cons	• removing a wrapper from the middle of the stack can be difficult • making decorator behavior independent of stack order can be hard • code gets messy as layers multiply

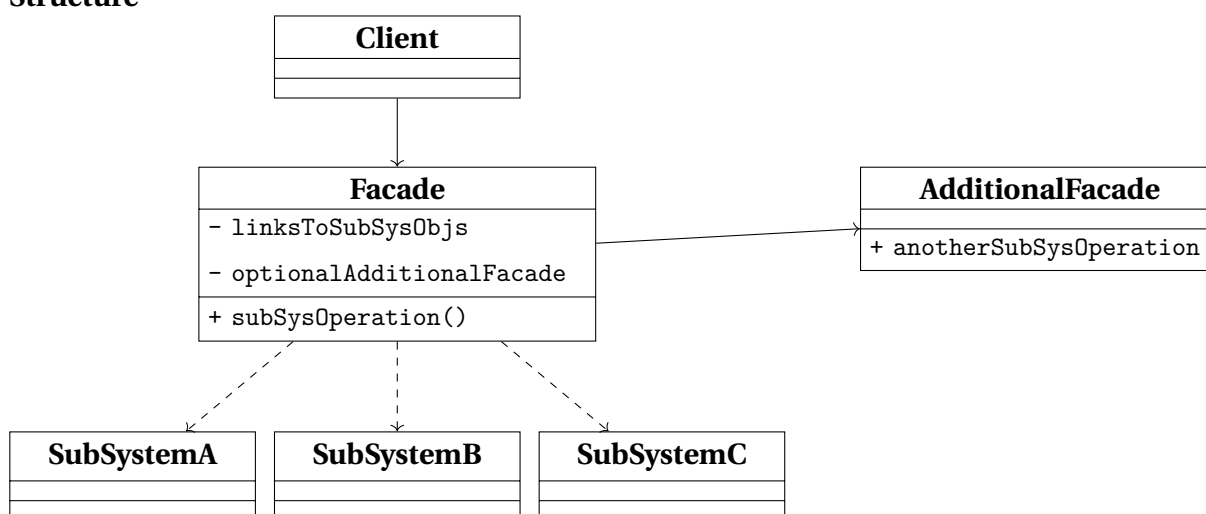
Interview tip: Decorator adds responsibilities; Proxy controls access or lifecycle — both wrap, but the intent differs.

19.3.5 Facade

A *Facade* hides a busy subsystem behind one simple entry point. Clients call a few high-level methods; the facade knows which subsystem objects to orchestrate. Use it to layer a codebase or to stop every caller from wiring the same complex dance.

What for	providing users with a <i>simplified interface to a complex set of classes</i>
How to	a facade class provides a simple interface to a complex subsystem
When needed	• a limited but straightforward interface to a complex subsystem is needed • a subsystem needs to be structured into layers

Structure



Facade	convenient access to part of the subsystem; knows where to direct client requests and how to drive the parts
AdditionalFacade	optional helper that shrinks what one facade must manage; usable by clients and by other facades
SubSystemA , ..., SubSystemC	subsystem classes that collaborate and must be understood in depth; they are unaware of the facade's existence
Client	calls subsystem classes through the facade only

Table 19.20: *Facade Class Diagram.*

Pros and Cons

Pros	client code is isolated from subsystem complexity
Cons	a facade can itself become a god object if too much of the system is funnelled through it

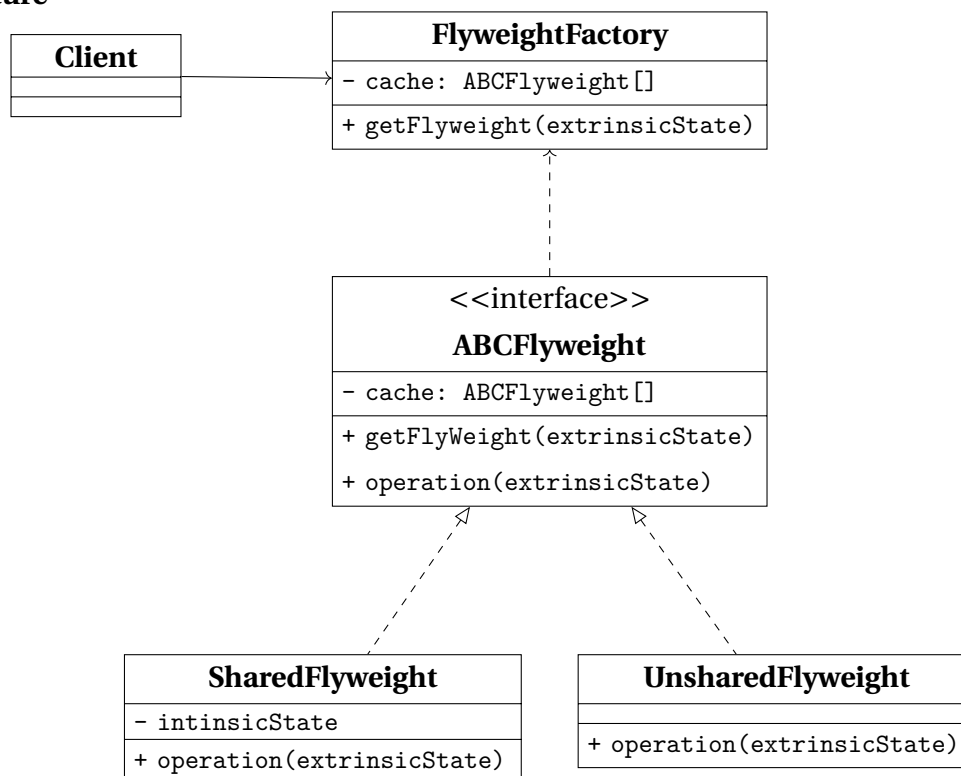
Interview tip: Facade defines a *new*, simpler API over many classes; Adapter reshapes one existing interface to another.

19.3.6 Flyweight

Flyweight cuts memory by sharing the immutable, context-free part of many similar objects (intrinsic state) and keeping the rest (extrinsic) outside or passed in. A factory hands out cached flyweights so identical intrinsic data is not duplicated per instance. Worth it when you create huge numbers of near-identical objects (glyphs, particles, tiles).

What for	reducing RAM consumption by <i>sharing common parts of state between multiple objects</i> , instead of keeping copies of the same data in each object
How to	extract the <i>intrinsic state</i> (invariant, context-independent, shareable) and pass the <i>extrinsic state</i> (variant, context-dependent, unshareable) into operations
When needed	• dealing with large numbers of objects with repeated elements that would use a large amount of memory if stored individually

Structure



ABCFlyweight	interface for shareable flyweight objects (intrinsic state + ops that take extrinsic state)
SharedFlyweight	concrete flyweight holding shared intrinsic state
UnsharedFlyweight	flyweight-shaped object that is not shared (keeps unique state)
FlyweightFactory	creates and shares a pool of flyweight objects
Client	configures flyweights at use sites by passing extrinsic context into their methods

Table 19.21: *Flyweight Class Diagram.*

Pros and Cons

Pros	saves RAM when managing many similar objects
Cons	• CPU cost can rise as RAM falls (lookup / extrinsic plumbing) • code complexity rises around the intrinsic/extrinsic split

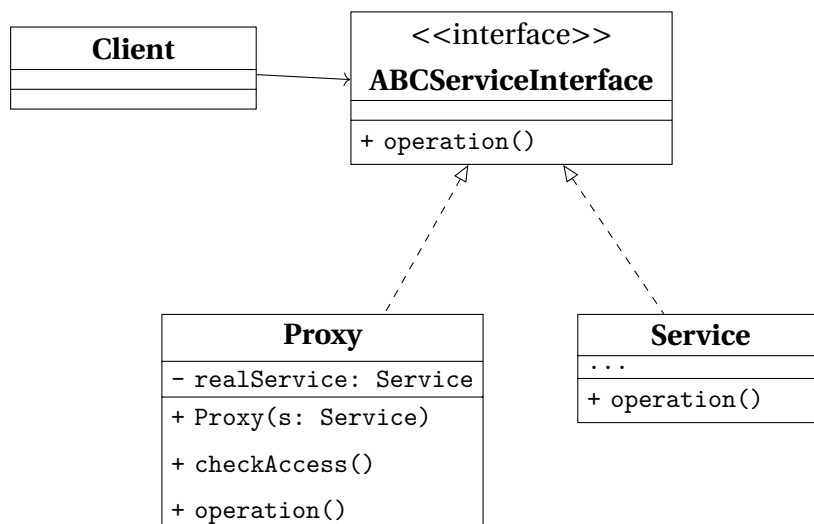
Interview tip: say “intrinsic vs extrinsic state” and that flyweights should be immutable so sharing is safe across contexts.

19.3.7 Proxy

A *Proxy* stands in for a real service object under the same interface, so clients need not know they are talking to a stand-in. It can delay creation, check access, cache results, or log calls before forwarding. Choose the proxy flavour that matches the need (virtual, protection, remote, smart reference).

What for	<i>substitute or placeholder for another object</i> , <i>controlling access</i> and allowing actions both before and after using the object
How to	a proxy class with the same interface as the original object accepts requests and forwards them to a real service object (often managing its lifecycle)
When needed	• lazy initialization • access control • local execution of a remote service • logging the request history through a proxy • caching request results, especially when results are large • reference counting so a service can be dismissed when no clients remain

Structure



ABCServiceInterface	interface of the service that needs a proxy; the proxy must implement it to stand in for the service
Service	class providing useful business logic
Proxy	holds a reference to a service object, may preprocess requests, then forwards them and may manage the service's lifecycle
Client	works with service or proxy interchangeably when both follow the same interface

Table 19.22: Proxy Class Diagram.

Pros and Cons

Pros	• control over service objects without knowledge of their internals • the client need not manage the service object's lifecycle • a proxy can answer while the service is still initializing • <i>Open/Closed Principle</i>: new proxies can be added without changing the service or its clients
Cons	• extra classes increase complexity • responses from the service can be delayed

Interview tip: Proxy and Decorator look alike — Proxy often owns lifecycle and access; Decorator is stacked by the client to add behavior.

19.4 Relations among Patterns (summary)

Per-pattern “relations” tables are omitted on purpose. The compact summary below collects the usual interview contrasts.

Pattern	Related to	How / difference
Factory Method	Abstract Factory	AF is often a set of factory methods for a <i>family</i> of products; FM creates one product hierarchy.
Factory Method	Template Method	A template step can be a factory method.
Factory Method	Iterator	Collection subclasses can return their own iterator types via FM.
Abstract Factory	Builder	AF returns products immediately as a family; Builder stages construction then releases.
Abstract Factory	Prototype, Singleton	Prototypes can supply AF variants; an AF is sometimes a Singleton.
Builder	Composite	Builder can assemble a composite step-by-step.
Prototype	Command, Composite, Decorator	Clone history entries or complex trees instead of rebuilding.
Singleton	Facade, Flyweight	Facade is often one object (Singleton-like); Flyweight has <i>many</i> shared immutable instances.

Table 19.23: *Design Patterns — relations and interview contrasts (1/2).*

Pattern	Related to	How / difference
Chain of Responsibility	Command, Mediator, Observer, Decorator, Composite	CoR = dynamic receivers; Command = sender→receiver; Mediator = hub; Observer = subscribe; Decorator shares recursive wrapping but extends behaviour; with Composite, responsibilities form a tree.
Command	Memento, Strategy, Visitor, Prototype	Memento helps undo; Strategy parameterizes behaviour without packaging a request/history; Visitor is a powerful multi-type Command; Prototype can clone commands into history.
Iterator	Composite, Visitor, Memento	Traverse composites; Visitor walks and operates; Memento can snapshot cursor state.
Mediator	Facade, Observer	Facade organizes a subsystem API; Mediator centralizes peer chatter; Observer is one-to-many pub/sub (a Mediator may itself be implemented as a publisher).
Memento	Prototype	Prototype can be a simpler alternative when state has no external resource links.
State	Strategy, Bridge	All three compose and delegate; State often owns transitions; Strategy is usually chosen by the client; Bridge separates orthogonal abstraction/implementation axes.
Strategy	Template Method, Decorator	TM uses inheritance for steps; Strategy uses composition for the whole algorithm; Decorator changes the “skin”, Strategy the “guts”.
Adapter	Bridge, Proxy, Decorator	Adapter makes interfaces match; Bridge plans split abstraction/impl; Proxy controls access; Decorator adds duties.
Composite	Decorator, CoR	Similar recursive structure; Decorator adds behaviour, CoR forwards a request along a chain.
Facade	Mediator	Facade simplifies access; Mediator routes communication without adding subsystem features.
Proxy	Decorator	Same interface wrapping; Proxy = access/lifecycle; Decorator = extra behaviour.

Table 19.24: *Design Patterns — relations and interview contrasts (2/2).*

Design Patterns Bibliography

- [197] *Design Patterns*. [Refactoring Guru](#).
- [198] *Factory Method Design Pattern*. [Wikipedia](#).
- [199] *Abstract Factory Design Pattern*. [Wikipedia](#).
- [200] *Builder Design Pattern*. [Wikipedia](#).
- [201] *Prototype Design Pattern*. [Wikipedia](#).
- [202] *Singleton Design Pattern*. [Wikipedia](#).
- [203] *Chain of Responsibility Design Pattern*. [Wikipedia](#).
- [204] *Command Design Pattern*. [Wikipedia](#).
- [205] *Iterator Design Pattern*. [Wikipedia](#).
- [206] *Mediator Design Pattern*. [Wikipedia](#).
- [207] *Memento Design Pattern*. [Wikipedia](#).
- [208] *Observer Design Pattern*. [Wikipedia](#).
- [209] *State Design Pattern*. [Wikipedia](#).
- [210] *Strategy Design Pattern*. [Wikipedia](#).
- [211] *Template Method Design Pattern*. [Wikipedia](#).
- [212] *Visitor Design Pattern*. [Wikipedia](#).
- [213] *Adapter Design Pattern*. [Wikipedia](#).
- [214] *Bridge Design Pattern*. [Wikipedia](#).
- [215] *Composite Design Pattern*. [Wikipedia](#).
- [216] *Decorator Design Pattern*. [Wikipedia](#).
- [217] *Facade Design Pattern*. [Wikipedia](#).
- [218] *Flyweight Design Pattern*. [Wikipedia](#).
- [219] *Proxy Design Pattern*. [Wikipedia](#).

20

CHAPTER

Solid Principles

SOLID is a mnemonic acronym for five design principles intended to make object-oriented designs more understandable, flexible, and maintainable.

The principles are a subset of many principles promoted by American software engineer and instructor **Robert C. Martin**, [220]- [221]- [222].

They were first introduced in his 2000 paper Design Principles and Design Patterns, while the acronym was introduced later, around 2004, by **Michael Feathers** [230].

Although the SOLID principles apply to any object-oriented design, they can also form a core philosophy for methodologies such as agile development or adaptive software development.

20.1 Single-Responsibility

"There should never be more than one reason for a class to change." [224]

In other words, every class should have only one responsibility. [225]

Smell: a “God” or kitchen-sink type that mixes persistence, formatting, and notification — any product change lands in the same file.

Fix: split along axes of change; keep orchestration thin and push each concern into its own type.

Interview tip: say *reason to change* (actor/concern), not “small class” — SRP is cohesion of change, not line count.

20.2 Open-Close

"Software entities ... should be open for extension, but closed for modification." [226]

New behaviour should plug in without editing stable, already-working code.

Smell: a growing switch/if-else on type tags, or scattered edits to a “core” module every time a variant appears.

Fix: extend via polymorphism, strategies, or plugins; seal the stable path and add new implementations beside it.

Interview tip: name what stays closed (the hot path / published API) and what opens (a new subtype or strategy) — OCP is not “never touch a file again”.

20.3 Liskov Substitution

"Functions that use pointers or references to base classes must be able to use objects of derived classes without knowing it." [227]

See also: [Design by contract](#).

Subtypes must honour the base contract so callers can swap implementations safely.

Smell: a derived type that weakens preconditions, strengthens postconditions the wrong way, throws unexpected exceptions, or no-ops a base method (classic square/rectangle and “not supported” overrides).

Fix: redesign the hierarchy or split interfaces so every subtype truly is-a usable substitute; prefer composition when inheritance breaks the contract.

Interview tip: walk one failing substitution (caller assumes base behaviour, derived surprises it) — LSP is behavioural substitutability, not just shared method names.

20.4 Interface Segregation

"Clients should not be forced to depend upon interfaces that they do not use." [228] [223]

Prefer small, role-focused interfaces over fat ones that dump unused methods on every client.

Smell: a fat interface where many implementers leave methods empty, throw, or ignore

half the API.

Fix: carve role interfaces (read vs write, poll vs push) so each client depends only on what it calls.

Interview tip: point at unused methods on an implementer — that unused surface is the ISP smell, not “too many files”.

20.5 Dependency Inversion

"Depend upon abstractions, [not] concretions." [229] [223]

High-level policy should not hard-wire low-level details; both should depend on shared abstractions.

Smell: domain logic that constructs concrete I/O, DB, or vendor SDKs directly — swapping a dependency means editing the core.

Fix: introduce an interface (or port) owned by the high level; inject the concrete adapter at the boundary.

Interview tip: draw who owns the abstraction — DIP inverts the usual “UI depends on DB” arrow so details plug into policy, not the reverse.

Solid Principles Bibliography

- [220] Robert C. Martin. *Principles of OOD*. [WayBack Machine](#). 2014.
- [221] Robert C. Martin. *Getting a SOLID start*. Uncle Bob Consulting LLC. 2009.
- [222] Sandu Metz. *SOLID Object-Oriented*. [Ghost Archive - Gotham Ruby Conference](#). 2009.
- [223] Robert C. Martin. *Design Principles and Design Patterns*. [Internet Archive - Objectmentor](#). 2000.
- [224] Robert C. Martin. *Single Responsibility Principle*. [Internet Archive - Objectmentor](#). 2015.
- [225] Robert C. Martin. *Agile Software Development, Principles, Patterns, and Practices*. [Internet Archive - Objectmentor](#). 2003.
- [226] *Open/Closed Principle*. [Internet Archive - Objectmentor](#).
- [227] *Liskov Substitution Principle*. [Internet Archive - Objectmentor](#).
- [228] *Interface Segregation Principle*. [Internet Archive - Objectmentor](#).
- [229] *Dependency Inversion Principle*. [Internet Archive - Objectmentor](#).
- [230] *Clean Architecture: A Craftman's Guide to Software Structure and Design*.
- [231] *Clean Architecture: A Craftman's Guide to Software Structure and Design*. [here](#).

UML and OOP

21.1 UML

21.1.1 Class Diagram

In Unified Modeling Language, a class diagram *describes the static structure of a system* by showing the system's classes, their attributes, operations (methods) and the relationship among them.

A class diagram is the *conceptual modeling of the structure of the software application*.

In case of detailed modeling, the class diagram can be *translated into programming code*.

Visibility

Visibility in a class diagram maps to language *access control*: who may read or call a member from outside the class (and package/module rules where they exist). Model it when you need a shared picture of encapsulation — design reviews, onboarding, or interview whiteboard sketches of an API surface. This notation is placed before methods

- + public
- private
- # protected
- ~ package

and attributes names, to indicate their visibility.

Scope

UML defines *instance and class scopes for members*.

Scope maps to *instance vs static*: per-object state and behaviour versus members shared by the type. The latter is represented by underlined names. Sketching scope on a diagram helps interviews and reviews catch “who owns this data” mistakes before code.

- **instance members** are scoped to a specific instance
- **class members** are the one commonly recognized as **static**

Arrows

The relationship defined in UML are represented by the following logical connectors

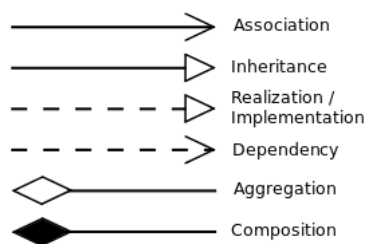


Figure 21.1: UML logical connectors

- **Association**

a *relationship between classes of objects* that allows one object instance to cause another *to perform an action on its behalf*.

This relationship is structural and does not represent any behavioural trait.

Code implementation usually requires the requesting object to invoke a method or member function using a *reference or a pointer to the controlled object*.

Association can be *bidirectional, unidirectional, aggregation and reflexive*.

Objects related via the association are considered to act in a role with respect to the association.

The ends of the association can have all the characteristics of a property:

multiplicity, name, visibility and a specification explaining whether the association is ordered and / or unique.

- **Dependency**

a relationship showing that *an element requires other model elements* for their specification or implementation.

The dashed lines from the dependent/client to the independent/supplier element specifies *the direction of a relationship and not of a process*.

- **Aggregation**

a variant of *has a association relationship and more specific than the latter*. It represents a part-whole or a part-of relationship. As a type of association, an aggregation can be named and have the same adornments an association can.

An aggregation *can occur when a class is a collection or a container of other classes*, but *the content of the container continues to exist when the container is destroyed*. In UML notation, the *hollow diamond shape is drawn on the containing class end* of the connecting line.

An example of aggregation is a Pond which has zero or more Ducks and Duck which has at most one Pond.

A duck can exist separately from a pond.

- **Composition**

refers to *combining objects within compound objects, ensuring the encapsulation* of each object by *using the well-defined interface without exposing the internals*. Differently, data structures do not enforce encapsulation.

Composition can be regarded as a *relationship between types*: an object of composite type *has* objects of other types

Composition must be distinguished from subtyping, which is the process of adding detail to a general data type to create a more specific one.

In UML notation, the *filled diamond shape is drawn on the containing class end* of the connecting line. An example of composition is a University which has one or more Departments, which belong exactly to one University.

A Department can not exist as a separate entity detached from a specific University.

Class-level relationships

- **Generalization/Inheritance**

a relationship between two classes, in which the *subclass* is considered to be a specialization from the *super type*.

Any instance of the subtype is also an instance of the superclass. As an example, *humans are a subclass of simian which is a subclass of mammal*.

Generalization is also known as **inheritance** or a *is a* relationship.

The base class is also known as parent, superclass, base class or base type.

The subtype is also known as child, subclass, derived class, derived type, inheriting class or inheriting type.

- **Realization/Implementation**

a relationship between two model elements, in which the *client* realizes (implements or executes) the behavior that the *supplier* specifies.

General relationship

- **Dependency**

a weaker form of bond that indicates that one class depends on another because it uses it at some point in time. Sometimes this relationship might be implemented as member function arguments.

- **Multiplicity**

an optional notation which can be useful to add details to the association relationship. At each end of the line, one of the following notation can be used to indicate the range of number of objects that participate in the association from the perspective of the other end.

0	No instances
0..1	No instances, or one instance
1	Exactly one instance
1..1	Exactly one instance
0..*	Zero or more instances
*	Zero or more instances
1..*	One or more instances

21.1.2 Sequence Diagram

A [sequence diagram](#) or [system sequence diagram](#) shows process interactions arranged in a time sequence. The diagram depicts the processes and objects involved and the sequence of messages exchanged as needed to carry out the functionality.

In the [4+1 architectural view model](#) of the system under development, sequence diagrams are typically associated with use case realizations [237].

Sequence diagrams are sometimes called [event diagrams](#) or [event scenarios](#).

Lifelines

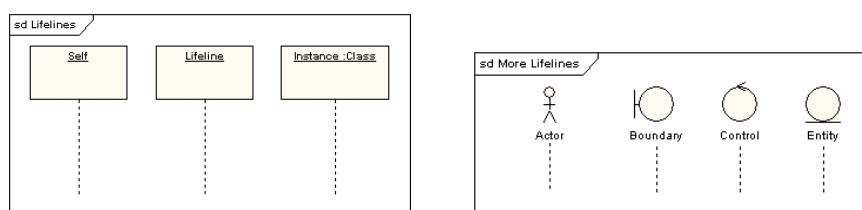


Figure 21.2: *Sequence Diagram Lifelines and Actors*

A Lifeline is shown using a symbol that represents its “head” followed by a vertical line (which may be dashed) that represents the lifetime of the participant.

The head of the line can also be the representation of an actor.

Execution Specification

Also known by the name of [Activation](#) is interaction fragment which represents a period in the participant’s lifetime when it is

- executing a unit of behavior or action within the lifeline,
- sending a signal to another participant,
- waiting for a reply message from another participant.

The duration of an execution is represented by two execution occurrences - the start occurrence and the finish occurrence.

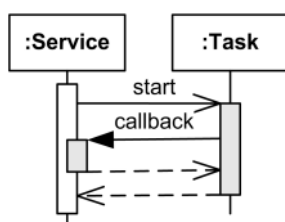


Figure 21.3: *Overlapping Execution*

Execution is represented as a thin grey or white rectangle on the lifeline.

Overlapping executions on the same lifeline are represented by overlapping rectangles.

Messages

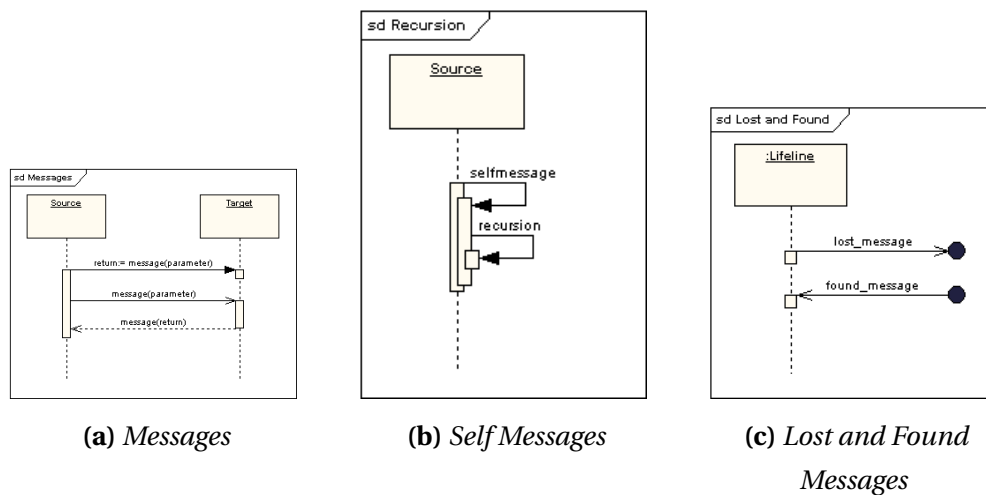


Figure 21.4: Sequence Diagram Messages

Message is a named element that defines one specific kind of communication between lifelines of an interaction. The message specifies not only the kind of communication, but also the sender and the receiver. Sender and receiver are normally two occurrence specifications (points at the ends of messages).

A message is shown as a line from the sender message end to the receiver message end. The line must be such that every line fragment is either horizontal or downwards when traversed from send event to receive event. The send and receive events may both be on the same lifeline. The form of the line or arrowhead reflects properties of the message. Depending on the type of action that was used to generate the message, message could be one of:

- synchronous call
- asynchronous call
- asynchronous signal
- create
- delete
- reply

Lifeline Start and End

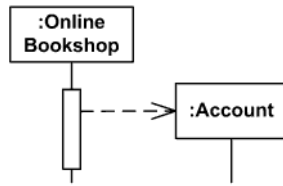


Figure 21.5: *Object-Lifetime Start*

Create message is sent to lifeline to create itself. Note, that it is weird but common practice in OOAD to send create message to a nonexisting object to create itself. In real life, create message is sent to some runtime environment.

Create message is shown as a dashed line with open arrowhead (same as reply), and pointing to created lifeline's head.

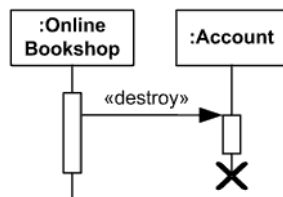


Figure 21.6: *Object-Lifetime End*

Delete message (called stop in previous versions of UML) is sent to terminate another lifeline. The lifeline usually ends with a cross in the form of an X at the bottom denoting destruction occurrence.

UML 2.3 specification provides neither specific notation for delete message nor a stereotype. Until they provide some notation, we can use custom «destroy» stereotype.

Duration and Time Constraints

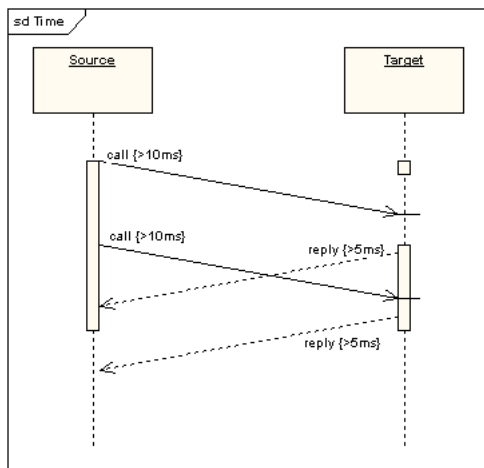


Figure 21.7

Combined Fragments

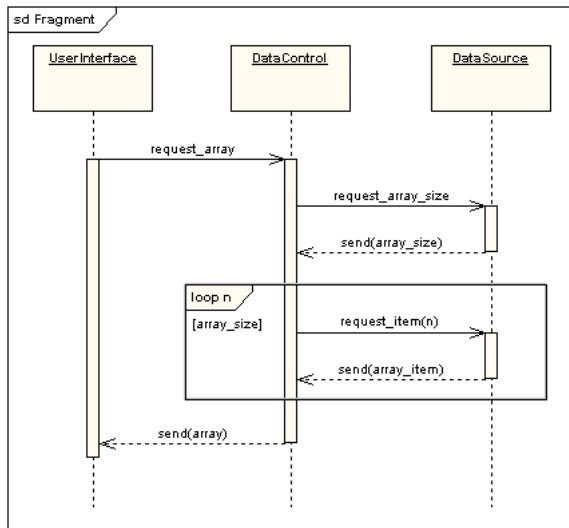


Figure 21.8: *Loop Fragment*

A combined fragment is one or more processing sequence enclosed in a frame and executed under specific named circumstances. In fig 21.8 there is an example of a loop fragment.

The available fragments are listed in table 21.1.

Label	Function
alt	fragment to model if...then...else constructs
opt	fragment to model switch constructs
loop	fragment to enclose a series of messages which are repeated
par	fragment to model concurrent processing
seq	weak sequencing fragment to enclose a number of sequences for which all the messages must be processed in a preceding segment before the following segment can start, but which does not impose any sequencing within a segment on messages that don't share a lifeline
strict	strict sequencing fragment to enclose a series of messages which must be processed in the given order
neg	negative fragment to enclose an invalid series of messages
assert	fragment to designate that any sequence not shown as an operand of the assertion is invalid
break	models an alternative sequence of events that is processed instead of the whole of the rest of the diagram
critical	encloses a critical section
ignore	declares a message or message to be of no interest if it appears in the current context
consider	fragment is in effect the opposite of the ignore fragment: any message not included in the consider fragment should be ignored

Table 21.1: *Available Fragments*

Gate

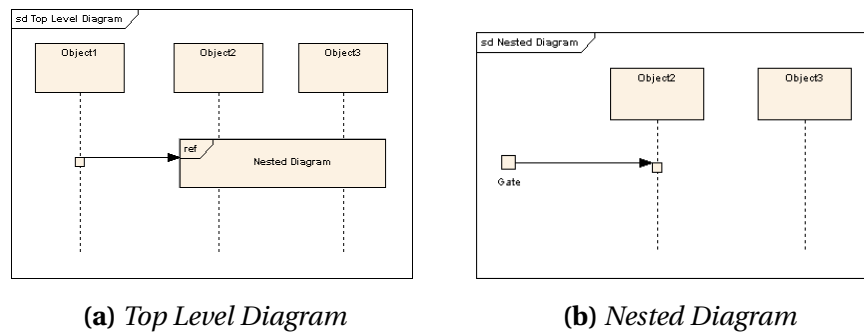


Figure 21.9: *Nested Sequence Diagrams*

A gate is a message end, connection point for relating a message outside of an interaction fragment with a message inside the interaction fragment.

The purpose of gates and messages between gates is to specify the concrete sender and receiver for every message. Gates play different roles:

- formal gates - on interactions
- actual gates - on interaction uses
- expression gates - on combined fragment

The gates are named implicitly or explicitly. Implicit gate name is constructed by concatenating the direction of the message ("in" or "out") and the message name, e.g. `in_search`, `out_read`.

Gates are notated just as message connection points on the frame.

Part Decomposition

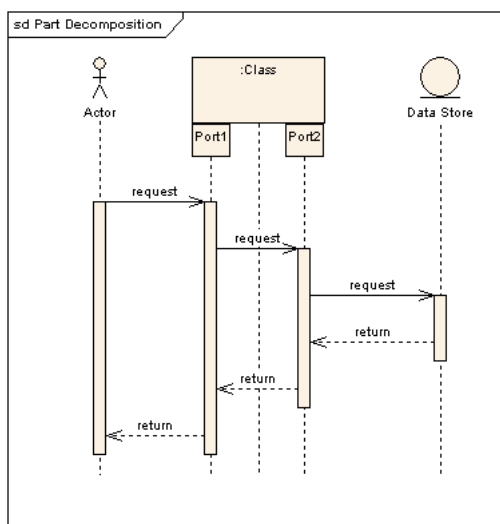


Figure 21.10: *Object with*

An object can have more than one lifeline coming from it.

This allows for inter and intra object messages to be displayed on the same diagram.

State Invariant

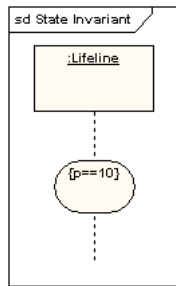


Figure 21.11: *State Invariant*

A state invariant is an interaction fragment which represents a run-time constraint on the participants of the interaction.

It may be used to specify different kinds of constraints, such as values of attributes or variables, internal or external states, etc.

State invariant is usually shown as a constraint in curly braces on the lifeline.

It could also be shown as a state symbol representing the equivalent of a constraint that checks the state of the object represented by the lifeline. This could be either the internal state of the classifier behavior of the corresponding classifier or some external state based on a "black-box" view of the lifeline.

Class and Sequence diagram Cheatsheet

Following [a summary of the information for class and sequence UML diagrams](#)

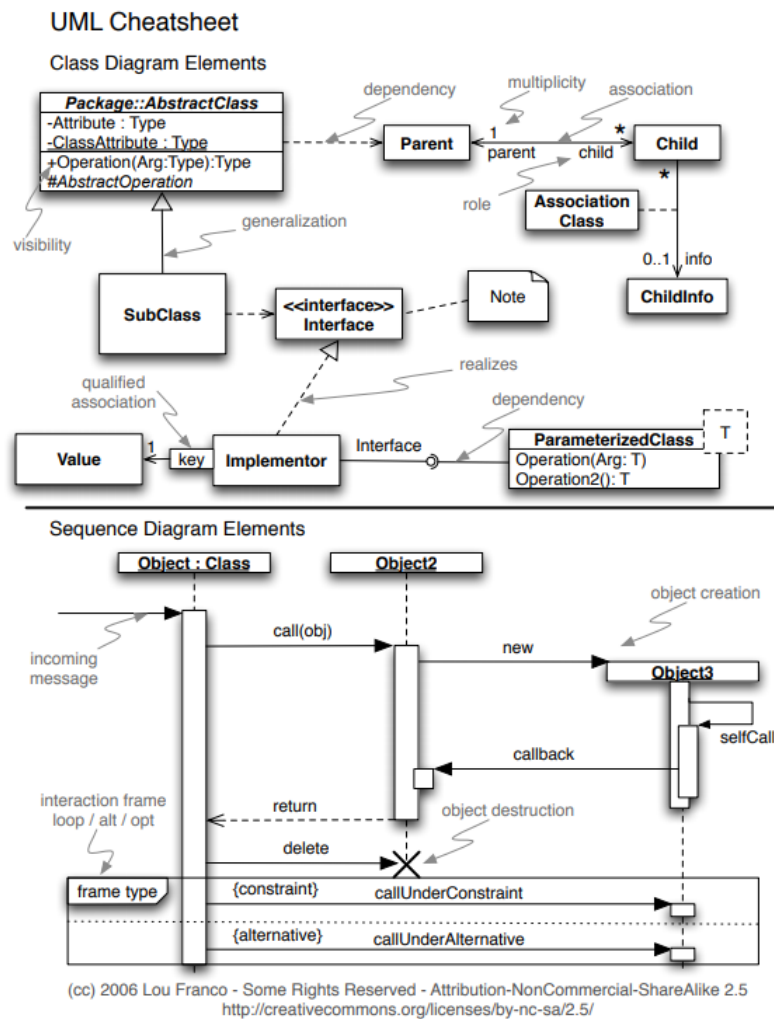


Figure 21.12: Class and Sequence diagrams cheatsheet

21.2 Object Oriented Programming

21.2.1 Polymorphism

Polymorphism means the [same syntactic call](#) can select [different implementations](#). In C++ it helps to separate two axes (Figure 21.13):

- **Binding time** — compile-time ([static](#) / early) vs run-time ([dynamic](#) / late).
- **Kind** (Cardelli) — [ad-hoc](#), [parametric](#), or [subtype](#).

Tutorial diagrams that only show “overloading vs virtuals” miss templates and CRTP; both are important in modern C++ (and in HFT hot paths, static dispatch is often preferred).

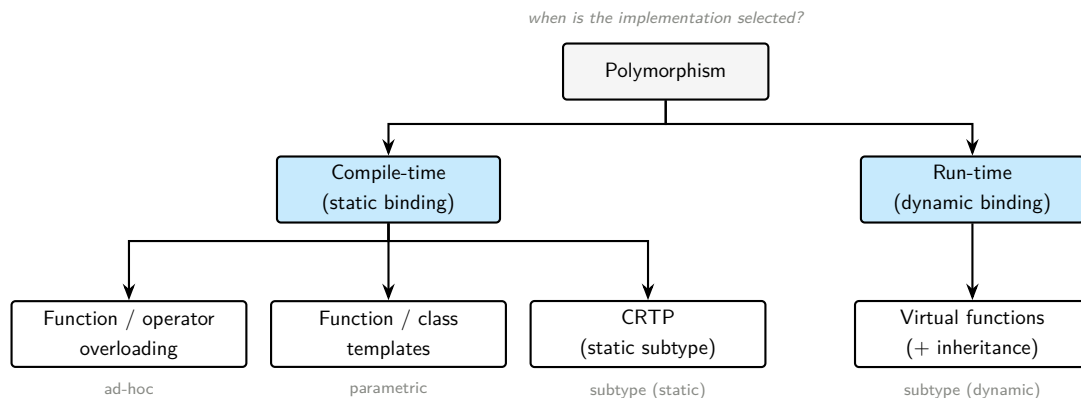


Figure 21.13: *Polymorphism in C++: binding time (top split) and kind (labels under leaves).*

Compile-time polymorphism

The implementation is fixed when the call is compiled ([static binding](#)). No vtable lookup on the hot path.

Ad-hoc: overloading

[Function](#) and [operator](#) overloading: the compiler picks an overload from the argument types (and count). Same name, different signatures — resolved entirely at compile time.

Parametric: templates

Function and class [templates](#) are [parametric](#) polymorphism: one definition is [monomorphized](#) per used type set. The generated code is specialized and usually inlinable. Cost: code size (and no single binary entry point usable from an unknown dynamic library client without type erasure).

Static subtype: CRTP

Inheritance can also be static: the Curiously Recurring Template Pattern (CRTP) lets a base call into the derived type via `static_cast`, without virtuals. See Section 2.1.12.

Interview tip: “compile-time polymorphism” is not only overloading — say `templates` (and CRTP when relevant). Prefer static dispatch on latency-critical paths when the set of types is known at compile time.

Run-time polymorphism

The implementation is selected `while the program runs` (*dynamic binding*), typically through a `virtual` function.

- Declare `virtual` in a base; override in derived types. Call through a `base pointer or reference` so the dynamic type can differ from the static type.
- Mechanically: object has a `vptr`; the call indexes a `vtable`. Extra indirection vs a direct/inlined call; harder for the optimizer to see through.
- A class with at least one pure virtual (= 0) is *abstract* and cannot be instantiated (Section 21.2.2).
- If you delete through a base pointer, the base destructor should be `virtual` (otherwise undefined behaviour).

Dynamic dispatch is the usual tool when the concrete type is not known until run time (plugins, heterogeneous containers of “instruments”, strategy objects swapped live). When the ABI must stay stable across a shared library boundary and clients are unknown, dynamic dispatch (or type erasure) is often necessary — templates alone cannot monomorphize what the library never saw.

Interview tip: virtual call = base ptr/ref + virtual + override; cost is indirection / missed inlining, not “virtual is always slow”. Use `override` / `final` to make intent checkable.

21.2.2 Interface vs Abstract Class

C++ has `no interface keyword`. In interview / Java / C# language, people still say “interface”; map that vocabulary onto C++ cleanly.

- An *abstract class* is any class with *at least one* pure virtual function (`virtual ... = 0`). It may also have data members, constructors, and methods with bodies. You cannot instantiate it directly.
- An *interface* (in the Java/C# sense) is modelled in C++ as a *pure abstract class*: typically *only* pure virtual functions (plus often a virtual destructor), *no* data members, and no implemented behaviour beyond defaults you deliberately allow (e.g. C++20 = `default` on the destructor).

	Interface (C++ idiom)	Abstract class
Language support	Convention (pure ABC), not a keyword	At least one = 0 virtual
Data members	Prefer none	Allowed (shared state)
Implemented methods	Prefer none (contract only)	Allowed (shared behaviour)
Multiple inheritance	Common for interface mix-in	Heavier; watch the diamond
Typical use	Stable API / capability surface	Partial implementation + extension points

```

1 // "Interface": pure abstract class (contract only)
2 struct IOrderSink {
3     virtual ~IOrderSink() = default;
4     virtual void on_fill(int id, int qty, double px) = 0;
5 };
6
7 // Abstract class: pure virtual + shared state / helpers
8 struct Instrument {
9     explicit Instrument(std::string s) : symbol_(std::move(s)) {}
10    virtual ~Instrument() = default;
11    virtual double mid() const = 0; // must
12    implement
13    const std::string& symbol() const { return symbol_; } //
14    shared
15 protected:
16     std::string symbol_;
17 };

```

Interview tip: say “C++ has no interface keyword; we use pure abstract classes.” Contrast with Java interface / abstract class only after that mapping is clear.

UML and OOP Bibliography

- [232] *The Unified Modeling Language*. uml-diagrams.org.
- [233] *UML Core Elements*. uml-diagrams.org.
- [234] *UML Association vs. Aggregation vs. Composition*. visual-paradigm.com.
- [235] *Sequence Diagrams Reference*. uml-diagrams.org.
- [236] *Object-Oriented Programming*. [Wikipedia - Object Oriented Programming](https://en.wikipedia.org/wiki/Object-oriented_programming).
- [237] *4+1 architectural view model*. [Wikipedia - 4+1 architectural view model](https://en.wikipedia.org/wiki/4+1_architectural_view_model).
- [238] *The Sequence Diagram*. [Sequence Diagram Templates and Examples](https://www.sequence-diagram.com).
- [239] *UML 2 Tutorial - Sequence Diagram*. [developer.ibm.com/articles](http://developer.ibm.com/articles/uml2tutorial) .
- [240] *The Sequence Diagram*. [developer.ibm.com/articles](http://developer.ibm.com/articles/uml2tutorial) .
- [241] *Object-Oriented Programming Using C++*. [Weber State University - School of Computing - Computer Science](http://www.weber.edu/~cs).
- [242] Allen Holub. *Allen Holub's UML Quick Reference*. [Allen Holub website](http://www.alanholub.com).
- [243] *Class Diagram*. [WikiPedia - Class Diagram](https://en.wikipedia.org/wiki/Class_Diagram).
- [244] *State Diagram*. [WikiPedia - State Diagram](https://en.wikipedia.org/wiki/State_Diagram).
- [245] *State Machine*. [WikiPedia - State Machine](https://en.wikipedia.org/wiki/State_machine).
- [246] *Programming paradigm*. [WikiPedia - Programming Paradigm](https://en.wikipedia.org/wiki/Programming_paradigm).
- [247] *Composition over Inheritance*. [WikiPedia - Composition over Inheritance](https://en.wikipedia.org/wiki/Composition_over_inheritance).
- [248] *Delegation (object-oriented programming)*. [WikiPedia - Delegation](https://en.wikipedia.org/wiki/Delegation_(object-oriented_programming)).
- [249] *Role-oriented programming*. [WikiPedia - Role-oriented programming](https://en.wikipedia.org/wiki/Role-oriented_programming).
- [250] James Rumbaugh. *Object-Oriented Modeling and Design*. [Internet Archive](http://www.internetarchive.org). 1991.
- [251] *Delegation (object-oriented programming)*. [WikiPedia - Delegation](https://en.wikipedia.org/wiki/Delegation_(object-oriented_programming)).



Other Topics

Python

22.1 PEP

22.2 Built-in Data Types

Python has several [types built in the interpreter](#); the principal ones are numerics, sequences, mappings, classes, instances and exceptions.

22.2.1 dict

22.2.2 list

22.2.3 set

22.2.4 frozenset

22.2.5 tuple

22.2.6 string

22.2.7 bytes

22.2.8 bytearray

22.3 unittest Module

22.3.1 Classes and functions

22.4 Decorator

A function decorator is a callable object that takes a function as its input parameter
[decorator](#)

[Decorators with parameters](#)

[PythonDecoratorLibrary](#)

22.5 Special Instance Method

Special Instance Method

22.6 Providing Multiple Ctors

multiple `__init__`

Python Bibliography

- [252] *Python Glossary*. docs.python.org.
- [253] *Providing Multiple Constructors in Your Python Classes*. [Real Python](#).
- [254] *Sorting Mini How To*. wiki.python.org.
- [255] *Thread-based parallelism*. docs.python.org.
- [256] *Process-based parallelism*. docs.python.org.
- [257] *Asynchronous I/O*. docs.python.org.
- [258] *Asyncio in Python - Full Tutorial*. [YouTube](#).

23

CHAPTER

Software Testing

23.1 Seven Testing Principles

23.2 SDLC vs. STLC

23.3 Python Unit Testing

23.3.1 unittest

`unittest` — Unit testing framework

It supports test automation, sharing of setup and shutdown code for tests, aggregation of tests into collections, and independence of the tests from the reporting framework.

To achieve this, unittest supports some important concepts in an object-oriented way:

- **test fixture** A test fixture represents the *preparation needed* to perform one or more tests, and any associated *cleanup actions*. This may involve, for example, creating temporary or proxy databases, directories, or starting a server process.
- **test case** A test case is the *individual unit of testing*. It checks for a specific response to a particular set of inputs. `unittest` provides a base class, `TestCase`, which may be used to create new test cases.
- **test suite** A test suite is a *collection of test cases, test suites, or both*. It is used to aggregate tests that should be executed together.

- **test runner** A test runner is a *component which orchestrates* the execution of tests and provides the outcome to the user. The runner may use a graphical interface, a textual interface, or return a special value to indicate the results of executing the tests.

A testcase is created by subclassing `unittest.TestCase` or using `FunctionTestCase` for legacy compatibility

Child class methods' names start by `test_` as a naming convention to inform the test runner.

`unittest.main` provides a command line interface to execute tests.

In the following tables 23.1, 23.2, 23.3 and 23.4 a *list of assert methods provided by TestCase*

Method	Checks that	New in
<code>assertEqual(a, b)</code>	<code>a == b</code>	
<code>assertNotEqual(a, b)</code>	<code>a != b</code>	
<code>assertTrue(x) bool(x)</code>	<code>is True</code>	
<code>assertFalse(x) bool(x)</code>	<code>is False</code>	
<code>assertIs(a, b)</code>	<code>a is b</code>	3.1
<code>assertIsNot(a, b)</code>	<code>a is not b</code>	3.1
<code>assertIsNone(x)</code>	<code>x is None</code>	3.1
<code>assertIsNotNone(x)</code>	<code>x is not None</code>	3.1
<code>assertIn(a, b)</code>	<code>a in b</code>	3.1
<code>assertNotIn(a, b)</code>	<code>a not in b</code>	3.1
<code>assertIsInstance(a, b)</code>	<code>isinstance(a, b)</code>	3.2
<code>assertNotIsInstance(a, b)</code>	<code>not isinstance(a, b)</code>	3.2

Table 23.1: *TestCase assert methods.*

Method	Checks that	New in
<code>assertRaises(exc, fun, *args, **kwds)</code>	fun raises exc	
<code>assertRaisesRegex(exc, r, fun, *args, **kwds)</code>	fun raises exc and the message matches regex r	3.1
<code>assertWarns(warn, fun, *args, **kwds)</code>	fun raises warn	3.2
<code>assertWarnsRegex(warn, r, fun, *args, **kwds)</code>	fun raises warn and the message matches regex r	3.2
<code>assertLogs(logger, level)</code>	The with block logs on logger with minimum level	3.4
<code>assertNoLogs(logger, level)</code>	The with block does not log on logger with minimum level	3.10

Table 23.2: *TestCase methods to check the production of exceptions.*

Method	Checks that	New in
<code>assertAlmostEqual(a, b)</code>	<code>round(a-b, 7) == 0</code>	
<code>assertNotAlmostEqual(a, b)</code>	<code>round(a-b, 7) != 0</code>	
<code>assertGreater(a, b)</code>	<code>a > b</code>	3.1
<code>assertGreaterEqual(a, b)</code>	<code>a >= b</code>	3.1
<code>assertLess(a, b)</code>	<code>a < b</code>	3.1
<code>assertLessEqual(a, b)</code>	<code>a <= b</code>	3.1
<code>assertRegex(s, r)</code>	<code>r.search(s)</code>	3.1
<code>assertNotRegex(s, r)</code>	<code>not r.search(s)</code>	3.2
<code>assertCountEqual(a, b)</code>	a and b have the same elements in the same number, regardless of their order.	3.2

Table 23.3: *TestCase methods to perform more specific checks*

Method	Used to compare	New in
<code>assertMultiLineEqual(a, b)</code>	strings	3.1
<code>assertSequenceEqual(a, b)</code>	sequences	3.1
<code>assertListEqual(a, b)</code>	lists	3.1
<code>assertTupleEqual(a, b)</code>	tuples	3.1
<code>assertSetEqual(a, b)</code>	sets or frozensets	3.1
<code>assertDictEqual(a, b)</code>	dicts	3.1

Table 23.4: *TesCase* type-specific methods automatically used by *assertEqual*

Organizing test code

`unittest.TestCase.setUp()`

In case of a considerable number of tests to run, it is possible to factor out the set-up using .

The `TestCase.setUp` method does nothing on default, but can be overridden by child classes in order to prepare the `test fixture`.

Any exception different than `AssertionError` or `SkipTest` raised by this method is interpreted as an error rather than a test failure.

`unittest.TestCase.tearDown()`

The `TestCase.tearDown()` method is called immediately after the test results are recorded, even if the test method raised an exception. The only requirement for this method to be called is the success of `setUp()` call.

Grouping Tests

23.3.2 doctest

`doctest`

23.4 C++ Unit Testing

23.4.1 gMock

`gMock for Dummies`

`gMock Cookbook`

23.5 Integration Test

Integration testing *checks the communication and data flow between two modules of the same system after integrating them.*

A software consists of *several small modules* that carry certain features and come together to build the system. Sometimes even when these modules pass the quality test

independently, they *might encounter data flow issues upon integration*.

Integration testing verifies the *compatibility between these individual parts* and ensures they work in unison *without any issues after integration*. It can be a black-box testing or white-box testing technique.

An example of integration testing would be checking the logical connection between the 'Proceed to Payment' page and the payment gateway for an e-commerce application.

23.6 Regression Test

Regression testing *re-checks software or an application after new changes are introduced*.

This testing process runs *functional and non-functional checks on the existing features of the software to ensure that no issues arise due to new code changes*.

The goal of regression testing is *to identify any defects in the present system that may occur due to updates/changes*. Regression testing can either be white box testing or black box testing.

You can *re-use the test cases* in regression testing to verify the existing functionalities. This includes running the same test cases and scripts to get the same output that validates the present features.

An example of regression testing is checking the Camera, Phone, and other applications in your mobile after a system update. Smartphones often receive regular system updates that can hamper the ongoing functions of the phone. Regression testing would assess if any OS or parts upgrade impacted your phone.

23.7 Behavior-Driven Development

23.8 terms from the youtube video

- decision coverage
- predicate coverage
- all path coverage
- boundary coverage
- black box testing
- white box testing

23.9 Continous Integration Systems

- BuildBot
- Jenkins

- [GitHub Actions](#)
- [AppVeyor](#)

Testing Bibliography

- [259] *Open Lecture by James Bach on Software Testing*. [Eesti Infotehnoloogia Kolledž YouTube Channel](#).
- [260] *Regression Testing*. [Wikipedia](#).
- [261] *Progression Testing*. [Wikipedia](#).
- [262] *Regression Testing vs Progression Testing*. [Test Sigma](#).
- [263] Kent Beck. *Simple Smalltalk Testing: With Patterns*. [webarchive](#).

24

CHAPTER

AI

Prompt Engineering Basics
ChatGPT Prompts Mastery
Intro to Generative AI
AI Introduction by Harvard
Microsoft GenAI Basics
Prompt Engineering Pro
Google's Ethical AI
Harvard Machine Learning
LangChain App Developer
Bing Chat Applications
Generative AI by Microsoft
Amazon's AI Strategy
GenAI for Everyone
AWS GenAI Foundation
OpenCV Bootcamp
Tensorflow Bootcamp

AI Bibliography

- [264] Implementazioni e spiegazioni da uno dei migliori ricercatori del campo.
- [265] IL libro di introduzione.
- [266] Come il primo link, più in depth con più matematica.
- [267] Ricetta per allenare modelli.
- [268] Pytorch, la libreria per ml in python.
- [269] ML fundamentals.



PART

Reference

25

CHAPTER

Dictionary

25.0.1 A

ABA • CAS failure mode where a location returns to an old value after intervening updates, so compare-and-swap succeeds incorrectly; mitigated with tags or safe reclamation (Chapter 8).

ADL • Argument Dependent Lookup

ADT • Abstract Data Type

AF_XDP • Address Family XDP: Linux socket type for high-performance packet I/O with zero-copy options; a kernel-bypass-adjacent path (Chapter 14).

Affinity • binding a thread (or IRQ) to a CPU set so it does not migrate; pairs with `isolcpus` and NUMA placement (Chapter 7).

Allocate • to assign, allot, distribute, or set apart for a particular purpose

Allocator (C++) • type that satisfies the Allocator requirements and supplies `allocate/deallocate` for con-

tainers; PMR variants use a runtime `memory_resource` (Section 3.1.2).

Amortized • cost of an operation averaged over a long sequence (e.g. `vector::push_back`); one step may be expensive, the sequence is cheap (Section 17.1.1).

Arena • preallocated memory region from which many objects are carved; avoids hot-path `malloc` (Chapter 3).

Atomic • Objects of atomic types are the only C++ objects that are free from data races; that is, if one thread writes to an atomic object while another thread reads from it, the behavior is well-defined. In addition, accesses to atomic objects may establish inter-thread synchronization and order non-atomic memory accesses as specified by `std::memory_order`.

25.0.2 B

BBO • Best Bid and Offer: top-of-book prices (and often sizes); the first level of the order book (Chapter 17).

Big O • asymptotic upper-bound notation for growth of time or space with input size (Chapter 17).

Black-Scholes • classic continuous-time option pricing model; basis for implied volatility (Chapter 16).

branch • a mean for teams to develop features giving them a separate workspace for their code. The branch to create are part of the branching strategy, which could be different depending on the specific svn used.

Busy polling • spinning on I/O or a queue instead of sleeping (e.g. `SO_BUSY_POLL`, `epoll timeout 0`); lower latency, higher CPU use (Chapter 15).

25.0.3 C

Cache coherence • protocol that keeps multiple CPU caches consistent for shared lines (e.g. MESI); basis of false sharing (Chapter 9).

Cache line • granularity of cache transfers and coherence (often 64 B); pad hot fields to avoid false sharing (Chapter 9).

CAS • Compare-And-Swap: atomic read-modify-write that updates a location only if it still holds an expected value (`compare_exchange_*` in C++); basis of many lock-free structures (Chapter 8).

CFS • Completely Fair Scheduler: default Linux policy (`SCHED_OTHER`); migrates work across CPUs unless isolated (Chapter 7).

Context switch • saving/restoring thread or process CPU state when the scheduler runs another task; costs μ s and adds jitter (Chapter 7).

constexpr • a C++ specifier which declares that it is possible to evaluate the value of the function or variable at compile time.

CPU affinity • see Affinity.

25.0.4 D

data race • undefined behavior in C++ when two threads access the same memory location, at least one access is a write, and the accesses are not ordered by the memory model (Chapter 8).

deadlock • any situation in which no member of some group of entities can proceed because each waits for another member, including itself, to take action, such as sending a message or, more commonly, releasing a lock

Dependency injection • design technique where a component receives its dependencies from the outside (constructor/setter/framework) instead of creating them itself; improves testability and decoupling.

DPDK • Data Plane Development Kit: user-space packet processing with poll-mode drivers; a common kernel-bypass stack (Chapter 14).

DSU • Disjoint Set Union (Union-Find): nearly $O(1)$ amortized connectivity queries (Section 18.11.5).

duck typing • a concept related to *dynamic typing*, where the type or the class of an object is less important than the methods it defines

double dispatch • choosing a method from the runtime types of *two* objects (e.g. Visitor): a single virtual call is not enough; typically one virtual on the element then a virtual on the visitor.

dynamic binding • selecting which function body runs at run time (typically via virtual functions); opposed to static / early binding (Section 21.2.1).

25.0.5 E

encapsulation • preventing unauthorized access to some piece of information or functionality

Epoch reclaim • safe memory reclamation for lock-free structures by retiring nodes only after all threads leave a shared epoch (Chapter 8).

EPOLLET • edge-triggered epoll: notify on state change; must drain until EAGAIN (Chapter 12).

epoll • Linux scalable I/O event notification (`epoll_create/ctl/wait`); preferred over `select/poll` for many file descriptors (Chapter 12).

25.0.6 F

False sharing • performance pathology where threads update different variables on the same cache line, causing coherence ping-pong (Chapter 9).

First-touch • NUMA page placement: a page is usually bound to the node of the CPU that first writes it; pin threads before warm-up (Chapter 10).

FIX • Financial Information eXchange: tag-value messaging standard for many order/broker links; not usually the colocated matching-engine hot path (Section 16.3).

FOK • Fill-or-Kill: execute the full quantity immediately or cancel (Chapter 16).

25.0.7 G

Greeks • option risk sensitivities (e.g. delta, gamma, theta, vega) (Chapter 16).

GTC • Good-Till-Cancel: order remains until filled or cancelled (Chapter 16).

25.0.8 H

Happens-before • memory-model relation that defines when one operation's side effects are visible to another; established by mutexes, atomics, etc. (Chapter 8).

Hazard pointers • safe reclamation scheme: readers publish pointers they may dereference; reclaimers skip those nodes (Chapter 8).

Heap • a large pool of memory used to satisfy memory request by allocating part.

Huge pages • large page sizes (e.g. 2 MiB) that reduce TLB pressure for big working sets (Chapter 9).

25.0.9 I

Idiom • a group of code fragments sharing an equivalent semantic role (meaning), which recurs frequently across software projects often expressing a special feature in one or more programming languages or libraries.

Implied vol • market price of an option expressed as the Black–Scholes volatility that matches it (Chapter 16).

Interrupt coalescing • NIC/driver (or OS) batches several packet arrivals into fewer hard IRQs to raise throughput and cut interrupt rate. That saves CPU but adds latency and jitter — usually the wrong trade for tick-to-trade paths (Chapter 15).

IOC • Immediate-or-Cancel: take what is available now, cancel the rest (Chapter 16).

I/O mux • waiting on many file descriptors for readiness (`select/poll/epoll`) (Chapter 12).

irqaffinity • kernel boot parameter setting the default CPU mask for IRQs; keep defaults on housekeeping cores so isolated CPUs stay quiet (Section 15.1.4).

irqbalance • userspace daemon that reshuffles IRQ-to-CPU affinity at runtime; disable or constrain on latency boxes or it undoes manual `smp_affinity` (Section 15.1.4).

isolcpus • kernel boot parameter that removes CPUs from the general scheduler balance set (scheduling exclusion); used with pinning for dedicated HFT cores (Section 15.1.1).

ITCH • family of binary multicast market-data protocols (order-book incremental updates) (Section 16.3).

Iter. invalid. • when a container operation makes iterators or references unusable (e.g. vector reallocation) (Section 5.2).

25.0.10 J

Jitter • variability of latency (not just the mean); isolation and allocation-free paths target lower jitter ([Chapter 15](#)).

25.0.11 K

Kernel bypass • moving packet path out of the kernel networking stack (DPDK, OpenOnload, AF_XDP, ...) to cut syscall/-copy overhead (Chapter 14).

25.0.12 L

Locality • spatial: nearby addresses used together; temporal: same addresses reused soon; drives cache-friendly layout (Chapter 9).

Lock convoy • many threads queueing on one mutex so throughput collapses under contention (Chapter 8).

Lock-free • algorithm that guarantees system-wide progress even if some threads are delayed; typically built with atomics/-CAS, not wait-free for every thread (Chapter 8).

L1 (MD) • market-data top-of-book (BBO) feed depth (Chapter 16).

L2 (MD) • market-data aggregated depth by price level (Chapter 16).

L3 (MD) • market-data full order-by-order / order-id book (Chapter 16).

25.0.13 M

Matching engine • venue component that matches orders and publishes trades / book updates (Chapter 16).

Memory order • C++ atomic ordering: relaxed, acquire, release, acq_rel, seq_cst; controls visibility and reordering (Chapter 8).

MESI • Modified / Exclusive / Shared / Invalid cache-coherence protocol states (Chapter 9).

method chaining • API style where setters / builders return `*this` (or a proxy) so calls can be written `obj.setA(1).setB(2);`; common in the Named Parameter idiom.

mmap • POSIX call that maps files or anonymous pages into a process address space; basis for memory-mapped I/O and some shared memory (Chapter 13).

mock • test double that stands in for a dependency and records / asserts how it was used (calls, arguments); contrasts with a stub, which mostly returns canned data.

most vexing parsing • C++ grammar ambiguity where a declaration looks like an object definition but is parsed as a function declaration (e.g. `TimeKeeper t(Timer());`); fix with extra parentheses or braces.

MPMC • Multi-Producer Multi-Consumer queue; heavier than SPSC (e.g. Michael–Scott) (Chapter 8).

25.0.14 N

Nagle • TCP algorithm that coalesces small sends; disable with `TCP_NODELAY` for latency paths (Chapter 11).

NAPI • Linux networking softirq path that polls the NIC after an interrupt; can still preempt user threads (Chapter 14).

nohz_full • kernel boot option enabling adaptive-ticks / tickless behaviour on listed CPUs (fewer scheduler timer interrupts when a single task runs); often paired with `isolcpus` / `rcu_nocbs` (Section 15.1.2).

NUMA • Non-Uniform Memory Access: multi-socket systems where local DRAM is faster than remote; place threads, memory, and NICs carefully (Chapter 10).

25.0.15 O

Object pool • reuse preconstructed objects instead of allocating on the hot path (Chapter 3).

OpenOnload • Solarflare/Xilinx kernel-bypass stack that accelerates sockets in user space (Chapter 14).

object • region of storage with associated semantics

Order book • local structure of resting bids and asks by price (and often order id); updated from market data (Chapter 17).

Overloading • a compile time polymorphism achieved specifying more than one function with the same name but different signature in the same scope.

Overriding • the act of redefining a method of a base class in a derived class, using the same name, return type and parameters.

25.0.16 P

Page fault • CPU trap when a virtual page is not mapped or not present; major faults hit disk/IO and blow hot-path latency (Chapter 7).

Parameterized type • another definition for [class templates](#).

perf • Linux profiling tool (perf record/stat) for cycles, cache misses, false sharing hunts (Chapter 9).

Pinning • see Affinity.

PMD • Poll-Mode Driver: NIC driver that busy-polls rings (DPDK-style) instead of interrupt-driven RX (Chapter 14).

PMR • Polymorphic Memory Resource (std::pmr): runtime-selected memory_resource behind containers (Section 3.1.2).

poll • POSIX I/O multiplexer; improves on select but still scales worse than epoll for large FD sets (Chapter 12).

Polymorphysm • ability of different objects to be accessed by a common interface.

25.0.17 R

Race Condition • undesirable situation which occurs when two instructions from different threads access the same memory location, at least one of these accesses is a write and there is no synchronization that is mandating any particular order among these accesses.

RAII • acronym for Resource Acquisition Is Initialization.

RBV optimization • see [270].

RCU • Read-Copy-Update: Linux synchronization for read-heavy shared data; writers defer freeing old versions via callbacks. Those callbacks can jitter isolated CPUs unless offloaded with `rcu_nocbs` (Section 15.1.3).

rcu_nocbs • kernel boot option that offloads RCU callback processing from listed CPUs onto housekeeping cores (Section 15.1.3).

Reactor • event-loop style: demultiplex I/O readiness then dispatch handlers (typical `epoll` server) (Chapter 12).

RTTI • acronym for Run-Time Type Information: language mechanism that exposes dynamic type of polymorphic objects (e.g. `typeid`, `dynamic_cast`). Requires a polymorphic type (usually a virtual function); typically implemented via type info reachable from the vtable. Can be disabled with compiler flags (`-fno-rtti`) to save space/-cost.

RVO • acronym for Return Value Optimization.

25.0.18 S

SBE • Simple Binary Encoding: compact, schema-driven binary messages for market data / OE (Section 16.3).

SCHED_FIFO • Linux real-time FIFO scheduling policy; use carefully on dedicated cores (Chapter 7).

SCHED_RR • Linux real-time round-robin scheduling policy (Chapter 7).

select • classic POSIX I/O multiplexer with an FD-count limit and $O(n)$ scans; prefer `poll/epoll` for scale (Chapter 12).

Seq. consistency • strongest C++ atomic default (`memory_order_seq_cst`): a single total order of all `seq_cst` ops (Chapter 8).

smp_affinity • per-IRQ CPU mask under `/proc/irq/<n>/smp_affinity` (or `smp_affinity_list`); steer hard IRQs (e.g. NIC queues) onto housekeeping cores (Section 15.1.4).

softirq • deferred kernel interrupt work (e.g. networking) that can run on a CPU and add jitter (Chapter 14).

Spinlock • busy-wait lock for very short critical sections; burns CPU while waiting (Chapter 8).

Spread • difference between best ask and best bid (Chapter 16).

static typing • types are checked at compile time; variables and expressions have types known to the compiler (as in C++), opposed to dynamic typing where much checking happens at run time.

SFINAE • acronym for **Substitution Failure Is Not An Error**, a technique used in template programming which discards the specialization from the overloaded set avoiding a compile error, when substituting the explicitly specified or deduced type for the template parameter fails.

SPSC • Single-Producer Single-Consumer

queue (often a ring buffer): lock-free handoff between exactly two threads (Section 8.3).

Stack • LIFO-ordered contiguous region of temporary memory whose size is known to the compiler, which manages to allocate and deallocate blocks of memory for the stack variables like functions' local data. . If a region of memory lies on the thread's stack, that memory is said to have been allocated on the stack, i.e. stack-based memory allocation (SBMA).

Stub • test double that provides canned answers to calls; lighter than a mock (which also verifies interaction). In broader software slang, also an unfinished placeholder implementation.

Syscall • trap from user space into the kernel (e.g. `recv`, `epoll_wait`); mode switch cost motivates bypass (Chapter 14).

25.0.19 T

Tail latency • high percentiles of latency (p99 / p999), not the mean; HFT cares about tails and jitter (Chapter 15).

TCP_NODELAY • socket option that disables Nagle coalescing for lower send latency (Chapter 11).

template function • instantiation of a function template

Test-driven Development • [wikipedia](#)

tick-to-trade • end-to-end latency from receiving a market event to sending an order (Chapter 15).

TLB • Translation Lookaside Buffer: CPU cache of virtual→physical address translations (Chapter 9).

Type erasure • hiding a concrete type behind a uniform interface (virtuals, `std::function/any`, or a custom eraser); related but not identical to opaque `void*` payloads (Section 2.1.13).

25.0.20 U

Union-Find • see DSU.

25.0.21 V

virtual member function • a mean to allow derived classes to replace the implementation provided by the base class. The compiler makes sure to call the correct replacement when the object is of the derived class and also when the object is accessed through a pointer to the base rather than to the derived class.

25.0.22 W

Wait-free • stronger than lock-free: every thread completes its operation in a bounded number of its own steps (Chapter 8).

25.0.23 X

XDP • eXpress Data Path: early in-kernel packet hook; AF_XDP exposes XDP rings to user space ([Chapter 14](#)).

25.0.24 Z

Zero-copy • I/O path that avoids copying packet/payload bytes between kernel and user buffers; for MD, parse in place on the ring with a non-owning view (Section 14.3).

Dictionary Bibliography

- [270] *Does return-by-value mean extra copies and extra overhead?* [ISOC++ Wiki](#).
- [271] *SFINAE - Substitution Failure Is Not An Error*. [Cppreference](#).

26

CHAPTER

References

- [1] *The Most Vexing Parse: How to Spot It and Fix It Quickly*. [Fluent C++](#).
- [2] R. Martinho Fernandes. *Rule of Zero*. [ISOC++ Blog](#). 2012.
- [3] Scott Meyers. *A Concern about the Rule of Zero*. [The View from Aristeia - Scott Meyers' Activities and Interests](#). 2014.
- [4] *A polymorphic class should suppress public copy/move*. [Standard Cpp Foundation Github](#).
- [5] *If you define or =delete any copy, move, or destructor function, define or =delete them all*. [Standard Cpp Foundation Github](#).
- [6] *References*. [ISOC++ Wiki](#).
- [7] *Most vexing parse*. [WikiPedia](#).
- [8] *Core C++ - lvalues and rvalues*. [Just Software Solutions](#). 2016.
- [9] *Rvalue References and Perfect Forwarding*. [Just Software Solutions](#). 2008.
- [10] Peter Dimov, Howard E. Hinnant, and Dave Abraham. *The Forwarding Problem: Arguments*. [open-std.org - N1385=02-0043](#).
- [11] Meyers, Sutter, and Alexandrescu. *Session Announcement: Adventures in Perfect Forwarding*. [C++ and Beyond](#). 2011.
- [12] *What are the main purposes of std::forward and which problems does it solve?* [StackOverflow Thread](#).
- [13] *What is the move semantic?* [StackOverflow Thread](#).

- [14] Howard Hinnant. *Everything you wanted to know about Move Semantics*. [Slideshare](#). 2014.
- [15] *If your function must not throw declare it noexcept*. Cpp Core Guidelines.
- [16] *What is the meaning of auto keyword*. [StackOverflow Thread](#).
- [17] Herb Sutter. *auto variables, Part 1*. [Guru of the week](#) - 92.
- [18] Herb Sutter. *auto variables, Part 2*. [Guru of the week](#) - 93.
- [19] Herb Sutter. *AAA style (Almost Always auto)*. [Guru of the week](#) - 94.
- [20] *What's In a Class? - The Interface Principle*. [Guru of the week](#) - C++ Report, 10(3), March 1998.
- [21] *Declaring non-type template arguments with auto*. Programming Language C++, Evolution Working Group - P0127R1. 2016.
- [22] *Declaring non-type template parameters with auto*. Programming Language C++, Evolution Working Group - P0127R2. 2016.
- [23] *Pros and Cons of Alternative Function Syntax in C++*. Petr Zemek's Blog of a Software Engineer.
- [24] *ADL - Argument-dependent lookup*. [Cppreference](#).
- [25] *Lambda Expressions*. [Cppreference](#).
- [26] Crowl Larvi Freeman. *Lambda Expressions and Closures: Wording for Monomorphic Lambdas (Revision 4)*. [open-std.org](#). 2008.
- [27] Vandevorde. *New wording for C++0x Lambdas*. [open-std.org](#). 2009.
- [28] *Closure (Computer Programming)*. [Wikipedia](#).
- [29] *Abbreviated function template*. [Cppreference](#).
- [30] *decltype* specifier. [Cppreference](#).
- [31] *Member Function Pointers and the Fastest Possible C++ Delegates*. [codeproject.com](#).
- [32] Stroustrup and Dos Reis. *Initializer list (Rev. 3)*. [open-std.org](#). 2007.
- [33] Merrill and Vandevorde. *Initializer Lists - Alternative Mechanism and Rationale (v.2)*. [Initializer Lists — Alternative Mechanism and Rationale \(v. 2\)](#). 2008.
- [34] Thorsten Ottosen. *Wording for range-based for-loop (revision 2)*. [open-std.org](#). 2007.
- [35] *Range-based for statements and ADL*. [open-std.org](#).
- [36] Svoboda. *Delete operators default to noexcept*. [open-std.org](#). 2010.
- [37] Mauer. *Deducing noexcept for destructors*. [open-std.org](#). 2010.
- [38] Abrahams, Sharoni, and Gregor. *Allowing Move Constructors to Throw (Rev. 1)*. [open-std.org](#). 2010.
- [39] Bjarne Stroustrup. *Bjarne Stroustrup's C++ Style and Technique FAQ*. [stroustrup.com](#).
- [40] Bjarne Stroustrup. *Exception Safety: Concepts and Techniques*. [stroustrup.com](#).

- [41] *Static Initialization Order Fiasco*. cppreference.com.
- [42] *Empty base optimization*. cppreference.com.
- [43] *Apache C++ Standard Template Library*. apache.org.
- [44] *Lapack: Linear Algebra Subroutine Library*. siam.org.
- [45] *Netlib*. netlib.org.
- [46] *Available C++ Libraries FAQ maintained by Nikki Locke*. trumphurst.com.
- [47] *Mathtools - A Resource for Technical Computing Community*. mathtools.net.
- [48] *boost Library*. boost.org.
- [49] *Most Vexing Parse*. Wiki [Most vexing parse](http://Mostvexingparse.com).
- [50] *Const Correctness*. [ISOC++ Wiki](http://ISOC++Wiki.com).
- [51] *More C++ Idioms*. [Wikibooks](http://Wikibooks.org).
- [52] *Pimpl Idiom*. Wiki [Wiki Web](http://WikiWeb.com).
- [53] *PImpl Idiom*. [cppreference](http://cppreference.com).
- [54] *Calling Virtuals from Constructors: Dynamic Binding During Initialization Idiom*. [ISOC++ Wiki](http://ISOC++Wiki.com).
- [55] *CRTP - Curiously Recurring Template Pattern*. Wiki [Wiki Web](http://WikiWeb.com).
- [56] *CRTP - Curiously Recurring Template Pattern*. Cplusplus.com.
- [57] Herb Sutter. *The Fast Pimpl Idiom*. [Guru of the Week](http://Guruoftheweek.com) - 28.
- [58] Herb Sutter. *Compilation Firewalls*. [Guru of the Week](http://Guruoftheweek.com) - 24.
- [59] Herb Sutter. *Compilation Firewalls (C++11-updated version of GotW #24)*. [Guru of the Week](http://Guruoftheweek.com) - 100.
- [60] Herb Sutter. *The Joy of Pimpls (or more about the Compiler-Firewall Idiom)*. [More About Compiler Firewalls](http://MoreAboutCompilerFirewalls.com).
- [61] *What is the copy and swap idiom?* [Stackoverflow](http://Stackoverflow.com).
- [62] Herb Sutter. *Exception-Safe Class Design, Part I: Copy Assignment*. [Guru of the Week](http://Guruoftheweek.com) - 59.
- [63] Herb Sutter. *Exception-Safe Class Design, Part II: Inheritance*. [Guru of the Week](http://Guruoftheweek.com) - 60.
- [64] *What is the Named Constructor Idiom?* [ISOC++ Wiki](http://ISOC++Wiki.com).
- [65] *Construct On First Use Idiom*. [ISOC++ Wiki](http://ISOC++Wiki.com).
- [66] *Nifty Counter Idiom*. [ISOC++ Wiki](http://ISOC++Wiki.com).
- [67] *Nifty Counter Idiom*. [Wikibook](http://Wikibook.org).
- [68] *Named Parameter Idiom*. [ISOC++ Wiki](http://ISOC++Wiki.com).
- [69] *Virtual Friend Function Idiom*. [ISOC++ Wiki](http://ISOC++Wiki.com).
- [70] *C++ Core Guidelines*. [Standard Cpp Foundation Github](http://StandardCppFoundationGithub.com).

- [71] Anton Eliens. *The Handle/Body Idiom*. Vrije Universiteit Amsterdam.
- [72] *Resource acquisition is initialization (RAII)*. Wikipedia.
- [73] *Type Erasure*. Wikibooks.
- [74] Andrii Polyanskyy. *C++ idioms everyone should know*. YouTube — Engineering Community.
- [75] Herb Sutter. *Leak-Freedom in C++ ... By Default* — CppCon 2016. CppCon YouTube Video. 2016.
- [76] *Memory Management: Stack and Heap*. Weber CS1410 notes.
- [77] *malloc*. Cppreference.
- [78] *calloc*. Cppreference.
- [79] *aligned alloc*. Cppreference.
- [80] *realloc*. Cppreference.
- [81] *free*. Cppreference.
- [82] *new expression*. Cppreference.
- [83] *delete expression*. Cppreference.
- [84] *operator new*. Cppreference.
- [85] *operator delete*. Cppreference.
- [86] *std::pmr::unsynchronized_pool_resource*. Cppreference.
- [87] *Object pool pattern*. Wikipedia.
- [88] *AllocatorAwareContainer / std::allocator*. Cppreference — Allocator.
- [89] *std::pmr (polymorphic memory resources)*. Cppreference — memory_resource.
- [90] *Dynamic Memory Management Library*. Cppreference.
- [91] *unique pointer*. Cppreference.
- [92] *shared pointer*. Cppreference.
- [93] *weak pointer*. Cppreference.
- [94] *auto pointer*. Cppreference.
- [95] *std::make_unique*. Cppreference.
- [96] *std::make_shared*. Cppreference.
- [97] *std::enable_shared_from_this*. Cppreference.
- [98] *Containers Library*. Cppreference.
- [99] *array*. Cppreference.
- [100] *vector*. Cppreference.
- [101] *deque*. Cppreference.
- [102] *forward list*. Cppreference.

- [103] *list*. [Cplusplusreference](#).
- [104] *set*. [Cplusplusreference](#).
- [105] *map*. [Cplusplusreference](#).
- [106] *multiset*. [Cplusplusreference](#).
- [107] *multimap*. [Cplusplusreference](#).
- [108] *unordered set*. [Cplusplusreference](#).
- [109] *unordered map*. [Cplusplusreference](#).
- [110] *unordered multiset*. [Cplusplusreference](#).
- [111] *unordered multimap*. [Cplusplusreference](#).
- [112] *stack*. [Cplusplusreference](#).
- [113] *queue*. [Cplusplusreference](#).
- [114] *priority queue*. [Cplusplusreference](#).
- [115] *flat set*. [Cplusplusreference](#).
- [116] *flat map*. [Cplusplusreference](#).
- [117] *flat multiset*. [Cplusplusreference](#).
- [118] *flat multimap*. [Cplusplusreference](#).
- [119] *std::span*. [Cplusplusreference](#).
- [120] *std::basic_string_view*. [Cplusplusreference](#).
- [121] Crawl McKenney Bohem. *C++ Data-Dependency Ordering: Atomics and Memory Model*. ISO/IEC JTC1 SC22 WG21 N2556. 2008.
- [122] *Memory model synchronization modes*. [GNU gcc Wiki](#).
- [123] *What do each memory_order mean?* [StackOverflow Thread](#).
- [124] *Atomic's memory orders, what for?* [YouTube](#).
- [125] *std::atomic memory_orders compared*. [YouTube](#).
- [126] *Standard library header <thread>*. [Cplusplusreference](#).
- [127] *std::memory_order*. [Cplusplusreference](#).
- [128] *std::mutex*. [Cplusplusreference](#).
- [129] *std::shared_mutex*. [Cplusplusreference](#).
- [130] *std::condition_variable*. [Cplusplusreference](#).
- [131] *fork(2)*. [Linux man-pages](#).
- [132] *clone(2)*. [Linux man-pages](#).
- [133] *getpid(2)*. [Linux man-pages](#).
- [134] *pthread_setaffinity_np(3)*. [Linux man-pages](#).
- [135] *sched_setaffinity(2)*. [Linux man-pages](#).

- [136] *sched_getaffinity(2)*. [Linux man-pages](#).
- [137] *sched_setscheduler(2)*. [Linux man-pages](#).
- [138] *pthread_setschedparam(3)*. [Linux man-pages](#).
- [139] *sched(7)*. [Linux man-pages](#).
- [140] *std::memory_order*. [Cplusplusreference](#).
- [141] *std::atomic*. [Cplusplusreference](#).
- [142] *Treiber stack*. [Wikipedia](#).
- [143] *Michael–Scott queue (Non-blocking linked list)*. [Wikipedia](#).
- [144] Maged M. Michael and Michael L. Scott. “Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms”. In: *PoPP*. 1996.
- [145] *perf: Linux profiling with performance counters*. [perf Wiki](#).
- [146] *Cache coherence (MESI / MESIF overview)*. [Wikipedia — MESI protocol](#).
- [147] *numa(3) / numa(7)*. [Linux man-pages](#).
- [148] *numactl(8)*. [Linux man-pages](#).
- [149] *socket(2)*. [Linux man-pages](#).
- [150] *fcntl(2)*. [Linux man-pages](#).
- [151] *tcp(7)*. [Linux man-pages](#).
- [152] *socket(7)*. [Linux man-pages](#).
- [153] Brian “Beej” Hall. *Beej’s Guide to Network Programming*. [beej.us](#).
- [154] *select(2)*. [Linux man-pages](#).
- [155] *poll(2)*. [Linux man-pages](#).
- [156] *epoll(7)*. [Linux man-pages](#).
- [157] *epoll_ctl(2)*. [Linux man-pages](#).
- [158] *open(2)*. [Linux man-pages](#).
- [159] *mmap(2)*. [Linux man-pages](#).
- [160] *munmap(2)*. [Linux man-pages](#).
- [161] *shm_open(3)*. [Linux man-pages](#).
- [162] *DPDK Documentation*. [doc.dpdk.org](#).
- [163] *OpenOnload / TCPDirect Documentation*. [AMD/Xilinx Onload](#).
- [164] *AF_XDP*. [Linux kernel docs](#).
- [165] *CPU isolation (isolcpus)*. [kernel.org admin-guide](#).
- [166] *NO_HZ: Reducing Scheduling-Clock Ticks*. [kernel.org](#).
- [167] *RCU and rcu_nocbs*. [kernel.org RCU](#).
- [168] *IRQ affinity (smp_affinity / irqaffinity)*. [kernel.org IRQ affinity](#).

- [169] *busy_poll* (socket busy polling). [socket\(7\)](#).
- [170] John C. Hull. *Options, Futures, and Other Derivatives*. Pearson, 2021.
- [171] Steven E. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer, 2004.
- [172] Paul Wilmott. *Paul Wilmott on Quantitative Finance*. Wiley, 2006.
- [173] *Black–Scholes model*. [Wikipedia](#).
- [174] *Greeks (finance)*. [Wikipedia](#).
- [175] *Put–call parity*. [Wikipedia](#).
- [176] *Volatility smile*. [Wikipedia](#).
- [177] *Monte Carlo methods for option pricing*. [Wikipedia](#).
- [178] *std::erfc*. [Cplusplusreference](#).
- [179] *Financial Information eXchange*. [Wikipedia](#).
- [180] *What is FIX?* [FIX Trading Community](#).
- [181] Thomas H. Cormen et al. *Introduction to Algorithms*. 3rd. MIT Press, 2009.
- [182] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley.
- [183] Steven S. Skiena. *The Algorithm Design Manual*. Springer.
- [184] *Quicksort*. [Wikipedia](#).
- [185] *Merge Sort*. [Wikipedia](#).
- [186] *Timsort*. [Wikipedia](#).
- [187] *Heapsort*. [Wikipedia](#).
- [188] *Bubble Sort*. [Wikipedia](#).
- [189] *Insertion Sort*. [Wikipedia](#).
- [190] *Selection Sort*. [Wikipedia](#).
- [191] *Shell Sort*. [Wikipedia](#).
- [192] *Bucket Sort*. [Wikipedia](#).
- [193] *Radix Sort*. [Wikipedia](#).
- [194] *NeetCode Roadmap*. [neetcode.io/roadmap](#).
- [195] Thomas H. Cormen et al. *Introduction to Algorithms*. 3rd. MIT Press, 2009.
- [196] Steven S. Skiena. *The Algorithm Design Manual*. Springer.
- [197] *Design Patterns*. [Refactoring Guru](#).
- [198] *Factory Method Design Pattern*. [Wikipedia](#).
- [199] *Abstract Factory Design Pattern*. [Wikipedia](#).
- [200] *Builder Design Pattern*. [Wikipedia](#).
- [201] *Prototype Design Pattern*. [Wikipedia](#).

- [202] *Singleton Design Pattern*. [Wikipedia](#).
- [203] *Chain of Responsibility Design Pattern*. [Wikipedia](#).
- [204] *Command Design Pattern*. [Wikipedia](#).
- [205] *Iterator Design Pattern*. [Wikipedia](#).
- [206] *Mediator Design Pattern*. [Wikipedia](#).
- [207] *Memento Design Pattern*. [Wikipedia](#).
- [208] *Observer Design Pattern*. [Wikipedia](#).
- [209] *State Design Pattern*. [Wikipedia](#).
- [210] *Strategy Design Pattern*. [Wikipedia](#).
- [211] *Template Method Design Pattern*. [Wikipedia](#).
- [212] *Visitor Design Pattern*. [Wikipedia](#).
- [213] *Adapter Design Pattern*. [Wikipedia](#).
- [214] *Bridge Design Pattern*. [Wikipedia](#).
- [215] *Composite Design Pattern*. [Wikipedia](#).
- [216] *Decorator Design Pattern*. [Wikipedia](#).
- [217] *Facade Design Pattern*. [Wikipedia](#).
- [218] *Flyweight Design Pattern*. [Wikipedia](#).
- [219] *Proxy Design Pattern*. [Wikipedia](#).
- [220] Robert C. Martin. *Principles of OOD*. [WayBack Machine](#). 2014.
- [221] Robert C. Martin. *Getting a SOLID start*. [Uncle Bob Consulting LLC](#). 2009.
- [222] Sandu Metz. *SOLID Object-Oriented*. [Ghost Archive - Gotham Ruby Conference](#). 2009.
- [223] Robert C. Martin. *Design Principles and Design Patterns*. [Internet Archive - Objectmentor](#). 2000.
- [224] Robert C. Martin. *Single Responsibility Principle*. [Internet Archive - Objectmentor](#). 2015.
- [225] Robert C. Martin. *Agile Software Development, Principles, Patterns, and Practices*. [Internet Archive - Objectmentor](#). 2003.
- [226] *Open/Closed Principle*. [Internet Archive - Objectmentor](#).
- [227] *Liskov Substitution Principle*. [Internet Archive - Objectmentor](#).
- [228] *Interface Segregation Principle*. [Internet Archive - Objectmentor](#).
- [229] *Dependency Inversion Principle*. [Internet Archive - Objectmentor](#).
- [230] *Clean Architecture: A Craftman's Guide to Software Structure and Design*.
- [231] *Clean Architecture: A Craftman's Guide to Software Structure and Design*. [here](#).

- [232] *The Unified Modeling Language*. uml-diagrams.org.
- [233] *UML Core Elements*. uml-diagrams.org.
- [234] *UML Association vs. Aggregation vs. Composition*. visual-paradigm.com.
- [235] *Sequence Diagrams Reference*. uml-diagrams.org.
- [236] *Object-Oriented Programming*. [Wikipedia - Object Oriented Programming](https://en.wikipedia.org/wiki/Object-oriented_programming).
- [237] *4+1 architectural view model*. [Wikipedia - 4+1 architectural view model](https://en.wikipedia.org/wiki/4+1_architectural_view_model).
- [238] *The Sequence Diagram*. [Sequence Diagram Templates and Examples](https://www.sequence-diagram.com).
- [239] *UML 2 Tutorial - Sequence Diagram*. [developer.ibm.com/articles](http://developer.ibm.com/articles/uml2tutorial) .
- [240] *The Sequence Diagram*. [developer.ibm.com/articles](http://developer.ibm.com/articles/uml2tutorial) .
- [241] *Object-Oriented Programming Using C++*. Weber State University - School of Computing - Computer Science.
- [242] Allen Holub. *Allen Holub's UML Quick Reference*. [Allen Holub website](http://www.alanhulub.com).
- [243] *Class Diagram*. [WikiPedia - Class Diagram](https://en.wikipedia.org/wiki/Class_Diagram).
- [244] *State Diagram*. [WikiPedia - State Diagram](https://en.wikipedia.org/wiki/State_Diagram).
- [245] *State Machine*. [WikiPedia - State Machine](https://en.wikipedia.org/wiki/State_machine).
- [246] *Programming paradigm*. [WikiPedia - Programming Paradigm](https://en.wikipedia.org/wiki/Programming_paradigm).
- [247] *Composition over Inheritance*. [WikiPedia - Composition over Inheritance](https://en.wikipedia.org/wiki/Composition_over_inheritance).
- [248] *Delegation (object-oriented programming)*. [WikiPedia - Delegation](https://en.wikipedia.org/wiki/Delegation_(object-oriented_programming)).
- [249] *Role-oriented programming*. [WikiPedia - Role-oriented programming](https://en.wikipedia.org/wiki/Role-oriented_programming).
- [250] James Rumbaugh. *Object-Oriented Modeling and Design*. [Internet Archive](http://www.internetarchive.org). 1991.
- [251] *Delegation (object-oriented programming)*. [WikiPedia - Delegation](https://en.wikipedia.org/wiki/Delegation_(object-oriented_programming)).
- [252] *Python Glossary*. [docs.python.org](https://docs.python.org/3/glossary.html).
- [253] *Providing Multiple Constructors in Your Python Classes*. [Real Python](https://realpython.com).
- [254] *Sorting Mini How To*. [wiki.python.org](https://wiki.python.org/moin/SortingMiniHowTo).
- [255] *Thread-based parallelism*. [docs.python.org](https://docs.python.org/3/library/threading.html).
- [256] *Process-based parallelism*. [docs.python.org](https://docs.python.org/3/library/multiprocessing.html).
- [257] *Asynchronous I/O*. [docs.python.org](https://docs.python.org/3/library/asyncio.html).
- [258] *Asyncio in Python - Full Tutorial*. [YouTube](https://www.youtube.com/watch?v=88e33333333).
- [259] *Open Lecture by James Bach on Software Testing*. [Eesti Infotehnoloogia Kolledž YouTube Channel](https://www.youtube.com/channel/UC88e33333333).
- [260] *Regression Testing*. [Wikipedia](https://en.wikipedia.org/wiki/Regression_testing).
- [261] *Progression Testing*. [Wikipedia](https://en.wikipedia.org/wiki/Progression_testing).
- [262] *Regression Testing vs Progression Testing*. [Test Sigma](https://testsigma.com).
- [263] Kent Beck. *Simple Smalltalk Testing: With Patterns*. [webarchive](http://web.archive.org).

- [264] [Implementazioni e spiegazioni da uno dei migliori ricercatori del campo.](#)
- [265] [IL libro di introduzione.](#)
- [266] [Come il primo link, più in depth con più matematica.](#)
- [267] [Ricetta per allenare modelli.](#)
- [268] [Pytorch, la libreria per ml in python.](#)
- [269] [ML fundamentals.](#)
- [270] [Does return-by-value mean extra copies and extra overhead? ISOC++ Wiki.](#)
- [271] [SFINAE - Substitution Failure Is Not An Error. Cppreference.](#)